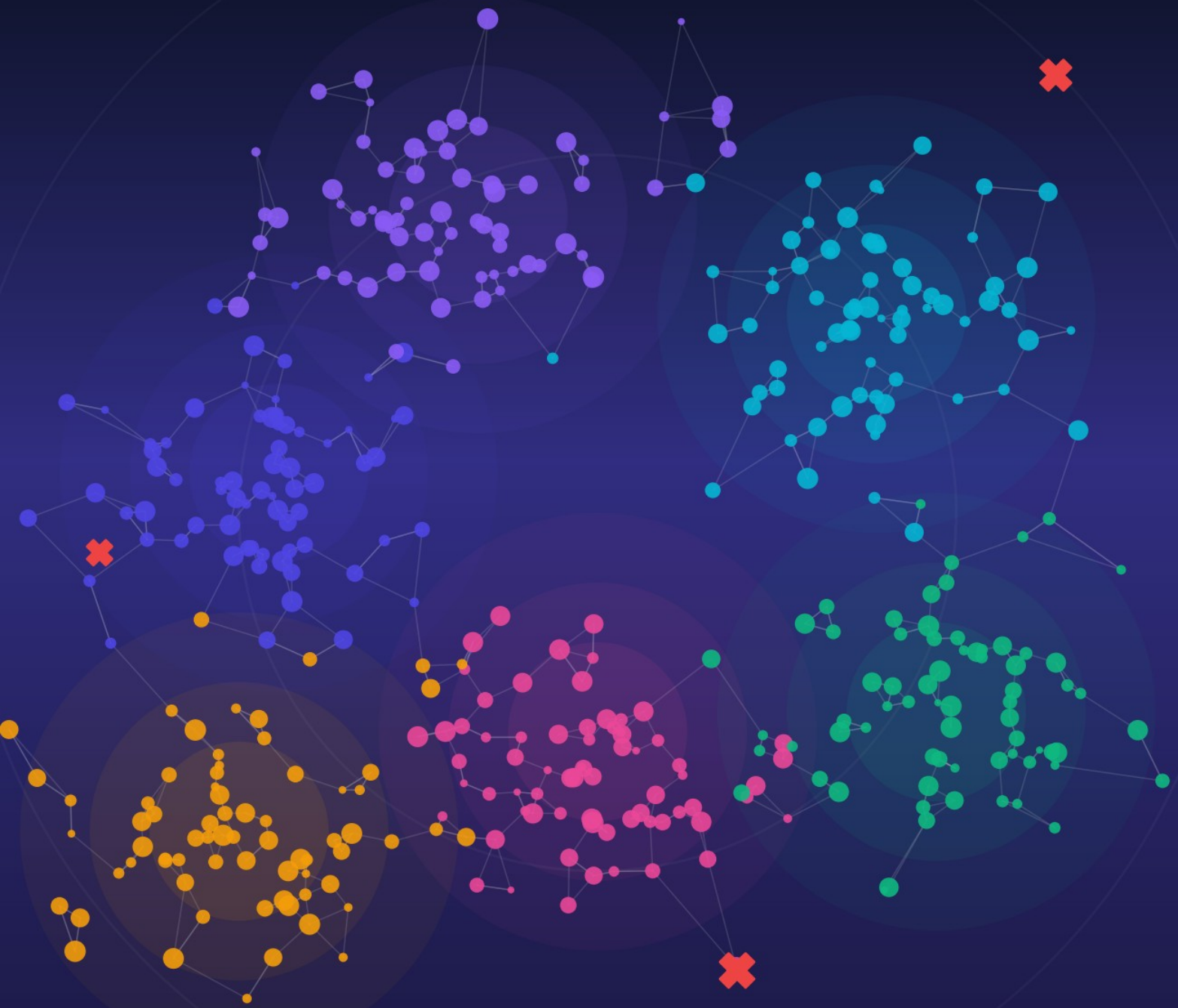
2026



APRENDIZADO NÃO SUPERVISIONADO

Do zero à prática com Python

Araken de Medeiros Santos

APRENDIZADO NÃO SUPERVISIONADO

Do zero à prática com Python

Araken de Medeiros Santos

2026

Sumário

Apresentação **15**
Para quem é este livro 15
Como o livro está organizado 15
Como estudar com este livro 15
Caixas e símbolos do livro 16
Sobre os códigos 16
Bases públicas da UCI e do Kaggle 17
Leituras recomendadas 17

Parte I — Fundamentos **18**

Capítulo 1 — Bem-vindo ao Aprendizado Não Supervisionado **19**
1.1 Uma história para começar 19
1.2 O que é aprender com dados? 20
1.3 Os grandes paradigmas de aprendizado 20
1.3.1 Aprendizado supervisionado 21
1.3.2 Aprendizado não supervisionado 21
1.3.3 Aprendizado semissupervisionado 21
1.3.4 Aprendizado auto-supervisionado 21
1.3.5 Aprendizado por reforço 22
1.4 Por que estudar aprendizado não supervisionado? 22
1.5 As quatro grandes famílias de tarefas 22
1.5.1 Agrupamento (clustering) 23
1.5.2 Redução de dimensionalidade 23
1.5.3 Regras de associação 23
1.5.4 Detecção de anomalias 23
1.6 Um pouco de história 24
1.7 Como é um projeto na prática? 24
1.8 Preparando o ambiente Python 25
1.9 Seu primeiro agrupamento 26
1.10 Cuidados éticos 27
1.11 Estrutura deste livro 27
Resumo do capítulo 28
Exercícios de múltipla escolha 29
Exercícios discursivos 33
Exercícios práticos em Python (scikit-learn) 34
Exercício 1 — Primeiro contato com o conjunto Wine 34
Exercício 2 — O efeito da escala 34
Exercício 3 — Enxergando os grupos com PCA 34
Exercício 4 — A sorte da inicialização 34
Exercício 5 — O algoritmo sempre responde 34
Projeto integrador do capítulo 35
Primeiro diagnóstico de um conjunto de dados 35

Capítulo 2 — Dados, Distâncias e Pré-processamento **36**
2.1 Por que começar pelos dados? 36
2.2 Dados como uma tabela de números 36
2.2.1 Tipos de atributos 37
2.3 Medindo semelhança: as distâncias 38
2.3.1 Distância Euclidiana 38
2.3.2 Distância de Manhattan 38

2.3.3 Distância de Chebyshev....38
2.3.4 Distância de Minkowski: a família completa....38
2.3.5 Distância (e similaridade) do cosseno....39
2.3.6 Distância de Hamming e de Jaccard....40
2.3.7 Dados mistos: a distância de Gower....40
2.4 Distâncias no Python....41
2.5 O problema da escala....41
2.5.1 Padronização (z-score)....42
2.5.2 Normalização min-max....42
2.5.3 Escalonamento robusto....42
2.5.4 Qual escolher?....42
2.6 Valores ausentes....43
2.7 Codificando categorias....43
2.7.1 Codificação one-hot....43
2.7.2 Codificação ordinal....44
2.8 Selecionando e criando atributos....44
2.9 A maldição da dimensionalidade....45
2.9.1 O espaço fica vazio....45
2.9.2 Todas as distâncias ficam parecidas....45
2.9.3 O que fazer?....46
2.10 Um pipeline limpo no scikit-learn....46
Resumo do capítulo....47
Exercícios de múltipla escolha....48
Exercícios discursivos....52
Exercícios práticos em Python (scikit-learn)....53
Exercício 1 — Matriz de distâncias entre clientes....53
Exercício 2 — Três escalonadores diante de um outlier....53
Exercício 3 — Imputar: mediana ou vizinhos?....53
Exercício 4 — Codificando variáveis categóricas....53
Exercício 5 — A maldição da dimensionalidade em números....53
Projeto integrador do capítulo....54
Pipeline de preparação para uma carteira de clientes....54
Parte II — Agrupamento....56
Capítulo 3 — K-means: o Clássico do Agrupamento....57
3.1 A analogia da pizzaria....57
3.2 O algoritmo, passo a passo....57
3.2.1 Exemplo numérico minúsculo....58
3.3 A matemática por trás (sem medo)....58
3.4 Colocando a mão na massa....59
3.5 O problema da inicialização....60
3.6 Quantos grupos? O método do cotovelo....60
3.7 Quando o K-means falha....61
3.8 Variantes importantes....62
3.8.1 Mini-Batch K-means....62
3.8.2 K-medoids (PAM)....62
3.8.3 K-modes....63
3.8.4 Fuzzy C-means....63
3.9 Estudo de caso 1: segmentação de clientes....63
3.10 Estudo de caso 2: compressão de imagem por cores....63
3.11 Boas práticas....64
Resumo do capítulo....65
Exercícios de múltipla escolha....66

Exercícios discursivos ... 70
Exercícios práticos em Python (scikit-learn) ... 71
Exercício 1 — K-means "do zero" com NumPy ... 71
Exercício 2 — Escolhendo k com inércia e silhueta ... 71
Exercício 3 — Aleatória ou k-means++? ... 71
Exercício 4 — Mini-Batch K-means em dados grandes ... 71
Exercício 5 — Quantização de cores de uma foto ... 71
Projeto integrador do capítulo ... 72
Segmentação RFM de uma loja ... 72
Capítulo 4 — Agrupamento Hierárquico: a Árvore de Semelhanças ... 73
4.1 A árvore genealógica dos dados ... 73
4.2 Duas estratégias ... 74
4.2.1 Aglomerativa (de baixo para cima) ... 74
4.2.2 Divisiva (de cima para baixo) ... 74
4.3 O dendrograma ... 74
4.3.1 Cortando a árvore ... 75
4.4 Como medir a distância entre grupos? As ligações ... 76
4.4.1 Ligação simples (vizinho mais próximo) ... 76
4.4.2 Ligação completa (vizinho mais distante) ... 76
4.4.3 Ligação média (UPGMA) ... 76
4.4.4 Ligação de Ward ... 77
4.5 Um exemplo à mão ... 77
4.6 O algoritmo e seu custo ... 77
4.7 Hierárquico no Python ... 78
4.7.1 Com SciPy: dendrograma completo ... 78
4.7.2 Com scikit-learn ... 78
4.7.3 Correlação cofenética ... 78
4.8 Mapas de calor com dendrogramas (clustermaps) ... 79
4.9 Vantagens e limitações ... 79
4.10 Hierárquico vs. K-means ... 80
Resumo do capítulo ... 80
Exercícios de múltipla escolha ... 81
Exercícios discursivos ... 85
Exercícios práticos em Python (scikit-learn) ... 86
Exercício 1 — O dendrograma à mão, conferido pelo computador ... 86
Exercício 2 — Qual ligação reproduz melhor as classes? ... 86
Exercício 3 — Cortando a árvore por altura ... 86
Exercício 4 — O efeito corrente ... 86
Exercício 5 — Agrupando exemplos e atributos ao mesmo tempo ... 86
Projeto integrador do capítulo ... 88
A árvore dos dígitos ... 88
Capítulo 5 — Agrupamento por Densidade: DBSCAN, OPTICS e HDBSCAN ... 89
5.1 Ilhas e oceano ... 89
5.2 O DBSCAN ... 90
5.2.1 Os dois parâmetros ... 90
5.2.2 Os três tipos de ponto ... 90
5.2.3 Alcançabilidade e conectividade ... 91
5.2.4 O algoritmo em palavras ... 91
5.3 DBSCAN no Python ... 91
5.4 Como escolher eps e min_samples? ... 92
5.4.1 Regra para min_samples ... 92
5.4.2 O gráfico de k-distâncias ... 92

5.4.3 Efeitos de eps....93
5.5 Pontos fortes e fracos do DBSCAN....94
5.6 OPTICS: a ordenação por alcançabilidade....94
5.7 HDBSCAN: densidade hierárquica....95
5.8 Outras ideias de densidade....96
5.9 Estudo de caso: pontos de táxi em uma cidade....96
5.10 Comparando os métodos....97
Resumo do capítulo....97
Exercícios de múltipla escolha....98
Exercícios discursivos....102
Exercícios práticos em Python (scikit-learn)....103
Exercício 1 — Varrendo o parâmetro eps....103
Exercício 2 — Achando o joelho do gráfico de k-distâncias....103
Exercício 3 — DBSCAN, K-means e espectral em anéis....103
Exercício 4 — Densidades variáveis: OPTICS e HDBSCAN....103
Exercício 5 — Pontos quentes com a distância haversine....103
Projeto integrador do capítulo....104
Pontos quentes de ocorrências urbanas....104
Capítulo 6 — Modelos de Mistura Gaussiana e Agrupamento Espectral....105
6.1 Dois olhares diferentes sobre "grupo"....105
6.2 A distribuição normal em dois minutos....105
6.2.1 A normal em várias dimensões....105
6.3 A ideia da mistura....106
6.4 O problema do "ovo e a galinha" e o algoritmo EM....107
6.4.1 Passo E (Esperança)....107
6.4.2 Passo M (Maximização)....107
6.4.3 Repetir....107
6.5 Agrupamento rígido e suave....107
6.6 Tipos de covariância....108
6.7 GMM no Python....108
6.8 Quantos componentes? AIC e BIC....109
6.9 Cuidados com o GMM....110
6.10 Agrupamento espectral....110
6.10.1 Passo 1: construir o grafo de similaridade....110
6.10.2 Passo 2: a matriz Laplaciana....110
6.10.3 Passo 3: usar os menores autovetores....111
6.10.4 Passo 4: K-means no novo espaço....111
6.10.5 O corte normalizado....112
6.10.6 Escolhendo o número de grupos: o eigengap....112
6.10.7 Custos e limitações....112
6.11 Comparação final....112
Resumo do capítulo....113
Exercícios de múltipla escolha....114
Exercícios discursivos....118
Exercícios práticos em Python (scikit-learn)....119
Exercício 1 — Tipos de covariância no Iris....119
Exercício 2 — Quantas gaussianas?....119
Exercício 3 — Pontos ambíguos e pertinência suave....119
Exercício 4 — GMM como detector de anomalias....119
Exercício 5 — Espectral: o papel da afinidade....119
Projeto integrador do capítulo....120
Perfis com probabilidades: uma operadora de telecomunicações....120

Capítulo 7 — Como Saber se os Grupos São Bons? Avaliação e Validação 121
7.1 O dilema do juiz sem regras 121
7.2 Antes de tudo: existe estrutura nos dados? 122
7.3 Validação interna 123
7.3.1 Coesão e separação 123
7.3.2 O coeficiente de silhueta 123
7.3.3 Índice de Calinski-Harabasz 124
7.3.4 Índice de Davies-Bouldin 124
7.3.5 Índice de Dunn 124
7.4 Escolhendo k de modo mais rigoroso 125
7.4.1 A estatística gap 125
7.4.2 Estabilidade 126
7.5 Validação externa: quando há um gabarito 127
7.5.1 Matriz de contingência 127
7.5.2 Pureza 128
7.5.3 Índice de Rand e Rand ajustado (ARI) 128
7.5.4 Informação mútua normalizada (NMI) 128
7.5.5 Homogeneidade, completude e V-measure 128
7.6 Validação pelo uso e pela interpretação 129
7.6.1 Perfil dos grupos 129
7.7 Um roteiro prático de avaliação 129
Resumo do capítulo 130
Exercícios de múltipla escolha 131
Exercícios discursivos 135
Exercícios práticos em Python (scikit-learn) 136
Exercício 1 — Há grupos para encontrar? A estatística de Hopkins 136
Exercício 2 — Silhueta por grupo e por ponto 136
Exercício 3 — Um painel de índices internos 136
Exercício 4 — Avaliação externa de três algoritmos 136
Exercício 5 — Estabilidade para escolher k 136
Projeto integrador do capítulo 137
Painel de avaliação de agrupamentos 137

Parte III — Redução de Dimensionalidade 138

Capítulo 8 — PCA e Métodos Lineares: SVD, ICA e NMF 139
8.1 Por que reduzir dimensões? 139
8.1.1 Seleção versus extração 139
8.2 A intuição do PCA 139
8.2.1 A analogia da sombra 140
8.2.2 Informação = variância 140
8.3 Como o PCA funciona, passo a passo 141
8.3.1 Um exemplo mínimo 141
8.4 A relação com a SVD 141
8.5 PCA no Python 141
8.6 Quantas componentes manter? 142
8.6.1 Variância acumulada 142
8.6.2 Gráfico scree (de declive) 142
8.6.3 Critério de Kaiser 143
8.6.4 Validação cruzada e análise paralela 143
8.7 Interpretando o resultado 143
8.7.1 Escores e cargas 143
8.7.2 O biplot 143
8.8 PCA para reconstruir e comprimir 144

8.8.1 Remoção de ruído....144
8.9 Pontos de atenção do PCA.... 144
8.10 Variações do PCA.... 145
8.10.1 PCA incremental....145
8.10.2 PCA aleatorizado....145
8.10.3 PCA esparso....145
8.10.4 PCA com kernel....145
8.11 Análise de Componentes Independentes (ICA).... 146
8.11.1 O problema da festa (cocktail party)....146
8.11.2 A ideia....146
8.12 Fatoração de Matrizes Não Negativas (NMF).... 147
8.12.1 A ideia: montar com peças....147
8.12.2 Aplicação em texto: descoberta de tópicos....147
8.13 Comparação das técnicas lineares.... 148
Resumo do capítulo.... 148
Exercícios de múltipla escolha.... 149
Exercícios discursivos.... 153
Exercícios práticos em Python (scikit-learn).... 154
Exercício 1 — Quantas componentes para 90% da variância?....154
Exercício 2 — Reconstrução e compressão de dígitos....154
Exercício 3 — PCA "na unha" com autovetores....154
Exercício 4 — Kernel PCA em anéis....154
Exercício 5 — Separando fontes com ICA....154
Projeto integrador do capítulo.... 155
Compressão, limpeza e partes de dígitos....155
Capítulo 9 — Aprendizado de Variedades: MDS, Isomap, t-SNE e UMAP....156
9.1 Quando a linha reta não basta.... 156
9.2 A hipótese da variedade.... 156
9.3 MDS: Escalonamento Multidimensional.... 157
9.3.1 A ideia do mapa de cidades....157
9.3.2 Dois sabores....157
9.4 Isomap: distâncias geodésicas.... 158
9.5 LLE: Embedding Linear Local.... 159
9.6 t-SNE.... 159
9.6.1 O problema que ele resolve....159
9.6.2 A ideia, em linguagem simples....159
9.6.3 Por que a distribuição t?....160
9.6.4 A perplexidade....160
9.6.5 Como ler um t-SNE (e como não ler)....161
9.6.6 Dicas práticas....161
9.7 UMAP.... 161
9.7.1 O que é....161
9.7.2 A ideia (sem a matemática pesada)....161
9.7.3 Parâmetros principais....161
9.7.4 UMAP versus t-SNE....162
9.8 Outras técnicas relevantes.... 163
9.9 Como avaliar uma projeção?.... 163
9.10 Um guia de escolha.... 163
Resumo do capítulo.... 163
Exercícios de múltipla escolha.... 165
Exercícios discursivos.... 169
Exercícios práticos em Python (scikit-learn).... 170

Exercício 1 — O mapa do Brasil a partir de distâncias....170
Exercício 2 — Desenrolando o rocambole....170
Exercício 3 — Perplexidade e qualidade do t-SNE....170
Exercício 4 — PCA antes do t-SNE: tempo e qualidade....170
Exercício 5 — Quem preserva melhor os vizinhos?....170
Projeto integrador do capítulo....172
Comparando mapas dos dígitos e dos vinhos....172

Parte IV — Padrões e Anomalias....173

Capítulo 10 — Regras de Associação: o que Costuma Aparecer Junto?....174
10.1 A lenda das fraldas e da cerveja....174
10.2 Vocabulário essencial....175
10.3 As três medidas que você precisa conhecer....175
10.3.1 Suporte: "com que frequência?"....176
10.3.2 Confiança: "quão confiável é a regra?"....176
10.3.3 Lift: "é mais que coincidência?"....176
10.3.4 Outras medidas....176
10.4 O desafio: explosão combinatória....177
10.5 O algoritmo Apriori....177
10.5.1 Passo a passo....177
10.5.2 Aplicando à nossa base (suporte mínimo = 0,5)....178
10.5.3 Custo....178
10.6 O FP-Growth....178
10.6.1 Ideia....178
10.7 Mãos à obra com mlxtend....179
10.7.1 Lendo a tabela de regras....180
10.8 Filtrando regras úteis....180
10.9 Extensões....181
10.10 Estudo de caso: recomendação em uma loja virtual....181
10.11 Armadilhas....182
Resumo do capítulo....182
Exercícios de múltipla escolha....183
Exercícios discursivos....187
Exercícios práticos em Python (scikit-learn)....188
Exercício 1 — Suporte, confiança e lift "na unha"....188
Exercício 2 — Apriori ou FP-Growth?....188
Exercício 3 — Como o suporte mínimo controla a explosão de regras....188
Exercício 4 — Regras em dados reais: o conjunto Wine....189
Exercício 5 — Filtrando regras e recomendando produtos....189
Projeto integrador do capítulo....190
Cesta de supermercado: encontrando regras de verdade....190

Capítulo 11 — Detecção de Anomalias: Encontrando o que Foge do Normal....192
11.1 Uma agulha no palheiro....192
11.2 Tipos de anomalia....193
11.3 Os três cenários de aprendizado....194
11.4 Métodos estatísticos simples....194
11.4.1 Z-score....194
11.4.2 Z-score modificado (MAD)....194
11.4.3 Regra do IQR (boxplot)....194
11.4.4 Distância de Mahalanobis....195
11.5 Isolation Forest....195
11.5.1 A ideia genial....195
11.5.2 Como funciona....195

11.6 LOF: Fator de Outlier Local ... 196
11.6.1 O problema da densidade variável ... 196
11.7 One-Class SVM ... 197
11.8 Outros métodos importantes ... 199
11.8.1 PCA para anomalias ... 199
11.9 Comparando os métodos ... 199
11.10 Definindo o limiar ... 200
11.11 Como avaliar? ... 200
11.12 Do alerta à ação ... 201
Resumo do capítulo ... 201
Exercícios de múltipla escolha ... 202
Exercícios discursivos ... 206
Exercícios práticos em Python (scikit-learn) ... 207
Exercício 1 — Regras estatísticas simples em uma medida real ... 207
Exercício 2 — Isolation Forest ... 207
Exercício 3 — LOF e a escolha de n_neighbors ... 207
Exercício 4 — One-Class SVM treinado só com o normal ... 207
Exercício 5 — Cinco detectores nos dígitos ... 207
Projeto integrador do capítulo ... 208
Detecção de fraude em transações de cartão ... 208

Parte V — Aprendizado Profundo ... 210

Capítulo 12 — Aprendizado Profundo Não Supervisionado: Autoencoders, VAEs, GANs e Auto-supervisão ... 211

12.1 Das técnicas clássicas às redes profundas ... 211
12.2 Redes neurais em cinco minutos ... 212
12.2.1 O neurônio artificial ... 212
12.2.2 Camadas e profundidade ... 212
12.2.3 Como a rede aprende ... 212
12.3 Autoencoders ... 213
12.3.1 A analogia do telegrama ... 213
12.3.2 As três partes ... 213
12.3.3 Relação com o PCA ... 213
12.3.4 Implementação em PyTorch ... 213
12.4 Variações do autoencoder ... 215
12.4.1 Subcompleto e com gargalo ... 215
12.4.2 Autoencoder esparso ... 215
12.4.3 Autoencoder de remoção de ruído (denoising) ... 215
12.4.4 Autoencoder convolucional ... 216
12.4.5 Aplicações ... 216
12.5 Autoencoders variacionais (VAE) ... 217
12.5.1 O limite do autoencoder comum ... 217
12.5.2 A ideia ... 217
12.5.3 O truque da reparametrização ... 218
12.5.4 VAE em PyTorch ... 218
12.5.5 Por que o VAE é importante? ... 219
12.6 Redes Adversariais Generativas (GANs) ... 219
12.6.1 A analogia do falsificador e da polícia ... 219
12.6.2 A formulação do jogo ... 220
12.6.3 Problemas conhecidos ... 220
12.6.4 Um esboço em PyTorch ... 220
12.7 Modelos de difusão (visão geral) ... 220
12.8 Aprendizado auto-supervisionado ... 221

12.8.1 A grande ideia....221
12.8.2 Depois do pré-treino....221
12.9 Aprendizado contrastivo....222
12.9.1 A ideia....222
12.9.2 A função de perda InfoNCE (NT-Xent)....222
12.9.3 Principais métodos....222
12.9.4 Um exemplo mínimo....223
12.10 Usando representações aprendidas....223
12.11 Boas práticas e limites....224
12.12 Fechamento do livro: a grande figura....225
Resumo do capítulo....225
Exercícios de múltipla escolha....226
Exercícios discursivos....230
Exercícios práticos em Python (scikit-learn)....231
Exercício 1 — Um autoencoder com MLPRegressor....231
Exercício 2 — Abrindo o autoencoder: extraindo o código....231
Exercício 3 — Autoencoder de remoção de ruído....231
Exercício 4 — Usando a RBM como extratora de atributos....231
Exercício 5 — Anomalias pelo erro de reconstrução....231
Projeto integrador do capítulo....232
Que representação ajuda mais a agrupar?....232

Parte VI — Projeto Final....233

Capítulo 13 — Projeto Final: Integrando Tudo....234
13.1 Chegou a hora de juntar as peças....234
13.2 A pergunta vem primeiro....235
13.3 Organizando o trabalho....235
13.3.1 Estrutura sugerida....235
13.3.2 Reprodutibilidade em cinco hábitos....236
13.3.3 Notebook ou script?....236
13.4 Comparar com justiça....236
13.5 O relatório final....236
13.5.1 Gráficos que comunicam....237
13.6 Ética e responsabilidade....237
13.7 Como será avaliado o seu projeto....237
13.8 Extensões para quem quer ir além....238
Resumo do capítulo....238
Exercícios de múltipla escolha....239
Exercícios discursivos....243
Projeto integrador final....244
Laboratório de Dígitos: do dado bruto à decisão....244

Apêndice A — Gabarito dos exercícios....245

Capítulo 1 — Bem-vindo ao Aprendizado Não Supervisionado....246
Múltipla escolha....246
Discursivas (respostas-modelo)....246

Capítulo 2 — Dados, Distâncias e Pré-processamento....248
Múltipla escolha....248
Discursivas (respostas-modelo)....248

Capítulo 3 — K-means....250
Múltipla escolha....250
Discursivas (respostas-modelo)....250

Capítulo 4 — Agrupamento Hierárquico 252
Múltipla escolha 252
Discursivas (respostas-modelo) 252
Capítulo 5 — Agrupamento por Densidade 254
Múltipla escolha 254
Discursivas (respostas-modelo) 254
Capítulo 6 — Modelos de Mistura Gaussiana e Agrupamento Espectral 256
Múltipla escolha 256
Discursivas (respostas-modelo) 256
Capítulo 7 — Como Saber se os Grupos São Bons? Avaliação e Validação 258
Múltipla escolha 258
Discursivas (respostas-modelo) 258
Capítulo 8 — PCA e Métodos Lineares 260
Múltipla escolha 260
Discursivas (respostas-modelo) 260
Capítulo 9 — Aprendizado de Variedades 262
Múltipla escolha 262
Discursivas (respostas-modelo) 262
Capítulo 10 — Regras de Associação 264
Múltipla escolha 264
Discursivas (respostas-modelo) 264
Capítulo 11 — Detecção de Anomalias 266
Múltipla escolha 266
Discursivas (respostas-modelo) 266
Capítulo 12 — Aprendizado Profundo Não Supervisionado 268
Múltipla escolha 268
Discursivas (respostas-modelo) 268
Capítulo 13 — Projeto Final 270
Múltipla escolha 270
Discursivas (respostas-modelo) 270
Apêndice B — Glossário 272
Apêndice C — Guia rápido de algoritmos e códigos 275
C.1 Mapa de decisão 275
C.2 Classes do scikit-learn e parâmetros principais 275
C.3 Modelo de código para um projeto completo 276
C.4 Lista de verificação final 277
Apêndice D — Soluções dos exercícios práticos 278
Capítulo 1 — Bem-vindo ao Aprendizado Não Supervisionado 279
Exercício 1 — Primeiro contato com o conjunto Wine 279
Exercício 2 — O efeito da escala 279
Exercício 3 — Enxergando os grupos com PCA 280
Exercício 4 — A sorte da inicialização 280
Exercício 5 — O algoritmo sempre responde 281
Capítulo 2 — Dados, Distâncias e Pré-processamento 282
Exercício 1 — Matriz de distâncias entre clientes 282
Exercício 2 — Três escalonadores diante de um outlier 282
Exercício 3 — Imputar: mediana ou vizinhos? 283
Exercício 4 — Codificando variáveis categóricas 283

Exercício 5 — A maldição da dimensionalidade em números ... 284
Capítulo 3 — K-means ... 286
Exercício 1 — K-means "do zero" com NumPy ... 286
Exercício 2 — Escolhendo k com inércia e silhueta ... 286
Exercício 3 — Aleatória ou k-means++? ... 287
Exercício 4 — Mini-Batch K-means em dados grandes ... 287
Exercício 5 — Quantização de cores de uma foto ... 288
Capítulo 4 — Agrupamento Hierárquico ... 289
Exercício 1 — O dendrograma à mão, conferido pelo computador ... 289
Exercício 2 — Qual ligação reproduz melhor as classes? ... 289
Exercício 3 — Cortando a árvore por altura ... 290
Exercício 4 — O efeito corrente ... 290
Exercício 5 — Agrupando exemplos e atributos ao mesmo tempo ... 291
Capítulo 5 — Agrupamento por Densidade ... 293
Exercício 1 — Varrendo o parâmetro eps ... 293
Exercício 2 — Achando o joelho do gráfico de k-distâncias ... 293
Exercício 3 — DBSCAN, K-means e espectral em anéis ... 294
Exercício 4 — Densidades variáveis: OPTICS e HDBSCAN ... 294
Exercício 5 — Pontos quentes com a distância haversine ... 295
Capítulo 6 — Modelos de Mistura Gaussiana e Agrupamento Espectral ... 297
Exercício 1 — Tipos de covariância no Iris ... 297
Exercício 2 — Quantas gaussianas? ... 297
Exercício 3 — Pontos ambíguos e pertinência suave ... 298
Exercício 4 — GMM como detector de anomalias ... 298
Exercício 5 — Espectral: o papel da afinidade ... 299
Capítulo 7 — Como Saber se os Grupos São Bons? Avaliação e Validação ... 301
Exercício 1 — Há grupos para encontrar? A estatística de Hopkins ... 301
Exercício 2 — Silhueta por grupo e por ponto ... 301
Exercício 3 — Um painel de índices internos ... 302
Exercício 4 — Avaliação externa de três algoritmos ... 303
Exercício 5 — Estabilidade para escolher k ... 304
Capítulo 8 — PCA e Métodos Lineares ... 305
Exercício 1 — Quantas componentes para 90% da variância? ... 305
Exercício 2 — Reconstrução e compressão de dígitos ... 305
Exercício 3 — PCA "na unha" com autovetores ... 306
Exercício 4 — Kernel PCA em anéis ... 306
Exercício 5 — Separando fontes com ICA ... 307
Capítulo 9 — Aprendizado de Variedades ... 308
Exercício 1 — O mapa do Brasil a partir de distâncias ... 308
Exercício 2 — Desenrolando o rocambole ... 309
Exercício 3 — Perplexidade e qualidade do t-SNE ... 309
Exercício 4 — PCA antes do t-SNE: tempo e qualidade ... 310
Exercício 5 — Quem preserva melhor os vizinhos? ... 310
Capítulo 10 — Regras de Associação ... 312
Exercício 1 — Suporte, confiança e lift "na unha" ... 312
Exercício 2 — Apriori ou FP-Growth? ... 312
Exercício 3 — Como o suporte mínimo controla a explosão de regras ... 313
Exercício 4 — Regras em dados reais: o conjunto Wine ... 314
Exercício 5 — Filtrando regras e recomendando produtos ... 315
Capítulo 11 — Detecção de Anomalias ... 317

Exercício 1 — Regras estatísticas simples em uma medida real 317
Exercício 2 — Isolation Forest 317
Exercício 3 — LOF e a escolha de n_neighbors 318
Exercício 4 — One-Class SVM treinado só com o normal 319
Exercício 5 — Cinco detectores nos dígitos 319

Capítulo 12 — Aprendizado Profundo Não Supervisionado 322
Exercício 1 — Um autoencoder com MLPRegressor 322
Exercício 2 — Abrindo o autoencoder: extraindo o código 322
Exercício 3 — Autoencoder de remoção de ruído 323
Exercício 4 — Usando a RBM como extratora de atributos 324
Exercício 5 — Anomalias pelo erro de reconstrução 324

Apêndice E — Soluções de referência dos projetos integradores 326

Capítulo 1 — Bem-vindo ao Aprendizado Não Supervisionado 327
Primeiro diagnóstico de um conjunto de dados 327

Capítulo 2 — Dados, Distâncias e Pré-processamento 329
Pipeline de preparação para uma carteira de clientes 329

Capítulo 3 — K-means 332
Segmentação RFM de uma loja 332

Capítulo 4 — Agrupamento Hierárquico 335
A árvore dos dígitos 335

Capítulo 5 — Agrupamento por Densidade 337
Pontos quentes de ocorrências urbanas 337

Capítulo 6 — Modelos de Mistura Gaussiana e Agrupamento Espectral 340
Perfis com probabilidades: uma operadora de telecomunicações 340

Capítulo 7 — Como Saber se os Grupos São Bons? Avaliação e Validação 342
Painel de avaliação de agrupamentos 342

Capítulo 8 — PCA e Métodos Lineares 345
Compressão, limpeza e partes de dígitos 345

Capítulo 9 — Aprendizado de Variedades 347
Comparando mapas dos dígitos e dos vinhos 347

Capítulo 10 — Regras de Associação 349
Cesta de supermercado: encontrando regras de verdade 349

Capítulo 11 — Detecção de Anomalias 351
Detecção de fraude em transações de cartão 351

Capítulo 12 — Aprendizado Profundo Não Supervisionado 354
Que representação ajuda mais a agrupar? 354

Capítulo 13 — Projeto Final 356
Laboratório de Dígitos: do dado bruto à decisão 356

Referências 361

Apresentação

Para quem é este livro

Este livro foi escrito para um **público misto**: estudantes de graduação e pós-graduação, profissionais de dados e curiosos que querem entender como as máquinas descobrem padrões **sem receber respostas prontas**.

Você **não precisa** de base prévia em matemática ou programação. Cada conceito técnico essencial é explicado no momento em que aparece. Quando uma fórmula surgir, ela virá com uma explicação "em palavras" e, quase sempre, com um exemplo numérico pequeno.

Ao mesmo tempo, o livro avança. Os capítulos começam no nível básico, passam pelo intermediário e chegam a temas avançados, como aprendizado de variedades, autoencoders variacionais, redes adversariais e aprendizado contrastivo.

Como o livro está organizado

O livro tem **seis partes** e **treze capítulos**.

- **Parte I — Fundamentos:** o que é aprendizado não supervisionado; dados, distâncias e pré-processamento.
- **Parte II — Agrupamento:** K-means, hierárquico, densidade (DBSCAN, OPTICS, HDBSCAN), modelos de mistura, agrupamento espectral e avaliação.
- **Parte III — Redução de dimensionalidade:** PCA, ICA, NMF, MDS, Isomap, t-SNE e UMAP.
- **Parte IV — Padrões e anomalias:** regras de associação e detecção de anomalias.
- **Parte V — Aprendizado profundo:** autoencoders, VAEs, GANs, difusão e aprendizado auto-supervisionado.
- **Parte VI — Projeto final:** um estudo completo que integra todas as técnicas em um mesmo conjunto de dados.

Cada capítulo segue o mesmo roteiro:

1. **Objetivos de aprendizagem**, no início.
2. **Conteúdo** com analogias, figuras, exemplos e códigos em Python.
3. **Resumo** em cartões de memorização.
4. **30 questões de múltipla escolha** e **20 questões discursivas**.
5. **Exercícios práticos em Python** com a biblioteca scikit-learn, em ordem crescente de dificuldade.
6. **Projeto integrador do capítulo**, com contexto, tarefas, entregáveis e critérios de avaliação.
7. Um quadro **"Para ir além"**, com conjuntos de dados públicos da UCI e do Kaggle para praticar.

O **gabarito de todas as questões** está no Apêndice A. As **soluções dos exercícios práticos** estão no Apêndice D, e a **solução de referência dos projetos integradores**, no Apêndice E. Tente resolver sozinho antes de consultá-las.

Como estudar com este livro

- **Estude em blocos curtos.** Sessões de 25 a 40 minutos, com pausas, ajudam a manter a atenção.
- **Pare e explique.** Após cada seção, tente explicar a ideia, em voz alta, como se falasse com um amigo.
- **Rode os códigos.** Altere um parâmetro por vez e observe o que muda.

- **Responda antes de olhar o gabarito.** Tentar lembrar fixa mais do que reler.
- **Volte aos cartões de memorização** dias depois. Revisões espaçadas ajudam a memória de longo prazo.

Caixas e símbolos do livro

> **Dica**
>
> Sugestões práticas para facilitar seu trabalho.

> **Atenção**
>
> Cuidados e erros comuns. Leia com calma.

> **No seu dia a dia**
>
> Exemplos do cotidiano que aproximam o conceito da sua realidade.

> **Saiba mais**
>
> Curiosidades e aprofundamentos para quem quer ir além.

> **Exemplo ou exercício guiado**
>
> Pequenas atividades para você experimentar.

> **Memorize**
>
> O essencial do assunto, em poucas linhas.

Sobre os códigos

Todos os trechos de código em Python deste livro foram executados durante a elaboração do material, com as versões de bibliotecas abaixo. Resultados numéricos podem variar um pouco em versões diferentes.

Componente	Versão utilizada
Python	3.13
NumPy	2.5
SciPy	1.18
pandas	3.0
scikit-learn	1.9
matplotlib	3.11
mlxtend	0.25
umap-learn	0.5
PyTorch	2.14

Os exemplos e projetos usam **conjuntos de dados embutidos no scikit-learn** ou **dados simulados** gerados no próprio código, com sementes fixas. Assim, você não precisa baixar arquivos externos e consegue reproduzir os resultados.

Os conjuntos embutidos vêm do repositório **UCI Machine Learning**: *Iris*, *Wine*, *Breast Cancer Wisconsin (Diagnostic)* e *Optical Recognition of Handwritten Digits* (a parte de teste, com 1.797 imagens). Os dados de clientes, de transações, de cestas de compras e de coordenadas geográficos usados em alguns projetos são **simulados** e servem apenas para fins didáticos; cada projeto informa a origem dos seus dados.

Bases públicas da UCI e do Kaggle

Ao final de cada capítulo, o quadro **"Para ir além"** sugere conjuntos de dados públicos do repositório **UCI Machine Learning** e do **Kaggle** para você repetir os exercícios com dados reais. Esses conjuntos **não foram usados nem testados nos códigos deste livro**: confira nos sites a descrição, a licença e as colunas antes de usá-los. Em geral, o fluxo é este:

- **UCI:** baixe o arquivo no site do repositório ou use o pacote `ucimlrepo`.
- **Kaggle:** baixe o arquivo no site (é preciso criar uma conta) ou use a API do Kaggle com um token pessoal.

Depois, carregue os dados com `pandas`, inspecione-os (formato, tipos, ausentes) e aplique o roteiro do capítulo.

Leituras recomendadas

Para aprofundar os estudos, alguns livros de referência são:

- Hastie, Tibshirani e Friedman (2009), *The Elements of Statistical Learning*, que dedica um capítulo ao aprendizado não supervisionado.
- Bishop (2006), *Pattern Recognition and Machine Learning*, com tratamento probabilístico de misturas gaussianas e do algoritmo EM.
- Murphy (2012), *Machine Learning: A Probabilistic Perspective*, uma visão ampla e rigorosa.
- Goodfellow, Bengio e Courville (2016), *Deep Learning*, para a Parte V.
- Aggarwal (2015), *Data Mining: The Textbook*, que cobre agrupamento, regras de associação e anomalias.
- Jain, Murty e Flynn (1999) e Jain (2010), duas revisões clássicas sobre agrupamento.

As referências completas estão ao final do livro. Priorizamos artigos publicados em periódicos de alto impacto, trabalhos apresentados em conferências de referência da área e livros reconhecidos.

Bons estudos!

PARTE I

Parte I — Fundamentos

Os conceitos que sustentam tudo o que vem depois: o que é aprender sem rótulos, como os dados são organizados, como medir semelhança e como preparar os dados.

Nesta parte:

- Cap. 1 — Bem-vindo ao Aprendizado Não Supervisionado
- Cap. 2 — Dados, Distâncias e Pré-processamento



Capítulo 1 — Bem-vindo ao Aprendizado Não Supervisionado

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar, com suas palavras, o que é aprendizado não supervisionado.
- Diferenciar aprendizado supervisionado, não supervisionado, semissupervisionado, auto-supervisionado e por reforço.
- Reconhecer as quatro grandes famílias de tarefas: agrupamento, redução de dimensionalidade, regras de associação e detecção de anomalias.
- Descrever as etapas de um projeto de aprendizado não supervisionado.
- Preparar o ambiente Python e executar seu primeiro agrupamento.

1.1 Uma história para começar

Imagine que você acaba de herdar uma biblioteca com **10 mil livros**. Nenhum deles tem etiqueta. Não há catálogo. Não há seções.

Ninguém lhe disse o que é "romance", "culinária" ou "física".

Mesmo assim, você começa a trabalhar. Pega um livro, folheia, e coloca perto de outro parecido. Livros com muitas receitas ficam juntos. Livros com fórmulas ficam em outra pilha. Livros com mapas formam uma terceira.

Sem receber **nenhuma resposta pronta**, você descobriu uma organização. Você fez, na prática, **aprendizado não supervisionado**.

NO SEU DIA A DIA **Onde isso aparece no seu dia a dia**

Quando o aplicativo de música sugere "playlists parecidas com a sua", quando o banco percebe uma compra estranha no seu cartão ou quando uma loja agrupa clientes por perfil de compra, há aprendizado não supervisionado trabalhando por trás.

Neste capítulo, você vai construir a base de todo o livro. Vamos com calma, passo a passo.

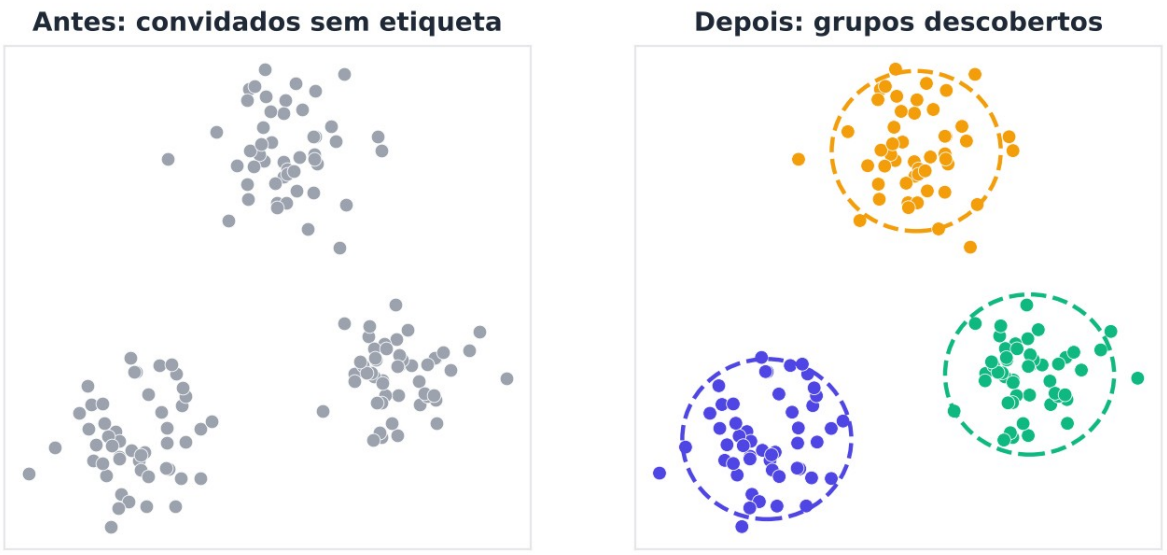


Figura 1.1 — Sem etiquetas, o algoritmo observa "quem se parece com quem" e descobre grupos sozinho.

1.2 O que é aprender com dados?

Aprendizado de máquina (*machine learning*) é a área da Inteligência Artificial que cria programas capazes de melhorar seu desempenho a partir de **dados**, sem que cada regra seja escrita à mão (Hastie; Tibshirani; Friedman, 2009).

Dois conceitos aparecerão o tempo todo. Vamos explicá-los já.

- **Exemplo** (ou amostra, ou instância): uma "linha" dos dados. Pode ser um cliente, uma foto, uma compra.
- **Atributo** (ou característica, ou *feature*): uma "coluna" dos dados. Pode ser idade, valor gasto, brilho de um pixel.

Os dados formam uma tabela. Chamamos de **X** a tabela de atributos. Se há n exemplos e d atributos, X tem n linhas e d colunas.

Às vezes existe também uma coluna especial, o **rótulo** (ou *label*, ou alvo, normalmente chamado y). O rótulo é a "resposta correta" para aquele exemplo. Por exemplo: "este e-mail é spam".

> **Memorize**
>
> **X** = os atributos que descrevem cada exemplo. **y** = a resposta correta, quando existe. A grande pergunta deste livro é: *o que podemos aprender quando só temos X?*

1.3 Os grandes paradigmas de aprendizado

Existem várias formas de uma máquina aprender. A diferença está no **tipo de "feedback"** que ela recebe.

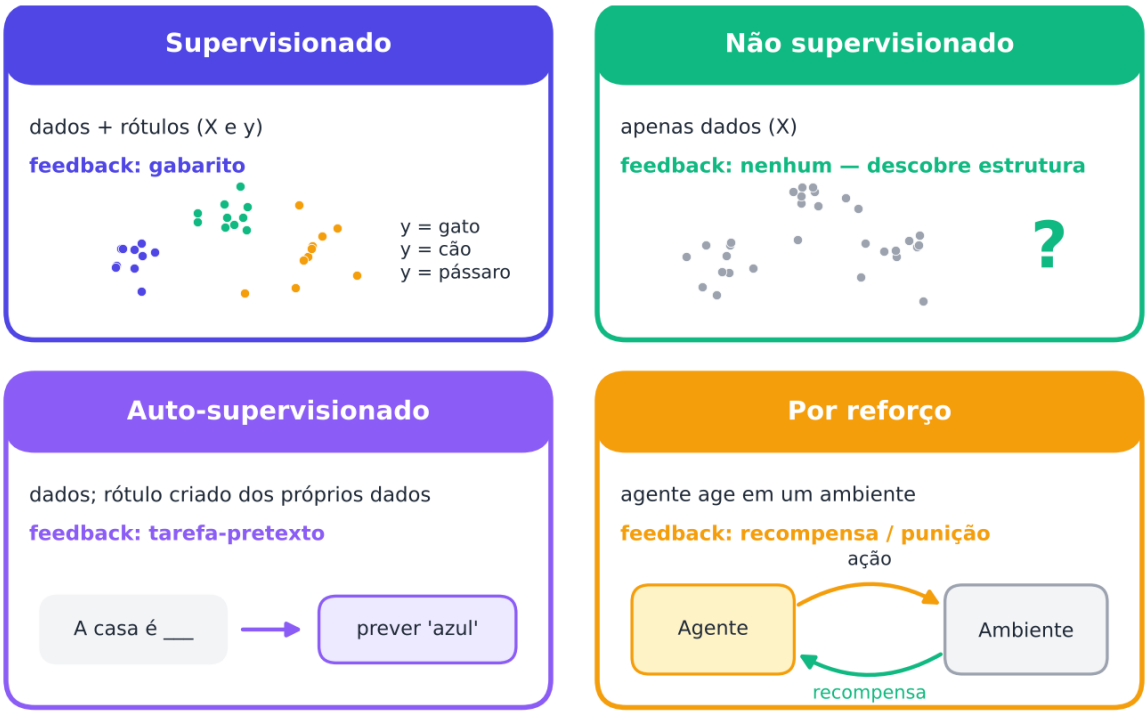


Figura 1.2 — Os paradigmas de aprendizado, comparados pelo tipo de feedback que recebem.

1.3.1 Aprendizado supervisionado

Aqui temos X **e** y. A máquina aprende a prever y a partir de X.

Analogia: um aluno que estuda com um gabarito. Ele tenta responder, confere o gabarito e corrige o erro.

Exemplos: prever o preço de uma casa (**regressão**) ou dizer se uma foto mostra um gato ou um cachorro (**classificação**).

1.3.2 Aprendizado não supervisionado

Aqui temos **apenas X**. Não há gabarito. O objetivo é **descobrir estrutura** escondida nos dados: grupos, padrões, regularidades, exceções.

Analogia: um explorador que chega a uma ilha desconhecida. Ninguém lhe explica nada. Ele observa e cria seu próprio mapa.

1.3.3 Aprendizado semissupervisionado

Uma pequena parte dos dados tem rótulo. A maior parte não tem. A máquina usa os dois tipos juntos.

Analogia: um professor corrige 20 provas e o aluno aprende com elas, mas também estuda 2.000 exercícios sem correção.

1.3.4 Aprendizado auto-supervisionado

A máquina **inventa seus próprios rótulos** a partir dos dados. Por exemplo, esconde uma palavra de uma frase e tenta adivinhá-la. Esse paradigma sustenta os grandes modelos de linguagem e será estudado no último capítulo (Devlin et al., 2019; Chen et al., 2020).

1.3.5 Aprendizado por reforço

Um agente age num ambiente e recebe **recompensas ou punições**. Ele aprende por tentativa e erro, como quem treina um cachorro com petiscos.

> **Atenção**
>
> As fronteiras entre os paradigmas não são rígidas. O aprendizado auto-supervisionado, por exemplo, usa dados sem rótulos humanos (parece não supervisionado), mas treina com uma "resposta" criada automaticamente (parece supervisionado). Por isso muitos autores o tratam como uma ponte entre os dois mundos.

1.4 Por que estudar aprendizado não supervisionado?

Existem pelo menos cinco boas razões.

1. **Rótulos são caros.** Pedir a especialistas que anotem milhões de exemplos custa tempo e dinheiro. Dados sem rótulo, por outro lado, são abundantes.
2. **Rótulos podem nem existir.** Quantos "tipos de cliente" existem? Ninguém sabe. É preciso descobrir.
3. **Exploração.** Antes de prever qualquer coisa, precisamos entender os dados. Gráficos, grupos e resumos ajudam muito.
4. **Pré-processamento.** Reduzir dimensões e criar boas representações melhora os modelos supervisionados.
5. **Segurança.** Fraudes e falhas são raras e mudam de forma. É difícil rotulá-las. Detectar o "diferente" é uma tarefa não supervisionada.

> **Saiba mais**
>
> Bengio, Courville e Vincent (2013) mostraram que aprender boas **representações** dos dados, isto é, formas mais úteis de descrevê-los, é uma das chaves do sucesso do aprendizado profundo. Muitas dessas representações nascem sem rótulos.

1.5 As quatro grandes famílias de tarefas

Quase tudo que veremos cabe em quatro famílias. Pense nelas como quatro "caixas de ferramentas".

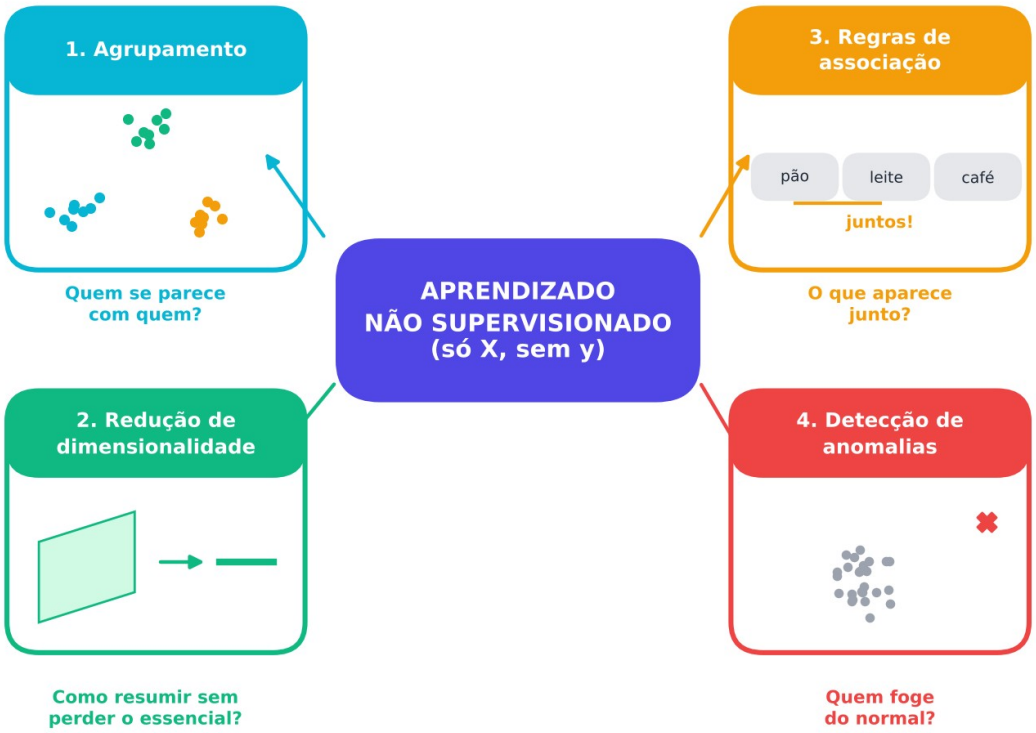


Figura 1.3 — O mapa das quatro famílias de tarefas do aprendizado não supervisionado.

1.5.1 Agrupamento (clustering)

Objetivo: dividir os exemplos em grupos, chamados *clusters*, de modo que exemplos do mesmo grupo sejam parecidos entre si e diferentes dos outros grupos (Jain; Murty; Flynn, 1999).

Exemplos: segmentar clientes; agrupar notícias por assunto; separar tipos de tecido em imagens médicas.

1.5.2 Redução de dimensionalidade

Dimensionalidade é o número de atributos. Uma foto pequena de 28 × 28 pixels tem 784 atributos!

Objetivo: representar os dados com **menos atributos**, perdendo o mínimo possível de informação. É como resumir um livro de 300 páginas em uma página, mantendo as ideias principais.

Exemplos: visualizar dados em 2D; comprimir imagens; remover ruído.

1.5.3 Regras de associação

Objetivo: descobrir relações do tipo "quem compra A costuma comprar B". É a famosa "análise da cesta de compras" (Agrawal; Imielinski; Swami, 1993).

Exemplo: supermercado descobre que clientes que levam fraldas e cerveja... também levam salgadinhos.

1.5.4 Detecção de anomalias

Anomalia (ou *outlier*) é um exemplo muito diferente do comportamento normal. **Objetivo:** encontrá-lo (Chandola; Banerjee; Kumar, 2009).

Exemplos: fraude em cartão; falha em máquina industrial; invasão em rede de computadores.

Família	Pergunta que responde	Exemplo curto
Agrupamento	Quem se parece com quem?	Perfis de clientes

Família	Pergunta que responde	Exemplo curto
Redução de dimensionalidade	Como resumir sem perder o essencial?	Mapa 2D de fotos
Regras de associação	O que costuma aparecer junto?	Cesta de compras
Detecção de anomalias	Quem foge do normal?	Fraude em cartão

1.6 Um pouco de história

A ideia de descobrir estrutura em dados é antiga. Veja alguns marcos.

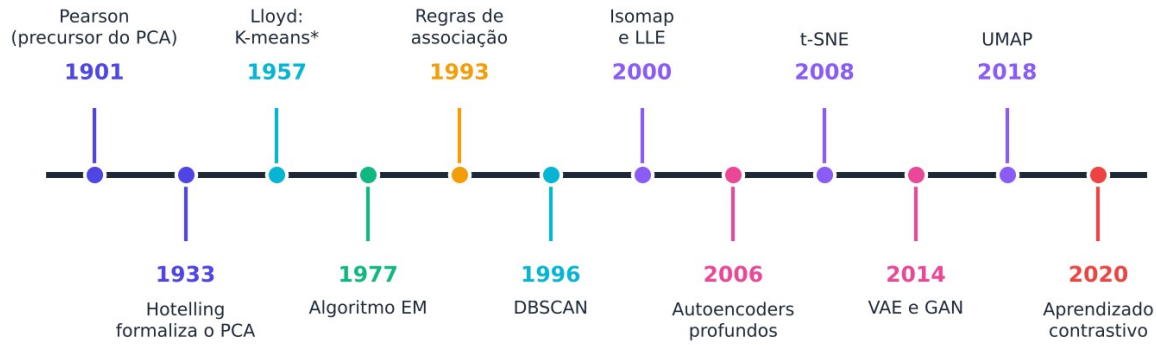


Figura 1.4 — Linha do tempo: de Pearson ao aprendizado auto-supervisionado.

- **1901:** Karl Pearson descreve o que hoje chamamos de Análise de Componentes Principais (PCA) (Pearson, 1901).
- **1933:** Harold Hotelling formaliza e nomeia o método (Hotelling, 1933).
- **1957 e 1982:** o algoritmo que depois seria chamado de *k-means* é proposto por Stuart Lloyd em 1957, mas só publicado em 1982 (Lloyd, 1982). O nome "k-means" vem de MacQueen (1967).
- **1977:** o algoritmo EM é apresentado por Dempster, Laird e Rubin (1977).
- **1993:** surgem as regras de associação (Agrawal; Imielinski; Swami, 1993).
- **1996:** o DBSCAN revoluciona o agrupamento por densidade (Ester et al., 1996).
- **2000:** Isomap e LLE inauguram o aprendizado de variedades (Tenenbaum; de Silva; Langford, 2000; Roweis; Saul, 2000).
- **2006:** redes profundas aprendem representações sem rótulos (Hinton; Salakhutdinov, 2006).
- **2008–2018:** t-SNE e UMAP popularizam a visualização de dados complexos (van der Maaten; Hinton, 2008; McInnes; Healy; Melville, 2018).
- **2020 em diante:** o aprendizado contrastivo e auto-supervisionado domina a pesquisa (Chen et al., 2020; He et al., 2020).

1.7 Como é um projeto na prática?

Todo projeto segue um caminho parecido. Guarde esta sequência.

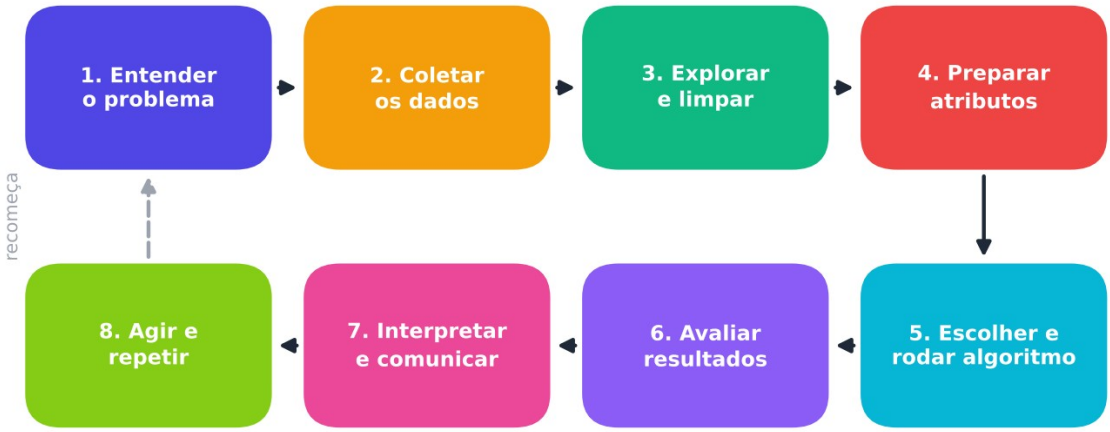


Figura 1.5 — O ciclo de um projeto de aprendizado não supervisionado.

1. **Entender o problema.** Qual decisão os resultados vão apoiar?
2. **Coletar os dados.** De onde vêm? Quem os produziu?
3. **Explorar e limpar.** Há valores ausentes? Erros? Repetições?
4. **Preparar os atributos.** Escolher, transformar, padronizar (Capítulo 2).
5. **Escolher e rodar o algoritmo.** Cada técnica tem pontos fortes e fracos.
6. **Avaliar.** Como saber se o resultado é bom *sem* gabarito? (Capítulo 7.)
7. **Interpretar e comunicar.** Dar nome aos grupos, explicar as descobertas.
8. **Agir e repetir.** Os dados mudam. O ciclo recomeça.

> **ATENÇÃO — O maior desafio**
>
> No aprendizado supervisionado, você mede o erro contra o gabarito. No não supervisionado, **não há gabarito**. A avaliação depende de critérios internos, de estabilidade e, principalmente, de **julgamento humano**. Por isso a etapa 7 é tão importante.

1.8 Preparando o ambiente Python

Usaremos **Python**, a linguagem mais popular em ciência de dados. Não se assuste se você nunca programou. Cada código será explicado linha por linha.

As bibliotecas que usaremos são:

- **NumPy:** cálculos com tabelas de números.
- **pandas:** tabelas de dados, como uma planilha dentro do Python.
- **matplotlib:** gráficos.
- **scikit-learn:** os algoritmos de aprendizado de máquina (Pedregosa et al., 2011).

Para instalar tudo, abra o terminal e digite:

```bash
pip install numpy pandas matplotlib scikit-learn scipy
```

> **Dica para quem está começando**
>
> Se preferir não instalar nada, use o **Google Colab** (colab.research.google.com). Ele roda Python no navegador, de graça, e já vem com essas bibliotecas.

1.9 Seu primeiro agrupamento

Vamos usar um conjunto de dados clássico: o **Iris**. Ele traz medidas de 150 flores de três espécies. Vamos *esconder* as espécies e pedir ao algoritmo que descubra os grupos sozinho.

```python
# 1) Importar as bibliotecas
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# 2) Carregar os dados (X = medidas; y = espécie)
iris = load_iris()
X = iris.data      # 150 linhas x 4 colunas
y = iris.target    # só vamos usar y no final, para conferir

print("Formato de X:", X.shape)

# 3) Pedir ao algoritmo 3 grupos, SEM mostrar y
modelo = KMeans(n_clusters=3, n_init=10, random_state=42)
grupos = modelo.fit_predict(X)

# 4) Reduzir de 4 para 2 dimensões, só para desenhar
X2d = PCA(n_components=2).fit_transform(X)

# 5) Desenhar
plt.scatter(X2d[:, 0], X2d[:, 1], c=grupos, cmap="viridis", s=40)
plt.title("Grupos descobertos pelo K-means")
plt.xlabel("Componente 1")
plt.ylabel("Componente 2")
plt.show()

# 6) Conferir (só para aprender!): quantas flores de cada espécie em cada grupo?
for g in range(3):
    especies = y[grupos == g]
    print("Grupo", g, "->", np.bincount(especies, minlength=3))
```

O que cada parte faz?

- `load_iris()` carrega os dados prontos.
- `KMeans(n_clusters=3)` cria o algoritmo e diz que queremos 3 grupos.
- `fit_predict(X)` "treina" e já devolve o grupo de cada flor.
- `PCA(n_components=2)` resume 4 medidas em 2, apenas para podermos desenhar.
- No final, comparamos com as espécies reais. Na prática, você não teria `y`!

Você deve ver um grupo perfeito (as flores da espécie *setosa*) e dois grupos que se misturam um pouco. Isso é normal e muito instrutivo. Algumas espécies realmente se parecem.

EXEMPLO **Exercício guiado**

Troque `n_clusters=3` por `2` e depois por `5`. O que muda no gráfico? Qual número parece "mais natural"? Anote suas impressões. No Capítulo 7 você aprenderá métodos para decidir melhor.

1.10 Cuidados éticos

Agrupar pessoas também é decidir sobre pessoas. Alguns cuidados:

- **Viés nos dados.** Se os dados refletem preconceitos do passado, os grupos também refletirão.
- **Privacidade.** Dados pessoais exigem proteção e base legal (no Brasil, a LGPD).
- **Rótulos que estigmatizam.** Dar um nome ofensivo a um grupo pode causar dano real.
- **Excesso de confiança.** Um grupo é uma hipótese, não uma verdade.

MEMORIZE **Regra de ouro**

O algoritmo **sugere**. Pessoas **decidem** e **respondem** pelas decisões.

1.11 Estrutura deste livro

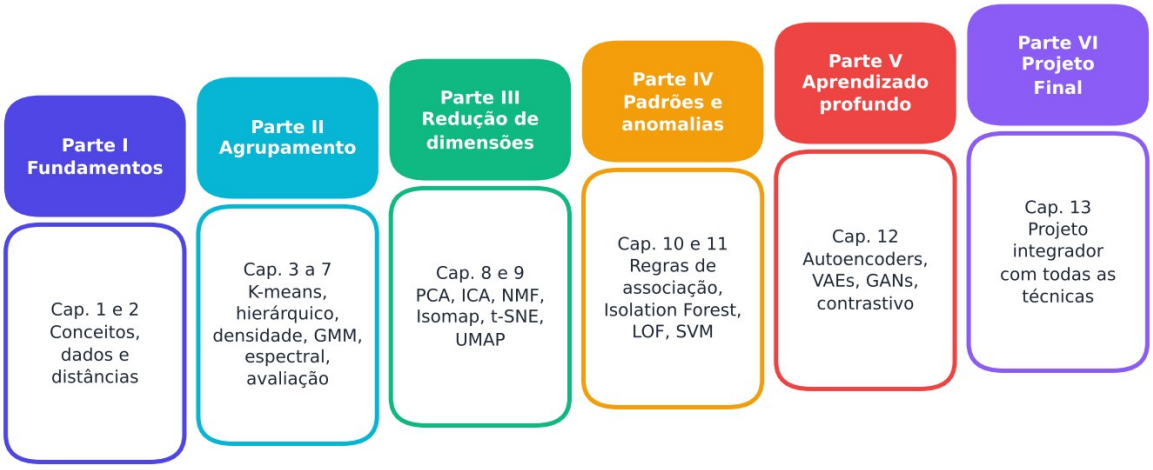


Figura 1.6 — Roteiro de estudo do livro: seis partes, treze capítulos.

- **Parte I — Fundamentos** (Capítulos 1 e 2): conceitos, dados e distâncias.
- **Parte II — Agrupamento** (Capítulos 3 a 7): K-means, hierárquico, densidade, modelos de mistura, espectral e avaliação.
- **Parte III — Redução de dimensionalidade** (Capítulos 8 e 9): PCA, ICA, NMF, Isomap, t-SNE e UMAP.
- **Parte IV — Padrões e anomalias** (Capítulos 10 e 11): regras de associação e detecção de anomalias.
- **Parte V — Aprendizado profundo** (Capítulo 12): autoencoders, VAEs, GANs e aprendizado auto-supervisionado.
- **Parte VI — Projeto final** (Capítulo 13): um estudo completo que integra todas as técnicas.

Cada capítulo traz **exercícios de múltipla escolha e discursivos**, **exercícios práticos em Python com scikit-learn** e um **projeto integrador**. Os códigos usam, sempre que possível, conjuntos de dados públicos do repositório UCI Machine Learning que já vêm no scikit-learn.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Não supervisionado** = só X, sem y. Descobre estrutura.
- **Quatro famílias:** agrupar, reduzir dimensões, associar, detectar anomalias.
- **Supervisionado** tem gabarito; **reforço** tem recompensa; **auto-supervisionado** cria o próprio gabarito.
- **Desafio central:** avaliar sem gabarito.
- **Projeto:** entender, coletar, limpar, preparar, modelar, avaliar, interpretar, agir.
- **Ética:** o algoritmo sugere; pessoas decidem.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. No aprendizado não supervisionado, os dados de treinamento contêm:

a) apenas atributos (X), sem rótulos.
b) apenas rótulos (y), sem atributos.
c) recompensas e punições.
d) atributos e rótulos (X e y).

Questão 2. Na analogia da biblioteca com 10 mil livros sem etiqueta, organizar os livros por semelhança corresponde a:

a) classificação supervisionada.
b) agrupamento.
c) regressão.
d) aprendizado por reforço.

Questão 3. Em uma tabela de dados, cada linha representa normalmente:

a) um atributo.
b) um rótulo.
c) um exemplo (instância).
d) um algoritmo.

Questão 4. A letra y, em aprendizado de máquina, designa em geral:

a) a distância entre pontos.
b) o número de grupos.
c) a matriz de atributos.
d) o rótulo ou resposta correta.

Questão 5. Qual paradigma usa um "gabarito" para corrigir os erros do modelo?

a) Não supervisionado.
b) Redução de dimensionalidade.
c) Supervisionado.
d) Por reforço.

Questão 6. Prever o preço de uma casa a partir de suas características é um exemplo de:

a) regressão supervisionada.
b) agrupamento.
c) detecção de anomalias.
d) regras de associação.

Questão 7. No aprendizado por reforço, o agente aprende principalmente por meio de:

a) rótulos fornecidos por especialistas.
b) recompensas e punições.
c) redução de dimensões.
d) distâncias entre exemplos.

Questão 8. O aprendizado semissupervisionado caracteriza-se por:

a) usar só recompensas.
b) usar apenas dados rotulados.
c) não usar nenhum dado.
d) usar poucos dados rotulados e muitos não rotulados.

Questão 9. No aprendizado auto-supervisionado, o rótulo:

a) é sempre fornecido por humanos.
b) é uma recompensa do ambiente.
c) é criado a partir dos próprios dados.
d) não existe nem é criado.

Questão 10. Qual das alternativas NÃO é uma das quatro grandes famílias de tarefas do aprendizado não supervisionado vistas no capítulo?

a) Agrupamento.
b) Classificação supervisionada.
c) Redução de dimensionalidade.
d) Detecção de anomalias.

Questão 11. A pergunta "quem se parece com quem?" corresponde à tarefa de:

a) agrupamento.
b) regras de associação.
c) aprendizado por reforço.
d) regressão.

Questão 12. Reduzir uma foto de 784 pixels a 2 números para visualização é uma aplicação de:

a) aprendizado por reforço.
b) regras de associação.
c) detecção de fraude.
d) redução de dimensionalidade.

Questão 13. A descoberta de que "quem compra fraldas costuma comprar cerveja" é um exemplo típico de:

a) regressão linear.
b) regras de associação.
c) agrupamento hierárquico.
d) PCA.

Questão 14. Um exemplo muito diferente do comportamento normal é chamado de:

a) rótulo.
b) anomalia (outlier).
c) centroide.
d) atributo.

Questão 15. Uma razão importante para estudar aprendizado não supervisionado é que:

a) rótulos podem ser caros ou inexistentes.
b) ele dispensa qualquer avaliação.
c) dados sem rótulo não existem.
d) rótulos são sempre baratos.

Questão 16. O maior desafio de avaliação no aprendizado não supervisionado é:

a) a ausência de gabarito.
b) o excesso de rótulos.
c) a impossibilidade de usar Python.
d) a falta de algoritmos.

Questão 17. Quem formalizou e nomeou a Análise de Componentes Principais em 1933 foi:

a) Ester.
b) MacQueen.
c) Hotelling.
d) Kingma.

Questão 18. O algoritmo DBSCAN foi apresentado em:

a) 1996.
b) 1957.
c) 2018.
d) 1977.

Questão 19. A primeira etapa recomendada de um projeto de aprendizado não supervisionado é:

a) comunicar os resultados.
b) entender o problema.
c) rodar o algoritmo.
d) escolher o número de grupos.

Questão 20. Em um projeto típico, a etapa "explorar e limpar" tem como objetivo:

a) criar rótulos manualmente.
b) identificar valores ausentes, erros e repetições.
c) escolher o algoritmo final.
d) publicar o livro.

Questão 21. Qual biblioteca Python reúne a maioria dos algoritmos de aprendizado de máquina usados no livro?

a) os.
b) matplotlib.
c) json.
d) scikit-learn.

Questão 22. No primeiro exemplo do capítulo, o conjunto de dados utilizado foi:

a) Wine Quality.
b) Titanic.
c) Iris.
d) MNIST.

Questão 23. No código do K-means do capítulo, o argumento n_clusters=3 indica que:

a) existem 3 atributos.
b) queremos 3 grupos.
c) o conjunto tem 3 exemplos.
d) o algoritmo deve rodar 3 vezes.

Questão 24. Por que o código do capítulo usa PCA(n_components=2) antes de desenhar o gráfico?

a) Para melhorar a precisão do K-means.
b) Para criar rótulos.
c) Para reduzir de 4 para 2 dimensões, só para visualizar.
d) Para remover o ruído.

Questão 25. Qual cuidado ético foi destacado no capítulo?

a) Agrupar pessoas é também decidir sobre pessoas, e os dados podem refletir preconceitos.
b) A privacidade só importa para dados públicos.
c) Nomes ofensivos para grupos não causam dano.
d) Dados reais nunca têm viés.

Questão 26. A "regra de ouro" do capítulo afirma que:

a) dados dispensam responsabilidade.
b) o algoritmo sugere; pessoas decidem e respondem pelas decisões.
c) pessoas não precisam interpretar resultados.
d) o algoritmo decide e pessoas obedecem.

Questão 27. No Brasil, a lei que trata da proteção de dados pessoais, mencionada no capítulo, é a:

a) LDB.
b) CLT.
c) LGPD.
d) LAI.

Questão 28. Segundo Bengio, Courville e Vincent (2013), uma chave do sucesso do aprendizado profundo é:

a) eliminar todos os dados.
b) aprender boas representações dos dados.
c) usar apenas rótulos manuais.
d) evitar redes neurais.

Questão 29. Na estrutura do livro, a Parte V trata de:

a) fundamentos.
b) agrupamento.
c) regras de associação.
d) aprendizado profundo.

Questão 30. Qual das situações abaixo é um exemplo de aprendizado não supervisionado?

a) Descobrir perfis de clientes sem nenhuma classificação prévia.
b) Prever se um e-mail é spam, tendo e-mails já marcados como spam.
c) Prever o preço de uma casa a partir de vendas anteriores.
d) Ensinar um robô a andar com recompensas.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com suas palavras e usando uma analogia do cotidiano, o que é aprendizado não supervisionado.
2. Diferencie aprendizado supervisionado de não supervisionado quanto aos dados e ao tipo de feedback.
3. Descreva as cinco principais formas de aprendizado apresentadas no capítulo, com um exemplo de cada.
4. Cite cinco motivos para estudar aprendizado não supervisionado.
5. Descreva as quatro famílias de tarefas e dê um exemplo de aplicação para cada.
6. O que é um exemplo e o que é um atributo? Mostre com uma tabela de clientes de uma loja.
7. Explique por que avaliar um agrupamento é mais difícil do que avaliar um classificador.
8. Descreva as oito etapas de um projeto de aprendizado não supervisionado.
9. Por que a etapa de interpretação e comunicação é especialmente importante em agrupamento?
10. Explique por que o aprendizado auto-supervisionado é visto como uma ponte entre o supervisionado e o não supervisionado.
11. No código do primeiro agrupamento, por que o vetor y foi usado só no final?
12. O que acontece com os grupos quando mudamos n_clusters de 3 para 2 ou 5 no exemplo do Iris? Como decidir o melhor valor?
13. Cite três riscos éticos ao agrupar pessoas e uma medida de mitigação para cada.
14. Explique a diferença entre regressão e classificação, citando um exemplo de cada.
15. Relacione três marcos históricos da linha do tempo do capítulo com as famílias de tarefas.
16. Por que o agrupamento pode ser considerado uma "hipótese" e não uma verdade?
17. Dê dois exemplos de aplicações de detecção de anomalias e explique por que são difíceis de resolver só com aprendizado supervisionado.
18. Escreva, com suas palavras, o que você espera aprender em cada uma das cinco partes do livro.
19. Descreva um problema real da sua área de interesse em que o aprendizado não supervisionado poderia ajudar, indicando a tarefa e os dados necessários.
20. Explique a diferença entre os termos "dados", "atributo" e "rótulo" usando o exemplo de um conjunto de fotos de animais.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Primeiro contato com o conjunto Wine

NÍVEL: BÁSICO

Carregue o conjunto **Wine** (`load_wine`). Mostre o formato de X, os nomes dos 13 atributos e quantos exemplos existem em cada classe. Depois, **esconda** o rótulo e agrupe os vinhos com `KMeans(n_clusters=3)`. Use `pd.crosstab` para comparar os grupos com as classes reais.

Pergunta: os grupos coincidem com as classes? Por que isso seria improvável?

Exercício 2 — O efeito da escala

NÍVEL: BÁSICO

Repita o agrupamento do exercício anterior, agora **padronizando** os dados com `StandardScaler`. Calcule o índice de Rand ajustado (`adjusted_rand_score`) entre os grupos e as classes reais, antes e depois da padronização.

Pergunta: o que mudou? Qual atributo dominava os cálculos antes?

Exercício 3 — Enxergando os grupos com PCA

NÍVEL: INTERMEDIÁRIO

Reduza o Wine padronizado a **2 componentes** com `PCA` e faça dois gráficos lado a lado: um colorido pelos grupos do K-means e outro colorido pelas classes reais. Imprima a variância explicada pelas duas componentes.

Exercício 4 — A sorte da inicialização

NÍVEL: INTERMEDIÁRIO

Rode o K-means **30 vezes** com `n_init=1` e sementes diferentes (`random_state` de 0 a 29) no Wine padronizado. Guarde a inércia de cada execução. Mostre mínimo, máximo e quantas soluções diferentes apareceram (arredonde a inércia a 1 casa decimal). Compare com `n_init=10`.

Pergunta: por que usar vários `n_init` é uma boa prática?

Exercício 5 — O algoritmo sempre responde

NÍVEL: AVANÇADO

Gere dados **puramente aleatórios** (uniformes) com o mesmo formato do Wine (178 linhas e 13 colunas). Agrupe-os com `KMeans(3)` e calcule a silhueta média. Compare com a silhueta do Wine padronizado.

Pergunta: o K-means entregou grupos para dados sem estrutura? O que isso ensina sobre avaliação?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Iris, Wine e Seeds (grãos de trigo, com 3 variedades).
> - **Kaggle:** Mall Customers (200 clientes de shopping, com renda e pontuação de gastos).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Primeiro diagnóstico de um conjunto de dados

Fonte dos dados. *Breast Cancer Wisconsin (Diagnostic)*, do repositório **UCI Machine Learning**, incluído no scikit-learn.

Contexto. Uma equipe de pesquisa recebeu o conjunto de dados de câncer de mama do scikit-learn (`load_breast_cancer`): 569 exames com 30 medidas cada. Pediram a você um **primeiro diagnóstico exploratório** usando apenas aprendizado não supervisionado.

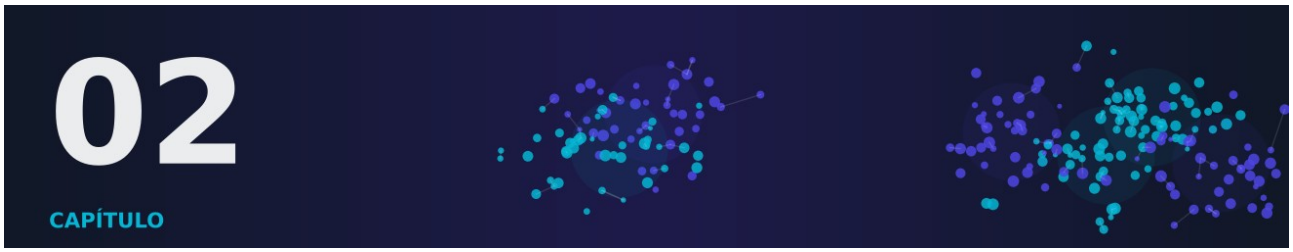
Atenção ética. Os dados são de uso didático. Os grupos que você encontrar **não são diagnóstico médico**.

Tarefas.

1. Carregue os dados, mostre o formato e verifique se há valores ausentes.
2. Padronize os atributos.
3. Agrupe com `KMeans(n_clusters=2)`. Calcule a silhueta média.
4. Reduza a 2 dimensões com PCA e desenhe os grupos.
5. **Só agora** compare os grupos com o rótulo real (`target`) usando `pd.crosstab` e o ARI.
6. Escreva um parágrafo (5 a 8 linhas) com suas conclusões e **três cuidados éticos** ao usar esse tipo de análise na saúde.

Entregáveis. Um notebook (ou script) comentado e o parágrafo de conclusões.

A solução de referência está no Apêndice E.



Capítulo 2 — Dados, Distâncias e Pré-processamento

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Representar dados como uma matriz de exemplos por atributos.
- Distinguir tipos de atributos: numéricos, categóricos e ordinais.
- Calcular e interpretar as distâncias Euclidiana, Manhattan, Chebyshev, Minkowski, cosseno e de Gower.
- Explicar por que escalar os dados é decisivo e aplicar padronização e normalização.
- Tratar valores ausentes e codificar variáveis categóricas.
- Descrever a "maldição da dimensionalidade" e suas consequências.

2.1 Por que começar pelos dados?

Existe uma frase famosa entre cientistas de dados: **"lixo entra, lixo sai"**. Se os dados forem ruins, o melhor algoritmo do mundo produzirá resultados ruins.

No aprendizado não supervisionado isso é ainda mais grave. Sem gabarito, você pode nem perceber o erro. O algoritmo sempre devolve *alguma* resposta, mesmo quando ela não faz sentido.

Neste capítulo você vai aprender três habilidades essenciais:

1. Entender como os dados são organizados.
2. Medir o quanto dois exemplos se parecem (**distância**).
3. Preparar os dados para que o algoritmo não seja enganado.

2.2 Dados como uma tabela de números

Todo algoritmo deste livro enxerga seus dados como uma **matriz**: uma tabela em que cada linha é um exemplo e cada coluna é um atributo.



Figura 2.1 — Os dados como matriz: linhas são exemplos; colunas são atributos. Cada linha também é um ponto no espaço.

Veja um exemplo simples, com clientes de uma padaria:

Cliente	Idade	Gasto mensal (R$)	Visitas por mês
Ana	25	120	8
Bruno	62	340	12
Carla	31	95	4
Diego	58	310	10

Cada cliente é um **ponto num espaço de 3 dimensões**. Ana está na posição (25, 120, 8). Bruno em (62, 340, 12). Dois clientes "parecidos" ficam **próximos** nesse espaço.

> MEMORIZE **A ideia central**
>
> Semelhança = proximidade. Quase todo algoritmo de agrupamento, e muitos de redução de dimensionalidade e de anomalias, depende de uma **medida de distância**.

2.2.1 Tipos de atributos

Nem todo atributo é um número comum. Conhecer o tipo orienta toda a preparação.

- **Numérico contínuo:** pode assumir infinitos valores. Ex.: altura, temperatura, preço.
- **Numérico discreto:** valores inteiros contáveis. Ex.: número de filhos.
- **Categórico nominal:** categorias sem ordem. Ex.: cor, cidade, time de futebol.
- **Categórico ordinal:** categorias com ordem. Ex.: "baixo, médio, alto"; nota de 1 a 5 estrelas.
- **Binário:** só dois valores. Ex.: comprou ou não comprou.

> ATENÇÃO **Cuidado com "números disfarçados"**
>
> O CEP e o número da camisa de um jogador são números, mas **não são quantidades**. O CEP 20000 não é "o dobro" do CEP 10000. Trate-os como categorias.

2.3 Medindo semelhança: as distâncias

Uma **distância** é uma regra que transforma dois pontos num número que diz "quão longe estão". Quanto maior, mais diferentes.

Para ser uma **métrica** (distância no sentido matemático), a regra deve cumprir quatro propriedades:

1. **Não negatividade:** d(a, b) ≥ 0.
2. **Identidade:** d(a, b) = 0 se, e somente se, a = b.
3. **Simetria:** d(a, b) = d(b, a).
4. **Desigualdade triangular:** d(a, c) ≤ d(a, b) + d(b, c). Ir direto nunca é mais longo que dar uma volta.

2.3.1 Distância Euclidiana

É a distância "em linha reta", a que você mediria com uma régua.

$$d(a, b) = \sqrt{((a_1 - b_1)^2 + (a_2 - b_2)^2 + \ldots + (a_d - b_d)^2)}$$

Em palavras: subtraia cada atributo, eleve ao quadrado, some tudo e tire a raiz.

Exemplo com Ana e Carla: diferenças (25−31 = −6; 120−95 = 25; 8−4 = 4). Quadrados: 36, 625, 16. Soma: 677. Raiz: ≈ 26,0.

2.3.2 Distância de Manhattan

Imagine andar de táxi em Manhattan, onde os quarteirões formam uma grade. Você não corta caminho pelos prédios. Anda em "L".

$$d(a, b) = |a_1 - b_1| + |a_2 - b_2| + \ldots + |a_d - b_d|$$

Também é chamada de distância L1 ou "do quarteirão". É mais robusta a valores extremos do que a Euclidiana, pois não eleva as diferenças ao quadrado.

2.3.3 Distância de Chebyshev

Considera apenas a **maior** diferença entre atributos.

$$d(a, b) = \max(|a_1 - b_1|, \ldots, |a_d - b_d|)$$

Analogia: o rei do xadrez. O número de jogadas que ele precisa para ir de uma casa a outra é a distância de Chebyshev.

2.3.4 Distância de Minkowski: a família completa

As três anteriores são casos de uma fórmula única, com um parâmetro *p*:

$$d_p(a, b) = (\Sigma |a_i - b_i|^p)^{1/p}$$

- p = 1 → Manhattan.
- p = 2 → Euclidiana.
- p → ∞ → Chebyshev.

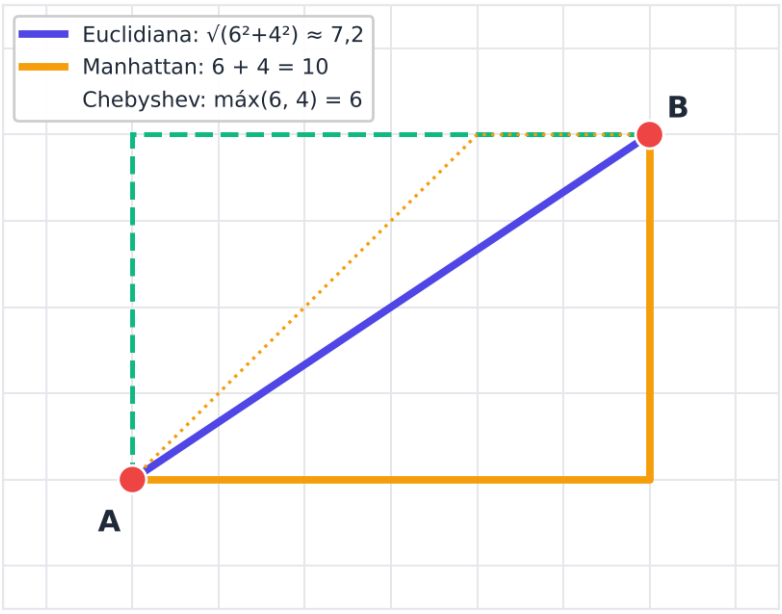


Figura 2.2 — Três formas de medir a distância entre dois pontos na mesma grade.

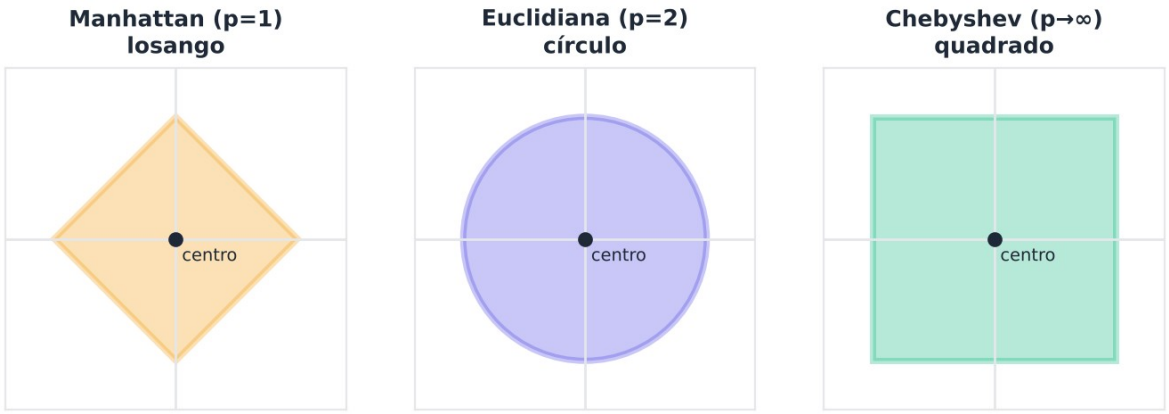


Figura 2.3 — O "círculo" de raio 1 muda de forma conforme a distância: losango (Manhattan), círculo (Euclidiana) e quadrado (Chebyshev).

2.3.5 Distância (e similaridade) do cosseno

Às vezes a **direção** importa mais que o tamanho. Pense em dois textos que falam do mesmo assunto, um curto e um longo. As palavras aparecem em proporções parecidas, mas as contagens do texto longo são maiores.

A **similaridade do cosseno** mede o ângulo entre os dois vetores:

$$\cos(a, b) = (a \cdot b) / (\|a\| \times \|b\|)$$

Vale 1 se apontam para a mesma direção, 0 se são perpendiculares e −1 se são opostos. A **distância do cosseno** é 1 − cos(a, b).

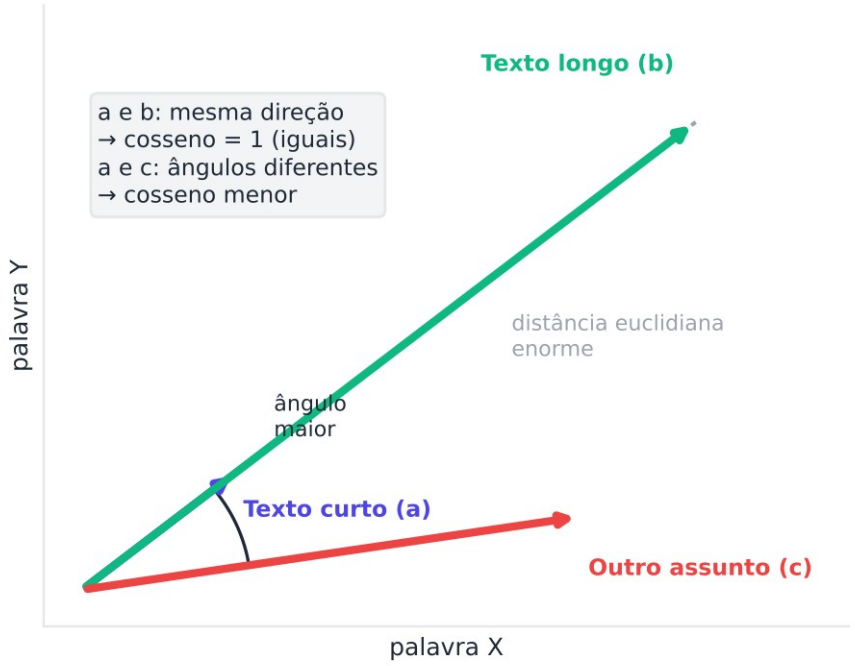


Figura 2.4 — Dois vetores com tamanhos muito diferentes, mas mesma direção: distância Euclidiana grande, distância do cosseno zero.

> SAIBA MAIS **Quando usar o cosseno?**
>
> É a escolha clássica para texto, recomendação e *embeddings* (representações numéricas aprendidas), porque ignora o "volume" e compara o "perfil" (Aggarwal, 2015).

2.3.6 Distância de Hamming e de Jaccard

Para dados binários ou conjuntos:

- **Hamming:** conta em quantas posições dois textos de mesmo tamanho diferem. "casa" e "cama" têm Hamming 1.
- **Jaccard:** compara dois conjuntos. É o tamanho da interseção dividido pelo tamanho da união. A distância de Jaccard é 1 menos esse valor.

Exemplo: o carrinho de Ana tem {pão, leite, café}. O de Bruno tem {pão, café, açúcar}. Interseção = 2 itens; união = 4 itens. Similaridade = 2/4 = 0,5. Distância = 0,5.

2.3.7 Dados mistos: a distância de Gower

E se o cliente tem idade (número), cidade (categoria) e plano (ordinal)? Gower (1971) propôs calcular uma distância parcial por atributo, de 0 a 1, e tirar a média.

- Para números: diferença dividida pela amplitude do atributo.
- Para categorias: 0 se iguais, 1 se diferentes.

É uma das soluções mais usadas quando os tipos de atributos se misturam.

Distância	Melhor quando...	Cuidado
Euclidiana	Atributos contínuos, mesma escala	Sensível a valores extremos
Manhattan	Há outliers ou muitos atributos	Depende dos eixos

Distância	Melhor quando...	Cuidado
Chebyshev	Só a maior diferença importa	Ignora o resto
Cosseno	Texto e embeddings	Ignora o tamanho
Jaccard	Conjuntos e cestas	Só presença/ausência
Gower	Dados mistos	Mais lenta

2.4 Distâncias no Python

```python
import numpy as np
from scipy.spatial import distance

ana   = np.array([25, 120, 8])
carla = np.array([31,  95, 4])

print("Euclidiana:", distance.euclidean(ana, carla))
print("Manhattan :", distance.cityblock(ana, carla))
print("Chebyshev :", distance.chebyshev(ana, carla))
print("Cosseno   :", distance.cosine(ana, carla))

# Matriz com a distância entre TODOS os pares de uma vez
from sklearn.metrics import pairwise_distances
X = np.array([[25, 120, 8],
              [62, 340, 12],
              [31,  95, 4],
              [58, 310, 10]])
D = pairwise_distances(X, metric="euclidean")
print(np.round(D, 1))
```

A matriz `D` é **simétrica** e tem zeros na diagonal. A distância de cada cliente a ele mesmo é zero.

2.5 O problema da escala

Observe a tabela da padaria. A coluna **Gasto** vai de 95 a 340. A coluna **Visitas** vai de 4 a 12. Ao calcular a distância Euclidiana, o gasto domina tudo.

Uma diferença de 4 visitas (muito!) pesa quase nada ao lado de uma diferença de R$ 25 no gasto (pouco!). O algoritmo "enxerga" quase só o dinheiro.

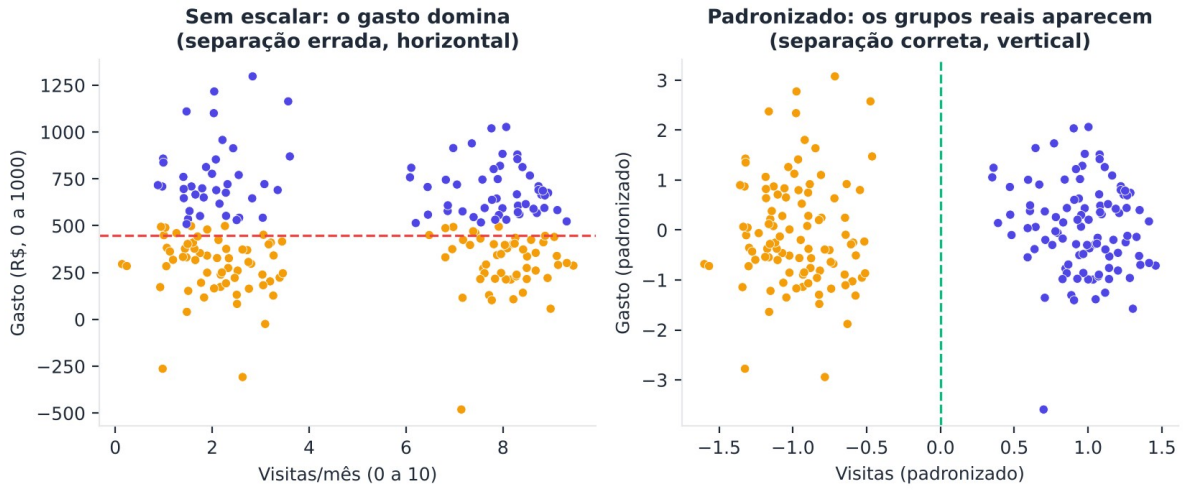


Figura 2.5 — Sem escalar, o atributo de maior amplitude domina os grupos. Depois de padronizar, ambos contribuem igualmente.

A solução é **colocar todos os atributos em escala comparável**.

2.5.1 Padronização (z-score)

Subtrai a média e divide pelo desvio padrão. O resultado tem média 0 e desvio padrão 1.

$$z = (x - \text{média}) \,/\, \text{desvio padrão}$$

O que significa? "Quantos desvios padrão acima ou abaixo da média este valor está?". Um z de 2 é um valor bem alto. Um z de −1 é um pouco abaixo da média.

2.5.2 Normalização min-max

Comprime cada atributo para o intervalo [0, 1].

$$x' = (x - \text{mínimo}) \,/\, (\text{máximo} - \text{mínimo})$$

2.5.3 Escalonamento robusto

Usa a **mediana** e o **intervalo interquartil** (a faixa que contém a metade central dos dados). Resiste bem a valores extremos.

$$x' = (x - \text{mediana}) \,/\, \text{IQR}$$

2.5.4 Qual escolher?

- **Padronização:** a escolha padrão para a maioria dos casos (K-means, PCA, GMM).
- **Min-max:** quando você precisa de limites fixos, como em redes neurais com entradas em [0, 1].
- **Robusto:** quando há muitos outliers.

```python
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

X = np.array([[25, 120, 8],
              [62, 340, 12],
              [31,  95, 4],
              [58, 310, 10]], dtype=float)
```

```
for nome, esc in [("Padronização", StandardScaler()),
                  ("Min-Max", MinMaxScaler()),
                  ("Robusto", RobustScaler())]:
    print(nome)
    print(np.round(esc.fit_transform(X), 2))
```

> ATENÇÃO **Regra de ouro contra vazamento**
>
> Se você for dividir os dados em treino e teste, **ajuste** o escalonador (`fit`) apenas no treino e depois **aplique** (`transform`) no teste. Caso contrário, informação do teste "vaza" para o treino.

2.6 Valores ausentes

Dados reais têm buracos. Alguém não informou a renda. Um sensor falhou.

Estratégias comuns:

1. **Remover** linhas ou colunas com muitos buracos. Simples, mas desperdiça dados.
2. **Imputar pela média ou mediana.** Rápido. A mediana é mais segura com outliers.
3. **Imputar pela moda** (valor mais frequente), para categorias.
4. **Imputar pelos vizinhos** (KNN): preenche com a média dos exemplos mais parecidos.
5. **Criar uma coluna indicadora** "estava ausente?". Às vezes a ausência é informativa.

PYTHON

```python
import numpy as np
from sklearn.impute import SimpleImputer, KNNImputer

X = np.array([[25, 120.0],
              [62, np.nan],
              [31,  95.0],
              [np.nan, 310.0]])

print(SimpleImputer(strategy="median").fit_transform(X))
print(KNNImputer(n_neighbors=2).fit_transform(X))
```

2.7 Codificando categorias

Os algoritmos só entendem números. Como transformar "azul, verde, vermelho"?

2.7.1 Codificação one-hot

Cria uma coluna para cada categoria, com 1 ou 0.

Cor	é_azul	é_verde	é_vermelho
azul	1	0	0
verde	0	1	0
vermelho	0	0	1

Assim, nenhuma cor parece "maior" que outra.

2.7.2 Codificação ordinal

Para categorias com ordem natural: baixo = 0, médio = 1, alto = 2.

> ATENÇÃO **Erro clássico**
>
> Codificar "azul = 0, verde = 1, vermelho = 2" faz o algoritmo achar que vermelho está "duas vezes mais longe" do azul. Para categorias **sem ordem**, use *one-hot*.

```python
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder

df = pd.DataFrame({"cor": ["azul", "verde", "vermelho", "azul"],
                   "tamanho": ["P", "M", "G", "M"]})

print(pd.get_dummies(df["cor"]))

ord_enc = OrdinalEncoder(categories=[["P", "M", "G"]])
print(ord_enc.fit_transform(df[["tamanho"]]))
```

2.8 Selecionando e criando atributos

Nem todo atributo ajuda. Um atributo irrelevante é **ruído**: ele aumenta distâncias sem trazer informação.

- **Remova atributos constantes** ou quase constantes.
- **Remova atributos duplicados** ou altamente correlacionados.
- **Crie atributos novos** a partir de conhecimento do problema (por exemplo, "gasto por visita" = gasto ÷ visitas).
- **Transforme escalas assimétricas.** Valores como renda têm cauda longa. O logaritmo "aproxima" os extremos.

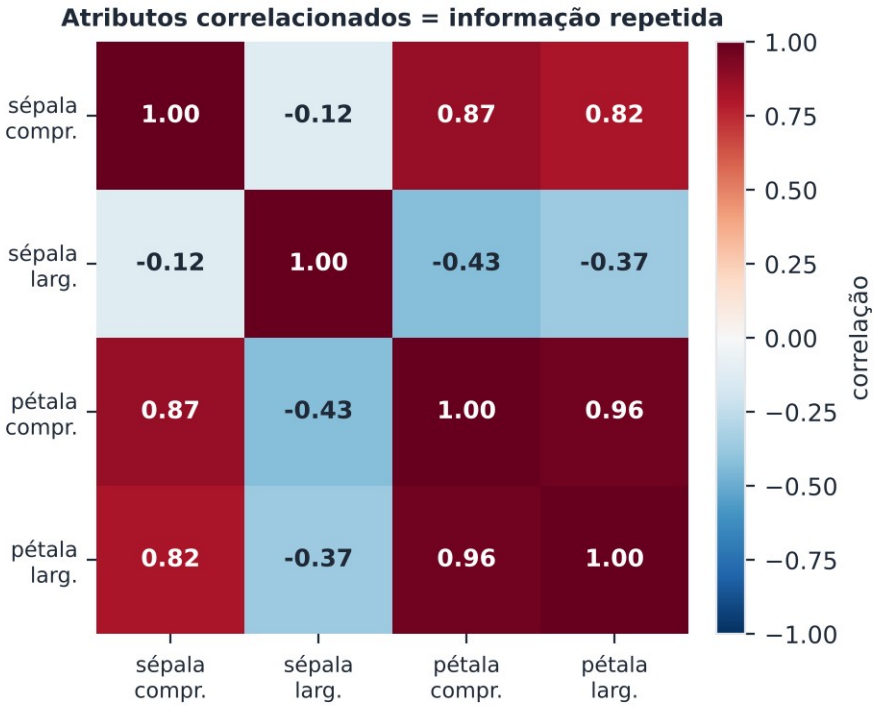


Figura 2.6 — Atributos muito correlacionados carregam informação repetida. Matriz de correlação como mapa de calor.

2.9 A maldição da dimensionalidade

Richard Bellman usou essa expressão em 1957 para descrever as dificuldades de trabalhar com muitos atributos.

2.9.1 O espaço fica vazio

Imagine 10 pontos numa reta de 1 metro: eles ficam bem próximos. Agora espalhe os mesmos 10 pontos num quadrado de 1 metro de lado: já ficam mais distantes. Num cubo, mais ainda. Em 100 dimensões, os pontos estão praticamente isolados no meio do nada.

Para "cobrir" o espaço com a mesma densidade, o número de exemplos cresce **exponencialmente** com a dimensão.

2.9.2 Todas as distâncias ficam parecidas

Beyer et al. (1999) mostraram que, em alta dimensão, a razão entre a maior e a menor distância tende a 1. Ou seja: **"o vizinho mais próximo" deixa de ser muito mais próximo que "o mais distante"**. Aggarwal, Hinneburg e Keim (2001) mostraram que normas com p menor (como Manhattan) sofrem menos com esse efeito.

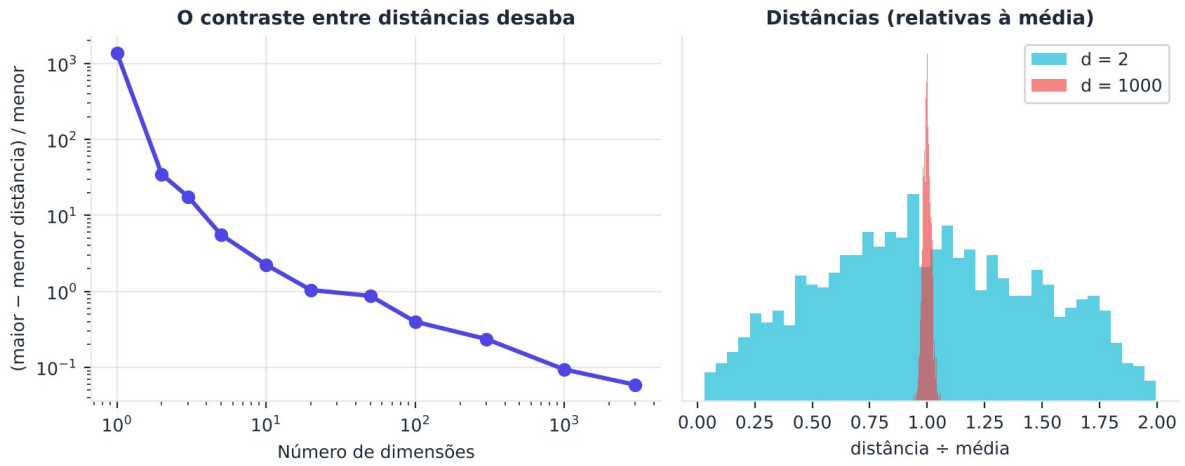


Figura 2.7 — Experimento: com o aumento do número de dimensões, a diferença relativa entre a menor e a maior distância encolhe.

2.9.3 O que fazer?

- Selecionar apenas atributos relevantes.
- Reduzir dimensionalidade (Capítulos 8 e 9).
- Usar métricas mais robustas (Manhattan, cosseno).
- Coletar mais dados, quando possível.

```python
import numpy as np

rng = np.random.default_rng(0)
for d in [2, 10, 100, 1000]:
    X = rng.random((500, d))
    ref = X[0]
    dist = np.linalg.norm(X[1:] - ref, axis=1)
    contraste = (dist.max() - dist.min()) / dist.min()
    print(f"d={d:5d} -> contraste relativo = {contraste:.2f}")
```

Veja como o contraste cai rapidamente quando *d* aumenta.

2.10 Um pipeline limpo no scikit-learn

O scikit-learn permite encadear etapas num **pipeline**. Assim, o mesmo tratamento é aplicado sempre, sem esquecer nenhum passo.

```python
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X = np.array([[25, 120.0, 8],
              [62, 340.0, 12],
              [31, np.nan, 4],
              [58, 310.0, 10],
              [29, 100.0, 5],
              [60, 330.0, 11]])
```

```
pipe = Pipeline([
    ("imputar", SimpleImputer(strategy="median")),
    ("escalar", StandardScaler()),
    ("agrupar", KMeans(n_clusters=2, n_init=10, random_state=0)),
])

print(pipe.fit_predict(X))
```

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Dados** = matriz: linhas são exemplos; colunas, atributos.
- **Semelhança = proximidade**. Escolha a distância certa para os seus dados.
- **Euclidiana** (régua), **Manhattan** (quarteirão), **Chebyshev** (rei do xadrez), **cosseno** (ângulo), **Jaccard** (conjuntos), **Gower** (dados mistos).
- **Escale os dados.** Padronização é a escolha padrão.
- **Ausentes:** remova, impute ou sinalize.
- **Categorias sem ordem:** *one-hot*.
- **Maldição da dimensionalidade:** em muitas dimensões, as distâncias se parecem.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Em uma matriz de dados com n exemplos e d atributos, a forma da matriz X é:

a) n linhas e n colunas.
b) d linhas e n colunas.
c) d linhas e d colunas.
d) n linhas e d colunas.

Questão 2. O CEP de uma cidade deve ser tratado, em geral, como:

a) quantidade contínua.
b) variável ordinal.
c) variável binária.
d) variável categórica.

Questão 3. "Baixo, médio, alto" é um exemplo de atributo:

a) ordinal.
b) binário.
c) nominal.
d) contínuo.

Questão 4. Qual propriedade NÃO é exigida de uma métrica de distância?

a) Simetria.
b) Distância sempre menor que 1.
c) Desigualdade triangular.
d) Não negatividade.

Questão 5. A distância Euclidiana entre os pontos (0, 0) e (3, 4) é:

a) 4.
b) 7.
c) 5.
d) 3.

Questão 6. A distância de Manhattan entre (0, 0) e (3, 4) é:

a) 12.
b) 4.
c) 5.
d) 7.

Questão 7. A distância de Chebyshev entre (0, 0) e (3, 4) é:

a) 7.
b) 5.
c) 4.
d) 3.

Questão 8. A distância de Minkowski com p = 1 equivale à distância:

a) Euclidiana.
b) de Chebyshev.
c) de Manhattan.
d) do cosseno.

Questão 9. A "bola unitária" (conjunto de pontos a distância 1 da origem) na distância de Manhattan, em duas dimensões, tem forma de:

a) losango.
b) círculo.
c) triângulo.
d) quadrado.

Questão 10. A similaridade do cosseno entre dois vetores com a mesma direção é:

a) 1.
b) 0,5.
c) −1.
d) 0.

Questão 11. A distância do cosseno é indicada principalmente para:

a) coordenadas geográficas.
b) texto e embeddings.
c) atributos binários simples apenas.
d) dados ordenados no tempo.

Questão 12. A distância de Hamming entre "casa" e "cama" é:

a) 0.
b) 4.
c) 1.
d) 2.

Questão 13. Os carrinhos {pão, leite, café} e {pão, café, açúcar} têm distância de Jaccard igual a:

a) 0,25.
b) 1.
c) 0,75.
d) 0,5.

Questão 14. A distância de Gower é apropriada quando os dados são:

a) todos numéricos e na mesma escala.
b) apenas imagens.
c) mistos (numéricos, categóricos, ordinais).
d) apenas texto.

Questão 15. Ao aplicar a distância Euclidiana sem escalar, o atributo que domina o resultado é aquele que:

a) tem mais valores ausentes.
b) é categórico.
c) tem maior amplitude de valores.
d) tem menor amplitude.

Questão 16. A padronização (z-score) produz atributos com:

a) mediana 1 e média 0.
b) mínimo 0 e máximo 1.
c) média 0 e desvio padrão 1.
d) soma igual a 1.

Questão 17. Um valor 70 em um atributo com média 50 e desvio padrão 10 tem z-score igual a:

a) 1.
b) 0,5.
c) 2.
d) 20.

Questão 18. Um valor 30, em um atributo com mínimo 10 e máximo 110, vale na normalização min-max:

a) 0,3.
b) 0,5.
c) 1.
d) 0,2.

Questão 19. O escalonamento robusto usa:

a) mediana e intervalo interquartil.
b) média e desvio padrão.
c) mínimo e máximo.
d) moda e variância.

Questão 20. Para evitar vazamento de dados (data leakage), o escalonador deve ser ajustado (fit):

a) apenas no conjunto de teste.
b) nos conjuntos de treino e teste juntos.
c) depois da avaliação.
d) apenas no conjunto de treino.

Questão 21. A imputação de valores ausentes pela mediana é preferível à média quando:

a) não há ausentes.
b) há muitos valores extremos (outliers).
c) os dados são categóricos.
d) os dados já estão padronizados.

Questão 22. O KNNImputer preenche um valor ausente com:

a) a média dos exemplos mais parecidos.
b) zero.
c) um valor aleatório.
d) o máximo da coluna.

Questão 23. Para a cor (azul, verde, vermelho), sem ordem natural, a codificação mais adequada é:

a) ordinal 0, 1, 2.
b) logarítmica.
c) min-max.
d) one-hot.

Questão 24. O problema da codificação ordinal aplicada a categorias sem ordem é que:

a) aumenta demais a memória.
b) o algoritmo trava.
c) o algoritmo supõe distâncias e ordens que não existem.
d) remove atributos.

Questão 25. Um atributo com cauda longa, como renda, costuma ser tratado com:

a) transformação logarítmica.
b) codificação one-hot.
c) distância de Hamming.
d) remoção da linha.

Questão 26. A "maldição da dimensionalidade" foi nomeada por:

a) Bellman.
b) Lloyd.
c) Pearson.
d) Hotelling.

Questão 27. Em alta dimensão, a razão entre a maior e a menor distância tende a:

a) infinito.
b) 100.
c) 1.
d) zero.

Questão 28. Qual medida é, segundo Aggarwal, Hinneburg e Keim (2001), menos afetada pela alta dimensão?

a) Normas com p menor, como Manhattan.
b) A distância de Hamming apenas.
c) Normas com p muito grande.
d) Nenhuma é afetada.

Questão 29. Um pipeline do scikit-learn serve para:

a) encadear etapas de pré-processamento e modelo, aplicadas sempre na mesma ordem.
b) substituir o Python.
c) plotar gráficos.
d) baixar dados da internet.

Questão 30. Atributos muito correlacionados entre si indicam, em geral:

a) informação repetida.
b) necessidade de aumentar k.
c) ausência de variância.
d) dados inválidos.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique por que "semelhança = proximidade" é a ideia central de muitos algoritmos deste livro.
2. Calcule a distância Euclidiana, de Manhattan e de Chebyshev entre Ana (25, 120, 8) e Carla (31, 95, 4).
3. Descreva a analogia usada para explicar as distâncias Euclidiana, Manhattan e Chebyshev.
4. Quando a distância do cosseno é preferível à Euclidiana? Dê um exemplo.
5. Calcule a similaridade e a distância de Jaccard entre {pão, leite, café} e {pão, café, açúcar}.
6. Explique o problema da escala usando o exemplo da padaria (gasto e visitas).
7. Compare padronização, normalização min-max e escalonamento robusto, indicando quando usar cada uma.
8. O que é vazamento de dados (data leakage) no pré-processamento e como evitá-lo?
9. Liste cinco estratégias para tratar valores ausentes e comente uma vantagem ou risco de cada.
10. Explique a codificação one-hot e por que ela é preferível à codificação ordinal para categorias sem ordem.
11. Explique o que são atributos constantes ou redundantes e por que removê-los ajuda o agrupamento.
12. Descreva a maldição da dimensionalidade e duas consequências para algoritmos baseados em distância.
13. Quais ações podem mitigar a maldição da dimensionalidade?
14. Qual é a diferença entre atributos nominais, ordinais e binários? Dê um exemplo de cada.
15. Explique as quatro propriedades de uma métrica e dê um exemplo de medida que as viole.
16. Explique como a distância de Gower trata atributos numéricos e categóricos.
17. Por que a transformação logarítmica ajuda em atributos como renda?
18. Descreva o experimento do contraste relativo de distâncias em função do número de dimensões e o que ele mostra.
19. Monte um pipeline em palavras (ou código) que impute ausentes, padronize e agrupe com K-means.
20. Por que valores extremos prejudicam a média e o desvio padrão, e que alternativa existe?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Matriz de distâncias entre clientes

NÍVEL: BÁSICO

Monte um `DataFrame` com quatro clientes (idade, gasto mensal, visitas por mês): Ana (25, 120, 8), Bruno (62, 340, 12), Carla (31, 95, 4) e Diego (58, 310, 10). Use `pairwise_distances` para calcular as matrizes de distância **Euclidiana**, **Manhattan** e **cosseno**. Qual par de clientes é o mais próximo em cada métrica?

Exercício 2 — Três escalonadores diante de um outlier

NÍVEL: BÁSICO

Crie um vetor com os valores 10, 12, 11, 13, 12, 11, 10, 500 (note o valor extremo). Aplique `StandardScaler`, `MinMaxScaler` e `RobustScaler`. Para cada um, mostre os valores transformados dos **7 primeiros** pontos. Qual deles "esmaga" os pontos normais por causa do outlier?

Exercício 3 — Imputar: mediana ou vizinhos?

NÍVEL: INTERMEDIÁRIO

Carregue o Wine, padronize e **apague aleatoriamente 10% dos valores** (semente fixa). Preencha os buracos com `SimpleImputer(strategy="median")` e com `KNNImputer(n_neighbors=5)`. Compare o erro (RMSE) entre os valores imputados e os valores verdadeiros.

Exercício 4 — Codificando variáveis categóricas

NÍVEL: INTERMEDIÁRIO

Crie um `DataFrame` com as colunas `cidade` (nominal: "SP", "RJ", "BH"), `plano` (ordinal: "básico" < "médio" < "premium") e `gasto` (numérica). Use `ColumnTransformer` para aplicar **one-hot** em `cidade`, **codificação ordinal** em `plano` e **padronização** em `gasto`. Mostre a matriz resultante e os nomes das colunas.

Exercício 5 — A maldição da dimensionalidade em números

NÍVEL: AVANÇADO

Para dimensões d = 2, 10, 100 e 1000, gere 500 pontos uniformes e calcule o **contraste relativo** (distância máxima − mínima) / mínima, a partir de um ponto de referência, usando as métricas Euclidiana e Manhattan. Mostre uma tabela. Qual métrica sofre menos?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Adult (censo, com valores ausentes e variáveis categóricas e numéricas), Statlog (German Credit Data) e Bank Marketing.
> - **Kaggle:** Telco Customer Churn (clientes de telecom, com dados mistos).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Pipeline de preparação para uma carteira de clientes

Fonte dos dados. Simulados pela função abaixo (nenhum dado real de clientes foi usado).

Contexto. Uma seguradora (fictícia) quer segmentar seus clientes, mas a base está "suja": há valores ausentes, outliers, atributos de escalas diferentes e variáveis categóricas. Seu trabalho é construir um **pipeline de preparação reprodutível** e mostrar que ele melhora o agrupamento.

Dados. Use a função abaixo para gerar a base (dados sintéticos, com semente fixa).

```python
import numpy as np
import pandas as pd


def gerar_clientes(n=600, seed=0):
    rng = np.random.default_rng(seed)
    g = rng.integers(0, 3, n)  # 3 perfis ocultos
    idade = np.where(
        g == 0,
        rng.normal(28, 4, n),
        np.where(g == 1, rng.normal(45, 6, n), rng.normal(63, 5, n)),
    )
    renda = np.exp(rng.normal(np.where(g == 0, 8.0, np.where(g == 1, 8.8, 8.4)), 0.35, n))
    sinistros = rng.poisson(np.where(g == 0, 2.0, np.where(g == 1, 0.6, 1.2)))
    cidade = rng.choice(["SP", "RJ", "BH", "POA"], n, p=[0.4, 0.3, 0.2, 0.1])
    plano = np.where(g == 0, "básico", np.where(g == 1, "premium", "médio"))
    df = pd.DataFrame(
        {
            "idade": idade,
            "renda": renda,
            "sinistros": sinistros,
            "cidade": cidade,
            "plano": plano,
        }
    )
    df.loc[rng.random(n) < 0.08, "idade"] = np.nan  # ausentes
    df.loc[rng.random(n) < 0.05, "cidade"] = np.nan
    df.loc[rng.choice(n, 5, replace=False), "renda"] *= 40  # outliers
    return df, g
```

Tarefas.

1. Inspecione a base: tipos, ausentes por coluna e estatísticas.
2. Faça uma abordagem **ingênua**: remova linhas com ausentes, use `LabelEncoder` (ou códigos inteiros) para as categorias e agrupe com `KMeans(3)` sem escalar. Meça a silhueta.
3. Construa um `Pipeline` com `ColumnTransformer`: imputação (mediana para numéricas, mais frequente para `cidade`), `log1p` na renda, `RobustScaler` nas numéricas, `OneHotEncoder` em `cidade` e codificação ordinal em `plano`. Termine com `KMeans(3)`.
4. Compare, para as duas abordagens, a silhueta e o ARI contra os perfis ocultos `g` (que só você, como autor da base, conhece).
5. Escreva 5 linhas explicando **o que cada etapa do pipeline resolveu**.

Entregáveis. Script com a função e o pipeline; tabela comparando as duas abordagens; texto de 5 linhas.

A solução de referência está no Apêndice E.

PARTE II

Parte II — Agrupamento

Como dividir exemplos em grupos: por distância (K-means, hierárquico), por densidade (DBSCAN, OPTICS, HDBSCAN), por probabilidade (GMM) e por grafos (espectral). E como saber se os grupos são bons.

Nesta parte:

- Cap. 3 — K-means: o Clássico do Agrupamento
- Cap. 4 — Agrupamento Hierárquico: a Árvore de Semelhanças
- Cap. 5 — Agrupamento por Densidade: DBSCAN, OPTICS e HDBSCAN
- Cap. 6 — Modelos de Mistura Gaussiana e Agrupamento Espectral
- Cap. 7 — Como Saber se os Grupos São Bons? Avaliação e Validação



Capítulo 3 — K-means: o Clássico do Agrupamento

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar a ideia do K-means com uma analogia do cotidiano.
- Executar, passo a passo, o algoritmo de Lloyd.
- Entender a função objetivo (inércia) que o K-means minimiza.
- Escolher *k* com o método do cotovelo e inicializar com k-means++.
- Reconhecer as limitações do K-means e conhecer suas variantes (Mini-Batch, K-medoids, K-modes).
- Aplicar K-means em segmentação de clientes e compressão de imagens.

3.1 A analogia da pizzaria

Você quer abrir **3 pizzarias** numa cidade. Cada morador vai pedir na pizzaria mais próxima. Onde colocá-las?

Uma boa ideia é:

1. Colocar as três em lugares aleatórios.
2. Cada morador escolhe a pizzaria mais próxima.
3. Cada pizzaria **muda de endereço** para o centro dos moradores que atende.
4. Repetir 2 e 3 até ninguém mudar de pizzaria.

Pronto, você acabou de executar o **K-means**. As pizzarias são os **centroides**. Os bairros que cada uma atende são os **grupos**.

MEMORIZE **Em uma frase**

K-means divide os dados em *k* grupos, cada um representado pela **média** (centro) de seus pontos.

3.2 O algoritmo, passo a passo

O método foi proposto por Lloyd (publicado em 1982) e batizado por MacQueen (1967). A versão mais usada é o **algoritmo de Lloyd**.

Entrada: os dados X e o número de grupos *k*.

1. **Inicialização:** escolha *k* pontos como centroides iniciais.
2. **Atribuição:** associe cada ponto ao centroide mais próximo.
3. **Atualização:** recalcule cada centroide como a **média** dos pontos do seu grupo.

4. **Repetição:** volte ao passo 2 até que as atribuições não mudem (convergência) ou se atinja um limite de iterações.

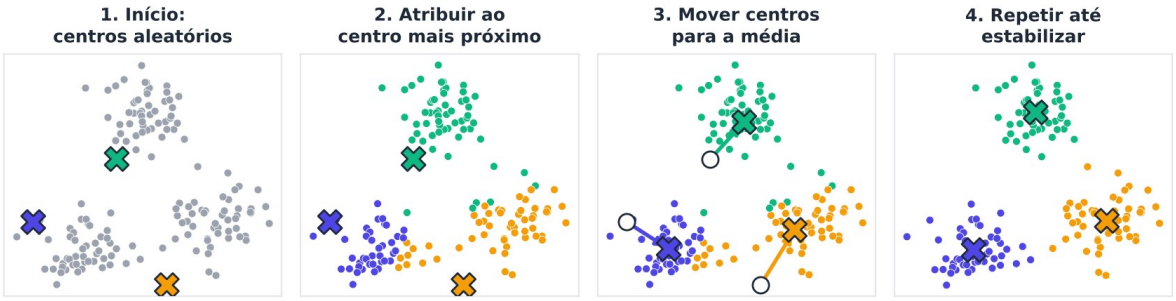


Figura 3.1 — O K-means em ação: iniciar, atribuir, atualizar e repetir até estabilizar.

3.2.1 Exemplo numérico minúsculo

Temos 6 pontos numa reta: **1, 2, 3, 10, 11, 12**. Queremos *k* = 2.

- **Início:** centroides em 1 e 2.
- **Rodada 1, atribuição:** o ponto 1 vai para o centroide 1. O ponto 2 vai para o centroide 2. O ponto 3 está a 2 do centroide 1 e a 1 do centroide 2, então vai para o 2. Os pontos 10, 11 e 12 também vão para o 2.
- **Rodada 1, atualização:** o grupo A = {1} tem centroide 1. O grupo B = {2, 3, 10, 11, 12} tem centroide 7,6.
- **Rodada 2, atribuição:** o ponto 2 está a 1 do centroide A e a 5,6 do B, então migra para A. O ponto 3 também migra para A (2 contra 4,6). Os pontos 10, 11 e 12 ficam em B.
- **Rodada 2, atualização:** A = {1, 2, 3} tem centroide 2. B = {10, 11, 12} tem centroide 11.
- **Rodada 3:** ninguém muda de grupo. **Convergiu!**

3.3 A matemática por trás (sem medo)

O K-means tenta minimizar a **inércia**, também chamada de soma dos quadrados dentro dos grupos (WCSS, do inglês *within-cluster sum of squares*):

$$J = \sum_{k} \sum_{x \in \text{grupo } k} \|x - \mu_k\|^2$$

Em palavras: para cada ponto, calcule o quadrado da distância até o centro do seu grupo. Some tudo. **Quanto menor, mais compactos os grupos.**

Duas propriedades importantes:

- A cada iteração, J **nunca aumenta**. Por isso o algoritmo sempre termina.
- Mas pode parar num **mínimo local**: uma solução boa, não necessariamente a melhor possível. O problema geral é NP-difícil (ou seja, não se conhece algoritmo rápido que garanta o ótimo) (Jain, 2010).

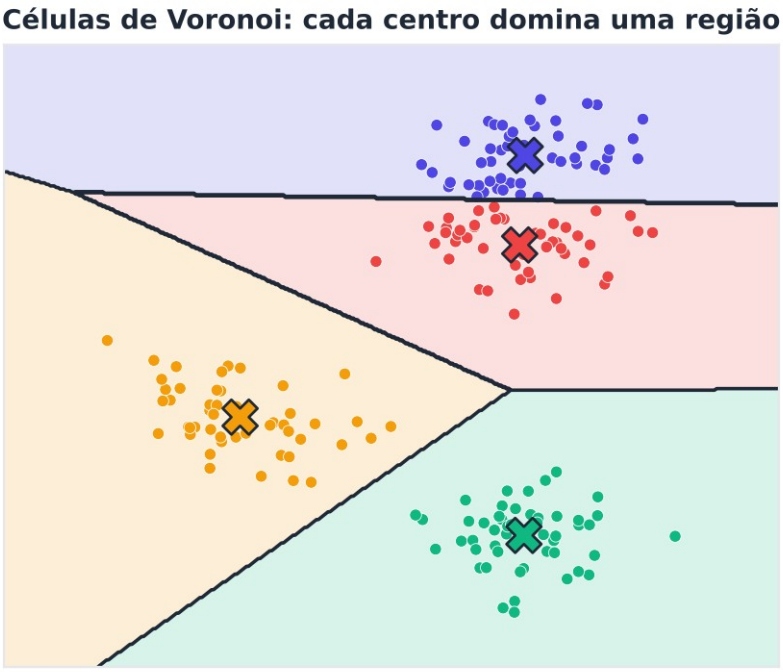


Figura 3.2 — Cada centroide "domina" uma região do plano chamada célula de Voronoi. As fronteiras entre grupos são retas.

> SAIBA MAIS **Por que a média?**
>
> Para a distância Euclidiana ao quadrado, a média é exatamente o ponto que minimiza a soma das distâncias dentro de um grupo. É por isso que o passo de atualização usa a média. Se mudar a distância, o centro ideal muda (veja K-medoids, na Seção 3.8).

3.4 Colocando a mão na massa

PYTHON

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Dados sintéticos: 3 "nuvens" de pontos
X, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=7)

km = KMeans(n_clusters=3, n_init=10, random_state=0)
rotulos = km.fit_predict(X)

print("Centroides:\n", km.cluster_centers_)
print("Inércia   :", km.inertia_)
print("Iterações :", km.n_iter_)

plt.scatter(X[:, 0], X[:, 1], c=rotulos, s=20)
plt.scatter(*km.cluster_centers_.T, c="red", marker="X", s=200)
plt.title("K-means com k=3")
plt.show()
```

Atributos úteis do objeto `KMeans`:

- `cluster_centers_`: as coordenadas dos centroides.

- `labels_`: o grupo de cada ponto.
- `inertia_`: o valor de J.
- `n_iter_`: quantas iterações foram necessárias.

Para classificar **novos** pontos, use `km.predict(novos_pontos)`.

3.5 O problema da inicialização

Os centroides iniciais importam muito. Se dois começarem no mesmo grupo natural, o algoritmo pode terminar com uma divisão ruim.

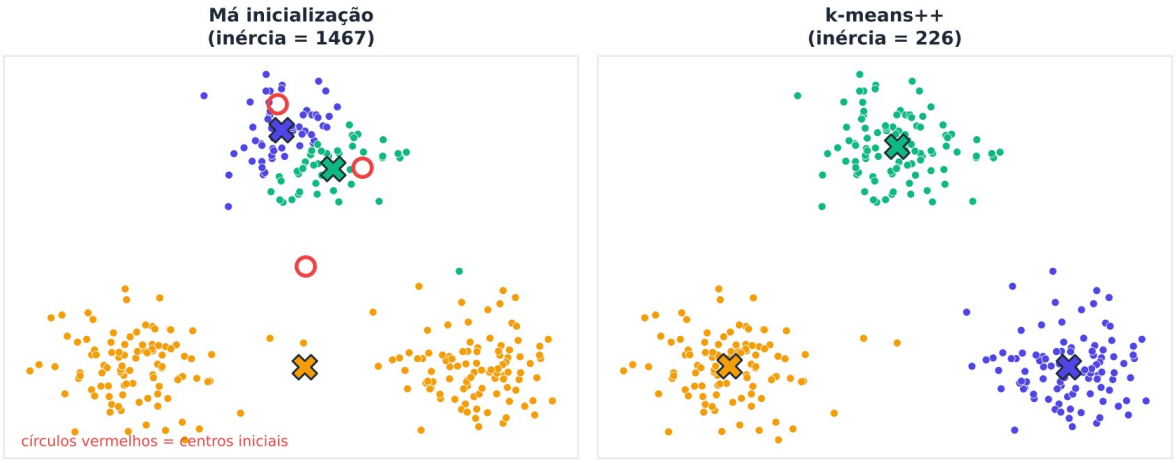


Figura 3.3 — Uma má inicialização leva a uma solução ruim (esquerda). O k-means++ espalha os centros iniciais e costuma acertar (direita).

Há três remédios.

1. **Rodar várias vezes** (`n_init`) com sementes diferentes e ficar com a menor inércia.
2. **k-means++** (Arthur; Vassilvitskii, 2007): escolhe o primeiro centro ao acaso. Cada centro seguinte é sorteado com probabilidade proporcional ao **quadrado da distância** ao centro mais próximo já escolhido. Isso espalha os centros. O método tem garantia teórica: o resultado é, em média, no máximo O(log *k*) vezes pior que o ótimo.
3. **Fornecer centros iniciais** com conhecimento do problema.

O scikit-learn usa `init="k-means++"` como padrão.

3.6 Quantos grupos? O método do cotovelo

O K-means exige que você escolha *k*. Como?

Uma técnica popular é rodar o algoritmo para vários valores de *k* e desenhar a inércia. Ela sempre cai quando *k* cresce (no limite, *k* = *n* dá inércia zero). Procure o ponto em que a queda **desacelera**, o "cotovelo".

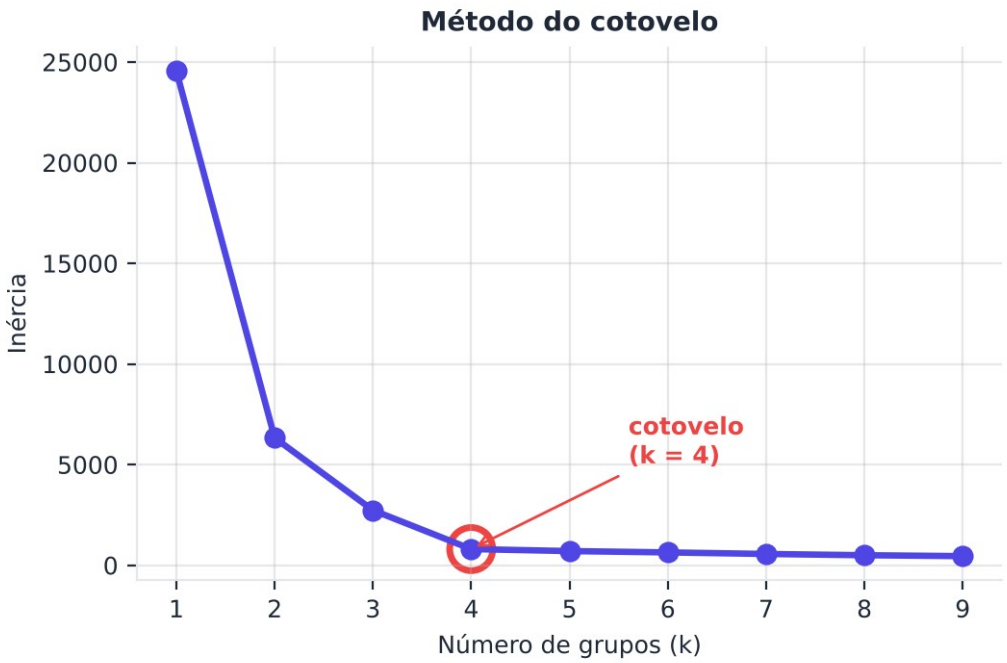


Figura 3.4 — Método do cotovelo: a curva dobra em k = 4, o número verdadeiro de grupos nesses dados.

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=400, centers=4, random_state=3)

ks = range(1, 10)
inercias = [KMeans(n_clusters=k, n_init=10, random_state=0).fit(X).inertia_
            for k in ks]

plt.plot(list(ks), inercias, "o-")
plt.xlabel("k")
plt.ylabel("Inércia")
plt.title("Método do cotovelo")
plt.show()
```

> **ATENÇÃO — O cotovelo nem sempre é nítido**
>
> Em dados reais, a curva costuma ser suave e a escolha fica subjetiva. Combine com outros critérios: **silhueta**, **estatística gap** e interpretabilidade (Capítulo 7). Tibshirani, Walther e Hastie (2001) propuseram a estatística *gap* para tornar essa decisão mais objetiva.

3.7 Quando o K-means falha

O K-means faz suposições fortes. Quando elas não valem, ele erra, mesmo executando perfeitamente.

1. **Grupos esféricos.** Ele busca "bolas" de tamanho parecido. Formas alongadas, curvas ou aneladas são mal capturadas.
2. **Tamanhos semelhantes.** Grupos muito desiguais em tamanho podem ser "cortados" de modo estranho.
3. **Densidades semelhantes.** Um grupo muito denso ao lado de outro difuso confunde o método.

4. **Sensível a outliers.** A média é puxada por valores extremos.
5. **Exige escalonamento.** Atributos de grande amplitude dominam.
6. **Precisa de *k* de antemão.**
7. **Só numérico.** A "média" de categorias não existe.

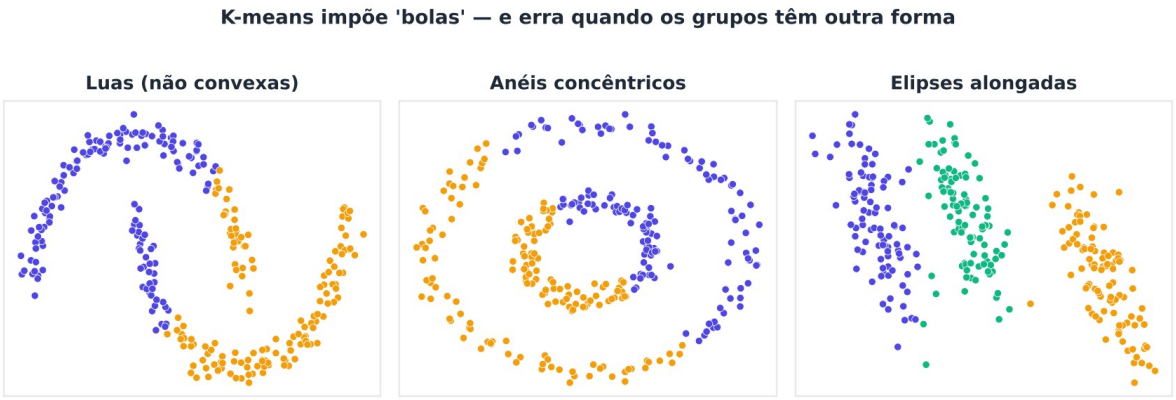


Figura 3.5 — Três cenários em que o K-means não encontra a estrutura "óbvia" para o olho humano: luas, anéis e elipses alongadas.

> **DICA Como se proteger?**
>
> Visualize os dados, escale os atributos, teste mais de um algoritmo e compare. Se os grupos não são "bolas", considere GMM (Capítulo 6), DBSCAN (Capítulo 5) ou agrupamento espectral (Capítulo 6).

3.8 Variantes importantes

3.8.1 Mini-Batch K-means

Para bases gigantes, usar todos os pontos em cada iteração é lento. O **Mini-Batch** (Sculley, 2010) usa pequenos lotes aleatórios a cada passo. É muito mais rápido, com pequena perda de qualidade.

```python
from sklearn.cluster import MiniBatchKMeans
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=100_000, centers=5, random_state=1)
mb = MiniBatchKMeans(n_clusters=5, batch_size=1024, n_init=3, random_state=0)
mb.fit(X)
print(mb.cluster_centers_.round(2))
```

3.8.2 K-medoids (PAM)

Em vez da média, usa um **medoide**: um ponto *real* do conjunto de dados que é o mais central do grupo. Vantagens: robusto a outliers e aceita qualquer distância (Kaufman; Rousseeuw, 1990). Desvantagem: custo maior.

Analogia: em vez de pôr a pizzaria no "centro geométrico" (que pode cair num lago), você a coloca no endereço de um morador real.

3.8.3 K-modes

Para dados **categóricos**: troca a média pela **moda** (valor mais frequente) e a distância por uma medida de discordância (Huang, 1998).

3.8.4 Fuzzy C-means

No agrupamento **difuso** (*fuzzy*), cada ponto pertence a todos os grupos, cada um com um **grau** entre 0 e 1. Um cliente pode ser 70 % "econômico" e 30 % "premium" (Bezdek, 1981).

Variante	Muda o quê?	Quando usar
Mini-Batch	Usa lotes	Dados enormes
K-medoids	Centro é ponto real	Outliers; qualquer distância
K-modes	Moda	Dados categóricos
Fuzzy C-means	Pertinência parcial	Fronteiras difusas

3.9 Estudo de caso 1: segmentação de clientes

Uma loja tem, por cliente, três informações: **recência** (dias desde a última compra), **frequência** (compras no ano) e **valor** (gasto total). Essa trinca é conhecida como análise **RFM**.

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

rng = np.random.default_rng(42)
# Três perfis simulados: fiéis, ocasionais e inativos
fieis = np.column_stack([rng.normal(15, 5, 100), rng.normal(30, 5, 100),
                         rng.normal(3000, 500, 100)])
ocasionais = np.column_stack([rng.normal(60, 15, 150), rng.normal(8, 3, 150),
                              rng.normal(700, 200, 150)])
inativos = np.column_stack([rng.normal(250, 40, 80), rng.normal(2, 1, 80),
                            rng.normal(150, 80, 80)])

df = pd.DataFrame(np.vstack([fieis, ocasionais, inativos]),
                  columns=["recencia", "frequencia", "valor"])

Xp = StandardScaler().fit_transform(df)
df["grupo"] = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(Xp)

# Perfil médio de cada grupo: é aqui que damos NOMES aos grupos
print(df.groupby("grupo").mean().round(1))
print(df["grupo"].value_counts())
```

Olhando as médias, você nomeia os grupos: "fiéis" (recência baixa, frequência e valor altos), "ocasionais" e "inativos". Cada um recebe uma estratégia: programa de fidelidade, cupons e campanha de reativação.

3.10 Estudo de caso 2: compressão de imagem por cores

Uma foto colorida tem milhares de cores. Podemos usar K-means para escolher só **8** cores representativas (a **paleta**) e repintar cada pixel com a cor mais próxima. É a chamada **quantização de cores**.

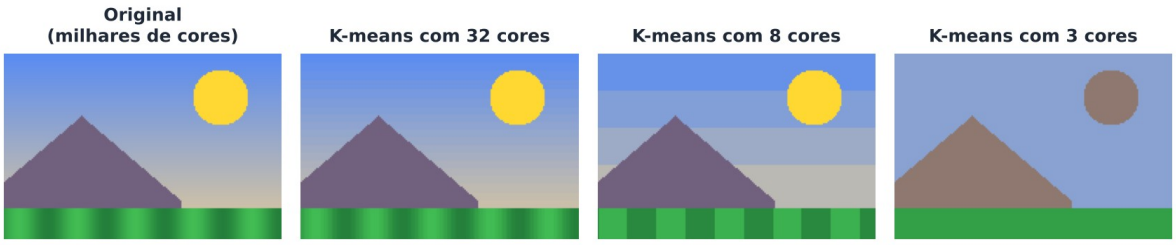


Figura 3.6 — Quantização de cores com K-means: da imagem original a versões com 32, 8 e 3 cores.

```python
import numpy as np
from sklearn.cluster import KMeans

# Imagem sintética 64x64 RGB (gradiente colorido)
h, w = 64, 64
img = np.zeros((h, w, 3))
img[..., 0] = np.linspace(0, 1, w)[None, :]
img[..., 1] = np.linspace(0, 1, h)[:, None]
img[..., 2] = 0.5

pixels = img.reshape(-1, 3)             # cada pixel vira um ponto 3D (R,G,B)
km = KMeans(n_clusters=8, n_init=3, random_state=0).fit(pixels)
img_q = km.cluster_centers_[km.labels_].reshape(h, w, 3)

print("Cores únicas antes:", len(np.unique(pixels.round(3), axis=0)))
print("Cores únicas depois:", len(np.unique(img_q.round(3), axis=0)))
```

Cada pixel passa a ser descrito por um **índice** (de 0 a 7) em vez de três números. Isso reduz muito o tamanho do arquivo. É a mesma ideia da **quantização vetorial** usada em compressão (Lloyd, 1982).

3.11 Boas práticas

- **Escale** os atributos antes.
- **Rode várias inicializações** (`n_init` ≥ 10).
- **Teste vários *k*** e use mais de um critério.
- **Inspecione** os grupos: tamanhos, perfis médios, exemplos reais.
- **Verifique a estabilidade:** mude a semente ou reamostre. Os grupos se mantêm?
- **Documente** como nomeou os grupos.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **K-means:** atribuir ao centro mais próximo → recalcular a média → repetir.
- **Minimiza** a inércia (soma dos quadrados dentro dos grupos).
- **Sempre converge**, mas talvez a um mínimo local.
- **k-means++** melhora a inicialização.
- **Cotovelo** ajuda a escolher *k*, mas não é infalível.
- **Falha** com grupos não esféricos, desiguais ou com outliers.
- **Variantes:** Mini-Batch, K-medoids, K-modes, Fuzzy C-means.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Na analogia da pizzaria, os centroides do K-means correspondem:

a) às ruas.
b) aos pedidos.
c) aos moradores da cidade.
d) às pizzarias.

Questão 2. Qual é a ordem correta dos passos do algoritmo de Lloyd?

a) Atribuir pontos, inicializar centros, atualizar centros.
b) Inicializar centros, atribuir pontos ao centro mais próximo, atualizar centros, repetir.
c) Atualizar centros, atribuir pontos, inicializar centros.
d) Escolher k, ordenar os pontos, calcular a mediana.

Questão 3. No passo de atualização do K-means, o novo centroide de cada grupo é:

a) a mediana dos pontos do grupo.
b) um ponto sorteado.
c) a média dos pontos do grupo.
d) o ponto mais distante do grupo.

Questão 4. A função objetivo (inércia) minimizada pelo K-means é:

a) o número de grupos.
b) a soma dos quadrados das distâncias dos pontos ao centro do seu grupo.
c) o maior diâmetro de grupo.
d) a soma das distâncias entre grupos.

Questão 5. Aplicando K-means com k = 2 aos pontos 1, 2, 3, 10, 11, 12, o resultado final é:

a) {1} e {2, 3, 10, 11, 12}.
b) {1, 2, 3, 10} e {11, 12}.
c) {1, 2} e {3, 10, 11, 12}.
d) {1, 2, 3} e {10, 11, 12}.

Questão 6. Sobre a convergência do K-means, é correto afirmar que:

a) o algoritmo sempre termina, mas pode parar em mínimo local.
b) sempre encontra o ótimo global.
c) a inércia nunca diminui.
d) nunca converge.

Questão 7. Cada região do plano dominada por um centroide é chamada de:

a) variedade.
b) eixo principal.
c) dendrograma.
d) célula de Voronoi.

Questão 8. No scikit-learn, o atributo que guarda o valor final da função objetivo do K-means é:

a) labels_.
b) inertia_.
c) cluster_centers_.
d) n_iter_.

Questão 9. O atributo que traz as coordenadas dos centroides é:

a) labels_.
b) cluster_centers_.
c) inertia_.
d) n_init.

Questão 10. A principal vantagem do k-means++ é:

a) eliminar a necessidade de escolher k.
b) espalhar os centros iniciais, melhorando a qualidade e a estabilidade.
c) detectar outliers.
d) funcionar com dados categóricos.

Questão 11. O parâmetro n_init do KMeans indica:

a) o número máximo de iterações.
b) o número de atributos.
c) quantas inicializações diferentes são testadas, mantendo a de menor inércia.
d) o número de grupos.

Questão 12. A inércia, quando k aumenta:

a) oscila sem padrão.
b) sempre aumenta.
c) permanece constante.
d) sempre diminui (ou se mantém).

Questão 13. O método do cotovelo escolhe k procurando:

a) o k em que a inércia é máxima.
b) o menor valor da inércia.
c) o ponto em que a queda da inércia desacelera.
d) o k mais alto possível.

Questão 14. Qual técnica foi proposta por Tibshirani, Walther e Hastie (2001) para escolher k de modo mais objetivo?

a) Corte do dendrograma.
b) Eigengap.
c) Estatística gap.
d) Critério de Kaiser.

Questão 15. O K-means tem dificuldade com:

a) bases pequenas.
b) grupos em forma de luas ou anéis.
c) grupos esféricos de mesmo tamanho.
d) dados numéricos padronizados.

Questão 16. Por que o K-means é sensível a outliers?

a) Porque a média é puxada por valores extremos.
b) Porque ignora distâncias.
c) Porque usa a mediana.
d) Porque exige dados categóricos.

Questão 17. O K-means exige escalonamento porque:

a) só aceita valores entre 0 e 1.
b) não aceita números negativos.
c) usa distância Euclidiana, e atributos de grande amplitude dominam.
d) usa a distância de Jaccard.

Questão 18. O Mini-Batch K-means é indicado para:

a) dados categóricos.
b) formas aneladas.
c) bases muito grandes, pois usa pequenos lotes aleatórios.
d) hierarquias.

Questão 19. O K-medoids usa como centro:

a) a média dos pontos.
b) a moda.
c) o ponto mais distante.
d) um ponto real do conjunto, o mais central do grupo.

Questão 20. Para dados categóricos, a variante apropriada do K-means é:

a) Fuzzy C-means.
b) Mini-Batch.
c) K-modes.
d) K-means++.

Questão 21. No Fuzzy C-means, cada ponto:

a) é descartado.
b) não tem pertinência.
c) pertence a um único grupo.
d) pertence a todos os grupos com graus entre 0 e 1.

Questão 22. Em segmentação RFM, as letras significam:

a) regressão, fidelidade e margem.
b) recência, frequência e valor monetário.
c) região, faixa etária e média.
d) risco, fraude e mercado.

Questão 23. Na quantização de cores com K-means, os pontos a agrupar são:

a) as colunas da imagem.
b) os nomes dos arquivos.
c) as bordas.
d) os pixels, vistos como pontos 3D (R, G, B).

Questão 24. Na compressão de uma imagem por K-means com 8 cores, cada pixel passa a ser descrito:

a) por um valor binário.
b) por nenhuma informação.
c) por três números reais.
d) por um índice de 0 a 7.

Questão 25. Em uma boa prática de uso do K-means, deve-se:

a) escalar os atributos e testar vários valores de k.
b) ignorar a inspeção dos grupos.
c) usar k fixo sem verificar.
d) rodar uma única inicialização.

Questão 26. Quem batizou o algoritmo como "k-means" em 1967 foi:

a) Ward.
b) Lloyd.
c) MacQueen.
d) Hotelling.

Questão 27. O problema geral de minimizar a inércia é:

a) linear.
b) trivial.
c) NP-difícil.
d) impossível de aproximar.

Questão 28. O que o atributo n_iter_ informa?

a) O número de iterações até a convergência.
b) O valor da inércia.
c) O número de pontos.
d) O número de grupos.

Questão 29. Para classificar novos pontos em grupos já aprendidos, usa-se no scikit-learn:

a) fit_predict nos novos pontos.
b) predict do modelo treinado.
c) inertia_.
d) score_samples.

Questão 30. Quando os grupos têm densidades e tamanhos muito diferentes, o K-means:

a) cria hierarquias.
b) detecta ruído.
c) pode dividi-los de modo inadequado.
d) funciona melhor.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique o K-means usando a analogia da pizzaria e identifique o que corresponde a centroide, grupo e convergência.
2. Descreva, passo a passo, o algoritmo de Lloyd.
3. Execute o K-means à mão para os pontos 1, 2, 3, 10, 11, 12 com k = 2 e centros iniciais 1 e 2. Mostre as iterações.
4. Defina inércia e explique por que ela sempre diminui (ou não aumenta) a cada iteração.
5. Por que o K-means pode parar em um mínimo local? Como reduzir esse problema?
6. Explique como funciona o k-means++ e por que ele é melhor que a inicialização aleatória.
7. Descreva o método do cotovelo e dois cuidados ao usá-lo.
8. Liste cinco limitações do K-means e uma forma de contorná-las.
9. Explique as diferenças entre K-means, K-medoids e K-modes.
10. O que é o agrupamento difuso (fuzzy) e em que situação é vantajoso?
11. Por que o Mini-Batch K-means é mais rápido e qual o custo dessa rapidez?
12. No estudo de caso RFM, explique como os grupos recebem nomes e como orientar ações de marketing.
13. Explique como o K-means realiza a quantização de cores de uma imagem e qual é o ganho.
14. Por que é recomendável escalar os atributos antes do K-means? Dê um exemplo.
15. Explique a célula de Voronoi e sua relação com as fronteiras do K-means.
16. O que significa dizer que o K-means "impõe bolas"? Dê um exemplo em que isso falha.
17. Por que a média é o centro ideal quando se usa a distância Euclidiana ao quadrado?
18. Proponha um roteiro de boas práticas para aplicar K-means a dados reais de clientes.
19. Compare o K-means com o hierárquico quanto à escalabilidade e à necessidade de definir k.
20. Dê um exemplo, fora dos citados no capítulo, de aplicação do K-means e justifique por que ele seria adequado.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — K-means "do zero" com NumPy

NÍVEL: INTERMEDIÁRIO

Implemente o algoritmo de Lloyd com NumPy (sem usar `KMeans`) para dados de `make_blobs` (3 grupos). Use 20 iterações e inicialização aleatória escolhendo 3 pontos dos dados. Calcule a inércia final e compare com a do `KMeans` do scikit-learn (`n_init=1`, mesmo tipo de inicialização).

Dica: a atribuição usa `np.argmin` sobre as distâncias; a atualização usa a média de cada grupo.

Exercício 2 — Escolhendo k com inércia e silhueta

NÍVEL: BÁSICO

Gere 500 pontos com 5 grupos (`make_blobs`, `random_state=3`, `cluster_std=0.9`). Para k de 2 a 9, calcule a inércia e a silhueta. Monte um `DataFrame` e indique o k de maior silhueta.

Exercício 3 — Aleatória ou k-means++?

NÍVEL: INTERMEDIÁRIO

Compare as inicializações `"random"` e `"k-means++"` rodando `KMeans` com `n_init=1` em 100 sementes diferentes sobre dados com 8 grupos (`make_blobs(1000, centers=8, random_state=1)`). Para cada método, calcule a média e o desvio padrão da inércia e a proporção de execuções que atingiram a melhor inércia observada (com tolerância de 1%).

Exercício 4 — Mini-Batch K-means em dados grandes

NÍVEL: INTERMEDIÁRIO

Gere 200 mil pontos (`make_blobs`, 6 grupos, 10 atributos). Compare `KMeans` e `MiniBatchKMeans` em **tempo** (use `time.perf_counter`) e **inércia**. Calcule a razão entre as inércias.

Exercício 5 — Quantização de cores de uma foto

NÍVEL: AVANÇADO

Carregue uma imagem de exemplo com `load_sample_images`. Transforme os pixels em pontos RGB e use K-means com 8, 16 e 32 cores (ajuste o modelo em uma **amostra de 10 mil pixels**, depois aplique a todos). Para cada k, calcule o erro quadrático médio da imagem reconstruída e a razão de compressão aproximada (24 bits por pixel antes; log2(k) bits por pixel depois, ignorando a paleta).

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Online Retail (transações de uma loja virtual, excelente para RFM) e Wholesale Customers.
> - **Kaggle:** Mall Customers e Customer Personality Analysis.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Segmentação RFM de uma loja

Fonte dos dados. Simulados pela função abaixo.

Contexto. Uma loja virtual (fictícia) quer criar campanhas diferentes para tipos diferentes de clientes. Você receberá o histórico de compras e deverá produzir uma **segmentação RFM** (recência, frequência, valor) com K-means e **propor ações de marketing**.

Dados. Gere o histórico com a função abaixo.

```python
import numpy as np
import pandas as pd


def gerar_compras(seed=0):
    rng = np.random.default_rng(seed)
    perfis = [
        ("fiel", 120, 22, 90.0),
        ("ocasional", 260, 5, 70.0),
        ("inativo", 150, 1.5, 55.0),
        ("vip", 60, 40, 220.0),
    ]
    linhas, cid = [], 0
    for nome, n, freq, ticket in perfis:
        for _ in range(n):
            cid += 1
            k = max(1, rng.poisson(freq))
            dias = np.sort(rng.integers(0, 365 if nome != "inativo" else 150, k))
            for d in dias:
                linhas.append((cid, int(d), float(rng.gamma(4, ticket / 4)), nome))
    return pd.DataFrame(linhas, columns=["cliente", "dia", "valor", "perfil_oculto"])
```

O campo `dia` indica o dia da compra (0 a 364; hoje é o dia 365). `perfil_oculto` só deve ser usado no final.

Tarefas.

1. Calcule, por cliente, a **recência** (dias desde a última compra), a **frequência** (nº de compras) e o **valor** (soma gasta).
2. Aplique `log1p` em frequência e valor (distribuições assimétricas) e padronize.
3. Escolha k entre 2 e 8 usando cotovelo e silhueta.
4. Treine o K-means final, calcule o **perfil médio** de cada segmento (em unidades originais) e dê um **nome** a cada um.
5. Verifique a **estabilidade**: repita a segmentação em 20 reamostragens (80% dos clientes) e calcule o ARI médio entre as soluções.
6. Proponha uma **ação de marketing** para cada segmento.
7. Só no final, compare com `perfil_oculto` (ARI).

Entregáveis. Script comentado, tabela de perfis, gráfico do cotovelo/silhueta e texto com ações propostas.

A solução de referência está no Apêndice E.



Capítulo 4 — Agrupamento Hierárquico: a Árvore de Semelhanças

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar a diferença entre agrupamento aglomerativo e divisivo.
- Ler e interpretar um dendrograma.
- Comparar os critérios de ligação: simples, completa, média e Ward.
- Escolher o número de grupos cortando o dendrograma.
- Aplicar agrupamento hierárquico com SciPy e scikit-learn, incluindo mapas de calor.
- Reconhecer custos computacionais e limitações do método.

4.1 A árvore genealógica dos dados

Pense numa **árvore genealógica**. Você e seu irmão se juntam num ramo (seus pais). Esse ramo se junta ao dos primos (seus avós). E assim por diante, até toda a família.

O **agrupamento hierárquico** faz algo parecido com os dados. Ele não entrega apenas *uma* divisão em grupos. Entrega uma **hierarquia completa**: desde cada ponto sozinho até todos juntos num único grupo.

Vantagens:

- **Não exige escolher *k* antes.** Você decide depois, olhando a árvore.
- **Mostra relações em vários níveis.** Pequenos grupos dentro de grupos maiores.
- **É determinístico.** Os mesmos dados dão sempre a mesma árvore.

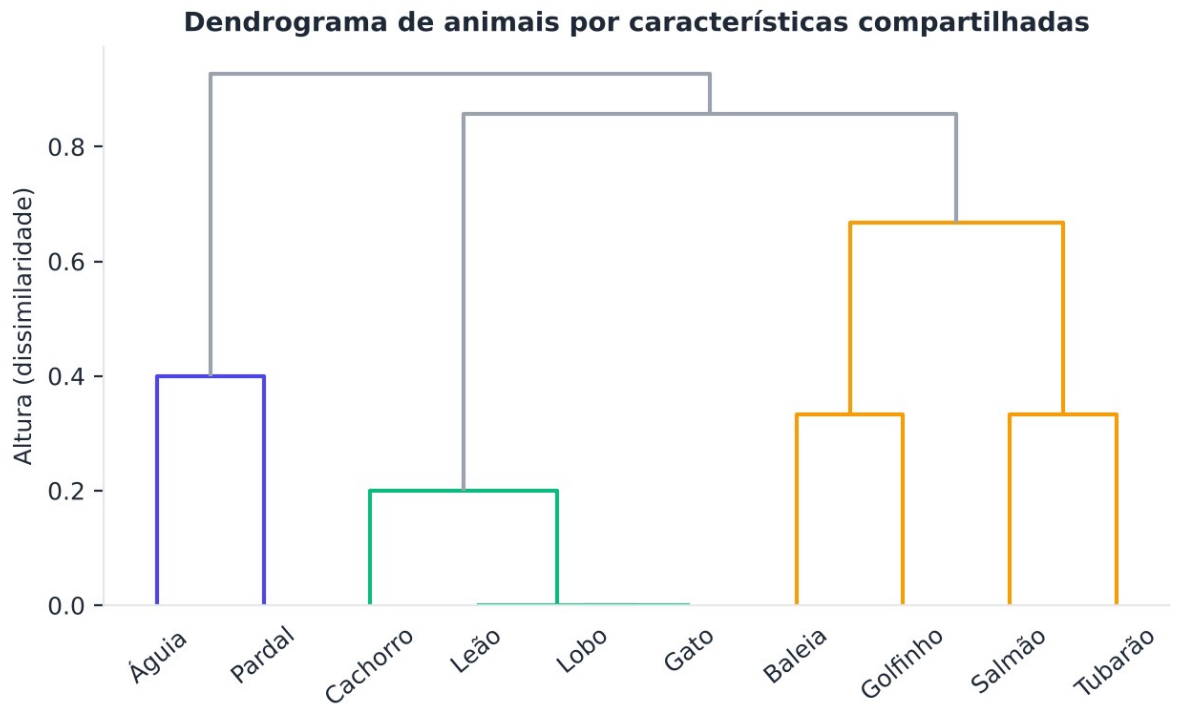


Figura 4.1 — Um dendrograma de animais: a mesma lógica da classificação biológica, de espécies a classes.

4.2 Duas estratégias

4.2.1 Aglomerativa (de baixo para cima)

É a mais usada. Passos:

1. Comece com **cada ponto em seu próprio grupo** (*n* grupos).
2. Encontre os **dois grupos mais próximos** e junte-os.
3. Repita o passo 2 até sobrar um único grupo.

4.2.2 Divisiva (de cima para baixo)

Faz o caminho inverso: começa com tudo num grupo único e o **divide** recursivamente. É mais cara e menos comum. Um exemplo clássico é o algoritmo DIANA (Kaufman; Rousseeuw, 1990).

> **Memorize**
>
> **Aglomerativo** = juntar (bottom-up). **Divisivo** = separar (top-down).

4.3 O dendrograma

O **dendrograma** é o gráfico em forma de árvore que registra a sequência de fusões.

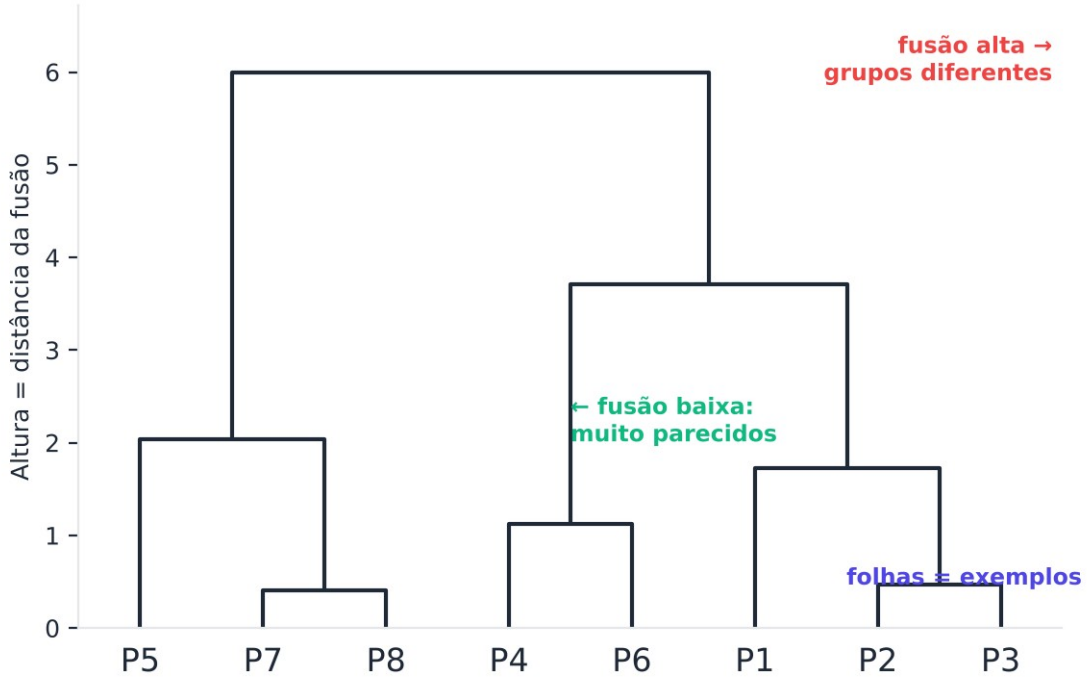


Figura 4.2 — Anatomia de um dendrograma: folhas, ramos, nós de fusão e altura.

Como ler:

- Cada **folha** (embaixo) é um exemplo.
- Cada **junção** é uma fusão entre dois grupos.
- A **altura** da junção indica a distância entre os grupos fundidos. Fusões baixas: grupos muito parecidos. Fusões altas: grupos diferentes.
- A **ordem horizontal** das folhas não tem significado por si só. Só importa *quem se junta com quem* e *a que altura*.

> ATENÇÃO **Erro comum**
>
> Duas folhas vizinhas no desenho **não** são necessariamente parecidas. Folhas só são próximas de fato se a altura da junção entre elas for baixa.

4.3.1 Cortando a árvore

Para obter grupos, trace uma **linha horizontal** (um "corte") no dendrograma. Cada ramo cortado é um grupo.

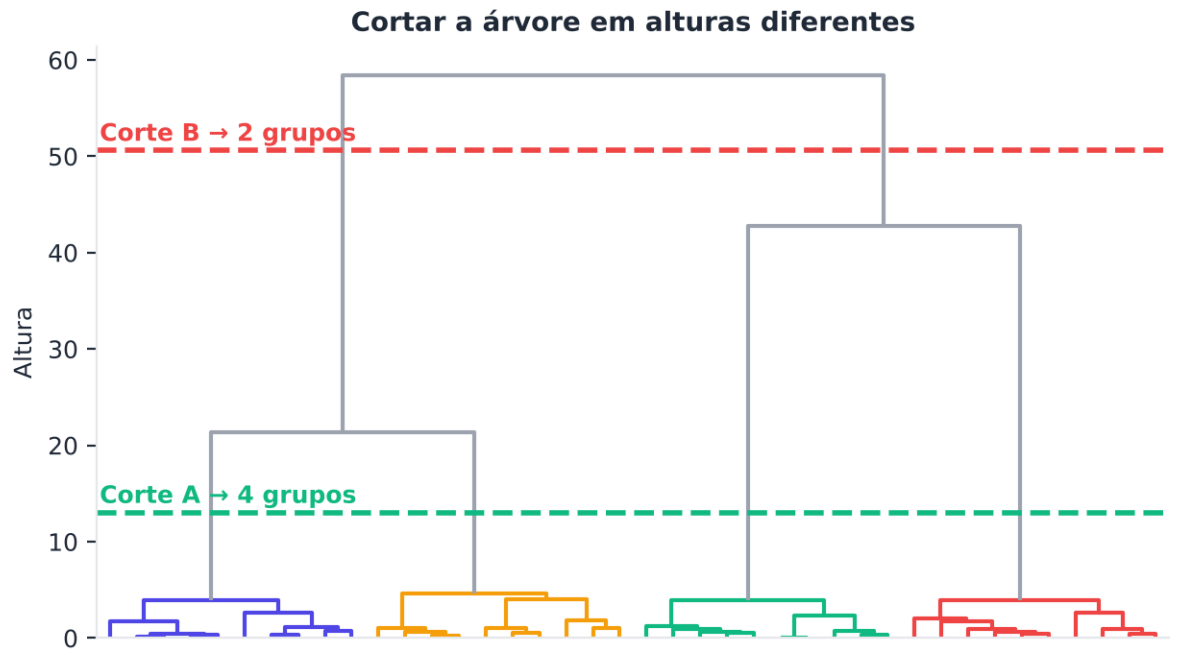


Figura 4.3 — Cortar em alturas diferentes produz números de grupos diferentes. Uma boa regra: corte onde há um "salto" grande entre fusões.

Uma regra prática: procure o **maior salto vertical** sem fusões. Ele indica que o próximo agrupamento juntaria coisas bem diferentes.

4.4 Como medir a distância entre grupos? As ligações

Medir a distância entre dois **pontos** é fácil (Capítulo 2). Mas entre dois **grupos**? Existem várias respostas. Cada uma é uma **ligação** (*linkage*).

4.4.1 Ligação simples (vizinho mais próximo)

Distância entre os dois pontos **mais próximos**, um de cada grupo.

Analogia: duas cidades estão "ligadas" se uma estrada curta conecta quaisquer duas casas de uma e outra.

- **Ponto forte:** captura formas alongadas e irregulares.
- **Ponto fraco:** o **efeito corrente** (*chaining*): uma fileira de pontos intermediários junta grupos distintos.

4.4.2 Ligação completa (vizinho mais distante)

Distância entre os dois pontos **mais distantes**.

- **Ponto forte:** gera grupos compactos, de diâmetro pequeno.
- **Ponto fraco:** sensível a outliers; pode quebrar grupos grandes.

4.4.3 Ligação média (UPGMA)

Média das distâncias entre **todos os pares** de pontos, um de cada grupo.

- Um meio-termo entre simples e completa.

4.4.4 Ligação de Ward

Junta os grupos cuja fusão causa o **menor aumento da variância total dentro dos grupos** (Ward, 1963). É a "irmã" hierárquica do K-means, pois também minimiza somas de quadrados.

- **Ponto forte:** grupos compactos e de tamanho parecido. Muito usada na prática.
- **Ponto fraco:** exige distância Euclidiana.

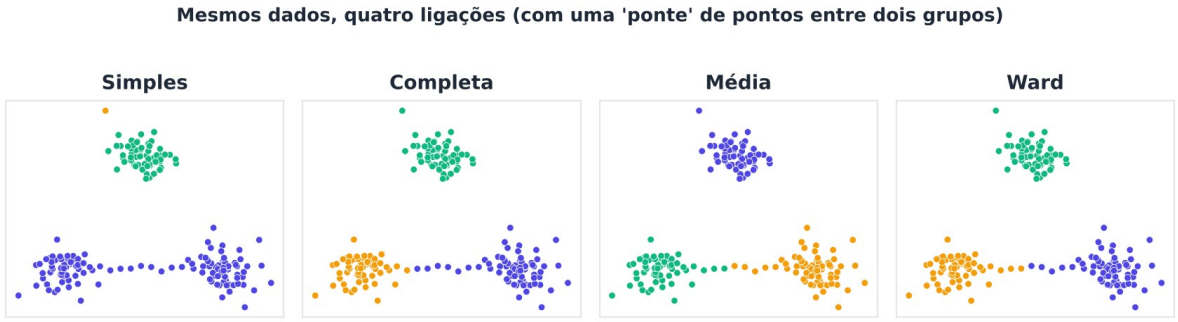


Figura 4.4 — As quatro ligações aplicadas aos mesmos dados. Observe como a ligação simples "puxa" correntes e Ward produz grupos equilibrados.

Ligação	Mede	Tende a gerar	Risco
Simples	Par mais próximo	Grupos alongados	Efeito corrente
Completa	Par mais distante	Grupos compactos	Sensível a outliers
Média	Média dos pares	Grupos intermediários	Meio-termo
Ward	Aumento da variância	Grupos esféricos e equilibrados	Só Euclidiana

4.5 Um exemplo à mão

Cinco pontos numa reta: **A = 1, B = 2, C = 6, D = 7, E = 15**. Usaremos a ligação simples.

1. Distâncias: AB = 1, CD = 1, BC = 4, DE = 8 etc.
2. A menor distância é 1 (empate entre AB e CD). Juntamos {A, B} em altura 1 e {C, D} em altura 1.
3. Distância entre {A, B} e {C, D} (ligação simples) = menor par = B a C = 4. Distância de {C, D} a E = D a E = 8.
4. Menor: 4. Juntamos {A, B, C, D} em altura 4.
5. Por fim, juntamos E em altura 8.

Alturas das fusões: 1, 1, 4, 8. O grande salto de 4 para 8 sugere que E é um grupo à parte. Um corte entre 4 e 8 dá **dois grupos**: {A, B, C, D} e {E}.

4.6 O algoritmo e seu custo

O algoritmo ingênuo:

1. Calcular a matriz de distâncias $n \times n$.
2. Em cada passo, achar o par mais próximo, fundir e atualizar a matriz.

O custo de memória é da ordem de n^2 e o de tempo, de n^3 na versão ingênua (podendo chegar a $n^2 \log n$ com estruturas eficientes). Por isso o método é ótimo para **centenas ou poucos milhares** de exemplos, mas inviável para milhões.

A atualização das distâncias após cada fusão segue a **fórmula de Lance-Williams**, que expressa todas as ligações acima numa só expressão com parâmetros diferentes (Lance; Williams, 1967).

> DICA **Como lidar com bases grandes?**
>
> Reduza antes (amostragem ou K-means com muitos grupos pequenos) e depois aplique o hierárquico sobre os centroides. Ou use o BIRCH (Zhang; Ramakrishnan; Livny, 1996), que constrói um resumo compacto dos dados em uma única passada.

4.7 Hierárquico no Python

4.7.1 Com SciPy: dendrograma completo

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data[::5]                    # 30 flores, para o desenho ficar legível

Z = linkage(X, method="ward")         # matriz de ligações
dendrogram(Z)
plt.title("Dendrograma (Ward) - Iris")
plt.xlabel("Flores")
plt.ylabel("Altura (distância de fusão)")
plt.show()

# Cortar a árvore em 3 grupos
grupos = fcluster(Z, t=3, criterion="maxclust")
print(grupos)
```

A matriz `Z` tem uma linha por fusão: os dois grupos juntados, a altura e o tamanho do novo grupo.

4.7.2 Com scikit-learn

```python
from sklearn.cluster import AgglomerativeClustering
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=200, centers=3, random_state=5)
modelo = AgglomerativeClustering(n_clusters=3, linkage="ward")
rotulos = modelo.fit_predict(X)
print(rotulos[:20])
```

Para cortar por **altura** em vez de por número de grupos, use `n_clusters=None, distance_threshold=valor`.

4.7.3 Correlação cofenética

Como saber se o dendrograma representa bem as distâncias originais? A **correlação cofenética** compara as distâncias originais com as "distâncias no dendrograma" (altura em que dois pontos se unem). Quanto mais perto de 1, melhor (Sokal; Rohlf, 1962).

```python
from scipy.cluster.hierarchy import linkage, cophenet
from scipy.spatial.distance import pdist
from sklearn.datasets import load_iris

X = load_iris().data
d = pdist(X)
for metodo in ["single", "complete", "average", "ward"]:
    c, _ = cophenet(linkage(X, method=metodo), d)
    print(f"{metodo:9s} correlação cofenética = {c:.3f}")
```

4.8 Mapas de calor com dendrogramas (clustermaps)

Em genômica, marketing e muitas outras áreas, é comum agrupar **linhas e colunas ao mesmo tempo** e exibir os valores como um mapa de calor. Isso revela blocos de comportamento: "estes clientes compram estes produtos".

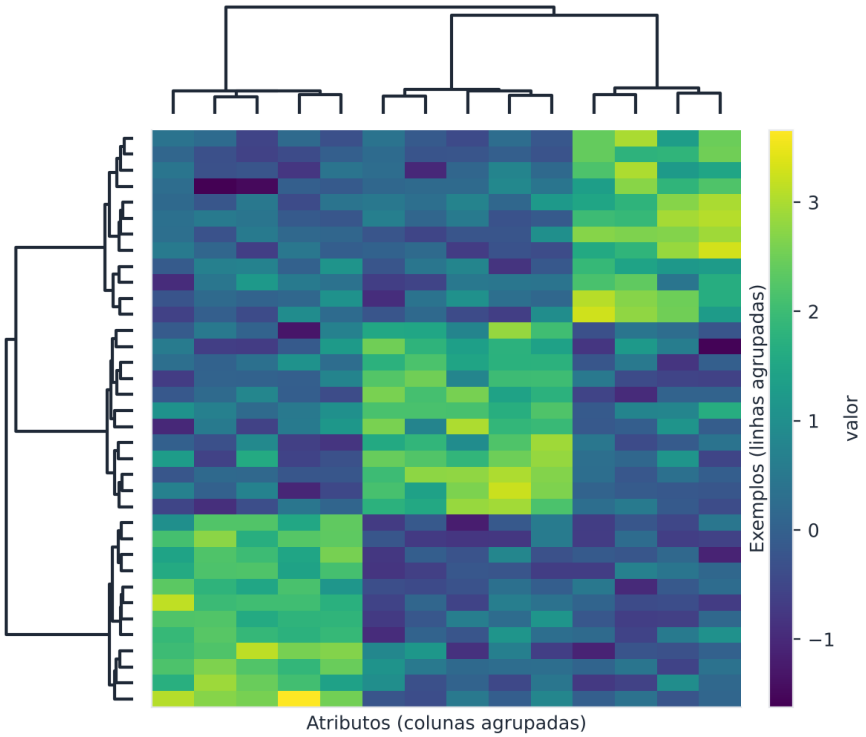


Figura 4.5 — Mapa de calor com dendrogramas nos dois eixos. Os blocos coloridos indicam subgrupos de linhas e colunas com padrão parecido.

Eisen et al. (1998) popularizaram essa visualização ao agrupar genes de levedura segundo sua expressão ao longo do tempo, num trabalho muito citado da biologia computacional.

4.9 Vantagens e limitações

Vantagens

- Entrega hierarquia completa.
- Não precisa fixar *k* antes.
- Funciona com qualquer distância (exceto Ward).

- Dendrograma é ótimo para comunicar.

Limitações

- Custo quadrático ou cúbico.
- Decisões são **irreversíveis**: uma fusão errada nunca é desfeita (algoritmo guloso).
- Sensível a ruído e outliers (dependendo da ligação).
- Escolha da ligação muda muito o resultado.

> EXEMPLO **Estudo de caso rápido: espécies do mesmo gênero**
>
> Biólogos usam hierárquico para construir **árvores fenéticas** a partir de medidas de forma. Ao usar a ligação média (UPGMA), obtêm uma árvore que resume a similaridade global entre espécies (Sokal; Michener, 1958).

4.10 Hierárquico vs. K-means

Aspecto	K-means	Hierárquico
Precisa de *k* antes?	Sim	Não
Resultado	Uma partição	Uma árvore de partições
Escalabilidade	Excelente	Limitada
Forma dos grupos	Esférica	Depende da ligação
Determinístico?	Não (depende da inicialização)	Sim
Melhor uso	Grandes bases	Exploração e bases médias

Resumo do capítulo

> MEMORIZE **Cartões de memorização**
>
- **Aglomerativo:** começa com cada ponto separado e junta os mais próximos.
- **Dendrograma:** árvore das fusões. A **altura** é a distância da fusão.
- **Corte horizontal** = escolha de *k*.
- **Simples** (mais próximo; corrente), **completa** (mais distante; compactos), **média** (equilíbrio), **Ward** (minimiza variância).
- **Custo alto** para muitos dados.
- **Fusões são irreversíveis.**
- **Correlação cofenética** avalia a fidelidade da árvore.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O agrupamento hierárquico produz como resultado:

a) uma hierarquia completa de partições, de cada ponto sozinho até todos juntos.
b) apenas os centroides.
c) somente pontos de ruído.
d) uma única partição fixa.

Questão 2. Na abordagem aglomerativa, o algoritmo começa:

a) com os outliers.
b) com todos os pontos em um único grupo.
c) com cada ponto em seu próprio grupo.
d) com k grupos aleatórios.

Questão 3. A abordagem divisiva funciona:

a) apenas com dois grupos.
b) de baixo para cima.
c) de cima para baixo, dividindo recursivamente o grupo inicial.
d) apenas com dados categóricos.

Questão 4. O gráfico em forma de árvore que registra as fusões chama-se:

a) biplot.
b) dendrograma.
c) histograma.
d) diagrama de Voronoi.

Questão 5. No dendrograma, a altura de uma junção indica:

a) a ordem de leitura dos dados.
b) a distância entre os grupos fundidos.
c) a densidade do grupo.
d) o número de pontos do grupo.

Questão 6. É correto afirmar que duas folhas vizinhas no desenho do dendrograma:

a) pertencem sempre ao mesmo grupo.
b) não são necessariamente parecidas; o que importa é a altura da junção entre elas.
c) têm distância zero.
d) são sempre muito parecidas.

Questão 7. Para obter k grupos a partir de um dendrograma, deve-se:

a) refazer o cálculo com k centroides.
b) remover as folhas.
c) trocar a distância.
d) traçar um corte horizontal na altura adequada.

Questão 8. Uma regra prática para escolher o corte é:

a) cortar sempre em 2 grupos.
b) cortar sempre na menor altura.
c) cortar na altura média.
d) cortar no maior salto vertical sem fusões.

Questão 9. A ligação simples define a distância entre grupos como a distância:

a) entre os pontos mais distantes.
b) média entre todos os pares.
c) entre os pontos mais próximos, um de cada grupo.
d) entre os centroides.

Questão 10. O "efeito corrente" (chaining) está associado à ligação:

a) de Ward.
b) completa.
c) média.
d) simples.

Questão 11. A ligação completa mede a distância entre grupos usando:

a) o par de pontos mais distantes.
b) a média dos pares.
c) o par de pontos mais próximos.
d) a variância.

Questão 12. A ligação média (UPGMA) calcula:

a) a média das distâncias entre todos os pares de pontos, um de cada grupo.
b) a distância entre medianas.
c) a menor distância entre grupos.
d) a variância total.

Questão 13. A ligação de Ward junta os grupos cuja fusão:

a) causa o menor aumento da variância dentro dos grupos.
b) tem o maior diâmetro.
c) ocorre aleatoriamente.
d) tem a menor distância entre centroides apenas.

Questão 14. A ligação de Ward exige:

a) distância Euclidiana.
b) distância de Hamming.
c) dados binários.
d) nenhum tipo de distância.

Questão 15. Para os pontos A=1, B=2, C=6, D=7, E=15, com ligação simples, a primeira fusão ocorre na altura:

a) 14.
b) 1.
c) 4.
d) 8.

Questão 16. No mesmo exemplo (A=1, B=2, C=6, D=7, E=15), a altura da fusão entre {A, B} e {C, D} na ligação simples é:

a) 1.
b) 8.
c) 5.
d) 4.

Questão 17. Ainda no mesmo exemplo, a última fusão (que inclui E) ocorre, na ligação simples, na altura:

a) 14.
b) 8.
c) 4.
d) 9.

Questão 18. O custo de memória do hierárquico ingênuo é da ordem de:

a) n.
b) $n \log n$.
c) n^2.
d) 2^n.

Questão 19. O agrupamento hierárquico é mais indicado para bases com:

a) milhões de exemplos.
b) apenas um exemplo.
c) bilhões de linhas.
d) centenas ou poucos milhares de exemplos.

Questão 20. A fórmula de Lance-Williams serve para:

a) escolher k.
b) calcular o silhouette.
c) calcular a inércia.
d) atualizar as distâncias após cada fusão, unificando as ligações.

Questão 21. O algoritmo BIRCH é indicado para:

a) detectar anomalias.
b) bases grandes, construindo um resumo compacto em uma passada.
c) reduzir dimensões.
d) dados categóricos.

Questão 22. No SciPy, a função que calcula a matriz de ligações é:

a) pdist.
b) fcluster.
c) cophenet.
d) linkage.

Questão 23. A função fcluster do SciPy é usada para:

a) escalar dados.
b) calcular distâncias.
c) desenhar o dendrograma.
d) cortar a árvore e obter os rótulos dos grupos.

Questão 24. A correlação cofenética mede:

a) a pureza dos grupos.
b) a inércia.
c) o quanto o dendrograma preserva as distâncias originais.
d) a quantidade de ruído.

Questão 25. Para cortar por altura no AgglomerativeClustering do scikit-learn, usa-se:

a) n_init.
b) n_clusters=None e distance_threshold.
c) eps.
d) perplexity.

Questão 26. Um mapa de calor com dendrogramas nos dois eixos serve para:

a) calcular a inércia.
b) escalonar dados.
c) remover outliers.
d) reordenar linhas e colunas e revelar blocos de padrão semelhante.

Questão 27. Uma limitação do hierárquico é que as fusões são:

a) reversíveis.
b) aleatórias.
c) sempre binárias em número par.
d) irreversíveis (algoritmo guloso).

Questão 28. Em comparação com o K-means, o hierárquico:

a) escala melhor para milhões de pontos.
b) é não determinístico.
c) é determinístico e não exige k de antemão.
d) só aceita grupos esféricos.

Questão 29. A ligação UPGMA é muito usada, em biologia, para construir:

a) regras de associação.
b) árvores fenéticas de similaridade entre espécies.
c) variedades.
d) redes neurais.

Questão 30. Qual conjunto de dados Eisen et al. (1998) agruparam, popularizando o mapa de calor com dendrogramas?

a) Imagens de dígitos.
b) Genes de levedura, segundo a expressão ao longo do tempo.
c) Séries financeiras.
d) Cestas de supermercado.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique a analogia da árvore genealógica para o agrupamento hierárquico.
2. Diferencie o agrupamento aglomerativo do divisivo, indicando qual é mais usado e por quê.
3. Explique como ler um dendrograma: o que significam folhas, ramos, altura e ordem horizontal.
4. Como escolher o número de grupos cortando um dendrograma? Quais cuidados?
5. Descreva as quatro ligações (simples, completa, média e Ward) e uma vantagem ou desvantagem de cada.
6. O que é o efeito corrente e por que ocorre?
7. Construa à mão o dendrograma, com ligação simples, para A=1, B=2, C=6, D=7, E=15 e indique o corte para 2 grupos.
8. Qual a complexidade do hierárquico e como lidar com bases grandes?
9. Descreva o que a correlação cofenética mede e como usá-la para escolher a ligação.
10. Explique como um mapa de calor com dendrogramas (clustermap) ajuda a analisar dados de expressão gênica ou de clientes.
11. Por que o hierárquico é considerado determinístico? Compare com o K-means.
12. Quais as vantagens e limitações do agrupamento hierárquico?
13. Explique a relação entre a ligação de Ward e o K-means.
14. Compare hierárquico e K-means em uma tabela ou texto: k, resultado, escalabilidade e determinismo.
15. Explique por que a ordem das folhas num dendrograma pode ser alterada sem mudar o agrupamento.
16. Mostre como obter 3 grupos de um conjunto de dados com SciPy ou scikit-learn.
17. Quando o hierárquico com ligação simples seria preferível a Ward? Justifique.
18. Dê um exemplo de aplicação do hierárquico em taxonomia e explique como a hierarquia é útil.
19. Explique o que são fusões irreversíveis e uma consequência prática.
20. Descreva a fórmula de Lance-Williams e sua utilidade no agrupamento hierárquico.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — O dendrograma à mão, conferido pelo computador

NÍVEL: BÁSICO

Aplique `linkage` com ligação **simples** aos pontos 1, 2, 6, 7 e 15 (uma dimensão). Mostre a matriz `Z` e confira se as alturas das fusões são 1, 1, 4 e 8. Depois corte a árvore em 2 grupos com `fcluster`.

Exercício 2 — Qual ligação reproduz melhor as classes?

NÍVEL: INTERMEDIÁRIO

No Wine padronizado, aplique `AgglomerativeClustering(n_clusters=3)` com as ligações `single`, `complete`, `average` e `ward`. Calcule o ARI contra as classes reais e a **correlação cofenética** (use `scipy` para a cofenética).

Exercício 3 — Cortando a árvore por altura

NÍVEL: INTERMEDIÁRIO

No Iris padronizado com ligação de Ward, use `AgglomerativeClustering(n_clusters=None, distance_threshold=h)` para h = 5, 8, 12 e 20. Quantos grupos surgem em cada corte? Depois use os saltos entre alturas consecutivas de `linkage` para sugerir um corte.

Exercício 4 — O efeito corrente

NÍVEL: INTERMEDIÁRIO

Crie dois grupos densos ligados por uma "ponte" de pontos (código abaixo) e agrupe com ligação `single` e `ward` (k = 2). Mostre o tamanho de cada grupo nos dois casos.

Pergunta: qual ligação separou corretamente os dois blocos? Por quê?

```python
import numpy as np

rng = np.random.default_rng(0)
A = rng.normal([0, 0], 0.4, (100, 2))
B = rng.normal([6, 0], 0.4, (100, 2))
ponte = np.c_[np.linspace(0.8, 5.2, 12), rng.normal(0, 0.05, 12)]
X = np.vstack([A, B, ponte])
```

Exercício 5 — Agrupando exemplos e atributos ao mesmo tempo

NÍVEL: AVANÇADO

No Wine padronizado, calcule a hierarquia dos **vinhos** (linhas) e a dos **atributos** (colunas, usando distância de correlação: `1 − |correlação|`). Reordene a matriz segundo as duas hierarquias e desenhe um mapa de calor. Imprima a ordem dos atributos e diga quais ficam juntos.

SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**

- **UCI:** Seeds, Glass Identification e Wine Quality.
- **Kaggle:** Spotify Tracks Dataset (agrupar gêneros ou artistas por características de áudio).

Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

A árvore dos dígitos

Fonte dos dados. *Optical Recognition of Handwritten Digits*, do repositório **UCI Machine Learning** (a parte de teste, com 1.797 imagens, vem incluída no scikit-learn).

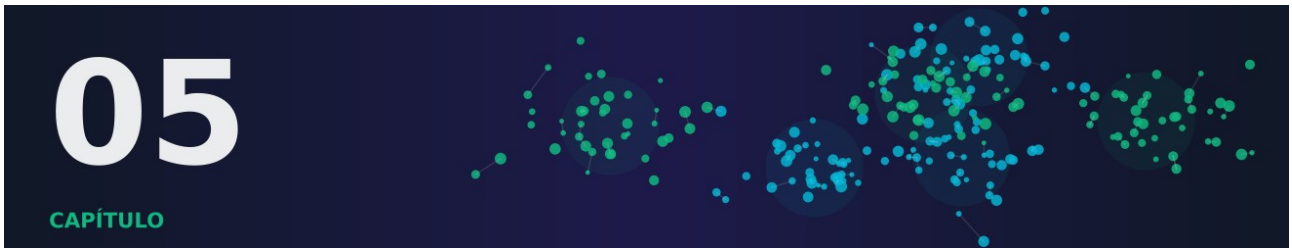
Contexto. Um laboratório de visão computacional quer entender **quais dígitos manuscritos se parecem mais entre si**. Você usará o conjunto `load_digits` (1.797 imagens de 8×8) e o agrupamento hierárquico para construir uma "árvore dos dígitos".

Tarefas.

1. Calcule a **imagem média** de cada dígito (10 vetores de 64 atributos) e aplique agrupamento hierárquico (ligação média, distância Euclidiana). Desenhe o dendrograma com os rótulos 0 a 9.
2. Corte a árvore em 3 grupos e descreva-os. Que dígitos parecem "primos"?
3. Agora agrupe as **1.797 imagens** com `AgglomerativeClustering` (ligação de Ward, k = 10). Calcule o ARI e o NMI contra os dígitos reais.
4. Monte a matriz de contingência e identifique os dois dígitos mais confundidos.
5. Compare Ward com ligação média e simples quanto ao ARI. Explique as diferenças.
6. Escreva um parágrafo relacionando suas conclusões entre os itens 2 e 4.

Entregáveis. Script, dendrograma, matriz de contingência e parágrafo.

A solução de referência está no Apêndice E.



Capítulo 5 — Agrupamento por Densidade: DBSCAN, OPTICS e HDBSCAN

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar a ideia de agrupar por regiões densas separadas por regiões vazias.
- Definir ponto central, ponto de borda e ruído.
- Aplicar o DBSCAN e ajustar seus parâmetros eps e min_samples.
- Usar o gráfico de k-distâncias para escolher eps.
- Compreender como OPTICS e HDBSCAN lidam com densidades variáveis.
- Comparar métodos baseados em densidade com K-means.

5.1 Ilhas e oceano

Olhe um mapa-múndi à noite, com as luzes das cidades. Você enxerga "manchas" brilhantes (cidades) separadas por áreas escuras. Nenhuma regra manda contar "3 manchas". Seus olhos simplesmente percebem **regiões densas** cercadas por **regiões vazias**.

Essa é a ideia do **agrupamento por densidade**:

> *Um grupo é uma região onde os pontos estão **bem próximos uns dos outros**, separada de outros grupos por regiões **pouco povoadas**.*

Consequências importantes:

- Os grupos podem ter **qualquer forma** (luas, espirais, anéis).
- Não é preciso fixar o número de grupos.
- Pontos isolados são tratados como **ruído**, e não forçados a entrar em algum grupo.

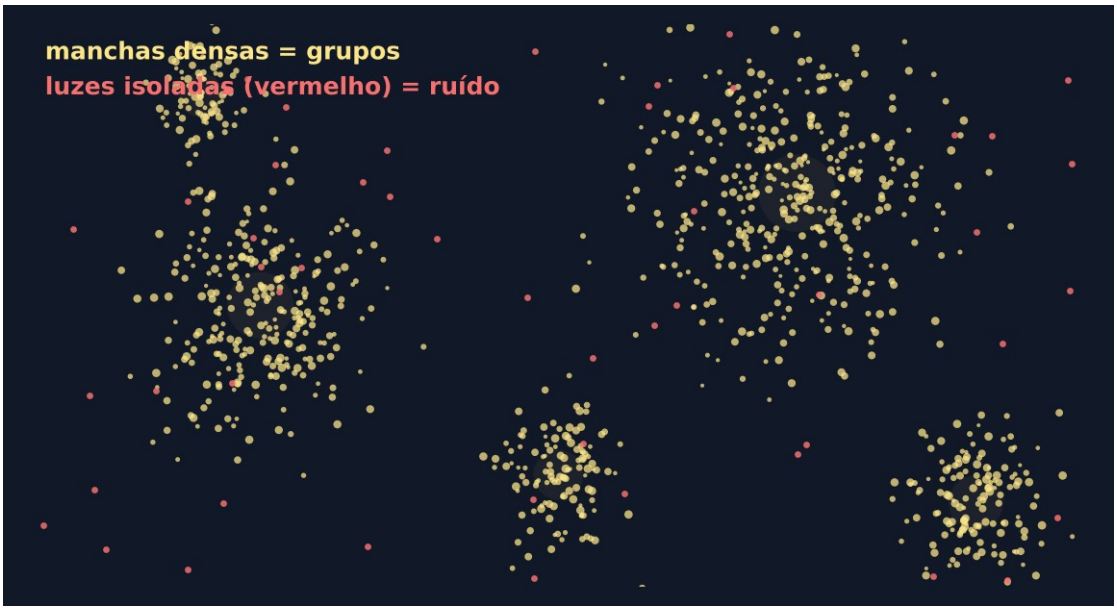


Figura 5.1 — Cidades à noite: grupos são regiões densas, separadas por vazio. O ruído são luzes isoladas.

5.2 O DBSCAN

O **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*) foi proposto por Ester, Kriegel, Sander e Xu (1996) e é um dos algoritmos de agrupamento mais citados da história. Em 2014, o artigo recebeu um prêmio de "teste do tempo" da conferência KDD.

5.2.1 Os dois parâmetros

- **eps (ε):** o raio da "vizinhança". Pense num círculo de ε de raio desenhado em volta de cada ponto.
- **min_samples (MinPts):** o número mínimo de pontos (incluindo o próprio) dentro desse círculo para a região ser considerada "densa".

5.2.2 Os três tipos de ponto

- **Ponto central (*core*):** tem pelo menos `min_samples` pontos no raio ε. Está "no coração" de uma região densa.
- **Ponto de borda (*border*):** não é central, mas está dentro do raio ε de algum ponto central. Fica na "praia" do grupo.
- **Ruído (*noise*):** não é central nem de borda. É um ponto solitário.

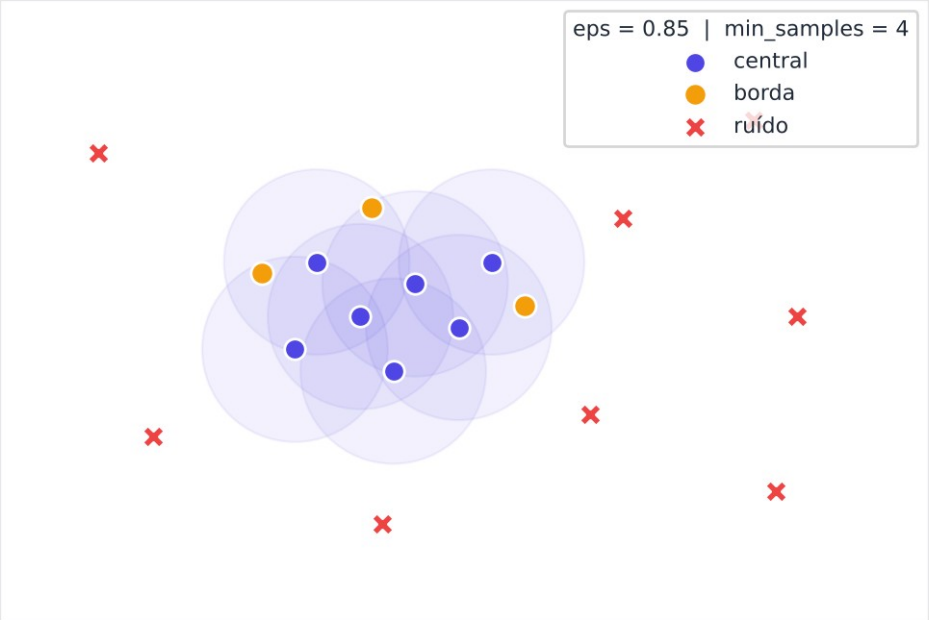


Figura 5.2 — Pontos centrais (azul), de borda (laranja) e ruído (vermelho), com min_samples = 4. Os círculos têm raio eps.

5.2.3 Alcançabilidade e conectividade

- Um ponto *q* é **diretamente alcançável por densidade** a partir de um ponto central *p* se *q* está no raio ε de *p*.
- Pontos centrais que estão no raio um do outro formam uma **cadeia**. Um grupo é uma cadeia máxima de pontos centrais, mais suas bordas.

Analogia: um boato que passa de pessoa a pessoa. Se você é "central" (tem pelo menos *m* amigos próximos), o boato passa a eles. Quem está na periferia ouve o boato, mas não o repassa.

5.2.4 O algoritmo em palavras

1. Escolha um ponto ainda não visitado.
2. Se ele é central, crie um novo grupo e **expanda**: adicione todos os vizinhos; para cada vizinho que também é central, adicione os vizinhos dele, e assim por diante.
3. Se não é central, marque-o temporariamente como ruído (ele ainda pode virar borda de algum grupo depois).
4. Repita até visitar todos.

5.3 DBSCAN no Python

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN, KMeans
from sklearn.preprocessing import StandardScaler

X, _ = make_moons(n_samples=400, noise=0.07, random_state=0)
X = StandardScaler().fit_transform(X)
```

```
db = DBSCAN(eps=0.3, min_samples=5).fit(X)
rotulos = db.labels_                    # -1 significa ruído

n_grupos = len(set(rotulos)) - (1 if -1 in rotulos else 0)
print("Grupos encontrados:", n_grupos)
print("Pontos de ruído   :", (rotulos == -1).sum())

# Comparação com K-means
km = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(X)

fig, ax = plt.subplots(1, 2, figsize=(9, 4))
ax[0].scatter(*X.T, c=km, s=15);      ax[0].set_title("K-means")
ax[1].scatter(*X.T, c=rotulos, s=15); ax[1].set_title("DBSCAN")
plt.show()
```

Como interpretar:

- `labels_` guarda o grupo de cada ponto. O valor **−1** marca ruído.
- `core_sample_indices_` lista os pontos centrais.
- O K-means corta as luas ao meio. O DBSCAN as separa corretamente.

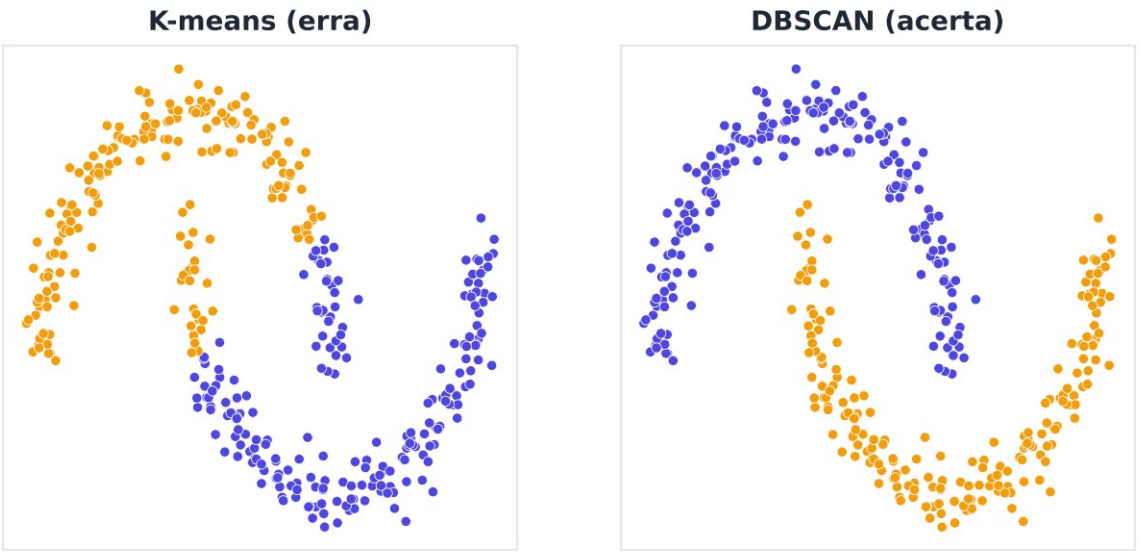


Figura 5.3 — Em formas não convexas, o DBSCAN acerta onde o K-means falha.

5.4 Como escolher eps e min_samples?

5.4.1 Regra para min_samples

Uma regra comum: `min_samples` ≥ $d + 1$, onde d é o número de atributos. Para dados com muito ruído, aumente o valor. Em duas dimensões, valores entre 4 e 10 costumam funcionar (Schubert et al., 2017).

5.4.2 O gráfico de k-distâncias

Para escolher eps:

1. Para cada ponto, calcule a distância até seu k-ésimo vizinho mais próximo (com k = `min_samples`).
2. Ordene essas distâncias em ordem crescente e faça o gráfico.
3. Procure o **joelho** da curva. O valor de y nesse ponto é um bom candidato a eps.

Intuição: pontos dentro de grupos têm vizinhos próximos (parte baixa da curva). Pontos de ruído têm vizinhos distantes (subida acentuada).

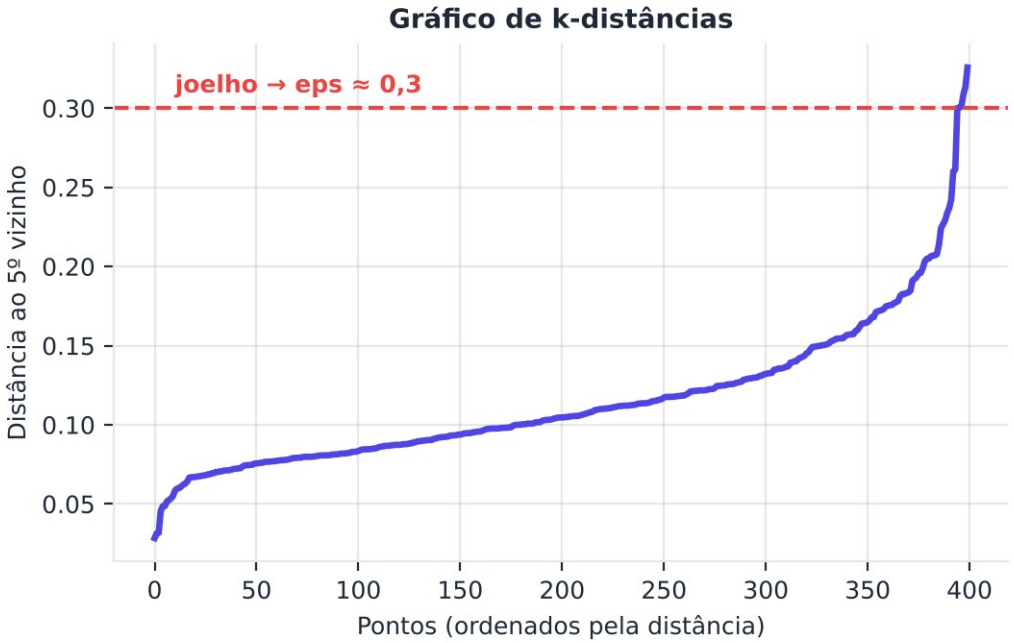


Figura 5.4 — Gráfico de k-distâncias: o joelho sugere o valor de eps.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import NearestNeighbors
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(make_moons(400, noise=0.07, random_state=0)[0])
k = 5
vizinhos = NearestNeighbors(n_neighbors=k).fit(X)
dist, _ = vizinhos.kneighbors(X)
dk = np.sort(dist[:, -1])              # distância ao k-ésimo vizinho

plt.plot(dk)
plt.xlabel("Pontos ordenados")
plt.ylabel(f"Distância ao {k}º vizinho")
plt.show()
```

5.4.3 Efeitos de eps

- **eps pequeno demais:** quase tudo vira ruído; muitos micro-grupos.
- **eps grande demais:** grupos distintos se fundem num só.

> **ATENÇÃO** **Escale os dados!**
>
> Como o DBSCAN usa um raio único, todos os atributos precisam estar na mesma escala. Padronize antes (Capítulo 2).

5.5 Pontos fortes e fracos do DBSCAN

Pontos fortes

- Descobre grupos de **formas arbitrárias**.
- **Detecta ruído** naturalmente.
- Não exige *k*.
- Eficiente: O(*n* log *n*) com índices espaciais.

Pontos fracos

- **Densidade única.** Com um só eps, não lida bem com grupos de densidades muito diferentes.
- **Sensível aos parâmetros.**
- **Alta dimensão:** o conceito de "vizinhança" perde sentido (Capítulo 2).
- Pontos de borda podem ser atribuídos a grupos diferentes dependendo da ordem de visita.

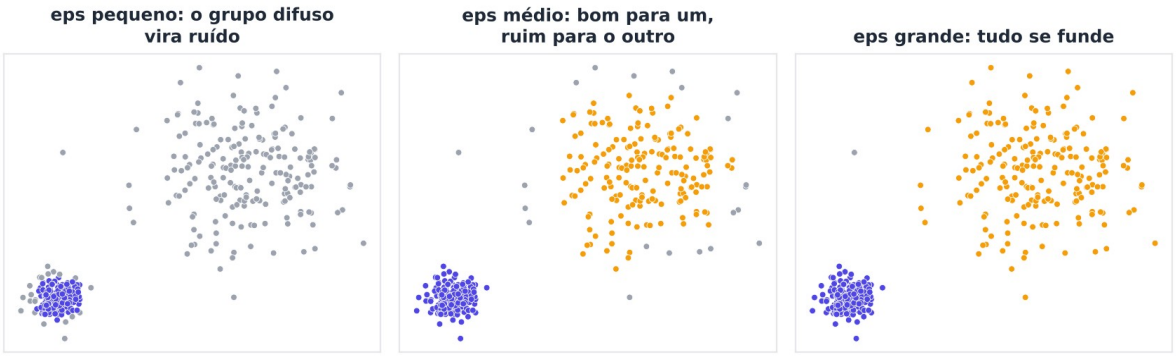


Figura 5.5 — O problema das densidades variáveis: um eps que funciona para o grupo denso transforma o grupo difuso em ruído.

5.6 OPTICS: a ordenação por alcançabilidade

O **OPTICS** (Ankerst et al., 1999) resolve a questão das densidades variáveis. Em vez de fornecer *uma* partição, ele produz uma **ordem dos pontos** e, para cada um, uma **distância de alcançabilidade**.

Ao desenhar essas distâncias na ordem calculada, obtém-se o **gráfico de alcançabilidade**.

- **Vales** = regiões densas = grupos.
- **Picos** = fronteiras entre grupos.
- Vales profundos são grupos bem densos; vales rasos, grupos difusos.

Analogia: uma paisagem de relevo. Os vales são cidades; as serras entre eles, áreas vazias. Você escolhe onde "alagar" para definir os grupos.

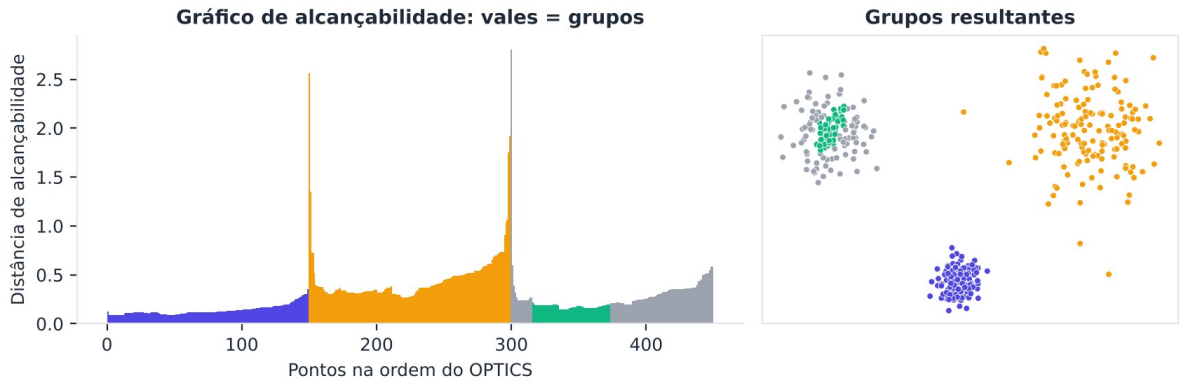


Figura 5.6 — Gráfico de alcançabilidade do OPTICS: cada vale corresponde a um grupo, com densidades diferentes.

```python
from sklearn.cluster import OPTICS
from sklearn.datasets import make_blobs
import numpy as np

# Três grupos com densidades bem diferentes
X1, _ = make_blobs(200, centers=[[0, 0]],  cluster_std=0.3, random_state=0)
X2, _ = make_blobs(200, centers=[[4, 4]],  cluster_std=1.0, random_state=1)
X3, _ = make_blobs(200, centers=[[-4, 4]], cluster_std=0.6, random_state=2)
X = np.vstack([X1, X2, X3])

op = OPTICS(min_samples=10, xi=0.05, min_cluster_size=0.05).fit(X)
print("Grupos:", len(set(op.labels_)) - (1 if -1 in op.labels_ else 0))
print("Ruído :", (op.labels_ == -1).sum())
# op.reachability_[op.ordering_] contém a curva do gráfico de alcançabilidade
```

> **ATENÇÃO — O OPTICS exige ajuste**
>
> Para extrair grupos do gráfico de alcançabilidade, o método `xi` e o tamanho mínimo de grupo precisam de ajuste. Com parâmetros inadequados, o OPTICS pode marcar muito ruído. Nos exercícios do capítulo, o HDBSCAN teve resultado muito melhor com poucos parâmetros.

5.7 HDBSCAN: densidade hierárquica

O **HDBSCAN** (Campello; Moulavi; Sander, 2013) combina duas ideias: densidade (como o DBSCAN) e hierarquia (como o Capítulo 4).

Em linhas gerais:

1. Calcula uma **distância de alcançabilidade mútua**, que "afasta" pontos em regiões esparsas.
2. Constrói a **árvore geradora mínima** e a transforma numa hierarquia (um dendrograma).
3. **Condensa** a árvore, descartando divisões que separam grupos minúsculos.
4. Escolhe os grupos **mais estáveis**: aqueles que "sobrevivem" por uma ampla faixa de valores de densidade.

Vantagens sobre o DBSCAN:

- Funciona com **densidades variáveis**.
- Só exige `min_cluster_size` (intuitivo: "qual o menor grupo que me interessa?").
- Fornece uma **probabilidade** de pertinência por ponto.

```python
from sklearn.cluster import HDBSCAN
from sklearn.datasets import make_blobs
import numpy as np

X1, _ = make_blobs(200, centers=[[0, 0]],  cluster_std=0.3, random_state=0)
X2, _ = make_blobs(200, centers=[[4, 4]],  cluster_std=1.0, random_state=1)
X = np.vstack([X1, X2])

h = HDBSCAN(min_cluster_size=20).fit(X)
print("Grupos:", len(set(h.labels_)) - (1 if -1 in h.labels_ else 0))
print("Ruído :", (h.labels_ == -1).sum())
print("Probabilidades (5 primeiros):", h.probabilities_[:5].round(2))
```

> **SAIBA MAIS** **Disponibilidade**
>
> O HDBSCAN passou a integrar o scikit-learn a partir da versão 1.3. Antes disso, era distribuído no pacote independente `hdbscan` (McInnes; Healy; Astels, 2017).

5.8 Outras ideias de densidade

- **Mean-shift** (Comaniciu; Meer, 2002): cada ponto "escala o morro" da densidade estimada até um pico. Os picos são os centros dos grupos. Não exige *k*.
- **DENCLUE:** modela a densidade como soma de funções de influência.
- **Grades de densidade (CLIQUE, STING):** dividem o espaço em células e unem as densas.

5.9 Estudo de caso: pontos de táxi em uma cidade

Imagine milhares de coordenadas (latitude e longitude) de embarque de passageiros. Queremos achar os **pontos quentes** (aeroportos, estações, bairros boêmios).

Por que DBSCAN (ou HDBSCAN) e não K-means?

- Os pontos quentes têm formas irregulares (seguem ruas e avenidas).
- Existem embarques isolados que são ruído, e não "pontos quentes".
- Não sabemos quantos pontos quentes existem.

Em geografia, usa-se a distância **haversine**, que mede distância sobre a esfera terrestre.

```python
import numpy as np
from sklearn.cluster import DBSCAN

rng = np.random.default_rng(0)
# Coordenadas simuladas (graus): 3 pontos quentes + ruído espalhado
centros = np.array([[-23.55, -46.63], [-23.43, -46.47], [-23.62, -46.70]])
pts = np.vstack([c + rng.normal(0, 0.005, (150, 2)) for c in centros])
ruido = np.column_stack([rng.uniform(-23.7, -23.4, 60), rng.uniform(-46.8, -46.4, 60)])
coords = np.vstack([pts, ruido])

rad = np.radians(coords)
```

```
raio_terra_km = 6371.0
eps_km = 0.5                              # 500 metros
db = DBSCAN(eps=eps_km / raio_terra_km, min_samples=10,
            metric="haversine", algorithm="ball_tree").fit(rad)

print("Pontos quentes:", len(set(db.labels_)) - (1 if -1 in db.labels_ else 0))
print("Ruído:", (db.labels_ == -1).sum())
```

5.10 Comparando os métodos

Critério	K-means	DBSCAN	OPTICS	HDBSCAN
Precisa de *k*?	Sim	Não	Não	Não
Forma dos grupos	Esférica	Qualquer	Qualquer	Qualquer
Densidades variáveis	Não se aplica	Fraco	Bom	Muito bom
Detecta ruído?	Não	Sim	Sim	Sim
Parâmetros-chave	*k*	eps, min_samples	min_samples, xi	min_cluster_size
Escalabilidade	Excelente	Boa	Média	Boa

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Grupo = região densa** separada por regiões vazias.
- **DBSCAN:** eps (raio) e min_samples (mínimo de vizinhos).
- **Central, borda, ruído.** Rótulo −1 = ruído.
- **Gráfico de k-distâncias** → joelho → eps.
- **Fraqueza:** densidades variáveis e alta dimensão.
- **OPTICS:** gráfico de alcançabilidade (vales = grupos).
- **HDBSCAN:** hierárquico + densidade; só pede `min_cluster_size`.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Um grupo, no agrupamento por densidade, é:

a) um conjunto de pontos próximos ao centroide.
b) um conjunto de pontos com mesmo rótulo.
c) um nó do dendrograma.
d) uma região densa separada de outras por regiões pouco povoadas.

Questão 2. Qual das vantagens abaixo é típica do DBSCAN?

a) Descobre grupos de formas arbitrárias e identifica ruído.
b) Exige o número de grupos de antemão.
c) Dispensa qualquer parâmetro.
d) Só funciona com grupos esféricos.

Questão 3. Os dois parâmetros principais do DBSCAN são:

a) perplexity e n_neighbors.
b) eps e min_samples.
c) linkage e metric.
d) k e n_init.

Questão 4. O parâmetro eps do DBSCAN representa:

a) a taxa de aprendizado.
b) o número de grupos.
c) o número de iterações.
d) o raio da vizinhança de cada ponto.

Questão 5. Um ponto central (core) do DBSCAN é aquele que:

a) tem pelo menos min_samples pontos no raio eps.
b) é o centroide do grupo.
c) está isolado.
d) tem o maior desvio padrão.

Questão 6. Um ponto de borda é aquele que:

a) está longe de todos.
b) não é central, mas está no raio eps de algum ponto central.
c) é o centro do grupo.
d) é central.

Questão 7. No scikit-learn, o rótulo −1 em labels_ do DBSCAN indica:

a) o primeiro grupo.
b) ponto central.
c) erro de execução.
d) ruído.

Questão 8. Se eps é pequeno demais, ocorre tipicamente:

a) convergência mais rápida sem efeitos.
b) um único grupo.
c) fusão de todos os grupos.
d) quase tudo vira ruído e surgem muitos micro-grupos.

Questão 9. Se eps é grande demais, ocorre tipicamente:

a) k aumenta.
b) o algoritmo trava.
c) grupos distintos se fundem em um só.
d) todos viram ruído.

Questão 10. O gráfico de k-distâncias serve para:

a) comparar dendrogramas.
b) escolher o valor de eps pelo "joelho" da curva.
c) escolher min_samples.
d) calcular a inércia.

Questão 11. Uma regra comum para min_samples é:

a) min_samples ≥ d + 1, onde d é o número de atributos.
b) sempre 1.
c) sempre igual a n.
d) igual a eps.

Questão 12. Por que é preciso padronizar os dados antes do DBSCAN?

a) Porque a padronização remove o ruído.
b) Porque o algoritmo é probabilístico.
c) Porque usa raio único, e todos os atributos devem estar na mesma escala.
d) Porque exige média zero para funcionar.

Questão 13. Uma fraqueza do DBSCAN é:

a) só aceitar dados categóricos.
b) exigir k de antemão.
c) não detectar ruído.
d) lidar mal com grupos de densidades muito diferentes.

Questão 14. A complexidade do DBSCAN com índices espaciais é da ordem de:

a) n!
b) n log n
c) n^3
d) 2^n

Questão 15. O OPTICS produz como saída principal:

a) uma ordenação dos pontos com distâncias de alcançabilidade.
b) a matriz de ligações.
c) os centroides.
d) uma partição única.

Questão 16. No gráfico de alcançabilidade, os vales correspondem a:

a) fronteiras entre grupos.
b) pontos centrais únicos.
c) grupos (regiões densas).
d) ruído.

Questão 17. O HDBSCAN combina duas ideias:

a) votação e regressão.
b) densidade e hierarquia.
c) média e mediana.
d) probabilidade e grafos.

Questão 18. O principal parâmetro do HDBSCAN é:

a) min_cluster_size.
b) eps.
c) k.
d) perplexity.

Questão 19. O HDBSCAN passou a integrar o scikit-learn a partir da versão:

a) 1.0.
b) 0.9.
c) 2.5.
d) 1.3.

Questão 20. O atributo probabilities_ do HDBSCAN informa:

a) o valor de eps.
b) o número de grupos.
c) a probabilidade de pertinência de cada ponto ao seu grupo.
d) a inércia.

Questão 21. O Mean-shift (Comaniciu e Meer, 2002) baseia-se em:

a) calcular autovetores.
b) cortar dendrogramas.
c) subir o "morro" da densidade estimada até um pico.
d) somar probabilidades.

Questão 22. Em um estudo de pontos quentes de embarque de táxi, usa-se a distância haversine porque:

a) mede distâncias sobre a esfera terrestre.
b) é uma distância binária.
c) ignora a latitude.
d) é a mais rápida.

Questão 23. No exemplo geográfico, eps = 0,5 km é convertido para radianos dividindo por:

a) o raio da Terra (6371 km).
b) 100.
c) o número de pontos.
d) 360.

Questão 24. Para dados em formato de luas, o resultado típico é:

a) Ambos erram sempre.
b) Ambos acertam sempre.
c) DBSCAN acerta; K-means erra.
d) K-means acerta; DBSCAN erra.

Questão 25. Qual combinação descreve melhor o HDBSCAN versus DBSCAN?

a) HDBSCAN exige eps.
b) HDBSCAN lida melhor com densidades variáveis.
c) HDBSCAN não detecta ruído.
d) HDBSCAN só funciona em 1D.

Questão 26. Em alta dimensão, métodos baseados em densidade:

a) funcionam melhor.
b) não são afetados.
c) sofrem porque o conceito de vizinhança perde sentido.
d) tornam-se determinísticos.

Questão 27. Qual das opções NÃO descreve ponto de ruído no DBSCAN?

a) Não está no raio eps de nenhum ponto central.
b) É o centro de um grupo.
c) Recebe o rótulo −1.
d) Não é central.

Questão 28. Uma diferença entre DBSCAN e K-means é que o DBSCAN:

a) exige k.
b) usa centroides.
c) não exige k e aceita formas arbitrárias.
d) é sempre mais lento.

Questão 29. Em duas dimensões, o gráfico de k-distâncias com k = min_samples é construído:

a) removendo os pontos centrais.
b) calculando a média das distâncias.
c) somando todas as distâncias.
d) ordenando as distâncias ao k-ésimo vizinho e procurando o joelho.

Questão 30. Qual é a interpretação do OPTICS "alagar" a paisagem de alcançabilidade?

a) Calcular a inércia.
b) Escolher o número de iterações.
c) Definir os grupos escolhendo um nível de corte na curva.
d) Remover o ruído.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com a analogia das luzes das cidades à noite, a ideia do agrupamento por densidade.
2. Defina ponto central, ponto de borda e ruído no DBSCAN, com um exemplo numérico (eps e min_samples).
3. Descreva, passo a passo, o algoritmo DBSCAN.
4. Explique a analogia do boato para pontos centrais e de borda.
5. Como escolher eps e min_samples na prática? Descreva o gráfico de k-distâncias.
6. Explique o efeito de eps muito pequeno e muito grande sobre o resultado.
7. Por que o DBSCAN falha com grupos de densidades muito diferentes? Que alternativas existem?
8. Como funciona o gráfico de alcançabilidade do OPTICS e como interpretá-lo?
9. Descreva as principais etapas do HDBSCAN.
10. Compare DBSCAN, OPTICS e HDBSCAN quanto aos parâmetros e às densidades variáveis.
11. Por que o DBSCAN exige padronização dos atributos?
12. Descreva um estudo de caso de pontos quentes geográficos e justifique a escolha de DBSCAN em vez de K-means.
13. Explique por que métodos de densidade sofrem em alta dimensão.
14. Qual é o papel do rótulo −1 e como contar grupos e ruído no scikit-learn?
15. Explique o Mean-shift e compare-o ao K-means quanto a k.
16. Por que pontos de borda podem ser atribuídos a grupos diferentes conforme a ordem de visita?
17. Proponha um roteiro para aplicar DBSCAN a um conjunto de dados novo.
18. Quando preferir HDBSCAN a DBSCAN? Justifique com duas razões.
19. Explique por que o ruído é uma informação valiosa e não apenas um descarte.
20. Compare o tratamento de formas de grupos por K-means, hierárquico com ligação simples e DBSCAN.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Varrendo o parâmetro eps

NÍVEL: BÁSICO

Gere duas luas (`make_moons(500, noise=0.07, random_state=0)`), padronize e aplique `DBSCAN(min_samples=5)` com eps = 0,05; 0,1; 0,2; 0,3 e 0,6. Para cada valor, mostre o número de grupos e de pontos de ruído.

Exercício 2 — Achando o joelho do gráfico de k-distâncias

NÍVEL: INTERMEDIÁRIO

Para o mesmo conjunto, calcule as distâncias ao 5° vizinho, ordene-as e encontre o **joelho** automaticamente: o ponto da curva mais distante da reta que liga o primeiro ao último ponto. Use esse valor como eps e rode o DBSCAN.

Exercício 3 — DBSCAN, K-means e espectral em anéis

NÍVEL: INTERMEDIÁRIO

Em `make_circles(600, factor=0.4, noise=0.05, random_state=0)`, compare K-means (k=2), agrupamento espectral (k=2, `affinity="nearest_neighbors"`) e DBSCAN (eps=0,15). Calcule o ARI contra os anéis reais.

Exercício 4 — Densidades variáveis: OPTICS e HDBSCAN

NÍVEL: AVANÇADO

Crie três grupos de densidades diferentes (desvios 0,3; 0,6 e 1,2) mais 5% de ruído uniforme. Aplique `DBSCAN(eps=0.5, min_samples=10)`, `OPTICS(min_samples=10, xi=0.05)` e `HDBSCAN(min_cluster_size=30)`. Para cada um, mostre o número de grupos, o ruído e o ARI (considerando apenas os pontos que não são ruído verdadeiro).

Exercício 5 — Pontos quentes com a distância haversine

NÍVEL: AVANÇADO

Simule 3 pontos quentes de ocorrências na região de São Paulo (latitude/longitude em graus, desvio de 0,004° em cada) e 80 ocorrências dispersas. Use `DBSCAN` com `metric="haversine"`, `eps` = 400 m e `min_samples=15`. Para cada ponto quente encontrado, mostre o tamanho e as coordenadas médias.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Dry Bean (medidas de forma de feijões) e Wireless Indoor Localization.
> - **Kaggle:** New York City Airbnb Open Data (latitude e longitude de imóveis, ótimo para DBSCAN e HDBSCAN com distância haversine).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Pontos quentes de ocorrências urbanas

Fonte dos dados. Simulados pela função abaixo.

Contexto. A secretaria de segurança de uma cidade (fictícia) quer decidir **onde posicionar patrulhas**. Você recebeu coordenadas de ocorrências registradas de dia e de noite. Seu trabalho é identificar os **pontos quentes**, ignorar ocorrências isoladas e sugerir a distribuição das patrulhas.

Dados. Use a função abaixo (dados sintéticos).

```python
import numpy as np
import pandas as pd


def gerar_ocorrencias(seed=0):
    rng = np.random.default_rng(seed)
    quentes = [
        (-23.550, -46.633, 0.003, 220, "dia"),
        (-23.520, -46.690, 0.006, 260, "noite"),
        (-23.610, -46.560, 0.004, 150, "dia"),
        (-23.480, -46.620, 0.010, 200, "noite"),
    ]
    linhas = []
    for lat, lon, s, n, turno in quentes:
        P = np.c_[rng.normal(lat, s, n), rng.normal(lon, s, n)]
        linhas += [(a, b, turno) for a, b in P]
    for _ in range(120):
        linhas.append(
            (
                rng.uniform(-23.70, -23.40),
                rng.uniform(-46.80, -46.45),
                rng.choice(["dia", "noite"]),
            )
        )
    return pd.DataFrame(linhas, columns=["lat", "lon", "turno"])
```

Tarefas.

1. Desenhe as ocorrências em um gráfico de dispersão. O que se vê a olho nu?
2. Aplique DBSCAN com distância haversine. Use o gráfico de k-distâncias para escolher eps (em km) e justifique `min_samples`.
3. Aplique também HDBSCAN (`metric="haversine"`). Compare: número de pontos quentes e de pontos de ruído.
4. Para cada ponto quente (DBSCAN), calcule o tamanho, o centro e a **densidade aproximada** (ocorrências por km² dentro do raio que contém 90% dos pontos do grupo).
5. Repita a análise separando os turnos "dia" e "noite". Algum ponto quente é exclusivo de um turno?
6. Proponha uma distribuição de 20 patrulhas **proporcional ao número de ocorrências**, por ponto quente. Discuta limites éticos (viés de policiamento, privacidade).

Entregáveis. Script, mapa com pontos quentes e ruído, tabela dos pontos quentes e texto com a proposta.

A solução de referência está no Apêndice E.



Capítulo 6 — Modelos de Mistura Gaussiana e Agrupamento Espectral

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar a ideia de que os dados vêm de uma mistura de distribuições.
- Descrever a distribuição normal (gaussiana) e a de várias variáveis.
- Entender o algoritmo EM (Esperança-Maximização) em linguagem simples.
- Usar GMM e escolher o número de componentes com BIC e AIC.
- Distinguir agrupamento rígido de agrupamento suave.
- Compreender o agrupamento espectral: grafos, matriz Laplaciana e autovetores.
- Saber quando preferir GMM ou agrupamento espectral ao K-means.

6.1 Dois olhares diferentes sobre "grupo"

Até aqui vimos grupos definidos por **distância** (K-means, hierárquico) ou por **densidade** (DBSCAN). Neste capítulo, vemos dois outros olhares.

1. **Olhar probabilístico (GMM):** os dados foram "sorteados" de várias distribuições misturadas. Descobrir os grupos é descobrir de qual distribuição cada ponto provavelmente veio.
2. **Olhar de grafo (espectral):** os pontos são pessoas numa rede social. Grupos são comunidades que têm muitas conexões internas e poucas externas.

6.2 A distribuição normal em dois minutos

A **distribuição normal** (ou gaussiana) é a famosa "curva do sino". Ela é descrita por dois números:

- **Média (μ):** o centro do sino.
- **Desvio padrão (σ):** a largura do sino. A **variância** é σ^2.

Cerca de 68 % dos valores caem entre $\mu - \sigma$ e $\mu + \sigma$. Cerca de 95 %, entre $\mu - 2\sigma$ e $\mu + 2\sigma$.

Exemplo: alturas de adultos de uma cidade. A maioria fica perto da média, e valores muito altos ou muito baixos são raros.

6.2.1 A normal em várias dimensões

Com vários atributos, a média vira um vetor μ e a variância vira uma **matriz de covariância** Σ. A covariância diz como os atributos variam *juntos* (por exemplo, altura e peso crescem juntos).

Em duas dimensões, os pontos formam uma "nuvem" em forma de **elipse**.

- Elipse na horizontal ou vertical: atributos sem correlação.
- Elipse inclinada: atributos correlacionados.
- Círculo: variâncias iguais e sem correlação.

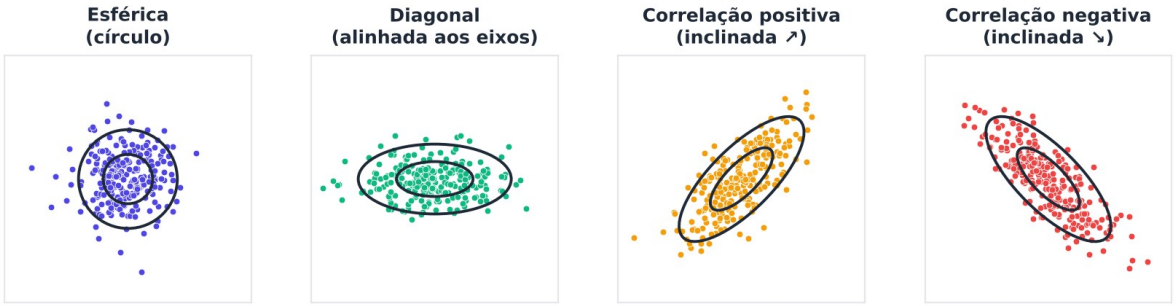


Figura 6.1 — Gaussianas bidimensionais: a matriz de covariância determina forma e inclinação da elipse.

6.3 A ideia da mistura

Imagine uma sala com **dois tipos de pessoas**: crianças e adultos. Se medirmos a altura de todos, o histograma terá **duas corcovas**. Cada corcova é uma distribuição normal. O conjunto é uma **mistura**.

Um **Modelo de Mistura Gaussiana** (GMM, *Gaussian Mixture Model*) escreve a densidade dos dados como uma soma ponderada de *K* gaussianas:

$$p(x) = \pi_1 \cdot \mathcal{N}(x \mid \mu_1, \Sigma_1) + \pi_2 \cdot \mathcal{N}(x \mid \mu_2, \Sigma_2) + \ldots + \pi_K \cdot \mathcal{N}(x \mid \mu_K, \Sigma_K)$$

- π_k são os **pesos** (a proporção de cada grupo; somam 1).
- μ_k e Σ_k são a média e a covariância do grupo *k*.

Aprender o modelo é estimar todos esses números a partir dos dados (Bishop, 2006; McLachlan; Peel, 2000).

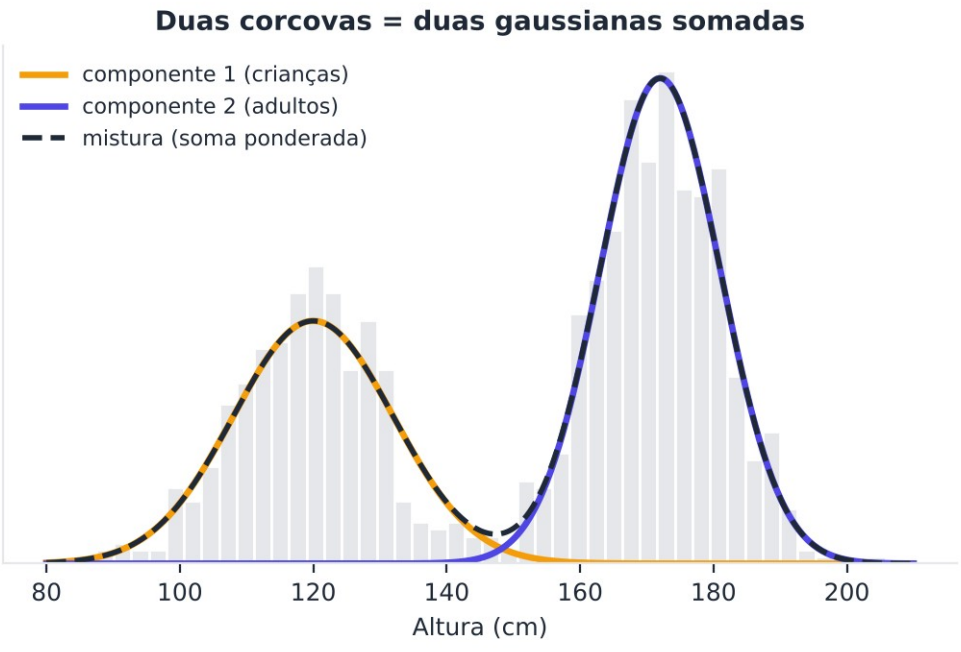


Figura 6.2 — Duas corcovas num histograma são duas gaussianas somadas. Descobrir as componentes é o objetivo do GMM.

6.4 O problema do "ovo e a galinha" e o algoritmo EM

Se soubéssemos **de qual grupo cada ponto veio**, estimar média e covariância de cada grupo seria trivial. Se soubéssemos **os parâmetros dos grupos**, calcular de qual grupo cada ponto veio seria fácil. Mas não sabemos nenhuma das duas coisas.

A solução é alternar. É o algoritmo **EM** (*Expectation-Maximization*), de Dempster, Laird e Rubin (1977).

6.4.1 Passo E (Esperança)

Dados os parâmetros atuais, calcule, para cada ponto, a **responsabilidade**: a probabilidade de ele pertencer a cada grupo.

Exemplo: um ponto tem 80 % de chance de ser do grupo A, 15 % do grupo B, 5 % do C.

6.4.2 Passo M (Maximização)

Dadas as responsabilidades, recalcule os parâmetros:

- Cada média é a **média ponderada** dos pontos (pesos = responsabilidades).
- Cada covariância é a **covariância ponderada**.
- Cada peso π_k é a **fração** total de responsabilidade do grupo.

6.4.3 Repetir

Alterne E e M até a verossimilhança parar de melhorar. A **verossimilhança** mede o quão "provável" é ver estes dados dado o modelo. Cada iteração garante que ela **não diminui**.

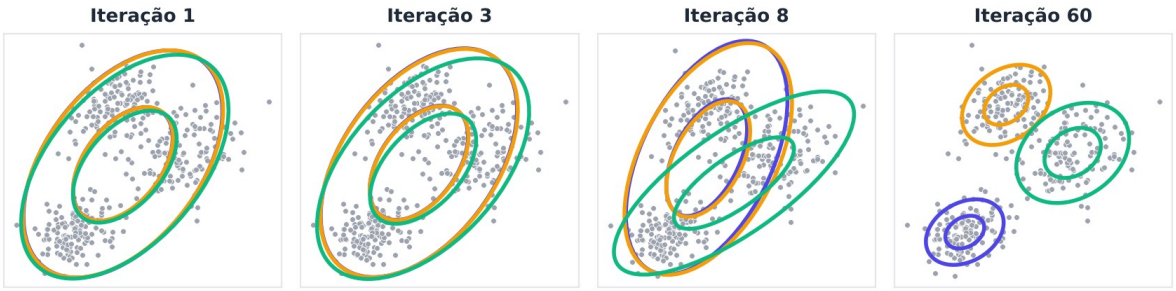


Figura 6.3 — Iterações do EM: as elipses começam mal posicionadas e se ajustam aos dados.

> MEMORIZE **K-means é um GMM "simplificado"**
>
> O K-means equivale a um GMM em que todas as gaussianas são esféricas, de mesma variância, e a atribuição é **rígida** (100 % ou 0 %). O GMM é uma generalização mais flexível.

6.5 Agrupamento rígido e suave

- **Rígido (*hard*):** cada ponto pertence a **um único** grupo. Ex.: K-means.
- **Suave (*soft*):** cada ponto tem uma **probabilidade** de pertencer a cada grupo. Ex.: GMM.

O agrupamento suave é útil quando existem fronteiras ambíguas. Um cliente de perfil misto pode receber duas campanhas.

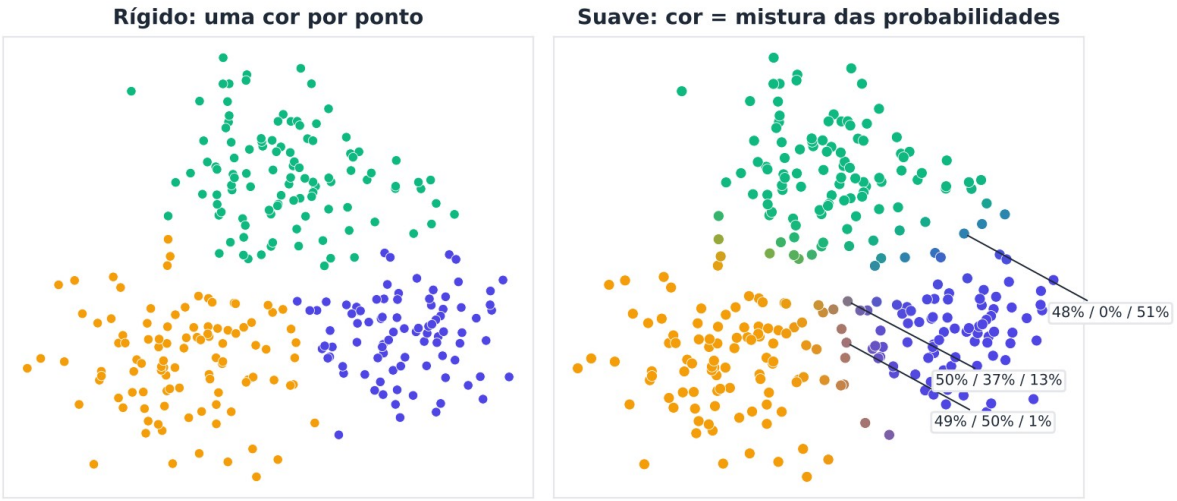


Figura 6.4 — Pertinência suave: a cor de cada ponto é uma mistura das cores dos grupos, proporcional às probabilidades.

6.6 Tipos de covariância

A matriz Σ pode ter muitos parâmetros. O scikit-learn oferece quatro versões, da mais restrita à mais flexível:

`covariance_type`	Forma das elipses	Nº de parâmetros
`spherical`	Círculos, tamanho distinto por grupo	Poucos
`diag`	Elipses alinhadas aos eixos	Moderado
`tied`	Todas com a mesma forma e inclinação	Moderado
`full`	Elipses livres, cada uma com sua forma	Muitos

Mais flexibilidade significa melhor ajuste, mas maior risco de **sobreajuste** (*overfitting*, memorizar o ruído).

6.7 GMM no Python

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture

# Dados com grupos elípticos (esticados)
X, _ = make_blobs(n_samples=500, centers=3, random_state=4)
X = X @ np.array([[0.8, -0.6], [-0.2, 0.9]])

gmm = GaussianMixture(n_components=3, covariance_type="full",
                      n_init=5, random_state=0).fit(X)

rotulos = gmm.predict(X)            # agrupamento rígido
probs   = gmm.predict_proba(X)      # agrupamento suave

print("Pesos       :", gmm.weights_.round(2))
print("Médias      :\n", gmm.means_.round(2))
print("Convergiu?  :", gmm.converged_)
print("Prob. do 1º ponto:", probs[0].round(3))
```

```
plt.scatter(X[:, 0], X[:, 1], c=rotulos, s=12)
plt.title("GMM com 3 componentes")
plt.show()
```

O método `score_samples(X)` devolve o log da densidade de cada ponto. Pontos com valor muito baixo são candidatos a **anomalias** (Capítulo 11). O método `sample(n)` **gera** novos dados sintéticos a partir do modelo.

6.8 Quantos componentes? AIC e BIC

Se aumentarmos *K*, a verossimilhança sempre cresce. Mas modelos muito complexos "decoram" os dados. Precisamos de uma **penalidade** pela complexidade.

- **AIC** (critério de informação de Akaike): AIC = 2p − 2 ln(L)
- **BIC** (critério de informação bayesiano, de Schwarz): BIC = p ln(n) − 2 ln(L)

onde *p* é o número de parâmetros, *n* o número de exemplos e *L* a verossimilhança. **Menor é melhor.** O BIC penaliza mais a complexidade e tende a escolher modelos mais simples (Schwarz, 1978; Akaike, 1974).

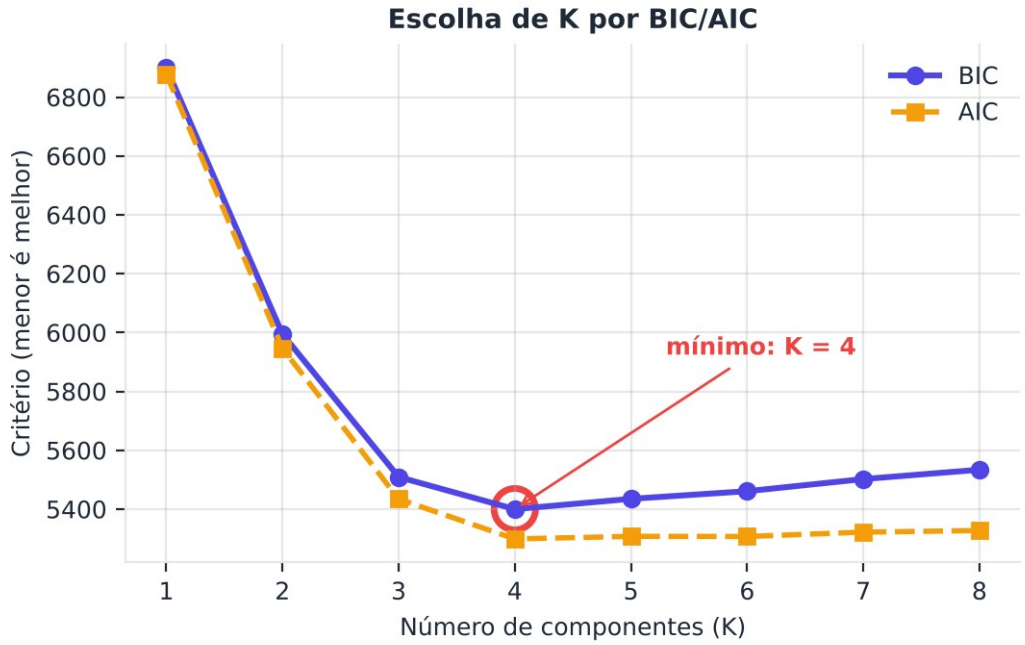


Figura 6.5 — BIC em função do número de componentes: o mínimo indica o melhor equilíbrio entre ajuste e simplicidade.

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture

X, _ = make_blobs(n_samples=600, centers=4, cluster_std=1.1, random_state=11)

for k in range(1, 8):
    g = GaussianMixture(n_components=k, n_init=3, random_state=0).fit(X)
    print(f"K={k}  BIC={g.bic(X):9.1f}  AIC={g.aic(X):9.1f}")
```

6.9 Cuidados com o GMM

- **Inicialização:** o EM pode ficar preso em máximos locais. Use `n_init` > 1 e inicialização por K-means (padrão).
- **Singularidades:** uma gaussiana pode "colapsar" sobre um único ponto, com variância quase zero. O parâmetro `reg_covar` adiciona um pequeno valor à diagonal para evitar isso.
- **Alta dimensão:** a covariância `full` tem ~ d^2 parâmetros. Prefira `diag` ou reduza antes a dimensão.
- **Suposição gaussiana:** grupos muito não gaussianos (luas, anéis) não são bem representados.

6.10 Agrupamento espectral

Chegamos ao segundo olhar. Imagine uma rede social. Existem comunidades de amigos próximos, ligadas entre si por poucos laços. Queremos **cortar** a rede nos pontos de menor conexão.

O **agrupamento espectral** (Shi; Malik, 2000; Ng; Jordan; Weiss, 2001; von Luxburg, 2007) usa a estrutura de **grafo** dos dados e a álgebra linear para encontrar esses cortes.

6.10.1 Passo 1: construir o grafo de similaridade

Cada ponto é um **nó**. Ligamos nós "semelhantes". Há três formas comuns:

- **Vizinhos mais próximos:** liga cada ponto aos seus *k* vizinhos.
- **Raio ε:** liga pontos a menos de ε de distância.
- **Kernel gaussiano (RBF):** pesa cada ligação por $e^{-\gamma\|x-y\|^2}$. Pontos próximos têm peso perto de 1; distantes, perto de 0.

O resultado é uma **matriz de afinidade** W: o valor W[i, j] diz o quão ligados estão os pontos *i* e *j*.

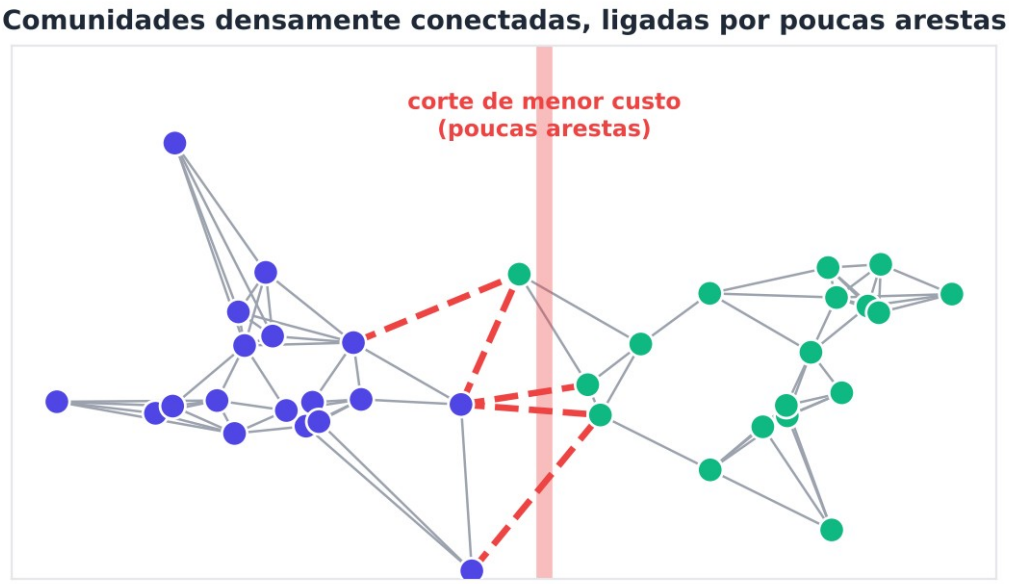


Figura 6.6 — Grafo de vizinhos: duas comunidades densamente conectadas por poucas arestas. O melhor corte separa as duas.

6.10.2 Passo 2: a matriz Laplaciana

Definimos:

- **D:** matriz diagonal em que cada elemento é a soma dos pesos das ligações do nó (o "grau").

- **L = D − W:** a **Laplaciana do grafo**.

O que ela tem de especial? Seus **autovalores** e **autovetores** (vetores especiais que a matriz só estica, sem girar) guardam a estrutura de conectividade do grafo. Se o grafo tem *k* comunidades perfeitamente separadas, a Laplaciana tem exatamente *k* autovalores iguais a zero (von Luxburg, 2007).

6.10.3 Passo 3: usar os menores autovetores

Pegue os *k* autovetores associados aos *k* menores autovalores. Eles formam uma nova tabela, com *n* linhas e *k* colunas. Cada ponto original ganha uma nova "coordenada" (a sua linha).

Mágica: nesse novo espaço, os grupos viram nuvens compactas e separadas, mesmo quando eram anéis entrelaçados no espaço original.

6.10.4 Passo 4: K-means no novo espaço

Aplique o K-means às linhas dessa tabela. Os rótulos obtidos valem para os pontos originais.

> MEMORIZE **Resumo em 4 passos**
>
> **Grafo → Laplaciana → autovetores → K-means.** O espectral transforma o problema difícil (formas estranhas) num problema fácil (nuvens compactas).

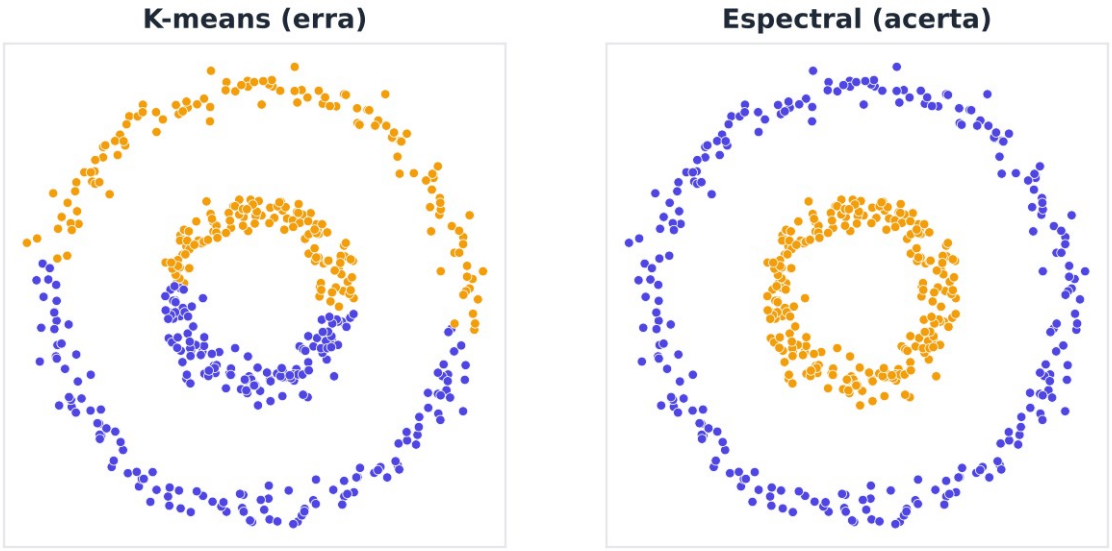


Figura 6.7 — Anéis concêntricos: o K-means falha; o agrupamento espectral os separa.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.cluster import SpectralClustering, KMeans

X, _ = make_circles(n_samples=500, factor=0.4, noise=0.05, random_state=0)

km = KMeans(n_clusters=2, n_init=10, random_state=0).fit_predict(X)
sc = SpectralClustering(n_clusters=2, affinity="nearest_neighbors",
                        n_neighbors=10, assign_labels="kmeans",
                        random_state=0).fit_predict(X)

fig, ax = plt.subplots(1, 2, figsize=(9, 4))
```

```
ax[0].scatter(*X.T, c=km, s=10); ax[0].set_title("K-means")
ax[1].scatter(*X.T, c=sc, s=10); ax[1].set_title("Espectral")
plt.show()
```

6.10.5 O corte normalizado

Shi e Malik (2000) mostraram que o espectral aproxima a minimização do **corte normalizado** (*normalized cut*). Cortar só pelo menor número de arestas pode isolar um único ponto (um corte "barato", mas inútil). O corte normalizado equilibra: ele penaliza grupos pequenos e favorece divisões balanceadas.

6.10.6 Escolhendo o número de grupos: o eigengap

Ordene os autovalores da Laplaciana. Procure o **maior salto** entre autovalores consecutivos. O número de autovalores antes do salto sugere k. Esse critério é chamado de **eigengap** (von Luxburg, 2007).

6.10.7 Custos e limitações

- Calcular autovetores de uma matriz $n \times n$ custa muito (cerca de n^3 na versão exata). Há aproximações (Nyström, vizinhos esparsos) que ajudam.
- É sensível à escolha da similaridade (γ ou número de vizinhos).
- Não classifica pontos novos de modo direto (precisa de extensões).

6.11 Comparação final

Critério	K-means	GMM	Espectral	DBSCAN
Pertinência	Rígida	Suave	Rígida	Rígida (+ ruído)
Forma dos grupos	Esféricos	Elípticos	Arbitrária	Arbitrária
Precisa de k	Sim	Sim (use BIC)	Sim (use eigengap)	Não
Modelo gerativo?	Não	**Sim**	Não	Não
Escalabilidade	Excelente	Boa	Baixa	Boa

> **DICA** **Roteiro de decisão**
>
> Grupos esféricos e base grande? **K-means.** Grupos elípticos e você quer probabilidades? **GMM.** Grupos de formas complicadas e base média? **Espectral.** Ruído e forma livre? **DBSCAN/HDBSCAN.**

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **GMM:** os dados vêm de uma soma ponderada de gaussianas.
- **EM:** alterna E (responsabilidades) e M (parâmetros).
- **Agrupamento suave** dá probabilidades de pertinência.
- **BIC/AIC:** escolhem K penalizando a complexidade.
- **Espectral:** grafo → Laplaciana → autovetores → K-means.
- **Eigengap** sugere o número de grupos.
- O espectral separa formas não convexas, a um custo alto.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. A distribuição normal é descrita por:

a) mínimo e máximo.
b) apenas a mediana.
c) média e desvio padrão.
d) moda e amplitude.

Questão 2. Aproximadamente que fração dos valores de uma normal cai entre $\mu - \sigma$ e $\mu + \sigma$?

a) 50%.
b) 68%.
c) 95%.
d) 99,9%.

Questão 3. Em várias dimensões, a variância é generalizada pela:

a) mediana.
b) distância de Hamming.
c) moda.
d) matriz de covariância.

Questão 4. Em duas dimensões, atributos correlacionados formam uma nuvem em forma de:

a) elipse inclinada.
b) reta vertical apenas.
c) quadrado.
d) círculo perfeito.

Questão 5. Um Modelo de Mistura Gaussiana escreve a densidade dos dados como:

a) uma árvore de decisão.
b) uma soma ponderada de gaussianas.
c) um produto de médias.
d) uma única gaussiana.

Questão 6. Em um GMM, os pesos π_k:

a) são os centroides.
b) somam zero.
c) somam 1 e representam a proporção de cada componente.
d) são distâncias.

Questão 7. O algoritmo EM alterna os passos:

a) L (limpeza) e T (treino).
b) A (atribuição) e C (corte).
c) D (divisão) e F (fusão).
d) E (esperança) e M (maximização).

Questão 8. No passo E do EM para GMM, calcula-se, para cada ponto:

a) a responsabilidade (probabilidade de pertencer a cada componente).
b) o número de componentes.
c) a média global.
d) o dendrograma.

Questão 9. No passo M do EM para GMM:

a) calculam-se autovalores.
b) escolhe-se k.
c) remove-se o ruído.
d) recalculam-se médias, covariâncias e pesos usando as responsabilidades.

Questão 10. A cada iteração, o EM garante que a verossimilhança:

a) fica sempre igual.
b) não diminui.
c) oscila livremente.
d) diminui.

Questão 11. O K-means pode ser visto como um GMM:

a) sem médias.
b) com dimensões ocultas.
c) com infinitos componentes.
d) em que as gaussianas são esféricas, de mesma variância, com atribuição rígida.

Questão 12. No agrupamento suave, cada ponto:

a) tem uma probabilidade de pertencer a cada grupo.
b) não é atribuído.
c) é ruído.
d) pertence a um único grupo.

Questão 13. No scikit-learn, o método que devolve as probabilidades de pertinência do GMM é:

a) predict.
b) inertia_.
c) fit_predict.
d) predict_proba.

Questão 14. O tipo de covariância mais flexível do GaussianMixture é:

a) spherical.
b) diag.
c) tied.
d) full.

Questão 15. O tipo "spherical" no GaussianMixture implica elipses:

a) livres.
b) inexistentes.
c) circulares, com tamanhos que podem variar por componente.
d) alinhadas ao eixo apenas.

Questão 16. Qual é a desvantagem de um modelo muito flexível (muitos parâmetros)?

a) Não funciona com números.
b) Nunca converge.
c) Risco de sobreajuste.
d) Aceita só uma dimensão.

Questão 17. Para escolher o número de componentes do GMM, usam-se:

a) corte do dendrograma.
b) eigengap.
c) BIC e AIC, menores sendo melhores.
d) inércia e cotovelo apenas.

Questão 18. O BIC, comparado ao AIC, tende a:

a) preferir sempre mais componentes.
b) penalizar menos.
c) penalizar mais a complexidade, escolhendo modelos mais simples.
d) ignorar o número de parâmetros.

Questão 19. O parâmetro reg_covar do GaussianMixture serve para:

a) escolher o kernel.
b) escolher o número de componentes.
c) definir o raio.
d) evitar covariâncias singulares, adicionando um pequeno valor à diagonal.

Questão 20. O método score_samples do GaussianMixture pode ser usado para:

a) desenhar dendrogramas.
b) padronizar dados.
c) escolher k.
d) detectar anomalias, pois pontos com baixa densidade são suspeitos.

Questão 21. No agrupamento espectral, o primeiro passo é:

a) cortar o dendrograma.
b) reduzir a dimensão com t-SNE.
c) construir um grafo de similaridade entre os pontos.
d) calcular centroides.

Questão 22. A matriz Laplaciana de um grafo é definida como:

a) $L = D + W$.
b) $L = D / W$.
c) $L = D - W$.
d) $L = W^2$.

Questão 23. Se o grafo tem k componentes perfeitamente separados, a Laplaciana possui:

a) exatamente k autovalores iguais a zero.
b) k^2 autovalores nulos.
c) nenhum autovalor nulo.
d) apenas um autovalor nulo.

Questão 24. No final do agrupamento espectral, o algoritmo aplicado nas linhas dos autovetores é:

a) o DBSCAN.
b) o PCA.
c) o Apriori.
d) o K-means.

Questão 25. O agrupamento espectral é particularmente útil para:

a) dados sem nenhuma estrutura.
b) detectar regras de associação.
c) formas não convexas, como anéis concêntricos.
d) bases com bilhões de pontos.

Questão 26. O critério "eigengap" sugere o número de grupos observando:

a) o maior salto entre autovalores consecutivos da Laplaciana.
b) a altura do dendrograma.
c) a silhueta.
d) o cotovelo da inércia.

Questão 27. O corte normalizado (normalized cut) evita:

a) o uso de grafos.
b) cortes que isolam poucos pontos, favorecendo divisões equilibradas.
c) qualquer corte.
d) o uso de autovetores.

Questão 28. Uma limitação do agrupamento espectral é:

a) ser determinístico em qualquer base.
b) só aceitar 2 grupos.
c) não usar grafo.
d) custo alto (autovetores de uma matriz $n \times n$).

Questão 29. O kernel gaussiano (RBF) em um grafo de afinidade atribui:

a) peso próximo de 1 a pontos próximos e perto de 0 a pontos distantes.
b) peso apenas a vizinhos de índice par.
c) o mesmo peso a todos.
d) peso negativo.

Questão 30. Para grupos elípticos e probabilidades de pertinência, a escolha mais adequada dentre as abaixo é:

a) K-means.
b) Mean-shift.
c) DBSCAN.
d) GMM.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique a ideia de mistura gaussiana com um exemplo do cotidiano (alturas de crianças e adultos).
2. Descreva o algoritmo EM aplicado ao GMM e o problema do "ovo e a galinha".
3. Diferencie agrupamento rígido de suave e dê um exemplo em que o suave é vantajoso.
4. Explique os quatro tipos de covariância do GaussianMixture e o compromisso entre flexibilidade e sobreajuste.
5. Como escolher o número de componentes com BIC e AIC? O que significa "menor é melhor"?
6. Cite três cuidados ao usar GMM.
7. Explique como o GMM pode ser usado para detectar anomalias e gerar dados sintéticos.
8. Descreva os quatro passos do agrupamento espectral.
9. Explique o papel da matriz Laplaciana e de seus autovalores no agrupamento espectral.
10. Por que o agrupamento espectral separa anéis concêntricos e o K-means não?
11. O que é o corte normalizado e por que é melhor que o corte mínimo simples?
12. Como o eigengap ajuda a escolher k?
13. Cite três formas de construir o grafo de similaridade.
14. Quais as limitações do agrupamento espectral e como contorná-las?
15. Compare K-means, GMM, espectral e DBSCAN quanto à pertinência, forma dos grupos e necessidade de k.
16. Explique por que a verossimilhança do GMM sempre cresce com mais componentes e como evitar o sobreajuste.
17. Explique o conceito de "responsabilidade" no EM com um exemplo numérico.
18. Dê um exemplo de aplicação do GMM em segmentação de clientes e comente o benefício da pertinência suave.
19. Escreva um roteiro de decisão entre K-means, GMM, espectral e DBSCAN/HDBSCAN.
20. Explique por que dizemos que o GMM é um modelo gerativo e o K-means não.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Tipos de covariância no Iris

NÍVEL: BÁSICO

Ajuste `GaussianMixture(n_components=3)` ao Iris padronizado, com `covariance_type` igual a `spherical`, `diag`, `tied` e `full`. Para cada um, mostre o BIC e o ARI contra as espécies. Qual tipo tem o menor BIC?

Exercício 2 — Quantas gaussianas?

NÍVEL: BÁSICO

Gere 600 pontos com 4 grupos (`make_blobs(600, centers=4, cluster_std=1.1, random_state=11)`). Ajuste GMMs com K de 1 a 8 e mostre BIC e AIC. Qual K cada critério escolhe?

Exercício 3 — Pontos ambíguos e pertinência suave

NÍVEL: INTERMEDIÁRIO

Gere dados com 3 grupos que se sobrepõem (`make_blobs(400, centers=[[0,0],[2.5,0.5],[1.2,3]], cluster_std=1.0, random_state=2)`). Ajuste um GMM com K = 3 e use `predict_proba`. Liste os 5 pontos mais ambíguos (menor probabilidade máxima) com suas probabilidades. Quantos pontos têm probabilidade máxima abaixo de 0,8?

Exercício 4 — GMM como detector de anomalias

NÍVEL: INTERMEDIÁRIO

Treine um GMM (K = 2) em 800 pontos normais (dois grupos). Gere 30 anomalias uniformes numa região ampla. Use `score_samples` e marque como anomalia os 3% de menor densidade. Quantas das 30 anomalias foram detectadas e quantos pontos normais foram marcados por engano?

Exercício 5 — Espectral: o papel da afinidade

NÍVEL: AVANÇADO

Em `make_moons(500, noise=0.07, random_state=0)`, aplique `SpectralClustering(2)` com `affinity="rbf"` e gamma = 0,1; 1; 10 e 100, e também `affinity="nearest_neighbors"` com 10 vizinhos. Compare o ARI com o K-means.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Dry Bean e Seeds.
> - **Kaggle:** Mall Customers e Credit Card Dataset for Clustering (comportamento de uso de cartões).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Perfis com probabilidades: uma operadora de telecomunicações

Fonte dos dados. Simulados pela função abaixo.

Contexto. Uma operadora (fictícia) quer entender o comportamento de seus clientes pelo **consumo de dados** (GB/mês) e pelo **tempo de chamadas** (minutos/mês). Os perfis se sobrepõem, e o time de marketing quer saber **com que confiança** cada cliente pertence a um perfil.

Dados.

```python
import numpy as np

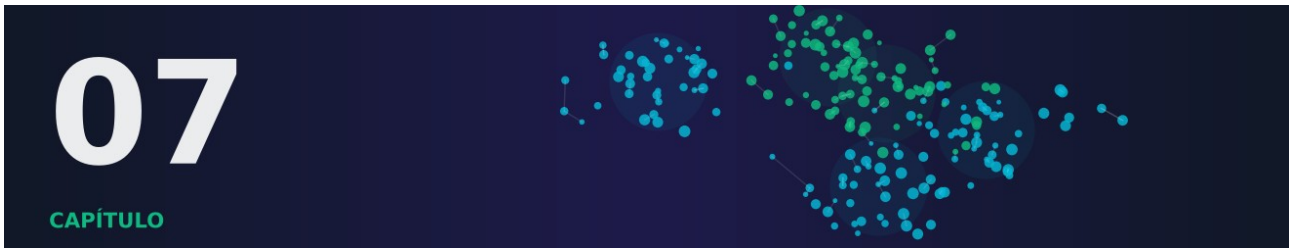

def gerar_telecom(seed=0):
    rng = np.random.default_rng(seed)
    a = rng.multivariate_normal([4, 300], [[3, 40], [40, 4000]], 350)  # voz
    b = rng.multivariate_normal([25, 120], [[30, -30], [-30, 1500]], 450)  # dados
    c = rng.multivariate_normal([14, 210], [[20, 10], [10, 2500]], 300)  # misto
    X = np.vstack([a, b, c])
    y = np.repeat([0, 1, 2], [350, 450, 300])
    return np.clip(X, 0, None), y
```

Tarefas.

1. Padronize os dados. Compare K-means (k = 3) e GMM (K = 3, covariância `full`) usando o ARI contra os perfis ocultos.
2. Escolha K de 1 a 7 por **BIC** e discuta se confirma 3 perfis.
3. Calcule a pertinência suave. Quantos clientes têm probabilidade máxima abaixo de 0,7? Chame-os "clientes de fronteira".
4. Proponha uma ação de marketing para os clientes de fronteira.
5. Use `gmm.sample(300)` para gerar 300 clientes sintéticos e compare as médias com as dos dados reais (dados sintéticos para testes e demonstrações).
6. Marque os 1% de clientes com menor `score_samples` como atípicos e descreva-os.

Entregáveis. Script, tabela de comparação, gráfico com elipses e texto de recomendações.

A solução de referência está no Apêndice E.



Capítulo 7 — Como Saber se os Grupos São Bons? Avaliação e Validação

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que avaliar agrupamentos é difícil sem gabarito.
- Diferenciar validação interna, externa e relativa.
- Calcular e interpretar silhueta, Calinski-Harabasz e Davies-Bouldin.
- Usar índices externos como ARI, NMI e pureza quando há rótulos de referência.
- Aplicar a estatística gap e testes de estabilidade para escolher o número de grupos.
- Verificar a tendência de agrupamento (estatística de Hopkins) antes de agrupar.

7.1 O dilema do juiz sem regras

No futebol, o juiz tem um livro de regras. No aprendizado supervisionado, também: o gabarito diz quem acertou.

No agrupamento, **não há livro de regras**. Se um algoritmo diz que há 4 grupos e outro diz que são 6, quem está certo? Talvez os dois, talvez nenhum.

ATENÇÃO **O algoritmo sempre responde**

Mesmo com dados puramente aleatórios, o K-means entrega *k* grupos. Isso não prova que existam grupos reais. Avaliar é separar descoberta de miragem.

Existem três famílias de avaliação (Jain; Dubes, 1988; Halkidi; Batistakis; Vazirgiannis, 2001):

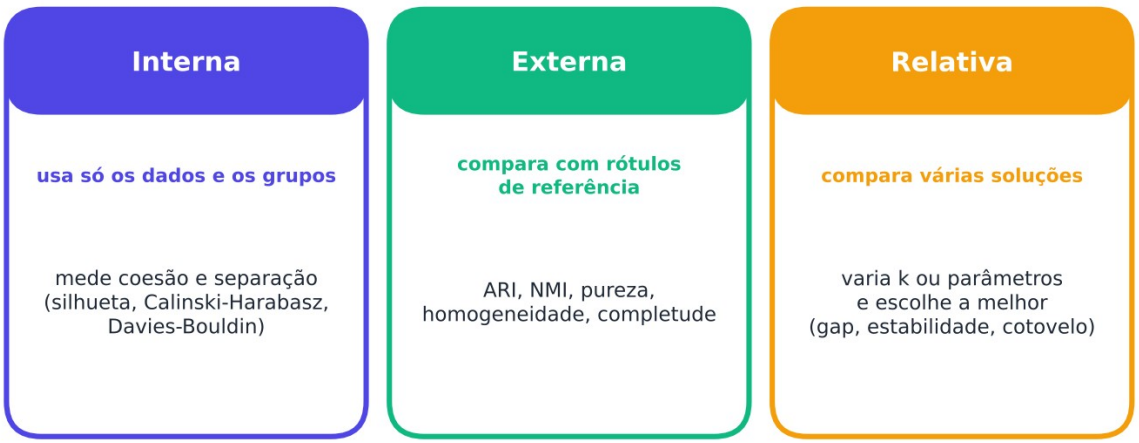


Figura 7.1 — As três famílias de avaliação: interna, externa e relativa.

1. **Validação interna:** usa apenas os dados e os grupos. Mede coesão e separação.
2. **Validação externa:** compara os grupos com rótulos de referência conhecidos (se houver).
3. **Validação relativa:** compara várias soluções (por exemplo, vários *k*) e escolhe a melhor.

7.2 Antes de tudo: existe estrutura nos dados?

Antes de avaliar grupos, pergunte: **há grupos para encontrar?**

A **estatística de Hopkins** testa a *tendência de agrupamento* (Hopkins; Skellam, 1954; Banerjee; Dave, 2004). A ideia:

1. Sorteie alguns pontos **reais** dos dados e meça a distância de cada um a seu vizinho mais próximo (w).
2. Sorteie o mesmo número de pontos **artificiais**, uniformemente espalhados na região, e meça a distância ao vizinho real mais próximo (u).
3. Calcule H = Σu^d / (Σu^d + Σw^d), em que *d* é o número de atributos (cada distância é elevada a *d*, como na definição original).

Interpretação:

- H ≈ 0,5: dados parecem uniformes (sem grupos).
- H próximo de 1: dados fortemente agrupáveis.
- H próximo de 0: dados muito regulares (como uma grade).

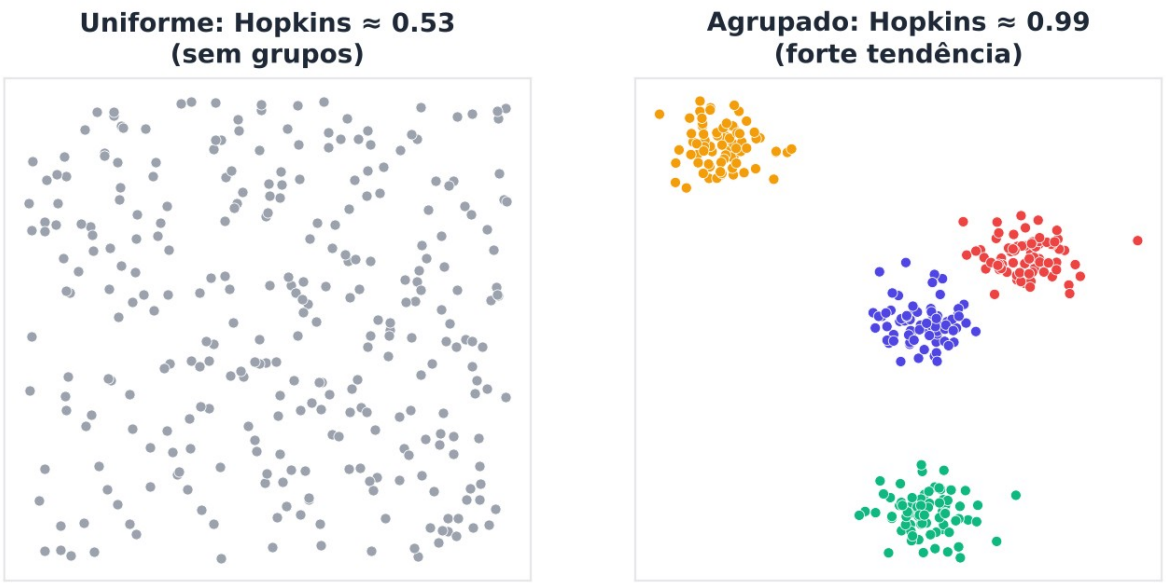


Figura 7.2 — Dados uniformes (esquerda) versus dados agrupados (direita). A estatística de Hopkins diferencia os dois.

```python
import numpy as np
from sklearn.neighbors import NearestNeighbors
from sklearn.datasets import make_blobs

def hopkins(X, m=None, seed=42):
    rng = np.random.default_rng(seed)
    n, d = X.shape
    m = m or max(10, int(0.1 * n))
```

```
    nn = NearestNeighbors(n_neighbors=2).fit(X)
    idx = rng.choice(n, m, replace=False)
    w = nn.kneighbors(X[idx], 2)[0][:, 1]          # vizinho mais próximo (exclui ele mesmo)
    U = rng.uniform(X.min(0), X.max(0), (m, d))
    u = nn.kneighbors(U, 1)[0][:, 0]
    return (u ** d).sum() / ((u ** d).sum() + (w ** d).sum())

rng = np.random.default_rng(0)
uniforme = rng.random((500, 2))
agrupado, _ = make_blobs(500, centers=4, random_state=2)

print("Hopkins uniforme:", round(hopkins(uniforme), 2))
print("Hopkins agrupado:", round(hopkins(agrupado), 2))
```

7.3 Validação interna

7.3.1 Coesão e separação

Um bom agrupamento tem:

- **Coesão:** pontos do mesmo grupo bem próximos entre si.
- **Separação:** grupos distintos bem afastados.

Analogia: uma festa de amigos. Cada rodinha de conversa deve ser unida (coesão) e as rodinhas devem estar distantes umas das outras para não se misturarem (separação).

7.3.2 O coeficiente de silhueta

Proposto por Rousseeuw (1987), é a medida interna mais popular. Para cada ponto *i*:

- **a(i):** distância média de *i* aos outros pontos **do seu grupo** (coesão).
- **b(i):** menor distância média de *i* aos pontos de **outro grupo** (o grupo vizinho mais próximo).

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

O valor fica entre −1 e 1.

- **Perto de 1:** o ponto está bem encaixado no seu grupo.
- **Perto de 0:** o ponto está na fronteira entre dois grupos.
- **Negativo:** o ponto provavelmente está no grupo errado.

A **silhueta média** resume o agrupamento. Valores acima de 0,5 costumam indicar estrutura razoável; abaixo de 0,25, estrutura fraca (Kaufman; Rousseeuw, 1990).

> ATENÇÃO **Silhueta alta não garante grupos úteis**
>
> Quando há valores extremos, o K-means pode isolar alguns outliers em grupos minúsculos e colocar quase todos os pontos em um só grupo gigante. A silhueta média desse resultado pode ser altíssima (acima de 0,9), embora ele não tenha utilidade nenhuma. Sempre olhe os **tamanhos dos grupos**, os perfis e a estabilidade. Você verá um caso real disso no projeto do Capítulo 2.

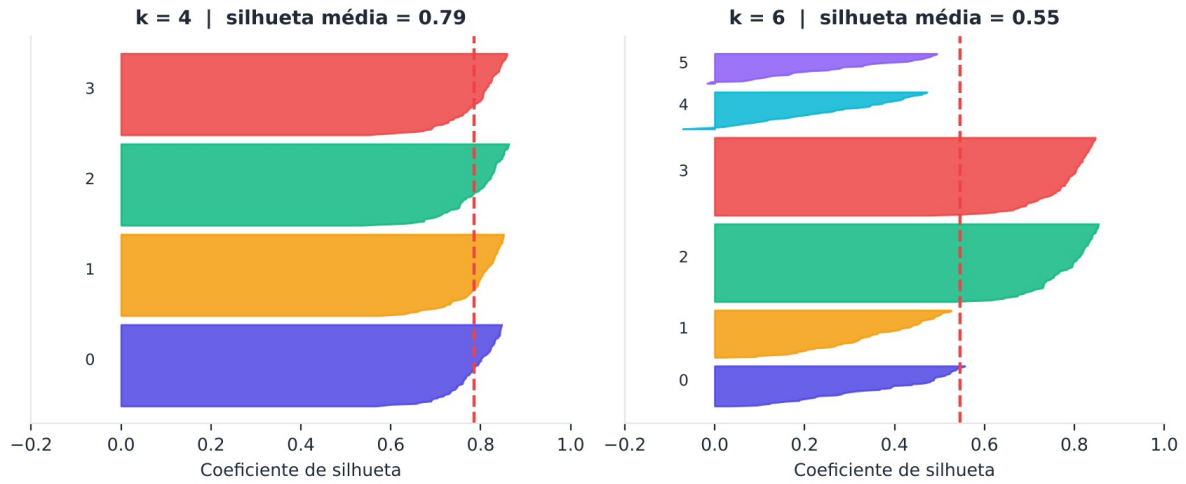


Figura 7.3 — Gráfico de silhueta para k = 3 e k = 5. Barras largas e longas indicam grupos coesos; barras que cruzam a linha vermelha (média) ou ficam negativas indicam problemas.

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, silhouette_samples

X, _ = make_blobs(n_samples=500, centers=4, random_state=8)

for k in range(2, 8):
    rot = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(X)
    print(f"k={k}  silhueta média = {silhouette_score(X, rot):.3f}")

# Silhueta individual de cada ponto
rot = KMeans(n_clusters=4, n_init=10, random_state=0).fit_predict(X)
s = silhouette_samples(X, rot)
print("Pontos com silhueta negativa:", (s < 0).sum())
```

7.3.3 Índice de Calinski-Harabasz

Razão entre a dispersão **entre** os grupos e a dispersão **dentro** dos grupos, ajustada pelo número de grupos e de pontos (Calinski; Harabasz, 1974).

- **Quanto maior, melhor.**
- Tende a favorecer grupos esféricos e compactos.

7.3.4 Índice de Davies-Bouldin

Para cada grupo, procura o grupo "mais parecido" e mede a razão entre a soma das dispersões e a distância entre os centros. Depois faz a média (Davies; Bouldin, 1979).

- **Quanto menor, melhor** (zero é o ideal).

7.3.5 Índice de Dunn

Razão entre a menor distância entre grupos e o maior diâmetro de um grupo (Dunn, 1974). **Quanto maior, melhor.** É muito sensível a outliers.

Índice	Melhor valor	O que mede	Limitação
Silhueta	Alto (→ 1)	Encaixe de cada ponto	Favorece grupos convexos
Calinski-Harabasz	Alto	Razão entre/dentro	Favorece grupos esféricos
Davies-Bouldin	Baixo (→ 0)	Similaridade entre grupos	Favorece grupos esféricos
Dunn	Alto	Separação/diâmetro	Sensível a outliers

> **ATENÇÃO — Índices internos têm viés**
>
> Quase todos presumem grupos **convexos e compactos**. Eles podem punir injustamente o resultado correto do DBSCAN em luas ou anéis. Para métodos baseados em densidade, o índice **DBCV** (Moulavi et al., 2014) é mais adequado.

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import calinski_harabasz_score, davies_bouldin_score, silhouette_score

X, _ = make_blobs(n_samples=500, centers=4, random_state=8)
print(" k  silhueta   CH       DB")
for k in range(2, 8):
    r = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(X)
    sil = silhouette_score(X, r)
    ch = calinski_harabasz_score(X, r)
    db = davies_bouldin_score(X, r)
    print(f"{k:2d}  {sil:7.3f}  {ch:8.1f}  {db:6.3f}")
```

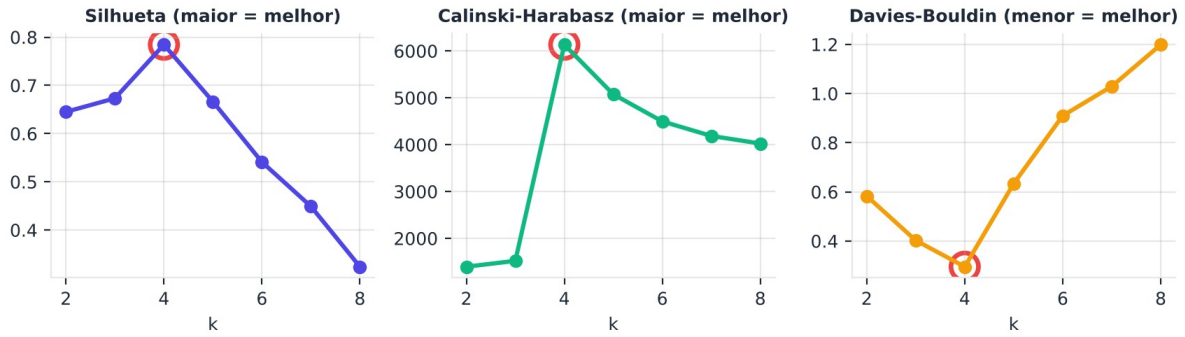


Figura 7.4 — Os três índices em função de k. Quando concordam (aqui, em k = 4), ganhamos confiança na escolha.

7.4 Escolhendo k de modo mais rigoroso

7.4.1 A estatística gap

Tibshirani, Walther e Hastie (2001) propuseram comparar a inércia observada com a esperada se os dados fossem **uniformes** (sem grupos):

$$\text{Gap}(k) = \text{E}[\log W_k \text{ (dados de referência)}] - \log W_k \text{ (dados observados)}$$

Escolhe-se o menor k tal que $\text{Gap}(k) \geq \text{Gap}(k+1) - \text{desvio padrão do Gap}(k+1)$.

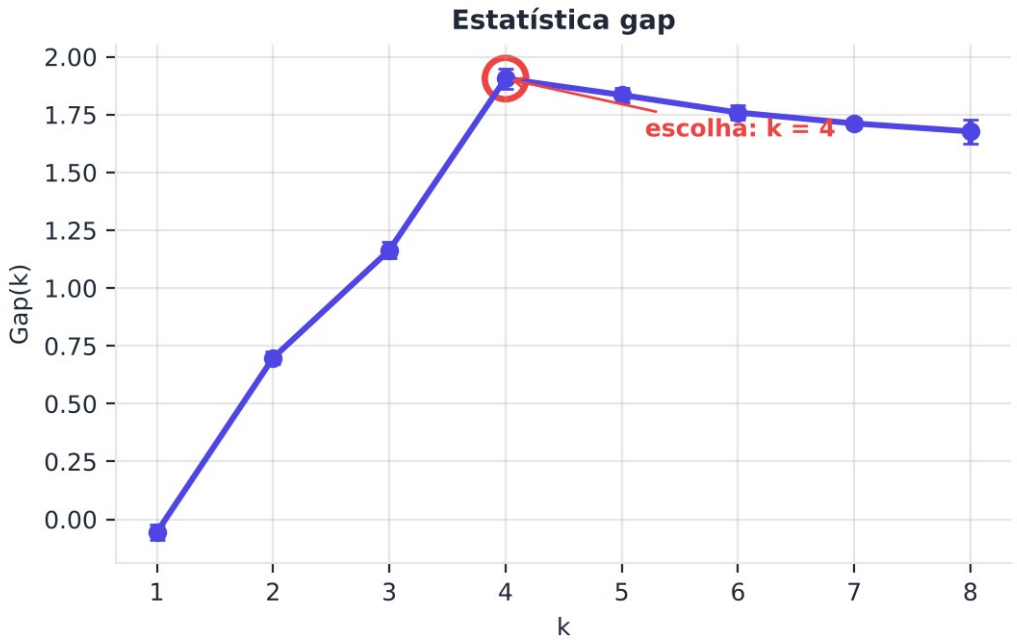


Figura 7.5 — Estatística gap: o primeiro k que satisfaz o critério é a escolha.

7.4.2 Estabilidade

Se os grupos são "reais", eles devem **resistir a pequenas perturbações**. Teste:

1. Sorteie várias subamostras (*bootstrap*).
2. Agrupe cada uma.
3. Compare os resultados entre si (com o ARI, a seguir).
4. Se a concordância é alta, o agrupamento é **estável** (Ben-Hur; Elisseeff; Guyon, 2002; Lange et al., 2004).

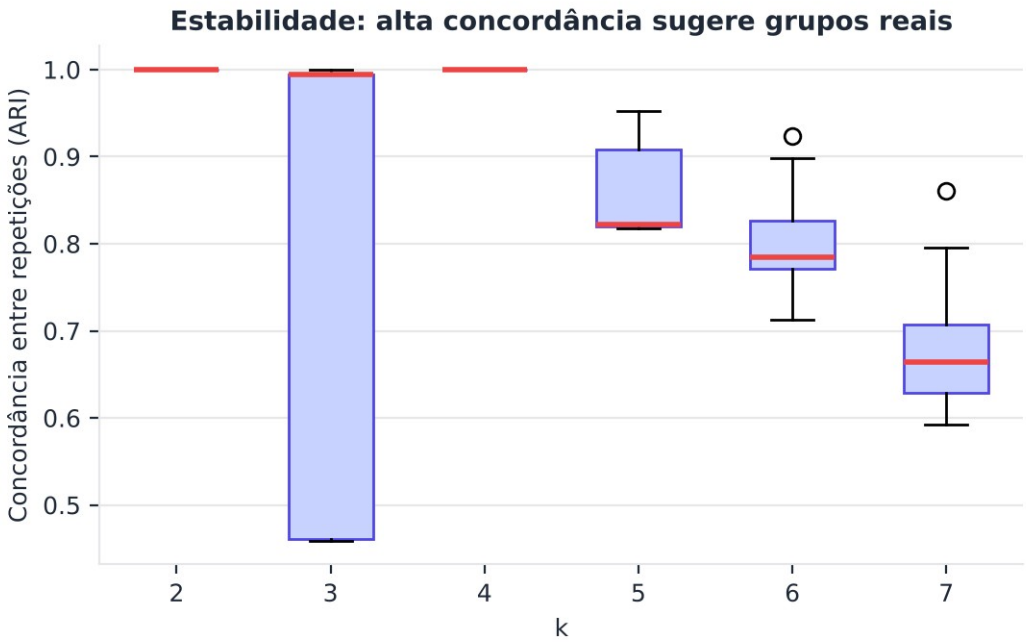


Figura 7.6 — Estabilidade: para o k correto, as repetições concordam entre si; para k errado, os resultados variam.

```python
import numpy as np
```

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

X, _ = make_blobs(n_samples=600, centers=4, random_state=8)
rng = np.random.default_rng(0)

def estabilidade(k, repeticoes=20):
    scores = []
    for _ in range(repeticoes):
        i1 = rng.choice(len(X), len(X) // 2, replace=False)
        i2 = rng.choice(len(X), len(X) // 2, replace=False)
        m1 = KMeans(k, n_init=5, random_state=0).fit(X[i1])
        m2 = KMeans(k, n_init=5, random_state=1).fit(X[i2])
        # Compara as duas soluções no conjunto inteiro
        scores.append(adjusted_rand_score(m1.predict(X), m2.predict(X)))
    return np.mean(scores)

for k in range(2, 8):
    print(f"k={k}  estabilidade (ARI médio) = {estabilidade(k):.3f}")
```

7.5 Validação externa: quando há um gabarito

Às vezes temos rótulos de referência, por exemplo em conjuntos de pesquisa, e queremos saber **o quanto o agrupamento os reproduz**. Atenção: os números dos grupos são arbitrários. "Grupo 0" pode ser a "Classe B". Por isso as medidas devem ser **invariantes à troca de nomes**.

7.5.1 Matriz de contingência

É uma tabela que cruza grupos (linhas) e classes (colunas), contando quantos pontos de cada classe caíram em cada grupo.

	Classe A	Classe B	Classe C
Grupo 0	48	2	0
Grupo 1	1	45	6
Grupo 2	1	3	44

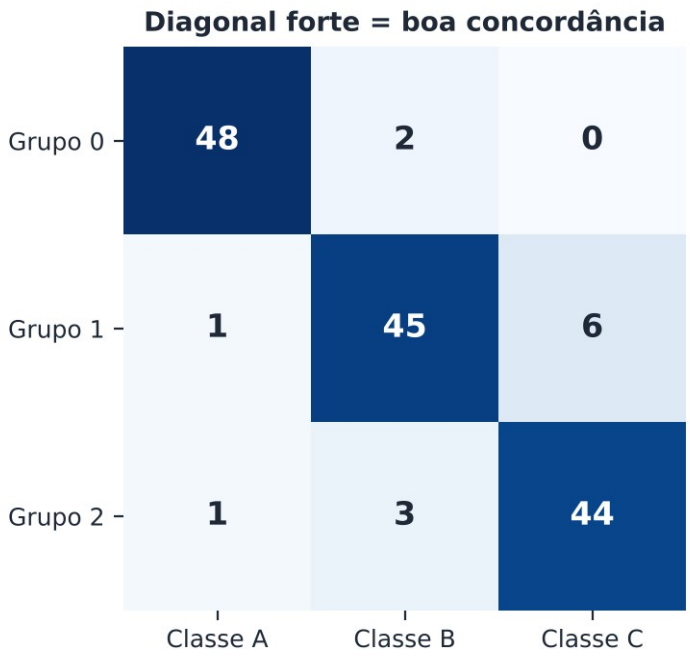


Figura 7.7 — Matriz de contingência como mapa de calor: uma diagonal forte indica boa concordância.

7.5.2 Pureza

Para cada grupo, conte a classe majoritária. Some e divida pelo total. Vai de 0 a 1. Defeito: cresce com o número de grupos (com *n* grupos, a pureza é 1).

7.5.3 Índice de Rand e Rand ajustado (ARI)

O **Rand** conta pares de pontos em que os dois agrupamentos **concordam** (juntos nos dois, ou separados nos dois). O **Rand ajustado (ARI)** desconta a concordância esperada ao acaso (Hubert; Arabie, 1985).

- **ARI = 1:** concordância perfeita.
- **ARI ≈ 0:** concordância ao nível do acaso.
- **ARI < 0:** pior que o acaso.

7.5.4 Informação mútua normalizada (NMI)

Vem da teoria da informação. Mede quanta informação sobre um agrupamento obtemos ao conhecer o outro, normalizada entre 0 e 1 (Strehl; Ghosh, 2002; Vinh; Epps; Bailey, 2010).

7.5.5 Homogeneidade, completude e V-measure

- **Homogeneidade:** cada grupo contém membros de uma única classe.
- **Completude:** todos os membros de uma classe estão no mesmo grupo.
- **V-measure:** média harmônica das duas (Rosenberg; Hirschberg, 2007).

```python
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import (adjusted_rand_score, normalized_mutual_info_score,
                             homogeneity_completeness_v_measure)

X, y = load_iris(return_X_y=True)
```

```
for nome, modelo in [("K-means", KMeans(3, n_init=10, random_state=0)),
                     ("Ward   ", AgglomerativeClustering(3, linkage="ward"))]:
    r = modelo.fit_predict(X)
    h, c, v = homogeneity_completeness_v_measure(y, r)
    print(f"{nome} ARI={adjusted_rand_score(y, r):.3f}  "
          f"NMI={normalized_mutual_info_score(y, r):.3f}  "
          f"H={h:.3f} C={c:.3f} V={v:.3f}")
```

SAIBA MAIS **Um aviso de Von Luxburg e colegas**

Von Luxburg, Williamson e Guyon (2012) lembram que "clustering" não tem uma definição única. A melhor solução depende do **objetivo** de quem agrupa. Rótulos de referência nem sempre representam o agrupamento "correto"; apenas um deles possível.

7.6 Validação pelo uso e pela interpretação

Nenhum número substitui estas perguntas:

- Os grupos fazem **sentido para o especialista**?
- São **acionáveis**? Cada grupo recebe uma ação diferente?
- Têm **tamanho útil**? Um grupo de 3 clientes dificilmente justifica uma campanha.
- São **estáveis** no tempo e entre amostras?
- Diferem de modo **relevante** nas variáveis de negócio (que não entraram no agrupamento)?

7.6.1 Perfil dos grupos

Uma boa prática é construir uma **tabela de perfil**: a média (ou mediana) de cada atributo, por grupo, e compará-la com a média geral. Gráficos de radar e de barras ajudam a contar a história.

PYTHON

```python
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

dados = load_wine(as_frame=True)
X = dados.data[["alcohol", "malic_acid", "flavanoids", "color_intensity"]]
grupos = KMeans(3, n_init=10, random_state=0).fit_predict(StandardScaler().fit_transform(X))

perfil = X.assign(grupo=grupos).groupby("grupo").mean()
relativo = (perfil / X.mean()).round(2)      # >1: acima da média geral
print(relativo)
```

7.7 Um roteiro prático de avaliação

1. **Hopkins:** há tendência de agrupamento?
2. **Varie *k*** (ou os parâmetros) e calcule silhueta, CH, DB e gap.
3. **Teste a estabilidade** com reamostragens.
4. **Compare algoritmos** (K-means, hierárquico, GMM, DBSCAN).
5. **Visualize** (Capítulos 8 e 9).

6. **Perfile** os grupos e valide com especialistas.
7. **Documente** as escolhas e os critérios.

> DICA **Se os critérios discordarem**
>
> É normal. Prefira o *k* em que **vários** critérios concordam, a solução **estável** e a que tenha **melhor interpretação**.

Resumo do capítulo

> MEMORIZE **Cartões de memorização**
>
- **Sem gabarito**, avaliar exige critérios internos, relativos e externos.
- **Hopkins:** os dados têm grupos para encontrar?
- **Silhueta** (−1 a 1): quanto maior, melhor encaixe.
- **Calinski-Harabasz:** maior é melhor. **Davies-Bouldin:** menor é melhor.
- **Gap** e **estabilidade** ajudam a escolher *k*.
- **ARI e NMI** comparam com rótulos de referência e ignoram os nomes dos grupos.
- **Interpretação e utilidade** têm a palavra final.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Por que avaliar agrupamentos é difícil?

a) Porque os algoritmos nunca retornam respostas.
b) Porque só existe uma forma de agrupar.
c) Porque não há um gabarito oficial.
d) Porque os dados são sempre rotulados.

Questão 2. A validação interna utiliza:

a) a opinião de um juiz externo.
b) recompensas.
c) apenas os dados e os grupos obtidos.
d) rótulos de referência.

Questão 3. A validação externa compara os grupos com:

a) o dendrograma.
b) o número de iterações.
c) a inércia.
d) rótulos de referência conhecidos.

Questão 4. A validação relativa:

a) usa só rótulos.
b) elimina atributos.
c) compara várias soluções (por exemplo, vários k) para escolher a melhor.
d) é sempre subjetiva.

Questão 5. A estatística de Hopkins testa:

a) a tendência de agrupamento dos dados.
b) a pureza.
c) a estabilidade.
d) a qualidade do dendrograma.

Questão 6. Uma estatística de Hopkins próxima de 0,5 sugere dados:

a) fortemente agrupados.
b) aproximadamente uniformes, sem grupos.
c) muito regulares como uma grade.
d) sem variação.

Questão 7. Um valor de Hopkins próximo de 1 indica:

a) forte tendência de agrupamento.
b) dados uniformes.
c) ruído puro.
d) dados constantes.

Questão 8. Na silhueta de um ponto, a(i) representa:

a) a distância média aos pontos do mesmo grupo (coesão).
b) a distância ao grupo mais distante.
c) a inércia total.
d) o número de grupos.

Questão 9. A fórmula da silhueta é:

a) s = a + b.
b) s = a / b.
c) s = (b − a) / máx(a, b).
d) s = máx(a, b).

Questão 10. Um coeficiente de silhueta negativo para um ponto indica que ele:

a) é ruído obrigatório.
b) provavelmente está no grupo errado.
c) está muito bem encaixado.
d) é o centroide.

Questão 11. Silhuetas médias acima de 0,5 costumam indicar:

a) erro de cálculo.
b) ausência de estrutura.
c) estrutura razoável.
d) dados idênticos.

Questão 12. O índice de Calinski-Harabasz é:

a) sempre igual a 1.
b) melhor quando maior.
c) melhor quando menor.
d) melhor quando negativo.

Questão 13. O índice de Davies-Bouldin é melhor quando:

a) maior.
b) negativo.
c) igual a 1.
d) menor (zero é o ideal).

Questão 14. O índice de Dunn é a razão entre:

a) inércia e silhueta.
b) k e n.
c) média e mediana.
d) a menor distância entre grupos e o maior diâmetro de grupo.

Questão 15. Os índices internos têm um viés comum:

a) exigem rótulos.
b) favorecem grupos convexos e compactos.
c) favorecem grupos de formas arbitrárias.
d) ignoram as distâncias.

Questão 16. Para avaliar agrupamentos baseados em densidade, é mais adequado o índice:

a) BIC.
b) DBCV.
c) Calinski-Harabasz.
d) Dunn.

Questão 17. A estatística gap compara a inércia observada com a esperada:

a) em dados ordenados.
b) em dados de referência uniformes, sem grupos.
c) em dados rotulados.
d) em dados duplicados.

Questão 18. A estabilidade de um agrupamento é avaliada:

a) pelo tamanho da imagem.
b) pelo número de iterações.
c) comparando soluções obtidas em reamostragens dos dados.
d) pela altura do dendrograma.

Questão 19. Para comparar duas soluções de agrupamento na estabilidade, usa-se com frequência:

a) a distância de Hamming.
b) o ARI.
c) o z-score.
d) o eigengap.

Questão 20. Em validação externa, as medidas devem ser invariantes:

a) ao idioma.
b) ao tipo de computador.
c) ao número de pontos.
d) à troca dos nomes dos grupos.

Questão 21. Uma matriz de contingência cruza:

a) grupos (linhas) e classes de referência (colunas).
b) atributos e exemplos.
c) k e inércia.
d) distâncias e ligações.

Questão 22. A pureza tem como defeito:

a) ser sempre zero.
b) depender do idioma.
c) crescer com o número de grupos (com n grupos, é 1).
d) ser sempre negativa.

Questão 23. O índice de Rand ajustado (ARI) vale aproximadamente zero quando:

a) os grupos são idênticos.
b) há um único grupo.
c) a concordância é perfeita.
d) a concordância é ao nível do acaso.

Questão 24. Um ARI igual a 1 indica:

a) desacordo total.
b) estrutura fraca.
c) concordância perfeita entre os dois agrupamentos.
d) pior que o acaso.

Questão 25. A informação mútua normalizada (NMI) vem:

a) da física de partículas.
b) da teoria da informação.
c) da geometria euclidiana.
d) da álgebra de matrizes esparsas.

Questão 26. A V-measure é:

a) o logaritmo da inércia.
b) o produto de ARI e NMI.
c) a média harmônica de homogeneidade e completude.
d) a diferença entre silhueta e Davies-Bouldin.

Questão 27. Homogeneidade alta significa que:

a) todos os membros de uma classe estão no mesmo grupo.
b) cada grupo contém membros de uma única classe.
c) todos os pontos são ruído.
d) há um único grupo.

Questão 28. Segundo von Luxburg, Williamson e Guyon (2012), a melhor solução de agrupamento depende:

a) apenas dos dados.
b) apenas do algoritmo.
c) do tamanho da tela.
d) do objetivo de quem agrupa.

Questão 29. Uma tabela de perfil dos grupos mostra:

a) a matriz de distâncias.
b) as probabilidades do EM.
c) o dendrograma.
d) a média (ou mediana) de cada atributo por grupo, comparada à média geral.

Questão 30. Qual conduta é recomendada quando os critérios de avaliação discordam entre si?

a) Escolher aleatoriamente.
b) Preferir o k em que vários critérios concordam, a solução estável e a mais interpretável.
c) Escolher sempre o menor k.
d) Escolher o maior k.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique por que um algoritmo de agrupamento sempre devolve uma resposta, mesmo com dados aleatórios, e o que isso implica.
2. Descreva as três famílias de validação (interna, externa e relativa) com um exemplo de índice de cada.
3. Explique a estatística de Hopkins e como interpretá-la.
4. Calcule a silhueta de um ponto com a = 2 e b = 6 e interprete.
5. Explique por que um ponto com s próximo de 0 está na fronteira entre grupos.
6. Compare silhueta, Calinski-Harabasz e Davies-Bouldin: o que medem, qual valor é melhor e que viés compartilham.
7. Por que índices internos podem penalizar injustamente o resultado correto do DBSCAN em luas?
8. Descreva a estatística gap e o critério para escolher k.
9. Descreva um procedimento de avaliação de estabilidade com bootstrap e ARI.
10. Interprete a matriz de contingência [[48,2,0],[1,45,6],[1,3,44]]. O agrupamento reproduz bem as classes?
11. Calcule a pureza do agrupamento da matriz de contingência acima.
12. Explique o ARI e por que é preferível ao índice de Rand simples.
13. Diferencie homogeneidade, completude e V-measure.
14. Dê três perguntas de validação pelo uso e pela interpretação dos grupos.
15. Por que perfilar os grupos com a razão (média do grupo)/(média geral) é útil?
16. Descreva um roteiro de avaliação com sete passos.
17. Por que, em validação externa, os rótulos de referência nem sempre são o agrupamento "correto"?
18. Dados os índices silhueta 0,62 (k = 3), 0,71 (k = 4) e 0,55 (k = 5), qual k seria sugerido e que outros passos você faria antes de decidir?
19. Explique por que usar o gap em vez de apenas o cotovelo pode ser mais objetivo para escolher k.
20. Explique a diferença entre coesão e separação e use a analogia da festa para ilustrá-la.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Há grupos para encontrar? A estatística de Hopkins

NÍVEL: BÁSICO

Implemente a estatística de Hopkins (use `NearestNeighbors`) e aplique-a a três conjuntos: dados uniformes (500 × 2), dados com 4 blobs e o Wine padronizado. Interprete os valores.

Exercício 2 — Silhueta por grupo e por ponto

NÍVEL: BÁSICO

No Wine padronizado, agrupe com K-means (k = 3). Calcule a silhueta de cada ponto (`silhouette_samples`). Mostre a silhueta média por grupo e o número de pontos com silhueta negativa.

Exercício 3 — Um painel de índices internos

NÍVEL: INTERMEDIÁRIO

No Wine padronizado, para k de 2 a 8, calcule a silhueta, o índice de Calinski-Harabasz e o de Davies-Bouldin para o K-means. Mostre uma tabela e indique o k preferido por cada índice. Eles concordam?

Exercício 4 — Avaliação externa de três algoritmos

NÍVEL: INTERMEDIÁRIO

Nos dígitos (`load_digits`), reduza a 20 componentes com PCA e agrupe em 10 grupos com K-means, Ward e GMM. Compare ARI, NMI, homogeneidade, completude e V-measure.

Exercício 5 — Estabilidade para escolher k

NÍVEL: AVANÇADO

Em `make_blobs(600, centers=4, random_state=8)`, para k de 2 a 7, repita 15 vezes: sorteie duas subamostras de 300 pontos, agrupe cada uma, e compare as duas soluções no conjunto inteiro com o ARI. Mostre o ARI médio e o desvio por k. Qual k é mais estável?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Seeds, Wine Quality, Dry Bean e Optical Recognition of Handwritten Digits (versão completa).
> - **Kaggle:** Mall Customers.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Painel de avaliação de agrupamentos

Fonte dos dados. Três conjuntos do repositório **UCI Machine Learning** incluídos no scikit-learn: Wine, Optical Digits e Iris.

Contexto. A sua equipe vai agrupar três conjuntos de dados com vários algoritmos e precisa de um **procedimento padronizado e justo de avaliação**, que não dependa de um único número.

Tarefas.

1. Escreva uma função `avaliar(X, modelos, ks)` que, para cada algoritmo (K-means, Ward, GMM) e cada k, calcule: silhueta, Calinski-Harabasz, Davies-Bouldin e **estabilidade** (ARI médio em 10 pares de subamostras de 70%).
2. Aplique a função ao **Wine** (padronizado), aos **dígitos** (PCA com 20 componentes) e ao **Iris** (padronizado), com k de 2 a 6.
3. Calcule a estatística de Hopkins de cada conjunto antes de agrupar.
4. Crie um **ranking combinado**: para cada linha, some os postos (ranks) dos quatro critérios (para DB, menor é melhor). Indique a melhor combinação algoritmo/k de cada conjunto.
5. Compare a recomendação do ranking com o ARI contra os rótulos reais (usados só no final).
6. Escreva um parágrafo de recomendação: o que você diria a um colega que só quer "o melhor k"?

Entregáveis. Função, tabelas por conjunto, ranking e parágrafo.

A solução de referência está no Apêndice E.

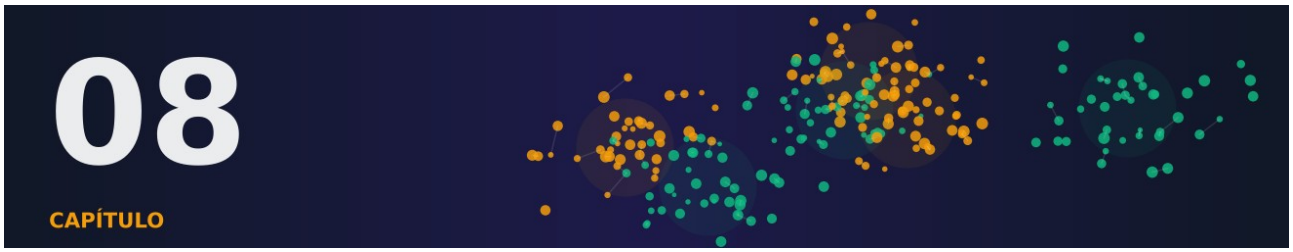
PARTE III

Parte III — Redução de Dimensionalidade

Como resumir e visualizar dados com muitos atributos: métodos lineares (PCA, ICA, NMF) e o aprendizado de variedades (MDS, Isomap, t-SNE, UMAP).

Nesta parte:

- Cap. 8 — PCA e Métodos Lineares: SVD, ICA e NMF
- Cap. 9 — Aprendizado de Variedades: MDS, Isomap, t-SNE e UMAP



Capítulo 8 — PCA e Métodos Lineares: SVD, ICA e NMF

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que reduzir dimensionalidade e a diferença entre seleção e extração de atributos.
- Compreender o PCA geometricamente: direções de máxima variância.
- Calcular o PCA via matriz de covariância e via SVD.
- Escolher o número de componentes com a variância explicada e o gráfico *scree*.
- Interpretar cargas, escores e biplots.
- Descrever PCA incremental, PCA com kernel, ICA e NMF e saber quando usá-los.

8.1 Por que reduzir dimensões?

Uma foto 28 × 28 tem 784 números. Um genoma tem milhares de genes. Um texto pode ter 50 mil palavras possíveis. São dados de **alta dimensão**.

Reduzir dimensões ajuda a:

1. **Visualizar.** Só enxergamos 2D ou 3D.
2. **Acelerar** algoritmos e economizar memória.
3. **Eliminar ruído e redundância.**
4. **Combater a maldição da dimensionalidade** (Capítulo 2).
5. **Entender** quais combinações de atributos importam.

8.1.1 Seleção versus extração

- **Seleção de atributos:** escolhe um subconjunto das colunas originais e descarta o resto. Mantém a interpretação.
- **Extração de atributos:** cria **novas colunas** como combinações das originais. É o caso do PCA.

Analogia: numa receita com 20 ingredientes, a **seleção** tira 8 ingredientes. A **extração** cria "misturas prontas" (ex.: "tempero base") que resumem vários ingredientes.

8.2 A intuição do PCA

A **Análise de Componentes Principais** (PCA) foi proposta por Pearson (1901) e desenvolvida por Hotelling (1933). Ainda hoje é a técnica de redução mais usada (Jolliffe; Cadima, 2016).

8.2.1 A analogia da sombra

Segure um objeto 3D diante de uma lâmpada. A sombra na parede é 2D. Dependendo do **ângulo**, a sombra revela muito ou pouco do objeto. Uma sombra de um lápis visto de frente é só um pontinho. De lado, mostra seu comprimento.

O PCA procura o **melhor ângulo de iluminação**: aquele cuja sombra preserva o máximo da informação do objeto.

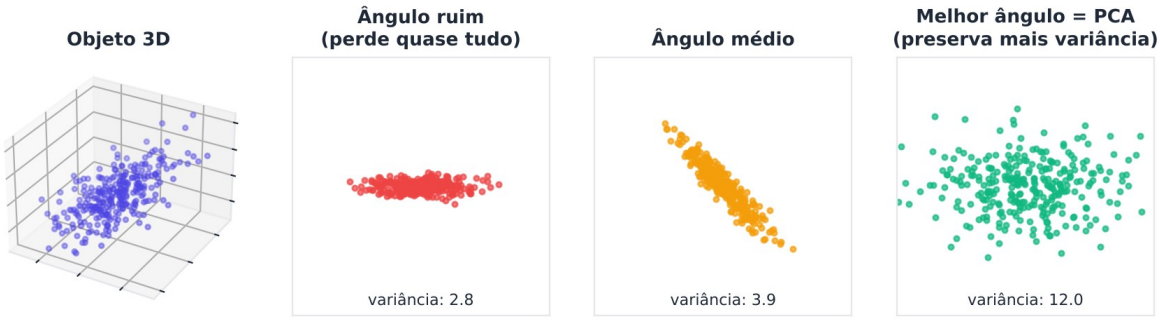


Figura 8.1 — Sombras: o mesmo objeto 3D projetado de ângulos diferentes. O PCA escolhe o ângulo que preserva mais informação.

8.2.2 Informação = variância

No PCA, "informação" significa **variância**: o quanto os dados se espalham. Um atributo que varia pouco quase não ajuda a distinguir exemplos. Um que varia muito, ajuda.

O PCA procura **direções** (eixos novos) nas quais os dados têm **máxima variância**:

- **PC1 (primeira componente principal):** a direção de maior variância.
- **PC2:** a direção de maior variância entre as **perpendiculares** a PC1.
- **PC3:** perpendicular às duas anteriores, e assim por diante.

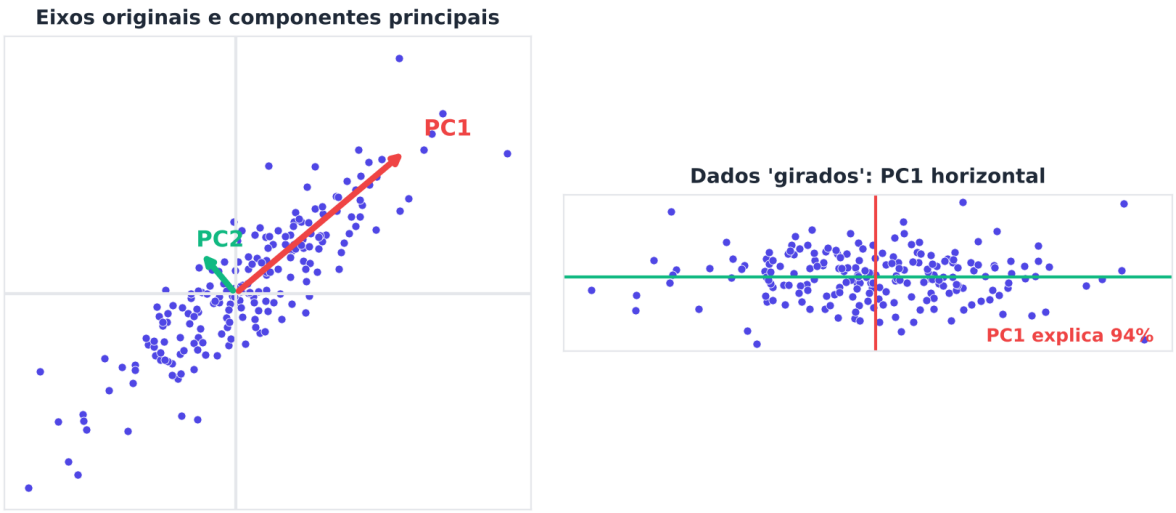


Figura 8.2 — PC1 aponta para onde a nuvem mais se estende. PC2 é perpendicular. Projetar nos eixos novos "gira" a nuvem.

> MEMORIZE **PCA em uma frase**
>
> Girar os eixos para alinhá-los às direções de maior espalhamento e ficar só com os primeiros.

8.3 Como o PCA funciona, passo a passo

1. **Centralize** os dados: subtraia a média de cada coluna. (Padronize também, se as escalas forem diferentes.)
2. **Calcule a matriz de covariância** C. Ela mostra como os atributos variam juntos.
3. **Calcule autovalores e autovetores** de C.

- Cada **autovetor** é uma direção (um eixo novo). - Cada **autovalor** é a variância dos dados ao longo daquele eixo.

1. **Ordene** os autovetores do maior para o menor autovalor.
2. **Escolha** os *k* primeiros: eles formam uma matriz W de projeção.
3. **Projete:** os novos dados são Z = X_centralizado × W.

8.3.1 Um exemplo mínimo

Dois atributos: altura e peso de 5 pessoas (já centralizados):

Pessoa	Altura	Peso
1	−2	−3
2	−1	−1
3	0	0
4	1	2
5	2	2

Os dados formam uma nuvem alongada, subindo da esquerda para a direita. O PC1 aponta nessa direção e combina altura e peso. O PC2 aponta na perpendicular e captura as pequenas diferenças restantes. Provavelmente PC1 explicará mais de 95 % da variância: **um único número** (a posição ao longo de PC1) resume bem cada pessoa.

8.4 A relação com a SVD

A **Decomposição em Valores Singulares** (SVD) fatora qualquer matriz X em três:

$$X = U \cdot S \cdot V^T$$

- **V:** as direções principais (autovetores da covariância).
- **S:** valores singulares (relacionados aos autovalores: autovalor = s^2 / (n − 1)).
- **U·S:** os escores (dados projetados).

Na prática, o scikit-learn calcula o PCA via SVD, que é mais estável e rápida. Um resultado clássico (Eckart; Young, 1936) garante que truncar a SVD nos *k* maiores valores singulares dá a **melhor aproximação de posto *k*** da matriz, no sentido dos mínimos quadrados.

8.5 PCA no Python

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
```

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X, y = load_iris(return_X_y=True)
Xp = StandardScaler().fit_transform(X)

pca = PCA(n_components=4).fit(Xp)

print("Variância explicada   :", pca.explained_variance_ratio_.round(3))
print("Variância acumulada   :", pca.explained_variance_ratio_.cumsum().round(3))
print("Cargas (componentes):\n", pca.components_.round(2))

Z = PCA(n_components=2).fit_transform(Xp)
plt.scatter(Z[:, 0], Z[:, 1], c=y)
plt.xlabel("PC1"); plt.ylabel("PC2"); plt.title("Iris em 2 componentes")
plt.show()
```

Como ler a saída:

- `explained_variance_ratio_`: a fração da variância total que cada componente explica.
- `components_`: cada linha é uma componente, com **cargas** (*loadings*), o peso de cada atributo original.
- No Iris, as duas primeiras componentes retêm cerca de 96 % da variância.

8.6 Quantas componentes manter?

8.6.1 Variância acumulada

Some a variância explicada das componentes em ordem. Escolha o menor número que atinja um limiar, como **90 % ou 95 %**.

```python
from sklearn.decomposition import PCA
# Pede ao PCA o nº de componentes suficiente para 95% da variância
pca95 = PCA(n_components=0.95)
```

8.6.2 Gráfico scree (de declive)

Desenhe a variância de cada componente. Procure o "cotovelo": a partir dele, as componentes acrescentam pouco (Cattell, 1966).

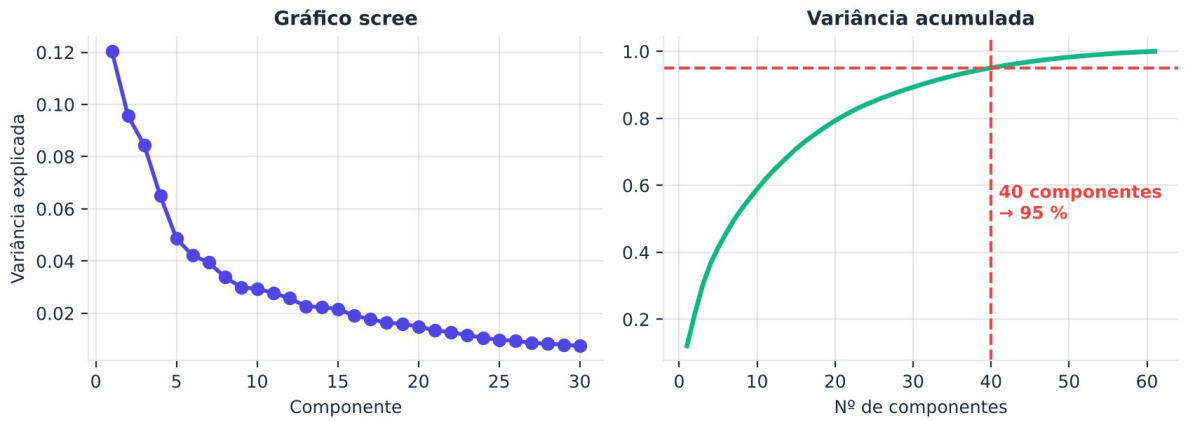


Figura 8.3 — Gráfico scree e variância acumulada: o cotovelo e a linha de 95 %.

8.6.3 Critério de Kaiser

Com dados padronizados, mantenha componentes com autovalor maior que 1: cada uma explica mais que um atributo original sozinho (Kaiser, 1960). Regra simples, mas discutível.

8.6.4 Validação cruzada e análise paralela

A **análise paralela** (Horn, 1965) compara os autovalores reais com os obtidos em dados aleatórios de mesmo tamanho. Mantém-se apenas o que supera o acaso. É considerada mais confiável que Kaiser.

8.7 Interpretando o resultado

8.7.1 Escores e cargas

- **Escores (*scores*):** as novas coordenadas de cada exemplo.
- **Cargas (*loadings*):** o quanto cada atributo original contribui para cada componente.

8.7.2 O biplot

O **biplot** (Gabriel, 1971) mostra num só gráfico os exemplos (pontos) e os atributos (setas). Setas que apontam para a mesma direção estão correlacionadas. Setas perpendiculares são independentes. Setas opostas, anticorrelacionadas.

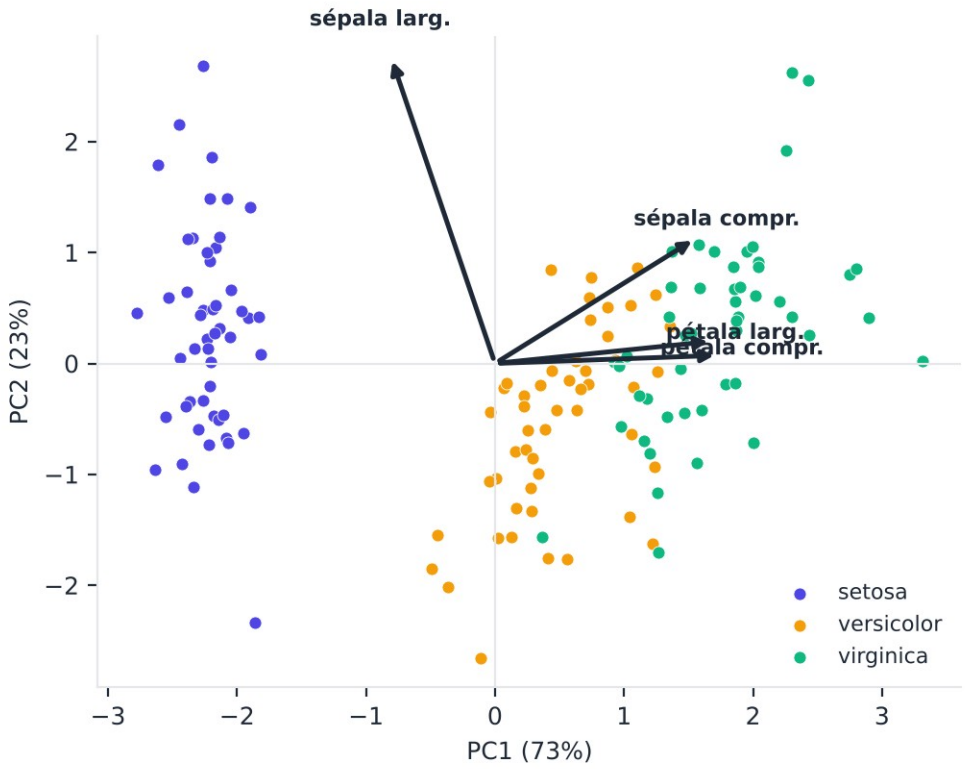


Figura 8.4 — Biplot do conjunto Iris: pontos são flores; setas, os atributos.

> **ATENÇÃO Cuidado**
>
> Os sinais das componentes são arbitrários: multiplicar uma componente por −1 não muda nada. E as componentes são combinações de *todos* os atributos, o que dificulta a explicação para leigos.

8.8 PCA para reconstruir e comprimir

Podemos voltar ao espaço original com `inverse_transform`. A imagem reconstruída com poucas componentes perde detalhes, mas guarda o essencial.

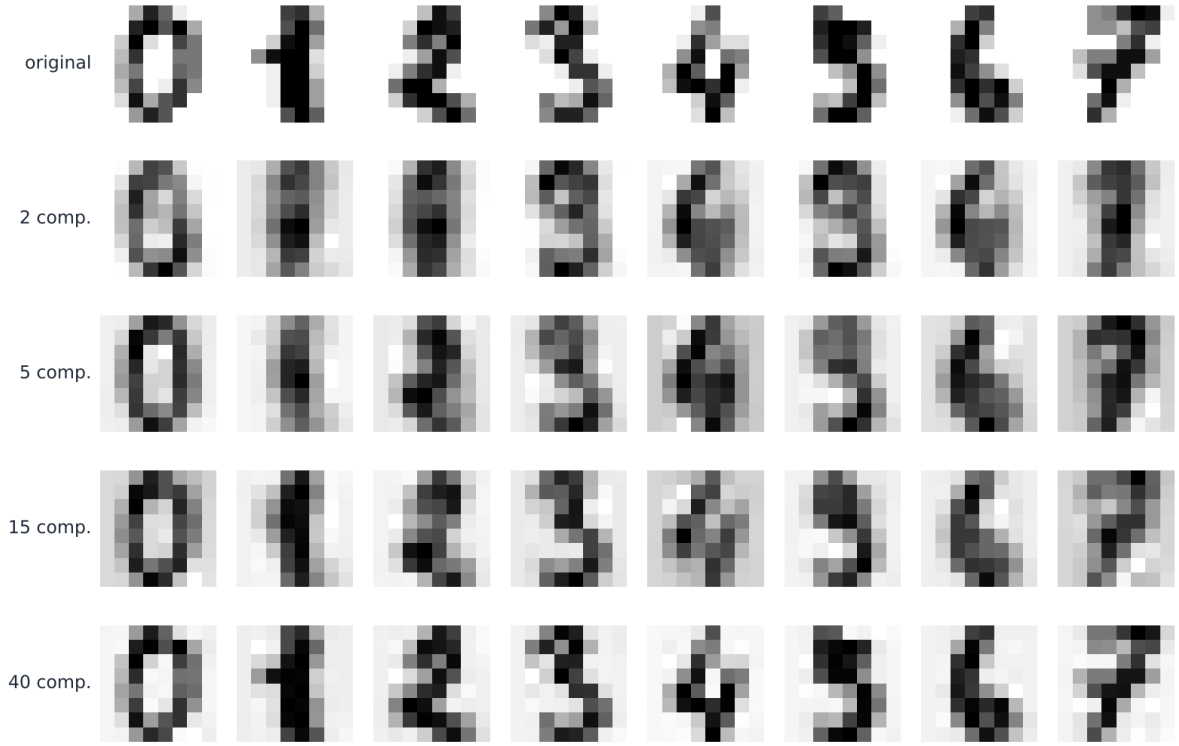


Figura 8.5 — Dígitos manuscritos reconstruídos com 2, 5, 15 e 40 componentes. Com 15 já reconhecemos os números.

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

X, y = load_digits(return_X_y=True)          # 1797 imagens de 8x8 = 64 atributos
print("Original:", X.shape)

for k in [2, 5, 15, 40]:
    pca = PCA(n_components=k).fit(X)
    Xr = pca.inverse_transform(pca.transform(X))
    erro = np.mean((X - Xr) ** 2)
    var = pca.explained_variance_ratio_.sum()
    print(f"{k:3d} componentes -> variância {var:.2%}  erro quadrático médio {erro:.2f}")
```

8.8.1 Remoção de ruído

Se o ruído é aleatório e a informação está concentrada em poucas direções, descartar as componentes de baixa variância **limpa** a imagem.

8.9 Pontos de atenção do PCA

- **Linear:** captura apenas relações lineares. Para estruturas curvas, veja o Capítulo 9.
- **Sensível à escala:** padronize.
- **Sensível a outliers:** eles puxam as direções. Há versões robustas (*Robust PCA*).

- **Variância ≠ importância:** uma direção de pouca variância pode separar classes importantes.
- **Sem rótulos:** não sabe nada sobre classes.

8.10 Variações do PCA

8.10.1 PCA incremental

Para bases que não cabem na memória, processa os dados em lotes.

```python
import numpy as np
from sklearn.decomposition import IncrementalPCA

rng = np.random.default_rng(0)
ipca = IncrementalPCA(n_components=5, batch_size=500)
for _ in range(10):                      # simula 10 lotes lidos do disco
    lote = rng.normal(size=(500, 50))
    ipca.partial_fit(lote)
print(ipca.explained_variance_ratio_.round(3))
```

8.10.2 PCA aleatorizado

Para matrizes enormes, algoritmos aleatórios aproximam a SVD com grande economia de tempo (Halko; Martinsson; Tropp, 2011). No scikit-learn: `PCA(svd_solver="randomized")`.

8.10.3 PCA esparso

Força muitas cargas a serem **zero**, de modo que cada componente envolva poucos atributos. Fica mais interpretável (Zou; Hastie; Tibshirani, 2006).

8.10.4 PCA com kernel

O **Kernel PCA** (Schölkopf; Smola; Müller, 1998) usa uma função *kernel* para fazer, implicitamente, o PCA num espaço de dimensão muito maior, onde relações não lineares viram lineares. Funciona bem para anéis e espirais.

```python
import numpy as np
from sklearn.datasets import make_circles
from sklearn.decomposition import PCA, KernelPCA

X, y = make_circles(n_samples=400, factor=0.3, noise=0.03, random_state=0)
Z_lin = PCA(2).fit_transform(X)
Z_rbf = KernelPCA(n_components=2, kernel="rbf", gamma=10).fit_transform(X)

# No PCA linear os dois anéis continuam misturados na 1ª componente;
# no Kernel PCA a 1ª componente já separa os anéis.
lin_int, lin_ext = Z_lin[y == 1, 0].mean(), Z_lin[y == 0, 0].mean()
rbf_int, rbf_ext = Z_rbf[y == 1, 0].mean(), Z_rbf[y == 0, 0].mean()
print("PC1 linear: anel interno", round(lin_int, 2), "| externo", round(lin_ext, 2))
print("PC1 kernel: anel interno", round(rbf_int, 2), "| externo", round(rbf_ext, 2))
```

8.11 Análise de Componentes Independentes (ICA)

8.11.1 O problema da festa (cocktail party)

Numa festa, dois microfones gravam a mistura de duas pessoas falando ao mesmo tempo. Como **separar** cada voz sem saber como a mistura foi feita? Esse é o problema da **separação cega de fontes**.

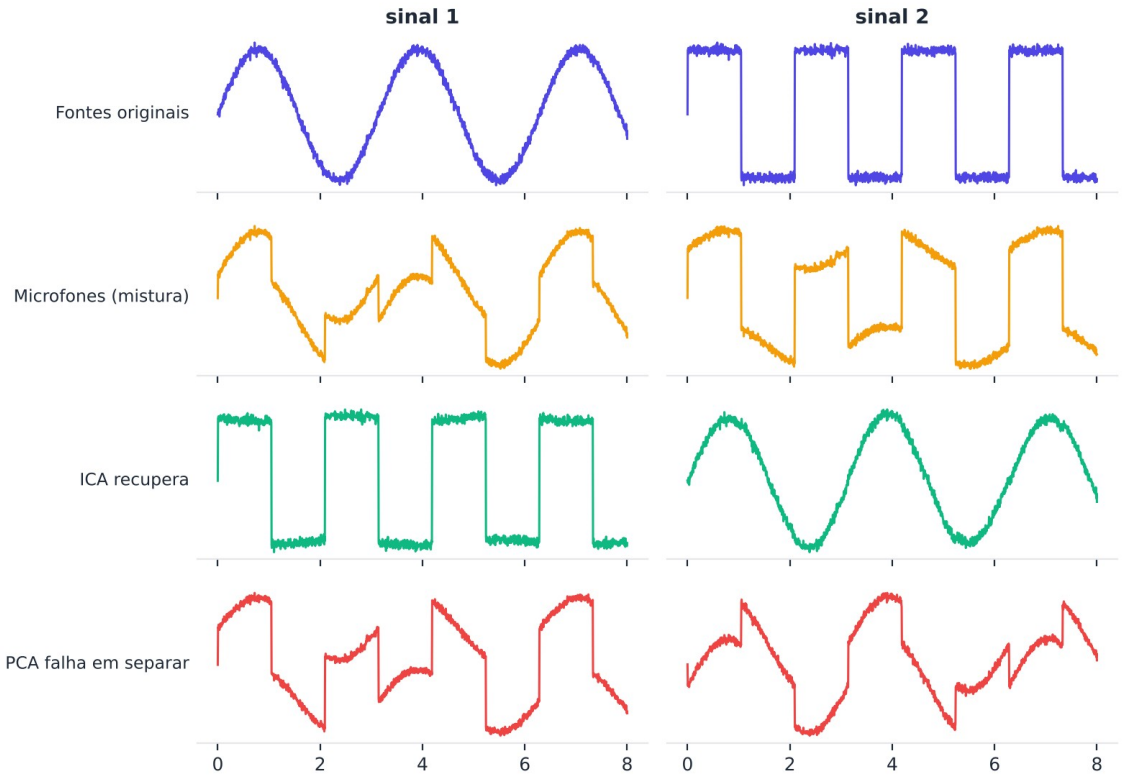


Figura 8.6 — Cocktail party: duas fontes independentes são misturadas em dois microfones. O ICA recupera as fontes originais; o PCA não.

8.11.2 A ideia

- **PCA** busca componentes **não correlacionadas**.
- **ICA** busca componentes **estatisticamente independentes**, um critério mais forte (Hyvärinen; Oja, 2000).

O ICA explora o fato de que sinais independentes e não gaussianos, quando somados, ficam "mais gaussianos" (pelo Teorema Central do Limite). O algoritmo procura direções que **maximizam a não gaussianidade**. O algoritmo FastICA é o mais popular.

```python
import numpy as np
from sklearn.decomposition import FastICA, PCA

t = np.linspace(0, 8, 2000)
s1 = np.sin(2 * t)                      # fonte 1: senoide
s2 = np.sign(np.sin(3 * t))             # fonte 2: onda quadrada
S = np.c_[s1, s2] + 0.05 * np.random.default_rng(0).normal(size=(2000, 2))

A = np.array([[1.0, 0.5], [0.4, 1.0]])   # mistura desconhecida
X = S @ A.T                              # sinais "gravados"

S_ica = FastICA(n_components=2, random_state=0).fit_transform(X)
S_pca = PCA(n_components=2).fit_transform(X)
```

```
# Correlação absoluta entre cada fonte original e cada componente recuperada
def melhor(S, R):
    c = np.abs(np.corrcoef(S.T, R.T)[:2, 2:])
    return c.max(axis=1).round(2)

print("ICA recupera as fontes? ", melhor(S, S_ica))
print("PCA recupera as fontes? ", melhor(S, S_pca))
```

Aplicações: separação de vozes, remoção de artefatos (piscadas) em eletroencefalogramas e análise de imagens cerebrais.

8.12 Fatoração de Matrizes Não Negativas (NMF)

8.12.1 A ideia: montar com peças

Muitos dados são naturalmente **não negativos**: pixels, contagens de palavras, notas de produtos. O **NMF** decompõe uma matriz X (não negativa) como o produto de duas matrizes também não negativas (Lee; Seung, 1999):

$$X \approx W \cdot H$$

- **H:** as "peças" básicas (partes de rostos, tópicos de texto).
- **W:** quanto de cada peça cada exemplo usa.

Como tudo é **aditivo** (não há subtração), as peças tendem a ser **partes** interpretáveis. Lee e Seung (1999) mostraram que, ao aplicar o NMF a rostos, as peças aprendidas eram olhos, narizes e bocas, enquanto o PCA gerava "rostos fantasmas" globais.

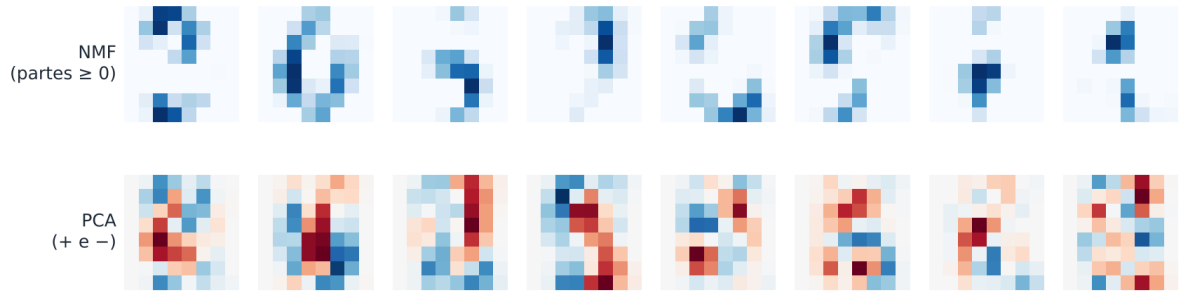


Figura 8.7 — NMF aprende partes (traços de dígitos); o PCA aprende padrões globais com valores positivos e negativos.

Analogia: o NMF é como montar um desenho com adesivos. Você só pode **adicionar** adesivos, nunca recortar.

8.12.2 Aplicação em texto: descoberta de tópicos

Se X conta palavras em documentos, cada linha de H é um **tópico** (um conjunto de palavras que aparecem juntas) e W indica o quanto cada documento fala de cada tópico.

```
PYTHON
from sklearn.decomposition import NMF
from sklearn.feature_extraction.text import TfidfVectorizer

docs = [
    "o time venceu o jogo com gol no final",
    "o campeonato de futebol tem jogo decisivo",
```

```
    "o banco aumentou a taxa de juros",
    "a inflação e os juros preocupam o mercado",
    "o atacante marcou gol e o time venceu",
    "o mercado reage ao aumento dos juros e da inflação",
]
vet = TfidfVectorizer()
X = vet.fit_transform(docs)

nmf = NMF(n_components=2, init="nndsvda", random_state=0, max_iter=500)
W = nmf.fit_transform(X)
H = nmf.components_

palavras = vet.get_feature_names_out()
for i, topico in enumerate(H):
    top = topico.argsort()[::-1][:5]
    print(f"Tópico {i}:", [palavras[j] for j in top])
```

8.13 Comparação das técnicas lineares

Técnica	Objetivo	Restrição	Melhor para
PCA	Máxima variância	Eixos ortogonais	Resumo, visualização, compressão
SVD truncada	Aproximação de posto baixo	Sem centralização	Matrizes esparsas e texto (LSA)
ICA	Independência estatística	Fontes não gaussianas	Separação de sinais
NMF	Partes aditivas	Valores ≥ 0	Imagens, texto, tópicos
Kernel PCA	PCA não linear	Escolha do kernel	Formas curvas

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **PCA:** gira os eixos para as direções de máxima variância.
- **Passos:** centralizar → covariância → autovetores → ordenar → projetar.
- **Escolha de k:** variância acumulada (90–95 %), scree, análise paralela.
- **Cargas** mostram o peso dos atributos; **escores** são as novas coordenadas.
- **SVD** é a forma numérica estável de calcular o PCA.
- **ICA:** fontes independentes. **NMF:** partes aditivas.
- O PCA é linear; para curvas, use Kernel PCA ou variedades.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual é uma diferença entre seleção e extração de atributos?

a) A seleção escolhe colunas originais; a extração cria novas colunas por combinação.
b) A seleção cria combinações; a extração apaga colunas.
c) Ambas criam novas colunas.
d) Ambas não alteram os dados.

Questão 2. O PCA foi proposto por Pearson em 1901 e desenvolvido por:

a) Hotelling, em 1933.
b) Ward, em 1963.
c) Lloyd, em 1957.
d) Kingma, em 2014.

Questão 3. No PCA, "informação" significa principalmente:

a) número de classes.
b) moda.
c) média.
d) variância.

Questão 4. A primeira componente principal (PC1) é:

a) a direção de maior variância dos dados.
b) o eixo y original.
c) a média dos atributos.
d) a direção de menor variância.

Questão 5. As componentes principais são, entre si:

a) idênticas.
b) paralelas.
c) aleatórias.
d) perpendiculares (ortogonais).

Questão 6. Qual é o primeiro passo do PCA?

a) Escolher k.
b) Centralizar (e, se necessário, padronizar) os dados.
c) Calcular a inércia.
d) Criar o dendrograma.

Questão 7. Em termos de álgebra, as componentes principais são:

a) os centroides.
b) autovetores da matriz de covariância.
c) a diagonal da matriz de dados.
d) as linhas da matriz identidade.

Questão 8. Na decomposição $X = U \cdot S \cdot V^T$ (SVD), as direções principais estão em:

a) S.
b) U.
c) V.
d) nenhuma delas.

Questão 9. O teorema de Eckart-Young garante que a SVD truncada nos k maiores valores singulares dá:

a) a pior aproximação.
b) a melhor aproximação de posto k, no sentido dos mínimos quadrados.
c) um agrupamento.
d) uma aproximação aleatória.

Questão 10. No scikit-learn, a fração da variância explicada por cada componente está em:

a) labels_.
b) explained_variance_ratio_.
c) inertia_.
d) cluster_centers_.

Questão 11. A variância acumulada serve para:

a) escolher o menor número de componentes que atinge um limiar (como 90 a 95%).
b) escolher a ligação.
c) escolher eps.
d) escolher o kernel.

Questão 12. O gráfico scree mostra:

a) a matriz de contingência.
b) a silhueta.
c) o dendrograma.
d) a variância de cada componente, para identificar o cotovelo.

Questão 13. O critério de Kaiser mantém componentes com autovalor:

a) maior que 1 (com dados padronizados).
b) negativo.
c) igual a zero.
d) menor que 1.

Questão 14. A análise paralela de Horn compara:

a) os autovalores reais com os de dados aleatórios de mesmo tamanho.
b) o K-means com o DBSCAN.
c) a precisão com a revocação.
d) o PCA com o t-SNE.

Questão 15. As cargas (loadings) do PCA indicam:

a) as novas coordenadas dos exemplos.
b) o peso de cada atributo original em cada componente.
c) o número de grupos.
d) a distância entre pontos.

Questão 16. O biplot mostra, no mesmo gráfico:

a) apenas o dendrograma.
b) exemplos (pontos) e atributos (setas).
c) apenas ruído.
d) apenas centroides.

Questão 17. Os sinais das componentes principais são:

a) sempre positivos.
b) importantes para a interpretação.
c) sempre negativos.
d) arbitrários: multiplicar por −1 não muda nada.

Questão 18. Reconstruir dados com inverse_transform após reduzir componentes:

a) apaga atributos.
b) devolve os dados exatamente iguais sempre.
c) devolve uma aproximação dos dados originais.
d) gera rótulos.

Questão 19. Uma limitação do PCA é que ele:

a) é insensível à escala.
b) exige rótulos.
c) não reduz dimensões.
d) captura apenas relações lineares.

Questão 20. O PCA é sensível:

a) à escala dos atributos e a outliers.
b) a nenhum fator.
c) apenas ao número de grupos.
d) apenas ao idioma.

Questão 21. O PCA incremental é útil quando:

a) há poucos dados.
b) os dados são categóricos.
c) só há uma coluna.
d) a base não cabe na memória.

Questão 22. O PCA esparso procura:

a) cargas com muitos zeros, para componentes mais interpretáveis.
b) menos exemplos.
c) mais dimensões.
d) cargas todas iguais.

Questão 23. O Kernel PCA é adequado para:

a) ordenar exemplos.
b) dados já lineares apenas.
c) estruturas não lineares, como anéis.
d) dados categóricos.

Questão 24. O ICA procura componentes:

a) iguais.
b) ordenadas por tamanho.
c) apenas não correlacionadas.
d) estatisticamente independentes.

Questão 25. O problema do "cocktail party" é um exemplo de:

a) detecção de ruído.
b) separação cega de fontes.
c) agrupamento hierárquico.
d) regras de associação.

Questão 26. O ICA explora o fato de que fontes independentes e não gaussianas, quando somadas:

a) ficam mais gaussianas.
b) desaparecem.
c) ficam constantes.
d) ficam menos gaussianas.

Questão 27. O NMF exige que os dados e os fatores sejam:

a) negativos.
b) binários.
c) inteiros.
d) não negativos.

Questão 28. Lee e Seung (1999) mostraram que o NMF, aplicado a rostos, aprende:

a) rostos fantasmas globais.
b) apenas bordas.
c) rótulos.
d) partes, como olhos, narizes e bocas.

Questão 29. Em tópicos de texto com NMF, cada linha de H representa:

a) um tópico (conjunto de palavras que aparecem juntas).
b) um grupo de distâncias.
c) um documento.
d) uma palavra isolada.

Questão 30. Se duas variáveis são fortemente correlacionadas, o PCA tende a:

a) exigir duas componentes de variância igual.
b) transformá-las em categorias.
c) descartá-las por completo.
d) representá-las bem com uma única componente principal.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique com a analogia da sombra a ideia do PCA.
2. Descreva os seis passos do PCA via matriz de covariância.
3. Por que a variância é usada como medida de informação no PCA? Em que situação essa suposição falha?
4. Explique a relação entre PCA e SVD e por que a SVD é usada na prática.
5. Como escolher o número de componentes? Compare variância acumulada, scree, Kaiser e análise paralela.
6. Interprete: um PCA em dados padronizados retorna variâncias explicadas de 62%, 25%, 8% e 5%. Quantas componentes manter para 90%?
7. Diferencie escores e cargas e explique como ler um biplot.
8. Explique como o PCA pode ser usado para compressão e remoção de ruído.
9. Cite quatro limitações ou cuidados do PCA.
10. Compare PCA incremental, aleatorizado, esparso e com kernel.
11. Descreva o problema do cocktail party e como o ICA o resolve de forma diferente do PCA.
12. Explique a decomposição $X \approx W \cdot H$ do NMF e por que as peças tendem a ser interpretáveis.
13. Mostre como o NMF poderia descobrir tópicos em um conjunto de textos.
14. Compare PCA e NMF quanto às restrições e à interpretação.
15. Por que é recomendado padronizar os dados antes do PCA?
16. Explique a diferença entre PCA e seleção de atributos usando uma analogia de receita.
17. Para dados de imagens de dígitos 8×8 (64 atributos), explique como usar PCA para reduzir a dimensão e avaliar a perda.
18. Explique o que é variância explicada e como o parâmetro n_components=0.95 do PCA a utiliza.
19. Dê um exemplo em que ICA seria preferível ao PCA e justifique.
20. Explique o que significa dizer que as componentes principais são "não correlacionadas" e por que isso é útil.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Quantas componentes para 90% da variância?

NÍVEL: BÁSICO

No **Wine** padronizado, ajuste um PCA completo. Mostre a variância explicada acumulada e o menor número de componentes que alcança 90%. Depois mostre os três atributos de maior peso (em valor absoluto) na primeira componente.

Exercício 2 — Reconstrução e compressão de dígitos

NÍVEL: BÁSICO

Nos **dígitos** (64 atributos), reduza com PCA a k = 2, 5, 10, 20 e 40 componentes e reconstrua as imagens. Para cada k, mostre a variância explicada, o erro quadrático médio (MSE) e a razão de compressão (64 / k).

Exercício 3 — PCA "na unha" com autovetores

NÍVEL: INTERMEDIÁRIO

No Wine padronizado, calcule o PCA manualmente: matriz de covariância (`np.cov`), autovalores e autovetores (`np.linalg.eigh`), ordenação decrescente e projeção. Compare as variâncias explicadas com as de `PCA` e confira se as componentes são ortogonais.

Exercício 4 — Kernel PCA em anéis

NÍVEL: INTERMEDIÁRIO

Em `make_circles(500, factor=0.3, noise=0.05, random_state=0)`, aplique PCA linear e `KernelPCA(kernel="rbf", gamma=10)` com 2 componentes. Treine uma regressão logística usando **apenas a primeira componente** de cada representação e compare a acurácia (use as classes reais só para essa avaliação).

Exercício 5 — Separando fontes com ICA

NÍVEL: AVANÇADO

Crie três sinais independentes (senoide, onda quadrada e dente de serra), misture-os com uma matriz aleatória e recupere as fontes com `FastICA`. Para cada fonte original, mostre a maior correlação absoluta com alguma componente recuperada e, para comparação, com alguma componente do PCA.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Pen-Based Recognition of Handwritten Digits, Wine Quality e Gas Sensor Array Drift (muitos atributos).
> - **Kaggle:** Fashion-MNIST (imagens de roupas, 28×28).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Compressão, limpeza e partes de dígitos

Contexto. Uma empresa de digitalização de documentos quer **reduzir o tamanho** de imagens de dígitos manuscritos, **limpar ruído** e entender as "partes" que formam os traços, sem perder muita capacidade de reconhecimento.

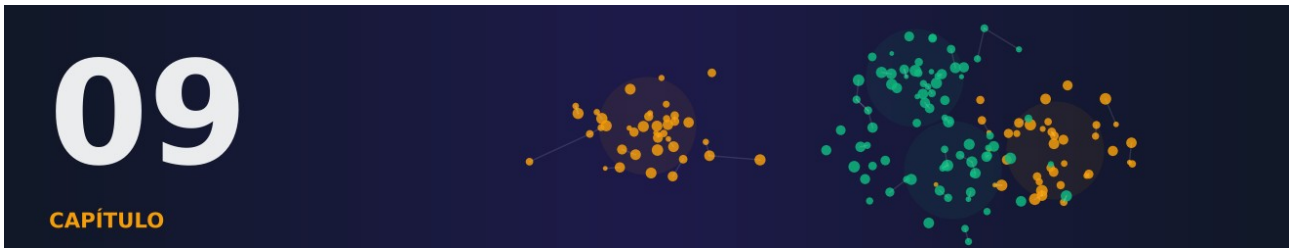
Fonte dos dados. *Optical Recognition of Handwritten Digits* (**UCI Machine Learning**), via `load_digits` do scikit-learn.

Tarefas.

1. Divida os dados em treino e teste (70/30, com `stratify` e semente fixa). **Ajuste todas as transformações só no treino.**
2. Treine uma linha de base: regressão logística nos 64 pixels. Anote a acurácia no teste.
3. Para k = 5, 10, 20 e 30, ajuste um PCA no treino, treine a regressão logística nas componentes e anote a acurácia no teste e a razão de compressão. Desenhe o gráfico acurácia × k.
4. **Limpeza de ruído:** adicione ruído gaussiano (desvio 4, em escala de 0 a 16) às imagens de teste, reconstrua com PCA de 15 componentes e compare o MSE contra a imagem limpa, antes e depois.
5. **Partes:** ajuste NMF com 12 componentes no treino e desenhe as 12 "peças". Compare a acurácia da regressão logística nas ativações do NMF com a do PCA de 12 componentes.
6. Escreva uma recomendação: qual k você usaria e por quê (compressão × acurácia)?

Entregáveis. Script, gráfico acurácia × k, figuras das peças do NMF, tabela de resultados e recomendação.

A solução de referência está no Apêndice E.



Capítulo 9 — Aprendizado de Variedades: MDS, Isomap, t-SNE e UMAP

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar a hipótese da variedade (*manifold hypothesis*).
- Distinguir distância em linha reta (euclidiana) de distância ao longo da variedade (geodésica).
- Descrever o MDS, o Isomap e o LLE.
- Entender como o t-SNE funciona, o papel da perplexidade e como interpretá-lo corretamente.
- Entender as ideias centrais do UMAP e compará-lo com o t-SNE.
- Evitar as armadilhas mais comuns na leitura de projeções 2D.

9.1 Quando a linha reta não basta

O PCA projeta os dados em **planos**. Mas e se os dados estiverem "enrolados"?

Pense num **tapete enrolado** (como um rocambole). Dois pontos em voltas vizinhas estão muito próximos em linha reta, mas, para ir de um ao outro **andando sobre o tapete**, é preciso percorrer um longo caminho.

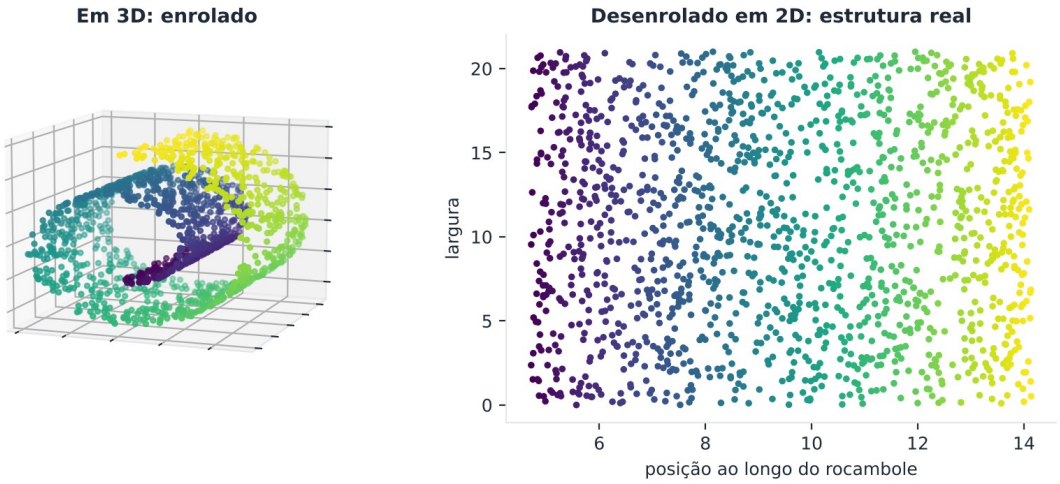


Figura 9.1 — O "rocambole suíço" (Swiss roll): em 3D, pontos de voltas diferentes parecem próximos. Desenrolado, vemos a estrutura real em 2D.

9.2 A hipótese da variedade

Uma **variedade** (*manifold*) é uma superfície que, vista de perto, parece plana, mesmo que globalmente seja curva. A superfície da Terra é o exemplo clássico: localmente parece um plano, mas é uma esfera.

A **hipótese da variedade** diz que dados de alta dimensão geralmente ficam sobre (ou perto de) uma variedade de **dimensão muito menor** (Bengio; Courville; Vincent, 2013; Roweis; Saul, 2000).

Exemplo: fotos de um rosto girando. Cada foto tem milhares de pixels, mas o que varia é basicamente **um parâmetro**: o ângulo de rotação. Os dados vivem numa curva de 1 dimensão dentro de um espaço de milhares.

O objetivo do aprendizado de variedades é **desenrolar** a estrutura e revelar essa dimensão intrínseca.

9.3 MDS: Escalonamento Multidimensional

9.3.1 A ideia do mapa de cidades

Suponha que você só tenha uma **tabela de distâncias** entre cidades, como nas antigas tabelas de quilometragem de estrada. Dá para reconstruir um **mapa**? Sim! Esse é o problema do **MDS** (*Multidimensional Scaling*).

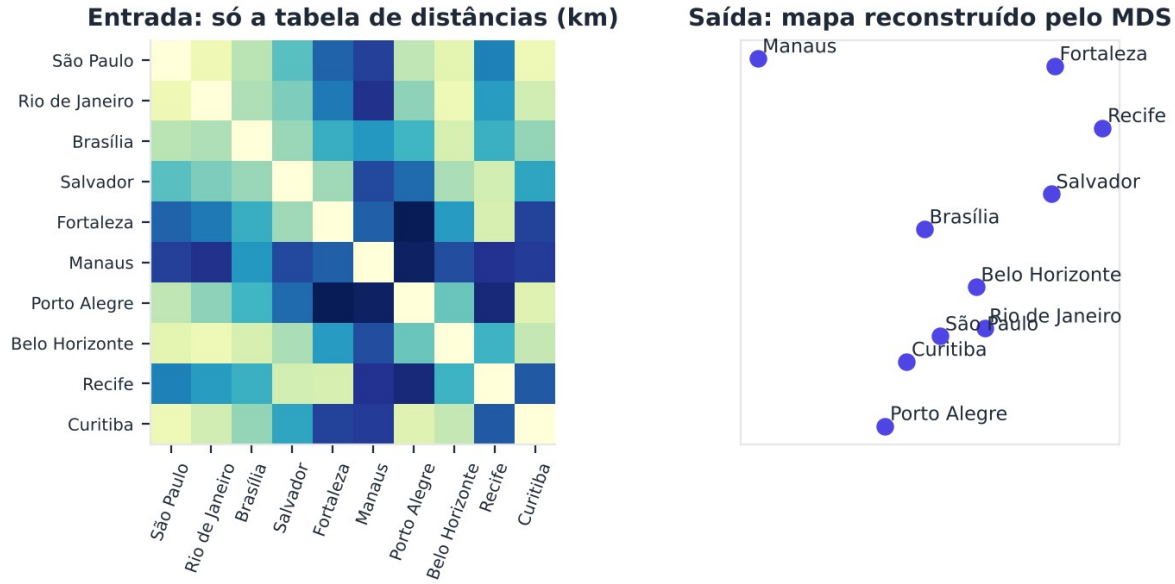


Figura 9.2 — MDS reconstrói o mapa de capitais brasileiras a partir apenas das distâncias entre elas. (O mapa pode sair girado ou espelhado.)

O MDS busca posições em 2D (ou 3D) tais que as distâncias no mapa sejam **o mais parecidas possível** com as distâncias originais. Mede-se o erro com o **stress**:

$$\text{stress} = \sqrt{\left(\Sigma \left(d_{\text{original}} - d_{\text{mapa}}\right)^2 / \Sigma \, d_{\text{original}}^2 \right)}$$

Quanto menor, melhor (Kruskal, 1964; Cox; Cox, 2001).

9.3.2 Dois sabores

- **MDS clássico (de Torgerson):** usa autovetores. Se as distâncias são Euclidianas, equivale ao PCA.
- **MDS métrico por stress (SMACOF):** otimiza iterativamente o stress.
- **MDS não métrico:** preserva apenas a **ordem** das distâncias, útil quando só há ranking de semelhança.

```python
import numpy as np
from sklearn.manifold import MDS
from sklearn.metrics import pairwise_distances

rng = np.random.default_rng(0)
```

```
pontos = rng.normal(size=(30, 2))                    # mapa "secreto"
D = pairwise_distances(pontos)                        # só conhecemos as distâncias

mds = MDS(n_components=2, dissimilarity="precomputed", random_state=0, n_init=4)
rec = mds.fit_transform(D)
print("Stress final:", round(mds.stress_, 4))
```

9.4 Isomap: distâncias geodésicas

O **Isomap** (Tenenbaum; de Silva; Langford, 2000) melhora o MDS usando a distância certa para variedades.

A **distância geodésica** é a distância medida **ao longo da superfície**. É a diferença entre a distância de voo (linha reta) e a distância por rodovia.

Passos do Isomap:

1. Conecte cada ponto aos seus *k* vizinhos mais próximos (grafo).
2. Calcule a distância do **menor caminho** no grafo entre todos os pares (aproxima a geodésica).
3. Aplique o MDS a essas distâncias.

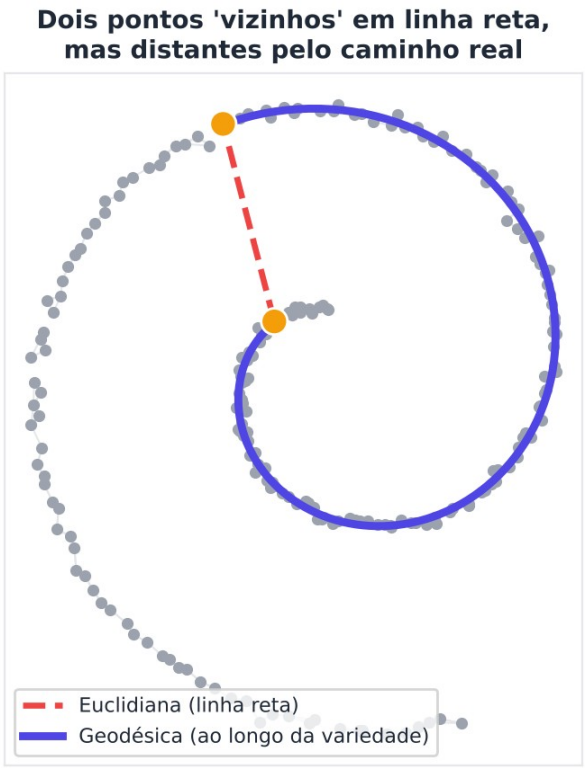


Figura 9.3 — Distância euclidiana (tracejada) versus geodésica (curva) entre dois pontos da variedade, aproximada por caminhos no grafo de vizinhos.

```python
import numpy as np
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import Isomap
from sklearn.decomposition import PCA

X, t = make_swiss_roll(n_samples=1500, noise=0.05, random_state=0)
```

```
iso = Isomap(n_neighbors=12, n_components=2).fit_transform(X)
pca = PCA(n_components=2).fit_transform(X)

# t = posição real ao longo do rocambole. Quanto maior a correlação, melhor desenrolou.
print("Corr(t, 1ª coord. Isomap):", abs(np.corrcoef(t, iso[:, 0])[0, 1]).round(2))
print("Corr(t, 1ª coord. PCA)   :", abs(np.corrcoef(t, pca[:, 0])[0, 1]).round(2))
```

Limitações: sensível a ruído (atalhos no grafo), exige que o grafo seja conectado, caro (n^2).

9.5 LLE: Embedding Linear Local

O **LLE** (*Locally Linear Embedding*; Roweis; Saul, 2000) parte da ideia: "cada ponto pode ser reconstruído como **combinação** dos vizinhos".

1. Para cada ponto, ache pesos que o reconstruam a partir dos seus *k* vizinhos.
2. Procure coordenadas de baixa dimensão que mantenham **esses mesmos pesos**.

Analogia: descrever a posição de cada casa por "um pouco à direita da padaria e bem perto da escola". Essas relações locais se mantêm quando o mapa é desenrolado.

9.6 t-SNE

9.6.1 O problema que ele resolve

Queremos um mapa 2D em que **vizinhos continuem vizinhos**. O t-SNE (*t-distributed Stochastic Neighbor Embedding*; van der Maaten; Hinton, 2008) é hoje uma das ferramentas de visualização mais usadas, especialmente em biologia e aprendizado profundo.

9.6.2 A ideia, em linguagem simples

1. **No espaço original**, para cada ponto, calcule a **probabilidade** de cada outro ponto ser seu "vizinho". Pontos próximos têm alta probabilidade; distantes, quase zero. Usa-se uma gaussiana em volta de cada ponto.
2. **No mapa 2D**, comece com posições aleatórias e calcule probabilidades parecidas, mas usando uma distribuição **t de Student** (de caudas pesadas).
3. **Aproxime** os dois conjuntos de probabilidades, minimizando a **divergência de Kullback-Leibler** (uma medida de diferença entre distribuições), por descida do gradiente.
4. Os pontos se movem no mapa: quem era vizinho se **atrai**; quem não era se **repele**.

Analogia: uma festa. No início, as pessoas estão espalhadas aleatoriamente. Quem tem afinidade se aproxima. Quem não tem se afasta. No fim, formam-se rodinhas.



Ilustração esquemática (painéis inicial e intermediário são ilustrativos): atração entre vizinhos e repulsão entre não vizinhos.

Figura 9.4 — O t-SNE alterna forças de atração (vizinhos) e repulsão (não vizinhos) até formar um mapa estável.

9.6.3 Por que a distribuição t?

Em baixa dimensão há **menos espaço**. Muitos pontos moderadamente distantes no espaço original não cabem todos "a uma distância moderada" no mapa e acabariam amontoados no centro (problema da *crowding*). As **caudas pesadas** da t de Student permitem que pontos não vizinhos fiquem bem afastados, abrindo espaço entre as rodinhas.

9.6.4 A perplexidade

O principal parâmetro é a **perplexidade**. Interprete-a como o "número aproximado de vizinhos que cada ponto considera". Valores típicos: **5 a 50**.

- **Baixa:** foca em estrutura muito local. Pode fragmentar grupos em pedaços.
- **Alta:** enxerga estruturas mais globais. Pode fundir grupos.

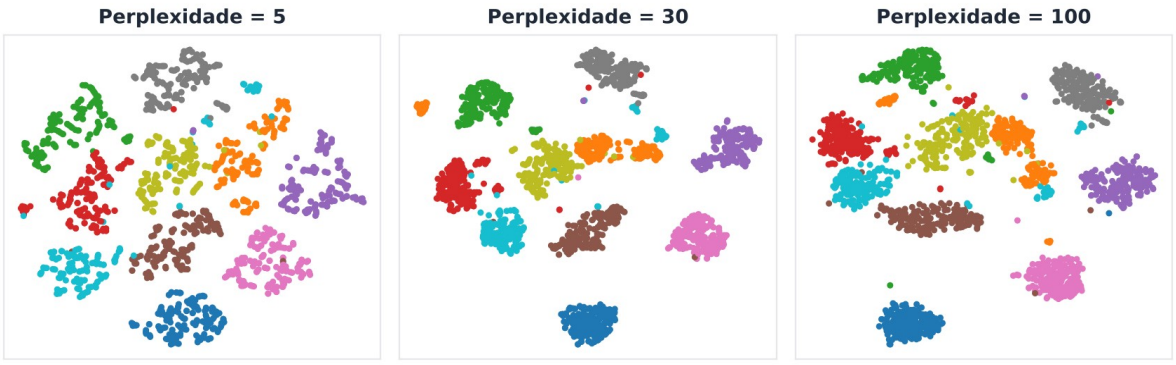


Figura 9.5 — O mesmo conjunto de dígitos projetado com perplexidades 5, 30 e 100. O resultado muda; teste vários valores.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE

X, y = load_digits(return_X_y=True)
Z = TSNE(n_components=2, perplexity=30, init="pca",
         learning_rate="auto", random_state=0).fit_transform(X)

plt.scatter(Z[:, 0], Z[:, 1], c=y, cmap="tab10", s=8)
plt.colorbar(label="Dígito")
```

```
plt.title("t-SNE dos dígitos manuscritos")
plt.show()
```

9.6.5 Como ler um t-SNE (e como não ler)

Kobak e Berens (2019) e outros autores alertam para armadilhas:

1. **O tamanho dos grupos não significa nada.** O t-SNE adapta a escala local; grupos densos e esparsos aparecem com tamanhos parecidos.
2. **A distância entre grupos distantes quase não significa nada.** Só a vizinhança local é confiável.
3. **A forma pode enganar.** Ruído pode parecer estrutura.
4. **O resultado varia** com a semente aleatória e a perplexidade. Rode mais de uma vez.
5. **Não há mapeamento para novos pontos** na versão original: ele não "aprende uma função".

> ATENÇÃO **Regra de segurança**
>
> Use o t-SNE para **explorar e comunicar**, não como prova. Confirme as descobertas com outras técnicas (como silhueta ou testes estatísticos).

9.6.6 Dicas práticas

- **Reduza antes** a 30–50 dimensões com PCA. Acelera e remove ruído.
- Use `init="pca"` para uma inicialização mais estável. Com ela, o resultado quase não muda com a semente; para avaliar a estabilidade, compare execuções com `init="random"` e sementes diferentes.
- Rode por tempo suficiente (`max_iter` ≥ 1000).
- Para bases grandes, use a implementação Barnes-Hut (padrão) ou FIt-SNE (Linderman et al., 2019).

9.7 UMAP

9.7.1 O que é

O **UMAP** (*Uniform Manifold Approximation and Projection*; McInnes; Healy; Melville, 2018) é uma técnica mais recente, que se tornou muito popular, sobretudo em análise de células individuais (Becht et al., 2019).

9.7.2 A ideia (sem a matemática pesada)

Possui base teórica em geometria e topologia, mas o procedimento prático tem dois passos:

1. **Construir um grafo "difuso" de vizinhos** em alta dimensão. Cada ligação tem um peso entre 0 e 1 (a "força" da vizinhança), calibrado localmente.
2. **Encontrar um layout** em baixa dimensão cujo grafo seja o mais parecido possível, usando atração e repulsão, como no t-SNE, mas com uma função de custo diferente (entropia cruzada).

9.7.3 Parâmetros principais

- **n_neighbors:** tamanho da vizinhança local (como a perplexidade). Pequeno → detalhes locais; grande → estrutura global. Padrão: 15.
- **min_dist:** o quão compactos os pontos podem ficar no mapa. Pequeno → blocos densos; grande → pontos mais espalhados. Padrão: 0,1.

- **metric:** a distância usada (euclidiana, cosseno etc.).

```python
import numpy as np
import umap
from sklearn.datasets import load_digits

X, y = load_digits(return_X_y=True)
Z = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=0).fit_transform(X)
print(Z.shape)

# Diferença importante: o UMAP aprende uma função, então transforma pontos novos.
modelo = umap.UMAP(n_neighbors=15, random_state=0).fit(X[:1500])
Z_novos = modelo.transform(X[1500:])
print("Pontos novos projetados:", Z_novos.shape)
```

> **DICA — Instalação**
>
> O UMAP é instalado à parte: `pip install umap-learn`.

9.7.4 UMAP versus t-SNE

Aspecto	t-SNE	UMAP
Velocidade	Mais lento	Mais rápido em bases grandes
Estrutura global	Fraca	Em geral melhor preservada
Pontos novos	Não (original)	Sim (`transform`)
Parâmetro principal	Perplexidade	n_neighbors, min_dist
Base teórica	Probabilidades e KL	Topologia e grafos difusos
Cuidados	Tamanho e distância entre grupos não significam nada	Idem; não confie cegamente na distância global

> **ATENÇÃO — Não há mágica**
>
> Estudos recentes mostram que boa parte da "melhor estrutura global" do UMAP vem da **inicialização** (por exemplo, PCA ou espectral). Com boa inicialização, o t-SNE se aproxima do UMAP na preservação global (Kobak; Linderman, 2021).

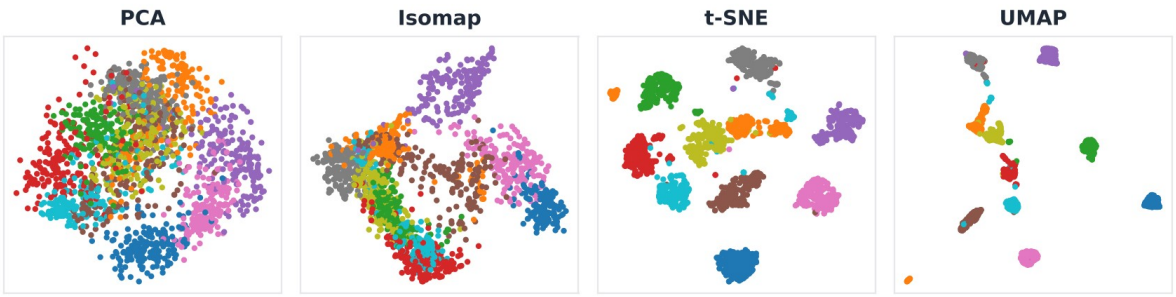


Figura 9.6 — Os mesmos dígitos projetados por PCA, Isomap, t-SNE e UMAP. Cada método conta uma parte da história.

9.8 Outras técnicas relevantes

- **LLE modificado e Hessian LLE:** variações mais estáveis do LLE.
- **Laplacian Eigenmaps** (Belkin; Niyogi, 2003): usa autovetores do grafo Laplaciano (veja o Capítulo 6!).
- **Diffusion maps** (Coifman; Lafon, 2006): usa "difusão" no grafo para capturar estrutura em várias escalas.
- **PHATE** (Moon et al., 2019): foca na preservação de trajetórias contínuas em dados biológicos.
- **Autoencoders** (Capítulo 12): a versão neural da redução não linear.

9.9 Como avaliar uma projeção?

Como saber se o mapa 2D é fiel? Algumas medidas:

- **Confiança (*trustworthiness*) e continuidade** (Venna; Kaski, 2006): medem se os vizinhos no mapa eram vizinhos no original e vice-versa.
- **Preservação dos k vizinhos:** proporção de vizinhos que se mantêm.
- **Correlação das distâncias:** entre original e mapa (mais útil para estrutura global).

```python
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE, trustworthiness

X, y = load_digits(return_X_y=True)
for nome, Z in [("PCA ", PCA(2).fit_transform(X)),
                ("t-SNE", TSNE(2, perplexity=30, init="pca", random_state=0).fit_transform(X))]:
    print(nome, "trustworthiness (k=10):", round(trustworthiness(X, Z, n_neighbors=10), 3))
```

9.10 Um guia de escolha

1. **Pensando em visualizar?** Comece por PCA (rápido, global). Depois t-SNE ou UMAP.
2. **Precisa transformar novos dados?** PCA, Kernel PCA ou UMAP.
3. **Base enorme?** UMAP ou PCA aleatorizado.
4. **Quer preservar a geometria global?** Isomap ou MDS.
5. **Precisa de componentes interpretáveis?** PCA, NMF.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Variedade:** os dados vivem numa superfície de dimensão menor, enrolada em alta dimensão.
- **Geodésica** = distância sobre a superfície.
- **MDS** reconstrói o mapa a partir de distâncias.
- **Isomap** = grafo + caminhos mínimos + MDS.
- **t-SNE** preserva vizinhanças locais; perplexidade ≈ nº de vizinhos.
- **No t-SNE**, tamanho e distância entre grupos não são confiáveis.
- **UMAP** é rápido, aprende função e costuma preservar mais estrutura global.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. A hipótese da variedade (manifold hypothesis) afirma que dados de alta dimensão:

a) são sempre lineares.
b) geralmente ficam sobre ou perto de uma variedade de dimensão muito menor.
c) ocupam uniformemente todo o espaço.
d) não têm estrutura.

Questão 2. Uma variedade é uma superfície que:

a) só existe em duas dimensões.
b) vista de perto parece plana, mesmo sendo globalmente curva.
c) não pode ser curva.
d) é sempre um plano.

Questão 3. Na estrutura "rocambole suíço", dois pontos de voltas diferentes:

a) não existem.
b) podem estar próximos em linha reta e distantes ao longo da superfície.
c) estão sempre distantes em linha reta.
d) são sempre idênticos.

Questão 4. A distância geodésica é a distância:

a) de Hamming.
b) em linha reta.
c) do cosseno.
d) medida ao longo da superfície (variedade).

Questão 5. O MDS (escalonamento multidimensional) recebe como entrada principal:

a) rótulos de classe.
b) uma imagem.
c) uma tabela de distâncias entre os objetos.
d) apenas o número de grupos.

Questão 6. O "stress" no MDS mede:

a) a variância.
b) o erro entre as distâncias originais e as do mapa.
c) o tempo de execução.
d) o número de grupos.

Questão 7. O MDS clássico, quando as distâncias são euclidianas, é equivalente ao:

a) PCA.
b) K-means.
c) DBSCAN.
d) t-SNE.

Questão 8. O MDS não métrico preserva:

a) densidades.
b) apenas a ordem das distâncias.
c) distâncias exatas.
d) rótulos.

Questão 9. O Isomap realiza, nesta ordem:

a) grafo de vizinhos, caminhos mínimos e MDS.
b) bootstrap e silhueta.
c) corte, regras e lift.
d) PCA, K-means e dendrograma.

Questão 10. Uma limitação do Isomap é:

a) sensibilidade a ruído (atalhos no grafo) e custo da ordem de n^2.
b) funcionar só com dois pontos.
c) exigir rótulos.
d) não usar vizinhos.

Questão 11. O LLE (Locally Linear Embedding) parte da ideia de que:

a) os dados são aleatórios.
b) cada ponto é reconstruído como combinação linear de seus vizinhos, e esses pesos devem ser preservados.
c) a distância global é a única relevante.
d) todos os pontos são independentes.

Questão 12. O t-SNE tem como objetivo principal:

a) preservar a variância global.
b) preservar vizinhanças locais em um mapa de baixa dimensão.
c) calcular regras de associação.
d) ordenar os pontos.

Questão 13. No t-SNE, as probabilidades no mapa de baixa dimensão usam:

a) uma distribuição t de Student, com caudas pesadas.
b) uma gaussiana apenas.
c) uma distribuição binária.
d) uma distribuição uniforme.

Questão 14. A medida que o t-SNE minimiza ao aproximar as duas distribuições é:

a) a divergência de Kullback-Leibler.
b) o BIC.
c) a soma dos quadrados.
d) o índice de Rand.

Questão 15. A perplexidade no t-SNE pode ser interpretada como:

a) a taxa de aprendizado.
b) o número de grupos.
c) o número aproximado de vizinhos considerados por ponto.
d) o número de iterações.

Questão 16. Perplexidade muito baixa tende a:

a) remover ruído.
b) fragmentar grupos em pedaços.
c) não alterar o resultado.
d) fundir tudo em um grupo.

Questão 17. Em um mapa t-SNE, o tamanho aparente dos grupos:

a) indica a variância.
b) indica a quantidade de pontos exatamente.
c) é sempre proporcional à densidade.
d) não tem significado confiável.

Questão 18. Em um mapa t-SNE, a distância entre grupos muito afastados:

a) quase não significa nada.
b) indica a ordem temporal.
c) é proporcional à similaridade original.
d) é sempre fiel.

Questão 19. Uma recomendação prática para o t-SNE é:

a) reduzir antes a 30–50 dimensões com PCA e usar init="pca".
b) usar apenas dados categóricos.
c) rodar uma única vez.
d) nunca padronizar dados.

Questão 20. O UMAP, ao contrário do t-SNE original:

a) não tem parâmetros.
b) não usa vizinhos.
c) só funciona em 1D.
d) aprende uma função e permite transformar pontos novos.

Questão 21. O parâmetro n_neighbors do UMAP controla:

a) a escala do eixo.
b) o tamanho da vizinhança local considerada.
c) o número de iterações.
d) o número de grupos.

Questão 22. O parâmetro min_dist do UMAP controla:

a) o quão compactos os pontos podem ficar no mapa.
b) a semente.
c) o número de componentes.
d) a métrica.

Questão 23. Segundo Kobak e Linderman (2021), a melhor preservação global do UMAP deve-se em grande parte:

a) a um segredo teórico.
b) ao número de cores.
c) ao tamanho do arquivo.
d) à inicialização (por exemplo, PCA ou espectral).

Questão 24. Os Laplacian Eigenmaps utilizam:

a) a árvore de isolamento.
b) autovetores do grafo Laplaciano.
c) a função sigmoide.
d) o algoritmo Apriori.

Questão 25. A medida trustworthiness (confiança) de uma projeção avalia:

a) a velocidade do algoritmo.
b) se os vizinhos no mapa eram de fato vizinhos no espaço original.
c) o número de grupos.
d) a inércia.

Questão 26. Se você precisa projetar novos dados em um mapa já aprendido, qual opção é mais direta?

a) Nenhuma técnica permite.
b) t-SNE original.
c) UMAP ou PCA, que possuem transform.
d) MDS clássico sem refazer o cálculo.

Questão 27. Para bases muito grandes, uma boa opção de visualização é:

a) Isomap com todos os pares.
b) MDS com todos os pares.
c) UMAP ou PCA aleatorizado.
d) dendrograma completo.

Questão 28. Para preservar a geometria global das distâncias, são indicados:

a) apenas t-SNE.
b) Isomap e MDS.
c) apenas DBSCAN.
d) apenas Apriori.

Questão 29. Qual técnica foi criada por Tenenbaum, de Silva e Langford (2000) e publicada na revista Science?

a) DBSCAN.
b) Apriori.
c) Isomap.
d) K-means.

Questão 30. No t-SNE, o resultado pode variar entre execuções porque:

a) os dados são alterados.
b) o algoritmo é analítico.
c) a perplexidade é sempre zero.
d) há inicialização aleatória e otimização por gradiente.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique a hipótese da variedade com o exemplo do tapete enrolado e das fotos de um rosto girando.
2. Diferencie distância euclidiana de geodésica e explique como o Isomap aproxima a geodésica.
3. Descreva o MDS e como ele reconstruiria um mapa de cidades a partir da tabela de distâncias.
4. Diferencie MDS clássico, métrico por stress e não métrico.
5. Descreva a ideia do LLE em linguagem simples.
6. Descreva o funcionamento do t-SNE em quatro passos.
7. Por que o t-SNE usa a distribuição t de Student no mapa?
8. Explique o papel da perplexidade e como escolher seu valor.
9. Liste cinco armadilhas na interpretação de um t-SNE.
10. Explique as duas etapas do UMAP e seus parâmetros n_neighbors e min_dist.
11. Compare t-SNE e UMAP quanto à velocidade, ao transform, ao parâmetro principal e à estrutura global.
12. Por que não se deve usar o t-SNE como prova de existência de grupos?
13. Explique como avaliar uma projeção usando trustworthiness e correlação das distâncias.
14. Por que PCA, Isomap, t-SNE e UMAP contam histórias diferentes sobre os mesmos dados?
15. Proponha um guia de escolha entre PCA, Isomap, t-SNE e UMAP para quatro situações.
16. Explique como os Laplacian Eigenmaps se relacionam com o agrupamento espectral.
17. Descreva por que reduzir antes com PCA ajuda o t-SNE.
18. Dê um exemplo de aplicação de UMAP ou t-SNE em biologia e comente cuidados.
19. Explique por que o rocambole suíço não pode ser bem representado pelo PCA.
20. Explique por que o resultado do t-SNE deve ser conferido com mais de uma semente e mais de um valor de perplexidade.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — O mapa do Brasil a partir de distâncias

NÍVEL: BÁSICO

Use as coordenadas aproximadas de dez capitais brasileiras (código abaixo), calcule a matriz de distâncias em km (fórmula haversine) e recupere um mapa 2D com `MDS(dissimilarity="precomputed")`. Mostre o *stress* e a correlação entre as distâncias originais e as do mapa.

PYTHON

```
cidades = {
    "São Paulo": (-23.55, -46.63),
    "Rio de Janeiro": (-22.91, -43.17),
    "Brasília": (-15.78, -47.93),
    "Salvador": (-12.97, -38.51),
    "Fortaleza": (-3.73, -38.53),
    "Manaus": (-3.12, -60.02),
    "Porto Alegre": (-30.03, -51.23),
    "Belo Horizonte": (-19.92, -43.94),
    "Recife": (-8.05, -34.88),
    "Curitiba": (-25.43, -49.27),
}
```

Exercício 2 — Desenrolando o rocambole

NÍVEL: INTERMEDIÁRIO

Gere `make_swiss_roll(1500, noise=0.05, random_state=0)`. Reduza a 2 dimensões com PCA, Isomap (12 vizinhos) e LLE (12 vizinhos). Calcule a correlação de Spearman entre a primeira coordenada de cada método e o parâmetro verdadeiro `t` (posição ao longo do rocambole). Qual desenrola melhor?

Exercício 3 — Perplexidade e qualidade do t-SNE

NÍVEL: INTERMEDIÁRIO

Nos **dígitos**, rode `TSNE` com perplexidade 5, 30 e 100 (`init="pca"`, `random_state=0`). Para cada uma, mostre a confiança (`trustworthiness`, k = 10) e a divergência de KL final (`kl_divergence_`).

Exercício 4 — PCA antes do t-SNE: tempo e qualidade

NÍVEL: INTERMEDIÁRIO

Compare o t-SNE aplicado aos 64 pixels com o t-SNE aplicado a 20 componentes de PCA: meça o tempo (`time.perf_counter`) e a confiança (k = 10) em relação aos dados originais.

Exercício 5 — Quem preserva melhor os vizinhos?

NÍVEL: AVANÇADO

Nos dígitos, projete em 2D com PCA, Isomap e t-SNE. Para cada projeção, calcule (a) a fração média de vizinhos preservados (os 10 vizinhos mais próximos no espaço original que continuam entre os 10 mais próximos na projeção) e (b) a acurácia de um KNN (5-fold) usando apenas as 2 coordenadas.

SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**

- **UCI:** Pen-Based Recognition of Handwritten Digits e Optical Recognition of Handwritten Digits (versão completa, 5.620 imagens).
- **Kaggle:** Fashion-MNIST e MNIST (Digit Recognizer).

Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Comparando mapas dos dígitos e dos vinhos

Contexto. Um grupo de pesquisa quer escolher **uma técnica de visualização padrão** para seus conjuntos de dados. Cabe a você fazer uma comparação justa e propor a escolha.

Fonte dos dados. *Optical Recognition of Handwritten Digits* e *Wine*, ambos do repositório **UCI Machine Learning**, incluídos no scikit-learn.

Tarefas.

1. Para cada conjunto (dígitos e Wine padronizado), gere projeções 2D com PCA, Isomap, LLE e t-SNE. (Se tiver o pacote `umap-learn`, inclua o UMAP; o código deve funcionar mesmo sem ele.)
2. Calcule, para cada projeção, a **confiança** (`trustworthiness`) e a **continuidade** (confiança com os papéis trocados) com k = 10, e a acurácia de um KNN nas coordenadas 2D.
3. Meça a **estabilidade do t-SNE**: repita com 5 sementes e calcule o desvio padrão da confiança.
4. Monte uma tabela por conjunto e desenhe um painel com as projeções.
5. Escreva uma recomendação (máximo de 8 linhas) que diga **quando** usar cada técnica e quais **armadilhas** de interpretação você destacaria para quem lê esses mapas.

Entregáveis. Script, tabelas, painel de gráficos e recomendação.

A solução de referência está no Apêndice E.

PARTE IV

Parte IV — Padrões e Anomalias

Como descobrir o que costuma aparecer junto (regras de associação) e o que foge do normal (detecção de anomalias).

Nesta parte:

- Cap. 10 — Regras de Associação: o que Costuma Aparecer Junto?
- Cap. 11 — Detecção de Anomalias: Encontrando o que Foge do Normal



Capítulo 10 — Regras de Associação: o que Costuma Aparecer Junto?

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar o problema da análise da cesta de compras.
- Definir itemset, suporte, confiança e lift.
- Executar o algoritmo Apriori e entender a propriedade da anti-monotonicidade.
- Descrever o FP-Growth e quando ele é mais eficiente que o Apriori.
- Gerar, filtrar e interpretar regras com mlxtend em Python.
- Reconhecer armadilhas: correlação não é causa, regras triviais e explosão combinatória.

10.1 A lenda das fraldas e da cerveja

Conta a lenda que um supermercado descobriu que, nas sextas-feiras à noite, quem comprava **fraldas** muitas vezes levava também **cerveja**. A explicação: pais enviados às pressas à loja aproveitavam para comprar sua bebida. A loja aproximou as prateleiras e as vendas subiram.

Seja a história verdadeira ou não, ela ilustra a ideia: **encontrar produtos que são comprados juntos**. Isso se chama **análise da cesta de compras** (*market basket analysis*).

A técnica foi formalizada por Agrawal, Imielinski e Swami (1993) e vai muito além de supermercados:

- **Varejo:** posicionamento de produtos, promoções combinadas.
- **Streaming e e-commerce:** "quem viu A também viu B".
- **Medicina:** sintomas e diagnósticos que ocorrem juntos; interações entre medicamentos.
- **Web:** páginas visitadas na mesma sessão.
- **Bioinformática:** genes expressos em conjunto.

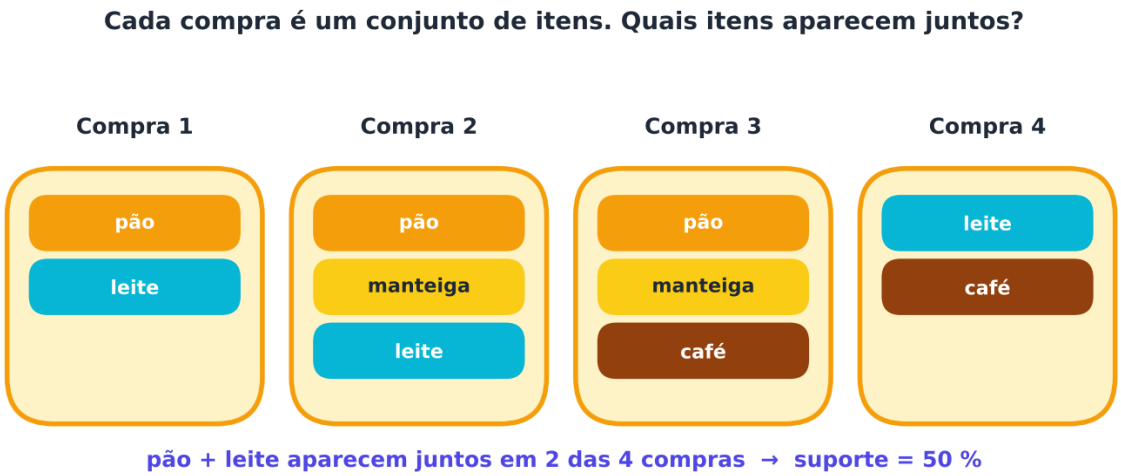


Figura 10.1 — A cesta de compras: cada compra é um conjunto de itens. A pergunta é: quais itens costumam aparecer juntos?

10.2 Vocabulário essencial

- **Item:** um produto, uma palavra, um sintoma.
- **Transação:** um conjunto de itens que ocorrem juntos (uma cesta, uma consulta, uma sessão).
- **Itemset:** qualquer conjunto de itens. Um itemset com *k* itens é um *k*-itemset.
- **Regra de associação:** uma afirmação do tipo **A → B** ("se A, então B"), onde A e B são itemsets sem itens em comum. A é o **antecedente**; B, o **consequente**.

Exemplo: {pão, manteiga} → {leite}. Leia: "quem leva pão e manteiga costuma levar leite".

> **ATENÇÃO** **A seta não é causa!**
>
> A → B significa **coocorrência**, não que A *cause* B. Sorvete e afogamentos aumentam juntos no verão, mas um não causa o outro. O calor explica os dois.

10.3 As três medidas que você precisa conhecer

Usaremos uma base com 10 compras:

#	Itens
1	pão, leite
2	pão, manteiga, leite
3	pão, manteiga
4	leite, café
5	pão, leite, café
6	pão, manteiga, café
7	leite, manteiga
8	pão, manteiga, leite, café
9	pão, café
10	pão, leite, manteiga

10.3.1 Suporte: "com que frequência?"

$$\text{suporte}(A) = (\text{nº de transações que contêm } A) / (\text{nº total de transações})$$

- suporte(pão) = 8/10 = 0,8
- suporte({pão, manteiga}) = 5/10 = 0,5

O suporte mede **quão comum** é o itemset. Evita regras baseadas em casos raros.

10.3.2 Confiança: "quão confiável é a regra?"

$$\text{confiança}(A \rightarrow B) = \text{suporte}(A \cup B) / \text{suporte}(A)$$

É a probabilidade de B aparecer, **dado que** A apareceu.

- confiança({pão, manteiga} → {leite}): pão e manteiga aparecem juntos em 5 compras (#2, #3, #6, #8 e #10). Dessas, 3 também têm leite (#2, #8 e #10). Então confiança = 3/5 = 0,6.

10.3.3 Lift: "é mais que coincidência?"

A confiança pode enganar: se o leite aparece em 90 % das compras, qualquer regra terminando em leite terá confiança alta, sem ter nenhum valor.

$$\text{lift}(A \rightarrow B) = \text{confiança}(A \rightarrow B) / \text{suporte}(B) = \text{suporte}(A \cup B) / (\text{suporte}(A) \cdot \text{suporte}(B))$$

- **lift = 1:** A e B são independentes.
- **lift > 1:** aparecem juntos **mais** do que o acaso (associação positiva).
- **lift < 1:** aparecem juntos **menos** que o acaso (associação negativa).

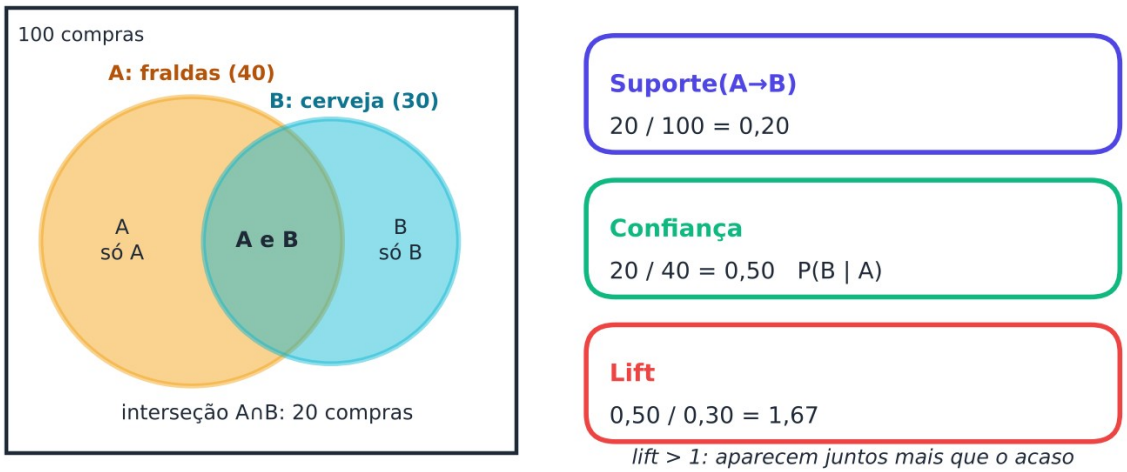


Figura 10.2 — Suporte, confiança e lift com diagramas de conjuntos: a interseção, comparada com o que se esperaria por acaso.

10.3.4 Outras medidas

- **Alavancagem (*leverage*):** suporte(A ∪ B) − suporte(A)·suporte(B). Diferença absoluta em relação à independência.
- **Convicção (*conviction*):** (1 − suporte(B)) / (1 − confiança). Mede o quanto a regra erra menos que o acaso (Brin et al., 1997).

- **Medida de Zhang e Jaccard**, entre outras (Tan; Kumar; Srivastava, 2004, comparam dezenas de medidas).

Medida	Responde	Faixa	Valor "neutro"
Suporte	É frequente?	0 a 1	—
Confiança	Quão confiável?	0 a 1	suporte(B)
Lift	Mais que o acaso?	0 a ∞	1
Alavancagem	Diferença do acaso	−0,25 a 0,25	0
Convicção	Quanto a regra "acerta"?	0 a ∞	1

10.4 O desafio: explosão combinatória

Numa loja com 1.000 itens, existem $2^{1000} - 1$ itemsets possíveis. É um número com mais de 300 dígitos. Impossível testar um por um!

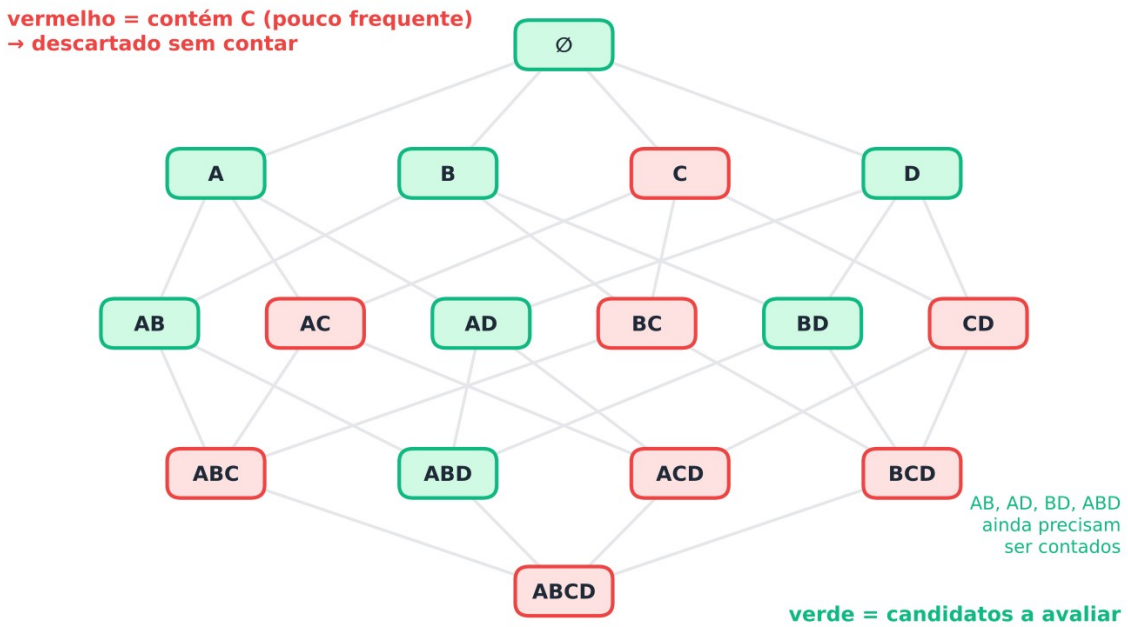


Figura 10.3 — O reticulado de itemsets. Se {A} é pouco frequente, todos os conjuntos acima dele (que contêm A) também são e podem ser descartados sem contagem.

10.5 O algoritmo Apriori

Agrawal e Srikant (1994) resolveram o problema com uma observação simples, a **propriedade Apriori** (ou anti-monotonicidade do suporte):

> *Se um itemset é **frequente**, todos os seus subconjuntos também são. Equivalentemente: se um itemset **não** é frequente, nenhum superconjunto dele pode ser.*

Analogia: se pouca gente compra "caviar", menos gente ainda compra "caviar e champanhe". Se caviar já é raro, não preciso nem testar as combinações com caviar.

10.5.1 Passo a passo

Defina um **suporte mínimo** (por exemplo, 0,3).

1. **Nível 1:** conte o suporte de cada item. Mantenha os que atingem o mínimo (itemsets frequentes de tamanho 1).
2. **Gerar candidatos:** combine itemsets frequentes de tamanho k para formar candidatos de tamanho $k + 1$. **Poda:** descarte candidatos com algum subconjunto não frequente.
3. **Contar:** percorra os dados e conte o suporte dos candidatos que sobraram. Mantenha os frequentes.
4. **Repetir** até não haver novos itemsets frequentes.
5. **Gerar regras:** para cada itemset frequente, teste divisões em antecedente e consequente. Mantenha as que atingem a confiança mínima.

10.5.2 Aplicando à nossa base (suporte mínimo = 0,5)

- **1-itemsets:** pão (0,8), leite (0,7), manteiga (0,6), café (0,5). Todos passam.
- **2-itemsets candidatos:** {pão, leite} = 0,5; {pão, manteiga} = 0,5; {pão, café} = 0,4 (não passa); {leite, manteiga} = 0,4 (não passa); {leite, café} = 0,3 (não passa); {manteiga, café} = 0,2 (não passa).
- **3-itemsets:** com {pão, leite} e {pão, manteiga} frequentes, o candidato {pão, leite, manteiga} exige {leite, manteiga}, que não é frequente → **podado**.
- **Fim.** Regras possíveis: pão → manteiga (confiança 5/8 = 0,625), manteiga → pão (5/6 ≈ 0,83), pão → leite (5/8 = 0,625), leite → pão (5/7 ≈ 0,71).

Veja como a poda poupou trabalho: nem contamos o suporte dos 3-itemsets.

10.5.3 Custo

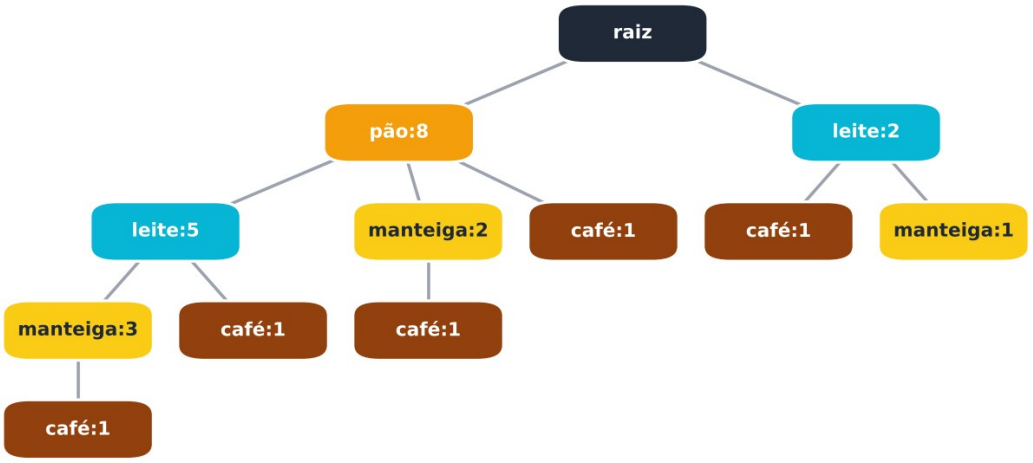
O Apriori pode exigir **muitas leituras** da base (uma por nível) e gerar **muitos candidatos**. Funciona bem para bases moderadas e suporte mínimo razoável.

10.6 O FP-Growth

Han, Pei e Yin (2000) propuseram o **FP-Growth** (*Frequent Pattern Growth*), que **não gera candidatos**.

10.6.1 Ideia

1. **Primeira leitura:** conte os itens e descarte os pouco frequentes. Ordene os itens de cada transação do mais frequente para o menos.
2. **Segunda leitura:** insira cada transação numa **árvore de prefixos** (a **FP-tree**). Transações que começam do mesmo jeito **compartilham** o caminho, e cada nó guarda uma contagem.
3. **Minerar a árvore:** para cada item, extraia seus caminhos (a "base condicional") e construa subárvores menores, recursivamente.



Itens ordenados por frequência: pão > leite > manteiga > café. Nó 'item:n' = n compras passam por aquele caminho.

Figura 10.4 — FP-tree: transações com início em comum compartilham caminhos, comprimindo a base numa estrutura compacta.

Vantagens: apenas duas leituras da base; sem candidatos; costuma ser muito mais rápido que o Apriori quando há muitos itens e suporte mínimo baixo. Em bases pequenas, com poucos itens, o Apriori pode ser tão rápido quanto (ou até mais rápido que) o FP-Growth: meça no seu caso.

Aspecto	Apriori	FP-Growth
Estratégia	Gerar e testar candidatos	Árvore compacta, sem candidatos
Leituras da base	Uma por nível	Duas
Memória	Menor	Precisa guardar a árvore
Velocidade	Mais lento com muitos itens	Em geral mais rápido com muitos itens
Facilidade	Muito simples	Mais elaborado

Outras abordagens: **Eclat** (Zaki, 2000), que usa listas verticais de identificadores de transações.

10.7 Mãos à obra com mlxtend

```python
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules

transacoes = [
    ["pão", "leite"],
    ["pão", "manteiga", "leite"],
    ["pão", "manteiga"],
    ["leite", "café"],
    ["pão", "leite", "café"],
    ["pão", "manteiga", "café"],
    ["leite", "manteiga"],
    ["pão", "manteiga", "leite", "café"],
    ["pão", "café"],
    ["pão", "leite", "manteiga"],
```

```
]

# 1) Transformar listas em tabela verdadeiro/falso (one-hot)
te = TransactionEncoder()
df = pd.DataFrame(te.fit(transacoes).transform(transacoes), columns=te.columns_)

# 2) Itemsets frequentes (suporte mínimo 0,3)
freq = apriori(df, min_support=0.3, use_colnames=True)
print(freq.sort_values("support", ascending=False))

# O FP-Growth devolve o mesmo resultado, normalmente mais rápido
freq2 = fpgrowth(df, min_support=0.3, use_colnames=True)
print(len(freq), len(freq2))

# 3) Regras de associação
regras = association_rules(freq, metric="confidence", min_threshold=0.6)
cols = ["antecedents", "consequents", "support", "confidence", "lift"]
print(regras[cols].sort_values("lift", ascending=False).round(2))
```

10.7.1 Lendo a tabela de regras

Cada linha traz o antecedente, o consequente e as métricas. Ordene por **lift** para ver as associações mais surpreendentes. Combine com suporte alto para garantir relevância.

10.8 Filtrando regras úteis

Um minerador pode produzir **milhares** de regras. Poucas são úteis. Boas práticas:

1. **Defina um suporte mínimo realista.** Muito baixo: explosão de regras. Muito alto: perde padrões raros mas valiosos.
2. **Priorize lift > 1,2** (ou outro limiar), além de boa confiança.
3. **Elimine regras redundantes:** se {A} → {C} já tem confiança parecida com {A, B} → {C}, a segunda adiciona pouco.
4. **Descarte regras triviais** (ex.: "feijão → arroz" numa loja brasileira) e **inexplicáveis** (podem ser acaso).
5. **Valide com especialistas** e, se possível, com um **teste A/B**.

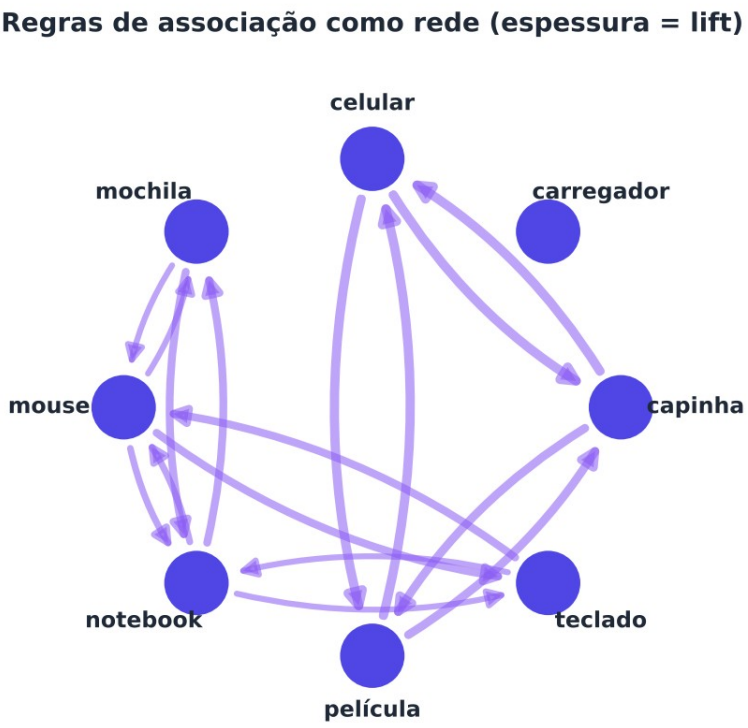


Figura 10.5 — Regras como rede: nós são produtos, setas são regras, a espessura indica o lift.

> SAIBA MAIS **Regras interessantes**
>
> Pesquisadores propuseram diversas medidas de "interesse" porque suporte e confiança sozinhos são insuficientes (Tan; Kumar; Srivastava, 2004; Geng; Hamilton, 2006). Nenhuma medida é a melhor para todos os casos.

10.9 Extensões

- **Regras com quantidades e categorias:** discretizam atributos numéricos (ex.: idade em faixas) para usá-los como itens (Srikant; Agrawal, 1996).
- **Regras com hierarquia de itens:** "refrigerante" e "refrigerante de cola" (Srikant; Agrawal, 1995).
- **Padrões sequenciais:** a ordem importa. "Comprou celular → comprou capinha → comprou película" (Agrawal; Srikant, 1995).
- **Regras negativas:** "quem compra A *não* compra B".
- **Mineração de itemsets de alta utilidade:** considera o lucro ou a quantidade de cada item, e não só a presença.

10.10 Estudo de caso: recomendação em uma loja virtual

Uma loja online quer sugerir "compre junto". Passos:

1. Cada **pedido** vira uma transação.
2. Mineram-se regras com suporte mínimo baixo (0,5 %) e lift > 2.
3. Para cada produto, recomendam-se os consequentes das melhores regras.
4. Compara-se, num teste A/B, a taxa de conversão com e sem as sugestões.

```python
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules

pedidos = [
    ["notebook", "mouse", "mochila"], ["notebook", "mouse"], ["celular", "capinha", "película"],
    ["celular", "capinha"], ["notebook", "mochila"], ["celular", "película"],
    ["notebook", "mouse", "teclado"], ["celular", "capinha", "película", "carregador"],
    ["mouse", "teclado"], ["notebook", "mouse", "mochila", "teclado"],
] * 5

te = TransactionEncoder()
df = pd.DataFrame(te.fit(pedidos).transform(pedidos), columns=te.columns_)
freq = fpgrowth(df, min_support=0.15, use_colnames=True)
r = association_rules(freq, metric="lift", min_threshold=1.2)

def recomendar(produto, regras, n=3):
    sel = regras[regras["antecedents"].apply(lambda s: s == frozenset([produto]))]
    sel = sel.sort_values(["lift", "confidence"], ascending=False).head(n)
    return [list(c)[0] for c in sel["consequents"] if len(c) == 1]

print("Quem compra 'celular' também compra:", recomendar("celular", r))
```

10.11 Armadilhas

1. **Correlação ≠ causa.**
2. **Paradoxo de Simpson:** uma associação pode inverter-se quando os dados são segmentados (por região, por tipo de cliente).
3. **Muitas regras, muita coincidência.** Com milhares de testes, algumas regras "aparecem" por puro acaso. Considere correções estatísticas (Webb, 2007).
4. **Itens muito frequentes** (sacola plástica) geram regras sem interesse.
5. **Mudanças no tempo.** Hábitos de consumo mudam; reavalie periodicamente.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Regra A → B:** coocorrência, não causa.
- **Suporte** = frequência. **Confiança** = P(B | A). **Lift** = quanto acima do acaso.
- **Apriori:** gera candidatos e poda com a propriedade "subconjunto de frequente é frequente".
- **FP-Growth:** comprime os dados numa árvore e minera sem candidatos.
- **Filtre** por lift, remova redundantes e valide.
- **Cuidado** com coincidências, triviais e Simpson.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. A análise da cesta de compras busca descobrir:

a) itens que costumam ser comprados juntos.
b) o estoque mínimo.
c) clientes com menor renda.
d) o preço ideal de cada item.

Questão 2. Em regras de associação, uma transação é:

a) um grupo de clientes.
b) um item isolado.
c) um conjunto de itens que ocorrem juntos (por exemplo, uma compra).
d) uma regra.

Questão 3. Em uma regra $A \rightarrow B$, A é chamado de:

a) lift.
b) consequente.
c) suporte.
d) antecedente.

Questão 4. A regra $A \rightarrow B$ significa que:

a) A e B são idênticos.
b) A e B coocorrem com certa frequência, sem implicar causalidade.
c) A causa B.
d) B causa A.

Questão 5. O suporte de um itemset é:

a) o número de itens.
b) a razão entre confiança e lift.
c) a probabilidade condicional de B dado A.
d) a fração de transações que o contêm.

Questão 6. Na base de 10 compras do capítulo, o suporte do pão é:

a) 0,6.
b) 0,8.
c) 0,5.
d) 1,0.

Questão 7. A confiança de $A \rightarrow B$ é calculada como:

a) suporte(A) / suporte(B).
b) suporte(A) × suporte(B).
c) suporte(B) − suporte(A).
d) suporte(A ∪ B) / suporte(A).

Questão 8. O lift de uma regra igual a 1 indica que A e B são:

a) fortemente associados.
b) independentes.
c) idênticos.
d) negativamente associados.

Questão 9. Um lift igual a 1,67 significa que A e B aparecem juntos:

a) exatamente como esperado.
b) 67% mais do que o esperado se fossem independentes.
c) 67% menos do que o esperado.
d) nunca juntos.

Questão 10. Por que o lift é útil além da confiança?

a) Porque a confiança pode ser alta só porque B é muito frequente.
b) Porque a confiança não existe.
c) Porque o lift não depende dos dados.
d) Porque o lift é sempre maior.

Questão 11. Na regra pão → manteiga da base de 10 compras, a confiança é:

a) 0,625.
b) 0,8.
c) 1,0.
d) 0,5.

Questão 12. Na mesma base, a confiança de manteiga → pão é aproximadamente:

a) 0,50.
b) 0,62.
c) 0,83.
d) 1,00.

Questão 13. A propriedade Apriori diz que:

a) todo subconjunto de um itemset frequente também é frequente.
b) todo superconjunto de um itemset frequente é frequente.
c) nenhum itemset é frequente.
d) todo itemset é frequente.

Questão 14. A propriedade Apriori permite:

a) calcular o lift diretamente.
b) padronizar atributos.
c) escolher k.
d) podar candidatos sem contar seu suporte.

Questão 15. No exemplo do capítulo, o candidato {pão, leite, manteiga} é podado porque:

a) o suporte do pão é zero.
b) pão não é frequente.
c) manteiga não existe.
d) {leite, manteiga} não é frequente (suporte 0,4 < 0,5).

Questão 16. O número de itemsets possíveis para n itens é:

a) 2n.
b) $2^n - 1$.
c) n^2.
d) n!

Questão 17. O FP-Growth difere do Apriori principalmente porque:

a) exige rótulos.
b) usa distância Euclidiana.
c) não gera candidatos, usando uma árvore de prefixos (FP-tree).
d) depende de k.

Questão 18. Em uma FP-tree, transações que começam do mesmo modo:

a) viram ruído.
b) são removidas.
c) compartilham o caminho, e cada nó guarda uma contagem.
d) são duplicadas.

Questão 19. O algoritmo Eclat usa:

a) árvores de isolamento.
b) kernels.
c) autovetores.
d) listas verticais de identificadores de transações.

Questão 20. No pacote mlxtend, os dados de transações precisam ser transformados com:

a) TransactionEncoder, gerando uma tabela verdadeiro/falso.
b) KMeans.
c) PCA.
d) StandardScaler.

Questão 21. A função que gera as regras a partir dos itemsets frequentes é:

a) score_samples.
b) association_rules.
c) linkage.
d) fit_predict.

Questão 22. A alavancagem (leverage) mede:

a) o número de transações.
b) a razão entre confiança e suporte.
c) suporte(A ∪ B) − suporte(A)·suporte(B).
d) a distância entre A e B.

Questão 23. Qual é um bom critério para filtrar regras úteis?

a) Escolher só as de suporte mínimo.
b) Escolher regras ao acaso.
c) Lift acima de um limiar (por exemplo, 1,2) combinado a suporte e confiança adequados.
d) Escolher só as triviais.

Questão 24. Uma regra "feijão → arroz" numa loja brasileira seria um exemplo de regra:

a) trivial.
b) anômala.
c) inválida.
d) de lift negativo.

Questão 25. O paradoxo de Simpson refere-se a:

a) uma associação que pode se inverter quando os dados são segmentados.
b) um tipo de kernel.
c) a distribuição de probabilidade.
d) uma regra de poda.

Questão 26. Com milhares de regras testadas, o risco é que:

a) o lift seja sempre 1.
b) nenhuma seja válida.
c) algumas apareçam por puro acaso.
d) o suporte seja negativo.

Questão 27. Padrões sequenciais diferem de regras de associação comuns porque:

a) a ordem dos itens importa.
b) exigem k.
c) usam rótulos.
d) usam distâncias.

Questão 28. Para usar atributos numéricos (como idade) em regras de associação, deve-se:

a) multiplicá-los por zero.
b) excluí-los sempre.
c) mantê-los contínuos sem tratamento.
d) discretizá-los em faixas para tratá-los como itens.

Questão 29. Em um teste A/B para validar recomendações "compre junto", compara-se:

a) a inércia dos grupos.
b) a perplexidade.
c) o tamanho da base.
d) a taxa de conversão com e sem as sugestões.

Questão 30. Qual é uma limitação do Apriori em bases grandes?

a) Muitas leituras da base e geração de muitos candidatos.
b) Exigir rótulos.
c) Funcionar só com dois itens.
d) Não calcular suporte.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique a análise da cesta de compras e cite três aplicações fora do varejo.
2. Defina item, transação, itemset e regra de associação com um exemplo.
3. Por que "A → B" não significa que A causa B? Dê um exemplo.
4. Calcule suporte, confiança e lift da regra {pão} → {leite}, sabendo que suporte(pão) = 0,8, suporte(leite) = 0,7 e suporte({pão, leite}) = 0,5.
5. Interprete um lift de 1,67 para fraldas → cerveja com suporte de 20%.
6. Explique por que a confiança sozinha pode enganar e como o lift corrige isso.
7. Descreva a propriedade Apriori e dê um exemplo de poda.
8. Descreva, passo a passo, o algoritmo Apriori.
9. Aplique o Apriori à base de 10 compras com suporte mínimo 0,5 e liste os itemsets frequentes.
10. Explique como o FP-Growth funciona e por que costuma ser mais rápido que o Apriori.
11. Compare Apriori e FP-Growth em estratégia, leituras da base, memória e velocidade.
12. Cite cinco boas práticas para filtrar regras úteis em um conjunto com milhares de regras.
13. Descreva um fluxo para construir um sistema "compre junto" em uma loja virtual.
14. Explique o paradoxo de Simpson no contexto das regras de associação.
15. Por que itens muito frequentes (como a sacola plástica) geram regras sem interesse?
16. Explique a importância de corrigir testes múltiplos ao minerar regras.
17. Dê dois exemplos de extensões das regras de associação e explique-as.
18. Por que os hábitos de consumo exigem reavaliação periódica das regras?
19. Explique a diferença entre alavancagem e lift.
20. Descreva como transformar uma lista de compras em tabela binária para usar no mlxtend e para que serve.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Suporte, confiança e lift "na unha"

NÍVEL: BÁSICO

Use as dez compras do Capítulo 10 (lista abaixo). Com `pandas`, calcule suporte, confiança e lift da regra **{pão} → {manteiga}** e confira os valores com `mlxtend` (`apriori` + `association_rules`).

PYTHON

```
compras = [
    ["pão", "leite"],
    ["pão", "manteiga", "leite"],
    ["pão", "manteiga"],
    ["leite", "café"],
    ["pão", "leite", "café"],
    ["pão", "manteiga", "café"],
    ["leite", "manteiga"],
    ["pão", "manteiga", "leite", "café"],
    ["pão", "café"],
    ["pão", "leite", "manteiga"],
]
```

Exercício 2 — Apriori ou FP-Growth?

NÍVEL: INTERMEDIÁRIO

Gere 20 mil cestas simuladas (código abaixo) e compare `apriori` e `fpgrowth` (suporte mínimo 0,02) quanto ao tempo e ao número de itemsets. Confirme que encontram os mesmos itemsets.

PYTHON

```
import numpy as np
import pandas as pd


def gerar_cestas(n=20000, seed=0):
    rng = np.random.default_rng(seed)
    p = np.array(
        [0.5, 0.4, 0.35, 0.3, 0.25, 0.2, 0.15, 0.12, 0.1, 0.08, 0.06, 0.05, 0.04, 0.03, 0.02]
    )
    M = rng.random((n, len(p))) < p
    M[M[:, 0], 1] |= rng.random(M[:, 0].sum()) < 0.5  # regra plantada: item0 -> item1
    return pd.DataFrame(M, columns=[f"item{i}" for i in range(len(p))])
```

Exercício 3 — Como o suporte mínimo controla a explosão de regras

NÍVEL: INTERMEDIÁRIO

Com as cestas do exercício anterior, varie `min_support` em 0,10; 0,05; 0,03 e 0,02 (use `fpgrowth`). Para cada valor, conte itemsets e regras (confiança mínima de 0,3).

Exercício 4 — Regras em dados reais: o conjunto Wine

NÍVEL: AVANÇADO

Transforme o **Wine** (UCI, via scikit-learn) em "itens": discretize cada atributo em 3 faixas (baixo, médio, alto) com `KBinsDiscretizer` (estratégia `quantile`) e acrescente a classe do vinho como item (`classe_0`, `classe_1`, `classe_2`). Minere regras com `fpgrowth` (suporte mínimo 0,1) e liste as 5 regras de maior lift cujo **consequente seja uma classe** e cuja confiança seja de pelo menos 0,9.

Exercício 5 — Filtrando regras e recomendando produtos

NÍVEL: AVANÇADO

Com as cestas do Exercício 2 (suporte mínimo 0,02), gere regras com lift de pelo menos 1,2 e confiança de pelo menos 0,3. Elimine redundâncias: descarte uma regra se existir outra com o **mesmo consequente**, antecedente **menor** (subconjunto) e confiança pelo menos igual. Escreva uma função `recomendar(itens_na_cesta, regras, n=3)` que devolva os consequentes das melhores regras aplicáveis.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Online Retail e Online Retail II (cada nota fiscal é uma cesta).
> - **Kaggle:** Market Basket Optimisation, Groceries Dataset e Instacart Market Basket Analysis.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Cesta de supermercado: encontrando regras de verdade

Contexto. O mercado (fictício) "Bom Preço" quer reorganizar prateleiras e criar combos. Você analisará o histórico de cestas e deverá separar **regras reais** de **coincidências**.

Fonte dos dados. Cestas **simuladas** pela função abaixo, em que **regras foram plantadas de propósito**. Assim você poderá medir se o seu método as recupera. (Para praticar com dados reais, veja o quadro "Para ir além" no final do capítulo.)

```python
import numpy as np
import pandas as pd

PRODUTOS = [
    "pão",
    "manteiga",
    "leite",
    "café",
    "açúcar",
    "arroz",
    "feijão",
    "macarrão",
    "molho",
    "queijo",
    "presunto",
    "cerveja",
    "salgadinho",
    "fralda",
    "refrigerante",
    "sabão",
    "detergente",
    "esponja",
]
PLANTADAS = [
    ("pão", "manteiga"),
    ("café", "açúcar"),
    ("macarrão", "molho"),
    ("cerveja", "salgadinho"),
    ("sabão", "detergente"),
]


def gerar_mercado(n=12000, seed=0):
    rng = np.random.default_rng(seed)
    p = rng.uniform(0.04, 0.35, len(PRODUTOS))
    M = rng.random((n, len(PRODUTOS))) < p
    ix = {nome: i for i, nome in enumerate(PRODUTOS)}
    for a, b in PLANTADAS:
        tem = M[:, ix[a]]
        M[tem, ix[b]] |= rng.random(tem.sum()) < 0.55
    return pd.DataFrame(M, columns=PRODUTOS)
```

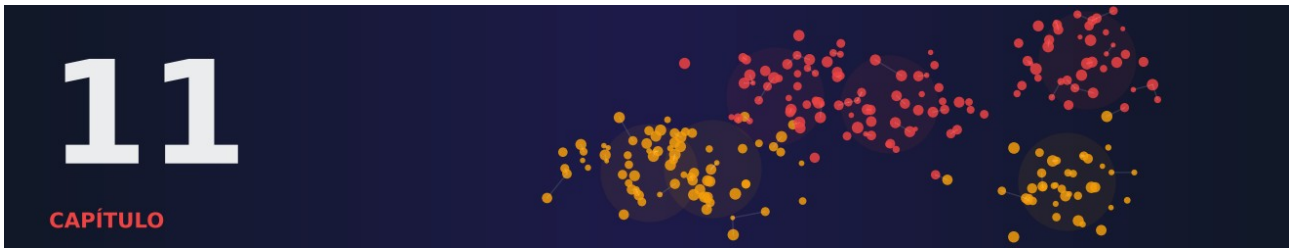
Tarefas.

1. Calcule o suporte de cada produto e liste os 5 mais frequentes.

2. Minere regras com `fpgrowth` (suporte mínimo 0,02) e confiança mínima de 0,3. Quantas regras surgem?
3. Filtre por **lift ≥ 1,5** e mantenha só regras de um item → um item. Compare com a lista de regras plantadas: calcule **precisão** (fração das regras mantidas que são plantadas, em qualquer sentido) e **revocação** (fração das plantadas recuperadas).
4. Varie o lift mínimo (1,2; 1,5; 2,0) e mostre como precisão e revocação mudam.
5. Proponha 3 combos de promoção a partir das melhores regras, com justificativa baseada em suporte, confiança e lift.
6. Discuta dois cuidados: correlação ≠ causa e o risco de regras espúrias com muitos testes.

Entregáveis. Script, tabela de precisão/revocação por limiar, 3 combos e texto de cuidados.

A solução de referência está no Apêndice E.



Capítulo 11 — Detecção de Anomalias: Encontrando o que Foge do Normal

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Definir anomalia e distinguir anomalias pontuais, contextuais e coletivas.
- Diferenciar abordagens supervisionada, semissupervisionada e não supervisionada.
- Aplicar métodos estatísticos simples (z-score, MAD, IQR).
- Explicar e usar Isolation Forest, LOF, One-Class SVM e GMM para detectar anomalias.
- Escolher limiares e avaliar detectores com métricas adequadas a classes desbalanceadas.
- Interpretar alertas e reduzir falsos positivos.

11.1 Uma agulha no palheiro

Imagine um guarda de segurança que olha mil câmeras. Quase tudo é rotina. Seu trabalho é perceber **aquela cena diferente**: alguém pulando o muro, uma mala abandonada.

Detectar anomalias é exatamente isso. Uma **anomalia** (ou *outlier*, ou "valor atípico") é um dado que **destoa do padrão esperado**. Hawkins (1980) a definiu como "uma observação que se desvia tanto das demais que desperta a suspeita de ter sido gerada por um mecanismo diferente".

Aplicações:

- **Finanças:** fraude em cartão, lavagem de dinheiro.
- **Indústria:** falhas de máquinas antes da quebra (manutenção preditiva).
- **Saúde:** exames atípicos, eletrocardiogramas anormais.
- **TI e segurança:** intrusões em redes, comportamento incomum de usuário.
- **Qualidade de dados:** erros de digitação e sensores defeituosos.



Figura 11.1 — Anomalias são raras e diferentes: o desafio é achar os poucos pontos que fogem do comportamento normal.

11.2 Tipos de anomalia

Chandola, Banerjee e Kumar (2009) organizam as anomalias em três tipos.

1. **Pontual:** um único ponto muito diferente do resto. *Exemplo:* uma compra de R$ 50.000 num cartão cujo gasto médio é R$ 300.
2. **Contextual:** um valor normal em um contexto, mas anormal em outro. *Exemplo:* 30 °C é normal em janeiro e anormal em julho no Sul do Brasil.
3. **Coletiva:** um **conjunto** de pontos é anormal, embora cada um, isolado, seja normal. *Exemplo:* uma sequência de batimentos cardíacos que, individualmente, parecem normais, mas juntos revelam arritmia.

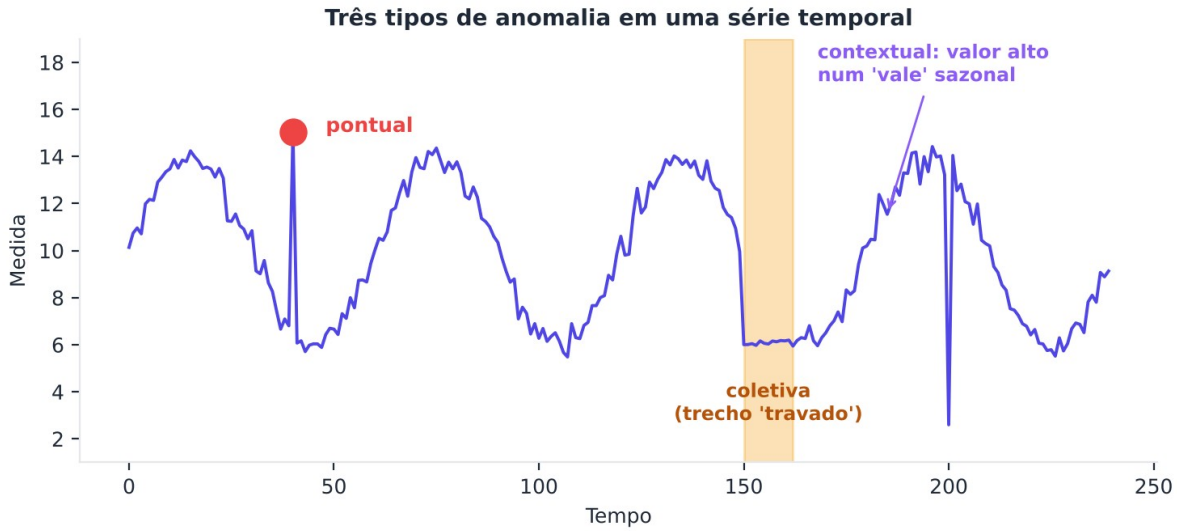


Figura 11.2 — Os três tipos de anomalia numa série temporal: pontual, contextual e coletiva.

11.3 Os três cenários de aprendizado

- **Supervisionado:** há exemplos rotulados de normal **e** de anômalo. Vira um problema de classificação, mas com classes muito desbalanceadas.
- **Semissupervisionado (*one-class*):** só há exemplos **normais** para treinar. Qualquer coisa muito diferente é suspeita.
- **Não supervisionado: nenhum rótulo.** Assume-se que anomalias são **raras** e **diferentes**. É o foco deste capítulo.

11.4 Métodos estatísticos simples

Antes de algoritmos sofisticados, experimente o básico. Funcionam surpreendentemente bem.

11.4.1 Z-score

Calcula quantos desvios padrão um ponto está da média. Regra comum: |z| > 3 é suspeito.

$$z = (x - \text{média}) \,/\, \text{desvio padrão}$$

Fraqueza: média e desvio são afetados pelos próprios outliers ("mascaramento").

11.4.2 Z-score modificado (MAD)

Usa a **mediana** e o **desvio absoluto mediano** (MAD), robustos a extremos (Iglewicz; Hoaglin, 1993):

$$z_{\text{mod}} = 0{,}6745 \cdot (x - \text{mediana}) \,/\, \text{MAD}$$

Regra: |z_mod| > 3,5 é suspeito.

11.4.3 Regra do IQR (boxplot)

Calcule o primeiro quartil (Q1, abaixo do qual estão 25 % dos dados) e o terceiro (Q3). O **IQR** = Q3 − Q1. São suspeitos os valores abaixo de Q1 − 1,5·IQR ou acima de Q3 + 1,5·IQR (Tukey, 1977).

```python
import numpy as np

rng = np.random.default_rng(1)
x = np.concatenate([rng.normal(100, 10, 200), [180, 195, 20]])   # 3 anomalias plantadas

z = (x - x.mean()) / x.std()
mad = np.median(np.abs(x - np.median(x)))
zmod = 0.6745 * (x - np.median(x)) / mad
q1, q3 = np.percentile(x, [25, 75])
iqr = q3 - q1

print("Z-score  |z|>3     :", np.where(np.abs(z) > 3)[0])
print("MAD      |z|>3.5   :", np.where(np.abs(zmod) > 3.5)[0])
print("IQR  (1,5 x IQR)   :", np.where((x < q1 - 1.5 * iqr) | (x > q3 + 1.5 * iqr))[0])
```

11.4.4 Distância de Mahalanobis

Para vários atributos, a distância Euclidiana ignora as correlações. A **distância de Mahalanobis** leva em conta a covariância: mede a distância em "desvios padrão" na direção do ponto, considerando a forma da nuvem (Mahalanobis, 1936).

Exemplo: uma pessoa de 1,90 m e 50 kg. Cada medida é comum, mas **a combinação** é incomum, pois altura e peso costumam andar juntos. Mahalanobis percebe isso; a Euclidiana, não.

```python
import numpy as np
from sklearn.covariance import EmpiricalCovariance, MinCovDet

rng = np.random.default_rng(0)
altura = rng.normal(170, 8, 300)
peso = 0.9 * (altura - 100) + rng.normal(0, 5, 300)
X = np.column_stack([altura, peso])
X = np.vstack([X, [190, 50]])                    # combinação atípica

robusto = MinCovDet(random_state=0).fit(X)       # estimador robusto de covariância
d2 = robusto.mahalanobis(X)                      # distância ao quadrado
print("Índices mais suspeitos:", np.argsort(d2)[-3:][::-1], " (o último é", len(X) - 1, ")")
```

11.5 Isolation Forest

11.5.1 A ideia genial

Liu, Ting e Zhou (2008) tiveram uma ideia diferente: em vez de modelar o "normal" e ver quem destoa, **isolar** os pontos.

Anomalias são **poucas e diferentes**, portanto, são **fáceis de isolar**.

Analogia: o jogo "20 perguntas". Para adivinhar uma pessoa comum numa multidão, você precisa de muitas perguntas. Para adivinhar o único homem de smoking numa festa de camiseta, basta uma.

11.5.2 Como funciona

1. Construa uma **árvore de isolamento**: escolha um atributo ao acaso e um valor de corte ao acaso entre o mínimo e o máximo. Divida os dados. Repita recursivamente até cada ponto ficar sozinho.
2. Repita para centenas de árvores com subamostras diferentes (a **floresta**).
3. Para cada ponto, calcule a **profundidade média** em que ele ficou isolado.
4. **Profundidade pequena = isolamento rápido = anomalia.**

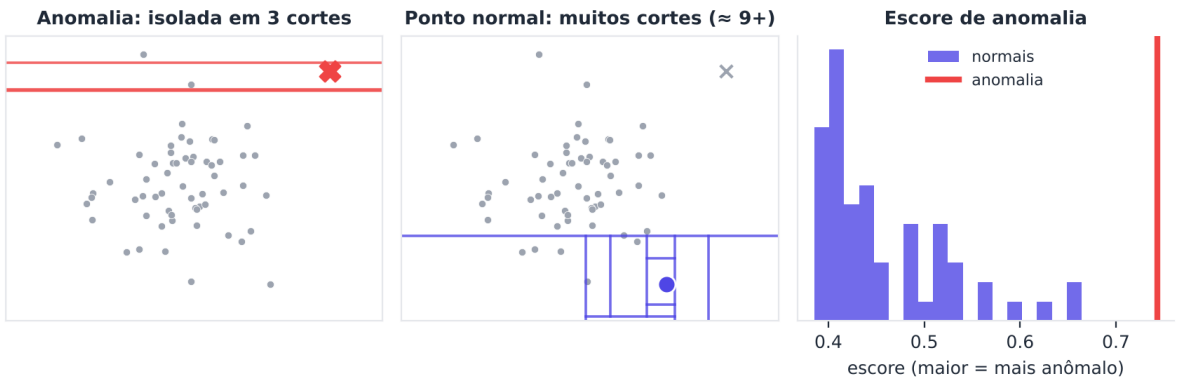


Figura 11.3 — Isolation Forest: o ponto anômalo é isolado com poucos cortes; um ponto normal, no meio da multidão, exige muitos.

O **escore de anomalia** é normalizado entre 0 e 1: perto de 1, anomalia; abaixo de 0,5, normal.

```python
import numpy as np
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(0)
normais = rng.normal(0, 1, size=(500, 2))
anomalias = rng.uniform(-6, 6, size=(15, 2))
X = np.vstack([normais, anomalias])

iso = IsolationForest(n_estimators=200, contamination=0.03, random_state=0).fit(X)
pred = iso.predict(X)                # +1 normal, -1 anomalia
escore = -iso.score_samples(X)       # quanto maior, mais anômalo

print("Anomalias apontadas:", (pred == -1).sum())
print("Acertos entre as 15 plantadas:", (pred[-15:] == -1).sum())
```

Vantagens: rápido (linear), poucos parâmetros, funciona em muitas dimensões. **Limites:** cortes paralelos aos eixos podem falhar com anomalias muito específicas; o parâmetro `contamination` precisa de estimativa.

11.6 LOF: Fator de Outlier Local

11.6.1 O problema da densidade variável

Em um bairro central, as casas são vizinhas de parede. Em um bairro rural, ficam a quilômetros. Uma casa a 200 m da vizinha é **normal** no campo, mas seria **estranha** no centro. A anomalia depende da **densidade local**.

O **LOF** (*Local Outlier Factor*; Breunig et al., 2000) compara a densidade em torno de um ponto com a densidade em torno de seus vizinhos.

- **LOF ≈ 1:** densidade parecida com a dos vizinhos (normal).
- **LOF >> 1:** o ponto está numa região muito menos densa que a dos vizinhos (anomalia).

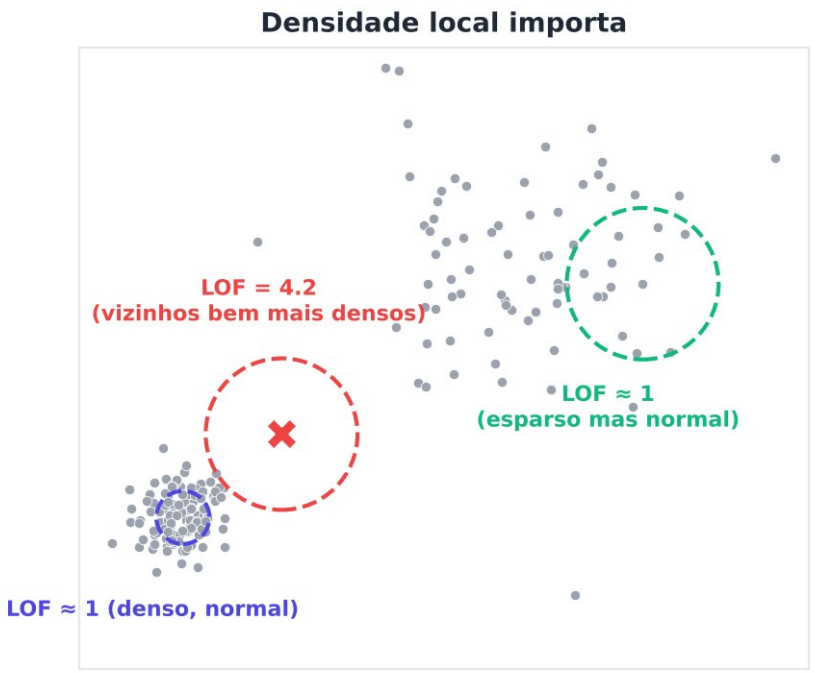


Figura 11.4 — LOF: o ponto vermelho está mais isolado que os vizinhos do grupo denso, então tem LOF alto, mesmo estando mais perto que pontos normais do grupo esparso.

```python
import numpy as np
from sklearn.neighbors import LocalOutlierFactor

rng = np.random.default_rng(0)
denso   = rng.normal([0, 0], 0.3, size=(300, 2))
esparso = rng.normal([5, 5], 1.5, size=(100, 2))
suspeito = np.array([[1.2, 1.2]])                  # perto do denso, mas fora dele
X = np.vstack([denso, esparso, suspeito])

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.01)
pred = lof.fit_predict(X)
escore = -lof.negative_outlier_factor_
print("Escore LOF do ponto suspeito:", round(escore[-1], 2))
print("Escore LOF mediano do resto :", round(float(np.median(escore[:-1])), 2))
```

Por padrão, o LOF só rotula os dados de treino. Para pontos novos, use `novelty=True` (e `predict` em dados novos).

> **ATENÇÃO — O LOF e os grupos de anomalias**
>
> O parâmetro `n_neighbors` precisa ser **maior que o tamanho dos grupos de anomalias**. Se há, por exemplo, 200 fraudes parecidas entre si e você usa 30 vizinhos, cada fraude só enxerga outras fraudes e parece normal: o LOF falha. No projeto do Capítulo 11, o AUPRC do LOF sobe de cerca de 0,02 (com 30 vizinhos) para cerca de 0,91 (com 300).

11.7 One-Class SVM

Schölkopf et al. (2001) propuseram aprender a **fronteira** que envolve os dados normais.

Ideia: usando um kernel (Capítulo 8), mapeie os dados para um espaço de alta dimensão e procure o hiperplano que separa os dados da **origem** com a maior margem. Tudo que ficar do lado da origem é anomalia.

Analogia: você desenha uma "cerca" em volta da multidão. Quem está fora é suspeito.

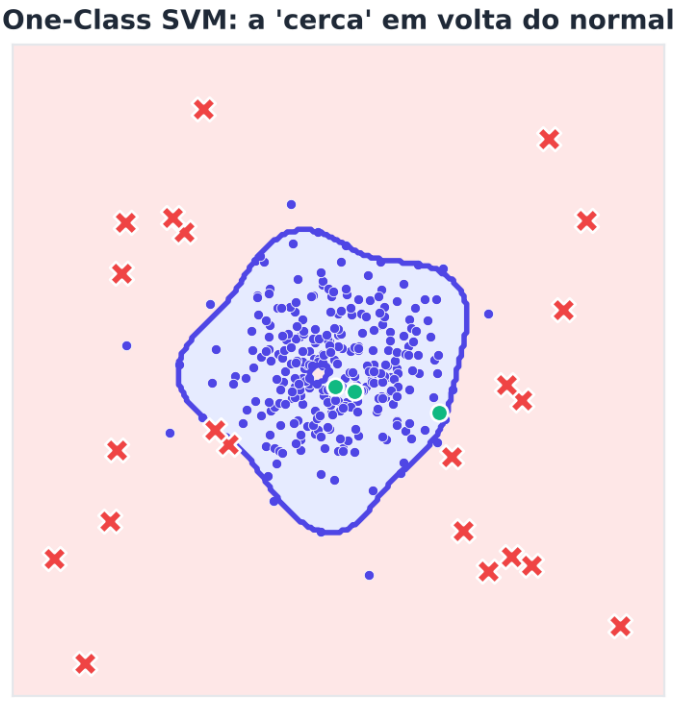


Figura 11.5 — One-Class SVM: a fronteira envolve a região normal e deixa de fora os pontos atípicos.

Parâmetros: `nu` (limite superior da fração de anomalias, entre 0 e 1) e `gamma` (flexibilidade da fronteira).

> **ATENÇÃO — O gamma faz muita diferença**
>
> Com `gamma="scale"`, o One-Class SVM pode ficar estreito demais e marcar muitos pontos normais como anomalias, ou ter desempenho pior que o acaso. Nos exercícios do capítulo, o mesmo modelo passa de AUC-ROC de 0,44 (gamma padrão) para 0,95 (gamma = 0,005). Ajuste e compare sempre.

```python
import numpy as np
from sklearn.svm import OneClassSVM
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
treino = rng.normal(0, 1, size=(400, 2))              # só dados normais
teste = np.vstack([rng.normal(0, 1, (50, 2)),         # normais
                   rng.uniform(-6, 6, (10, 2))])      # possíveis anomalias

esc = StandardScaler().fit(treino)
modelo = OneClassSVM(kernel="rbf", gamma=0.3, nu=0.05).fit(esc.transform(treino))
pred = modelo.predict(esc.transform(teste))
print("Marcados como anomalia:", (pred == -1).sum(), "de", len(teste))
```

11.8 Outros métodos importantes

- **Densidade com GMM** (Capítulo 6): pontos com baixa densidade sob o modelo são suspeitos. Use `score_samples`.
- **DBSCAN** (Capítulo 5): os pontos de ruído (rótulo −1) são candidatos.
- **kNN:** a distância ao *k*-ésimo vizinho como escore (Ramaswamy; Rastogi; Shim, 2000).
- **PCA:** o **erro de reconstrução** é alto para pontos que não seguem a estrutura principal.
- **Autoencoders** (Capítulo 12): treinados com dados normais, reconstroem mal os anômalos.
- **ECOD e COPOD** (Li et al., 2023; Li et al., 2020): métodos rápidos, baseados em caudas das distribuições, sem parâmetros.
- **Séries temporais:** modelos de previsão (ARIMA, redes recorrentes) e o erro de previsão como escore.

11.8.1 PCA para anomalias

```python
import numpy as np
from sklearn.decomposition import PCA

rng = np.random.default_rng(0)
base = rng.normal(size=(500, 3))
ruido = 0.1 * rng.normal(size=(500, 10))
X = base @ rng.normal(size=(3, 10)) + ruido    # dados em ~3 dimensões
X[0] += 5 * rng.normal(size=10)                # ponto que quebra a estrutura

pca = PCA(n_components=3).fit(X)
erro = np.sum((X - pca.inverse_transform(pca.transform(X))) ** 2, axis=1)
print("Ponto mais anômalo:", erro.argmax())
print("Erro dele:", erro.max().round(1), "| erro mediano:", np.median(erro).round(2))
```

11.9 Comparando os métodos

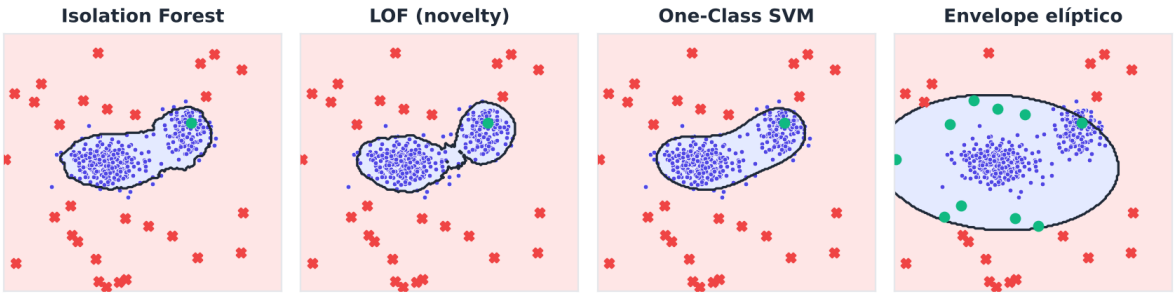


Figura 11.6 — Quatro detectores no mesmo conjunto de dados: cada um desenha uma fronteira diferente.

Método	Ideia	Pontos fortes	Pontos fracos
Z-score / MAD / IQR	Estatística simples	Rápido, explicável	Pouco para multivariado
Mahalanobis	Distância com covariância	Capta correlações	Supõe formato elíptico
Isolation Forest	Facilidade de isolar	Escalável, poucos parâmetros	Parâmetro de contaminação
LOF	Densidade local	Densidades variáveis	Escolha de *k*, custo
One-Class SVM	Fronteira em espaço kernel	Fronteiras flexíveis	Sensível a gamma/nu, lento

Método	Ideia	Pontos fortes	Pontos fracos
GMM	Densidade probabilística	Probabilidades	Supõe gaussianas
Autoencoder	Erro de reconstrução	Alta dimensão, imagens	Treino, ajuste

11.10 Definindo o limiar

Detectores devolvem um **escore**. Quem decide o que é anomalia é o **limiar**.

Opções:

1. **Fração esperada** (contaminação): marque os 1 % maiores escores.
2. **Percentil:** 99° percentil dos escores no treino.
3. **Teoria de valores extremos**: modela a cauda dos escores (Siffer et al., 2017).
4. **Custo:** escolha o limiar que minimiza o custo total de falsos positivos e falsos negativos.
5. **Capacidade operacional:** o time só consegue investigar 50 alertas por dia → o limiar é o 50° maior escore.

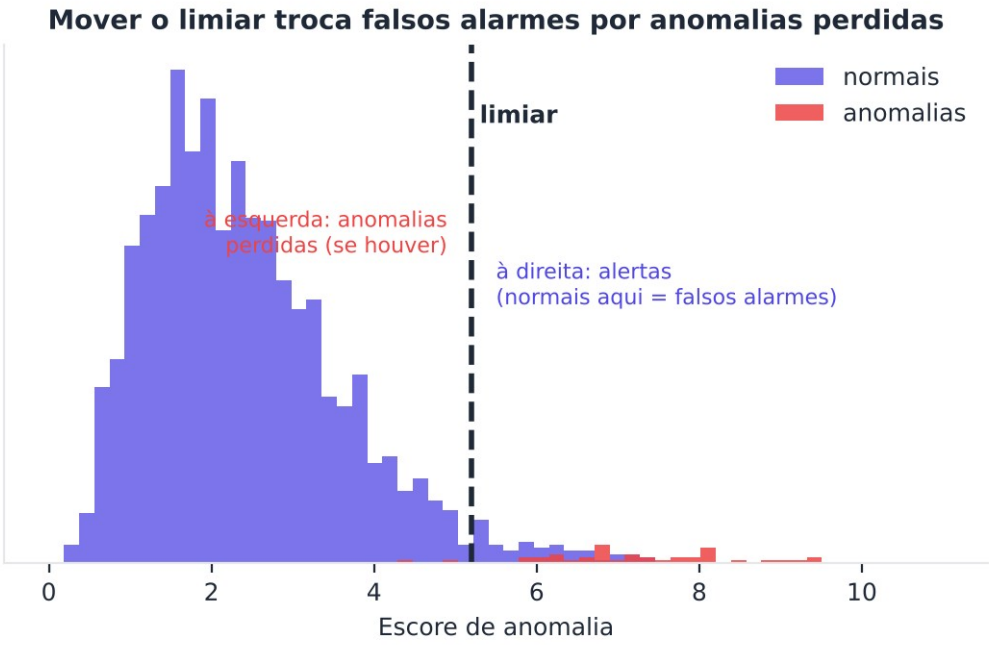


Figura 11.7 — Histograma dos escores: o limiar separa o "normal" do "suspeito". Mover o limiar troca falsos alarmes por anomalias perdidas.

11.11 Como avaliar?

Em anomalias, as classes são muito desbalanceadas (ex.: 0,1 % de fraudes). Um modelo que sempre diz "normal" teria 99,9 % de **acurácia** e seria inútil!

Prefira:

- **Precisão (*precision*):** dos alertas, quantos eram anomalias reais?
- **Revocação (*recall*):** das anomalias reais, quantas foram pegas?
- **F1:** média harmônica de precisão e revocação.
- **Curva Precisão-Revocação e AUPRC:** mais informativas que a ROC em dados muito desbalanceados (Saito; Rehmsmeier, 2015).

- **AUC-ROC:** útil, mas otimista com desbalanceamento extremo.
- **Precisão@k:** dos *k* maiores escores, quantos são reais?

```python
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.metrics import roc_auc_score, average_precision_score

rng = np.random.default_rng(0)
X = np.vstack([rng.normal(0, 1, (1000, 4)), rng.normal(3, 1.5, (20, 4))])
y = np.r_[np.zeros(1000), np.ones(20)]                    # só para avaliar!

escore = -IsolationForest(random_state=0).fit(X).score_samples(X)
print("AUC-ROC:", round(roc_auc_score(y, escore), 3))
print("AUPRC  :", round(average_precision_score(y, escore), 3))
print("Linha de base AUPRC (acaso):", round(y.mean(), 3))
```

11.12 Do alerta à ação

- **Explique o alerta.** Quais atributos mais contribuíram? Técnicas como SHAP ajudam (Lundberg; Lee, 2017).
- **Reduza falsos positivos** com regras de negócio e um segundo estágio de verificação.
- **Feedback humano:** guarde o que os analistas confirmam e use depois para treinar um modelo supervisionado.
- **Monitore a deriva** (*drift*): o "normal" muda com o tempo.
- **Cuidado com viés:** um detector pode marcar sistematicamente grupos minoritários.
- **Segurança:** adversários podem tentar imitar o normal.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Anomalia** = rara e diferente; pode ser pontual, contextual ou coletiva.
- **Estatística simples** (z-score, MAD, IQR, Mahalanobis) é um bom ponto de partida.
- **Isolation Forest:** anomalias são fáceis de isolar (caminho curto).
- **LOF:** compara a densidade local com a dos vizinhos.
- **One-Class SVM:** cerca a região normal.
- **Limiar** é uma decisão de negócio.
- **Avalie** com precisão, revocação e AUPRC, **não** com acurácia.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Hawkins (1980) definiu outlier como uma observação que:

a) se desvia tanto das demais que desperta a suspeita de ter sido gerada por outro mecanismo.
b) é sempre um erro de digitação.
c) é a média dos dados.
d) é o ponto mais frequente.

Questão 2. Uma compra de R$ 50.000 em um cartão que gasta em média R$ 300 é uma anomalia:

a) contextual.
b) coletiva.
c) pontual.
d) inexistente.

Questão 3. Uma temperatura de 30 °C é normal em janeiro e anormal em julho no Sul do Brasil. Trata-se de anomalia:

a) pontual.
b) coletiva.
c) supervisionada.
d) contextual.

Questão 4. Uma sequência de batimentos que, individualmente, parecem normais, mas juntos revelam arritmia, é uma anomalia:

a) contextual.
b) coletiva.
c) pontual.
d) de rótulo.

Questão 5. No cenário semissupervisionado (one-class), o treino usa:

a) exemplos normais e anômalos rotulados em igual número.
b) apenas anomalias.
c) apenas exemplos normais.
d) nenhum dado.

Questão 6. A regra $|z| > 3$ aponta como suspeitos os valores:

a) a mais de três desvios padrão da média.
b) com variância zero.
c) abaixo da mediana.
d) iguais à média.

Questão 7. Uma fraqueza do z-score é que:

a) não usa números.
b) média e desvio são afetados pelos próprios outliers (mascaramento).
c) só funciona em 3 dimensões.
d) não é padronizado.

Questão 8. O z-score modificado usa:

a) o mínimo e a moda.
b) a mediana e o desvio absoluto mediano (MAD).
c) a média e o máximo.
d) o suporte e a confiança.

Questão 9. Pela regra do IQR (Tukey), um valor é suspeito se estiver:

a) igual à mediana.
b) abaixo de zero.
c) abaixo de Q1 − 1,5·IQR ou acima de Q3 + 1,5·IQR.
d) entre Q1 e Q3.

Questão 10. A distância de Mahalanobis, ao contrário da euclidiana, considera:

a) apenas a média.
b) a ordem das colunas.
c) a covariância entre atributos.
d) o número de grupos.

Questão 11. A ideia central do Isolation Forest é que anomalias:

a) são poucas e diferentes, logo fáceis de isolar.
b) estão sempre no centro.
c) são difíceis de isolar.
d) são numerosas e iguais.

Questão 12. No Isolation Forest, um ponto é considerado anômalo se sua profundidade média de isolamento for:

a) pequena.
b) negativa.
c) igual à altura da árvore.
d) grande.

Questão 13. Os cortes das árvores do Isolation Forest são escolhidos:

a) ao acaso (atributo e valor aleatórios).
b) para maximizar a variância.
c) por votação.
d) pelo menor erro.

Questão 14. Em IsolationForest do scikit-learn, predict devolve:

a) 0 para normal e 1 para anomalia.
b) a probabilidade.
c) +1 para normal e −1 para anomalia.
d) o escore contínuo.

Questão 15. O parâmetro contamination do IsolationForest representa:

a) a fração esperada de anomalias.
b) a profundidade.
c) o número de árvores.
d) a escala dos dados.

Questão 16. O LOF (Local Outlier Factor) compara:

a) a variância total.
b) a densidade em torno de um ponto com a de seus vizinhos.
c) a distância ao centroide.
d) a média global com a mediana.

Questão 17. Um LOF aproximadamente igual a 1 indica:

a) densidade parecida com a dos vizinhos (normal).
b) anomalia forte.
c) ponto isolado.
d) erro.

Questão 18. O LOF é útil sobretudo quando:

a) há apenas um grupo.
b) todos os pontos têm a mesma densidade.
c) os dados são categóricos.
d) as densidades locais variam.

Questão 19. O One-Class SVM aprende:

a) o número de grupos.
b) as regras de associação.
c) a fronteira que envolve os dados normais.
d) a matriz de ligações.

Questão 20. No One-Class SVM, o parâmetro nu é:

a) um limite superior para a fração de anomalias.
b) o raio.
c) o número de vizinhos.
d) a profundidade.

Questão 21. O erro de reconstrução do PCA pode detectar anomalias porque:

a) o PCA cria rótulos.
b) pontos que não seguem a estrutura principal são mal reconstruídos.
c) o PCA aumenta a dimensão.
d) o PCA elimina todos os outliers.

Questão 22. Um autoencoder treinado só com dados normais, ao receber uma anomalia, tende a:

a) falhar ao executar.
b) ter erro de reconstrução alto.
c) reconstruir perfeitamente.
d) criar novos grupos.

Questão 23. A principal razão para não usar a acurácia em detecção de anomalias é que:

a) as classes são muito desbalanceadas.
b) anomalias são frequentes.
c) a acurácia não existe.
d) o modelo é supervisionado.

Questão 24. Precisão (precision) é:

a) a fração dos alertas que eram anomalias reais.
b) a média dos escores.
c) a fração das anomalias reais detectadas.
d) o número de falsos negativos.

Questão 25. A revocação (recall) é:

a) a fração das anomalias reais que foram detectadas.
b) a fração dos alertas corretos.
c) o total de alertas.
d) o limiar.

Questão 26. Para dados muito desbalanceados, é geralmente mais informativa que a curva ROC:

a) o dendrograma.
b) o gráfico scree.
c) a curva Precisão-Revocação (AUPRC).
d) o histograma.

Questão 27. A baseline de AUPRC para um classificador aleatório é igual:

a) a 1.
b) a 0,5.
c) a 0.
d) à proporção de anomalias nos dados.

Questão 28. Qual é uma forma de definir o limiar de alerta com base em capacidade operacional?

a) Ignorar a capacidade.
b) Usar o escore do 50º maior alerta se o time só investiga 50 por dia.
c) Escolher o menor escore.
d) Escolher sempre 0,5.

Questão 29. Para explicar um alerta de anomalia, pode-se usar:

a) apenas a média.
b) o número de grupos.
c) a silhueta.
d) técnicas como SHAP, que indicam quais atributos mais contribuíram.

Questão 30. Qual técnica usa a distância ao k-ésimo vizinho como escore de anomalia?

a) Apriori.
b) Ward.
c) PCA incremental.
d) kNN (Ramaswamy, Rastogi e Shim, 2000).

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Defina anomalia e descreva os três tipos (pontual, contextual e coletiva) com exemplos.
2. Diferencie os cenários supervisionado, semissupervisionado e não supervisionado na detecção de anomalias.
3. Calcule o z-score de um valor 150 em dados com média 100 e desvio padrão 20. Ele seria suspeito?
4. Compare z-score, z-score modificado (MAD) e IQR quanto à robustez.
5. Explique por que a distância de Mahalanobis detecta a combinação 1,90 m e 50 kg como atípica.
6. Explique o Isolation Forest usando a analogia do jogo das 20 perguntas.
7. Descreva a construção do Isolation Forest e como o escore é obtido.
8. Explique o LOF e por que ele funciona com densidades variáveis.
9. Explique a ideia do One-Class SVM e o papel do kernel.
10. Compare Isolation Forest, LOF e One-Class SVM em ideia, pontos fortes e fracos.
11. Como o PCA e o autoencoder podem ser usados para detectar anomalias?
12. Descreva cinco formas de definir o limiar de alerta.
13. Explique por que acurácia é enganosa em detecção de anomalias e que métricas usar.
14. Interprete: AUC-ROC de 0,997 e AUPRC de 0,95 com linha de base de 0,02.
15. Descreva uma estratégia de tratamento de falsos positivos em um sistema real.
16. Por que é preciso monitorar a deriva (drift) em detectores de anomalia?
17. Cite riscos éticos e de segurança de detectores de anomalia.
18. Como o GMM e o DBSCAN podem ser usados para detectar anomalias?
19. Explique a diferença entre anomalia e ruído e por que ela importa.
20. Proponha um pipeline para detectar falhas em máquinas industriais usando sensores.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Regras estatísticas simples em uma medida real

NÍVEL: BÁSICO

No **Breast Cancer Wisconsin** (UCI, via scikit-learn), monte um conjunto em que os casos **malignos** são "anomalias": use todos os 357 benignos e uma amostra de 18 malignos (semente fixa). Na medida `mean area`, aplique três regras: $|z| > 3$, z-score modificado (MAD) > 3,5 e a regra do IQR (1,5 × IQR). Para cada regra, mostre quantos pontos foram marcados e quantos eram malignos.

Exercício 2 — Isolation Forest

NÍVEL: BÁSICO

Com o mesmo conjunto (357 benignos + 18 malignos), padronize os 30 atributos e aplique `IsolationForest(n_estimators=300, random_state=0)`. Calcule o escore de anomalia (`-score_samples`), o AUC-ROC e o AUPRC (`average_precision_score`). Compare o AUPRC com a linha de base (proporção de malignos).

Exercício 3 — LOF e a escolha de n_neighbors

NÍVEL: INTERMEDIÁRIO

Com os mesmos dados padronizados, aplique `LocalOutlierFactor` para n_neighbors = 5, 10, 20 e 50. Calcule o AUPRC de cada um.

Exercício 4 — One-Class SVM treinado só com o normal

NÍVEL: INTERMEDIÁRIO

Divida os 357 benignos em treino (70%) e teste (30%) e acrescente os 18 malignos ao teste. Padronize com o treino. Treine `OneClassSVM(gamma=0.002)` **só com os benignos de treino** para nu = 0,01; 0,05; 0,1 e 0,3. No teste, calcule precisão e revocação para detectar os malignos. (Depois, repita com `gamma="scale"` e compare.)

Exercício 5 — Cinco detectores nos dígitos

NÍVEL: AVANÇADO

Use os **dígitos** (UCI, via scikit-learn): considere "normais" todos os dígitos de 0 a 8 e "anomalias" 30 imagens do dígito 9. Reduza a 15 componentes com PCA (ajustado nos normais de treino). Compare `IsolationForest`, `LocalOutlierFactor(novelty=True)`, `OneClassSVM`, `EllipticEnvelope` e o **erro de reconstrução do PCA** pelo AUC-ROC e pelo AUPRC no teste.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Statlog (Shuttle), Ionosphere, Breast Cancer Wisconsin e Default of Credit Card Clients.
> - **Kaggle:** Credit Card Fraud Detection (ULB), o conjunto clássico de fraude em cartões, com cerca de 285 mil transações e 0,17% de fraudes.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Detecção de fraude em transações de cartão

Contexto. Um banco (fictício) quer ser alertado sobre transações suspeitas. A equipe de análise só consegue investigar **no máximo 50 alertas a cada 10.000 transações**. Quase não há fraudes rotuladas, por isso você usará **aprendizado não supervisionado**: os rótulos serão usados apenas para avaliar.

Fonte dos dados. Transações **simuladas** pela função abaixo, com 1% de fraudes. (Para praticar com dados reais, veja o quadro "Para ir além".)

```python
import numpy as np
import pandas as pd


def gerar_transacoes(n=40000, taxa_fraude=0.01, seed=0):
    rng = np.random.default_rng(seed)
    fraude = rng.random(n) < taxa_fraude
    valor = np.where(fraude, np.exp(rng.normal(5.8, 0.9, n)), np.exp(rng.normal(3.9, 0.8, n)))
    hora = np.where(
        fraude,
        rng.choice([0, 1, 2, 3, 4, 23], n),
        rng.choice(24, n, p=np.r_[np.full(6, 0.01), np.full(18, 0.94 / 18)]),
    )
    distancia = np.where(fraude, rng.gamma(3, 120, n), rng.gamma(2, 6, n))
    n24h = np.where(fraude, rng.poisson(6, n), rng.poisson(1.5, n))
    minutos = np.where(fraude, rng.exponential(15, n), rng.exponential(400, n))
    exterior = np.where(fraude, rng.random(n) < 0.35, rng.random(n) < 0.03).astype(int)
    df = pd.DataFrame(
        {
            "valor": valor,
            "hora": hora,
            "distancia_km": distancia,
            "n_24h": n24h,
            "min_desde_ultima": minutos,
            "exterior": exterior,
        }
    )
    return df, fraude.astype(int)
```

Tarefas.

1. Aplique `log1p` às variáveis assimétricas (valor, distância, minutos), represente a hora por seno e cosseno e padronize. Separe 50% dos dados para treino e 50% para teste (sem usar rótulos no treino).
2. Treine `IsolationForest`, `LocalOutlierFactor(novelty=True)`, `GaussianMixture` (use o log da densidade) e `OneClassSVM` (em uma subamostra de 5.000 pontos). Calcule AUC-ROC e AUPRC no teste. **No LOF, compare n_neighbors = 30, 100 e 300 e explique a diferença** (dica: quantas fraudes há no treino?).
3. Crie um **escore combinado** pela média dos postos (ranks) dos detectores e avalie-o. Ele supera o melhor detector isolado?
4. Escolha o limiar pela **capacidade**: 50 alertas por 10.000 transações. No teste, quantos alertas, quantas fraudes capturadas, qual a precisão e a revocação?
5. Para os 5 alertas de maior escore, **explique** o motivo: liste os 3 atributos com maior valor absoluto de z-score robusto (mediana/MAD).

6. Compare com um limiar escolhido por **custo** (falso negativo custa 100 vezes um falso positivo). Qual a diferença no número de alertas?
7. Escreva um parágrafo com riscos éticos e operacionais (viés, deriva dos dados, adversários).

Entregáveis. Script, tabela de desempenho, análise de limiares e parágrafo.

A solução de referência está no Apêndice E.

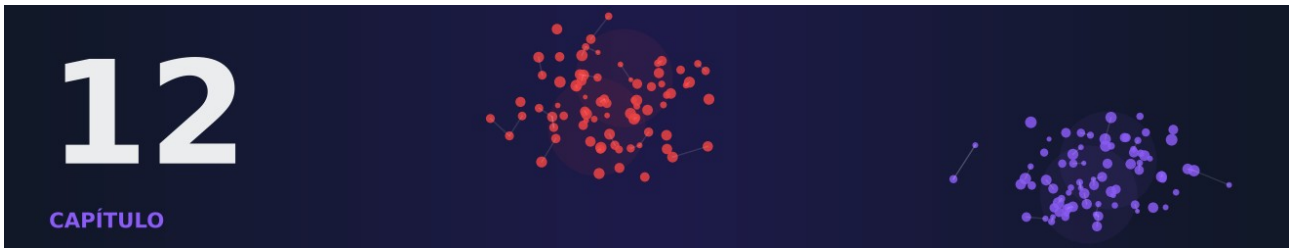
PARTE V

Parte V — Aprendizado Profundo

Como redes neurais aprendem representações sem rótulos: autoencoders, VAEs, GANs, difusão e aprendizado auto-supervisionado e contrastivo.

Nesta parte:

- Cap. 12 — Aprendizado Profundo Não Supervisionado: Autoencoders, VAEs, GANs e Auto-supervisão



Capítulo 12 — Aprendizado Profundo Não Supervisionado: Autoencoders, VAEs, GANs e Auto-supervisão

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar o que é uma rede neural e como ela aprende, em linguagem simples.
- Descrever um autoencoder: codificador, espaço latente e decodificador.
- Distinguir autoencoders subcompletos, esparsos, de remoção de ruído e variacionais.
- Entender a ideia dos VAEs (truque da reparametrização) e das GANs (jogo entre falsificador e polícia).
- Explicar o aprendizado auto-supervisionado e o contrastivo (SimCLR, MoCo).
- Implementar um autoencoder e um VAE simples em PyTorch.
- Relacionar os métodos profundos às técnicas clássicas vistas ao longo do livro.

12.1 Das técnicas clássicas às redes profundas

Nos capítulos anteriores, vimos técnicas clássicas: K-means, PCA, t-SNE. Elas funcionam muito bem, mas dependem de **atributos bem escolhidos**. Para imagens, textos e áudios, o "pixel cru" ou a "palavra crua" raramente é o melhor atributo.

O **aprendizado profundo** (*deep learning*) permite que a própria máquina **aprenda os atributos** a partir dos dados (LeCun; Bengio; Hinton, 2015). Quando isso é feito sem rótulos, temos o **aprendizado de representações não supervisionado** (Bengio; Courville; Vincent, 2013).

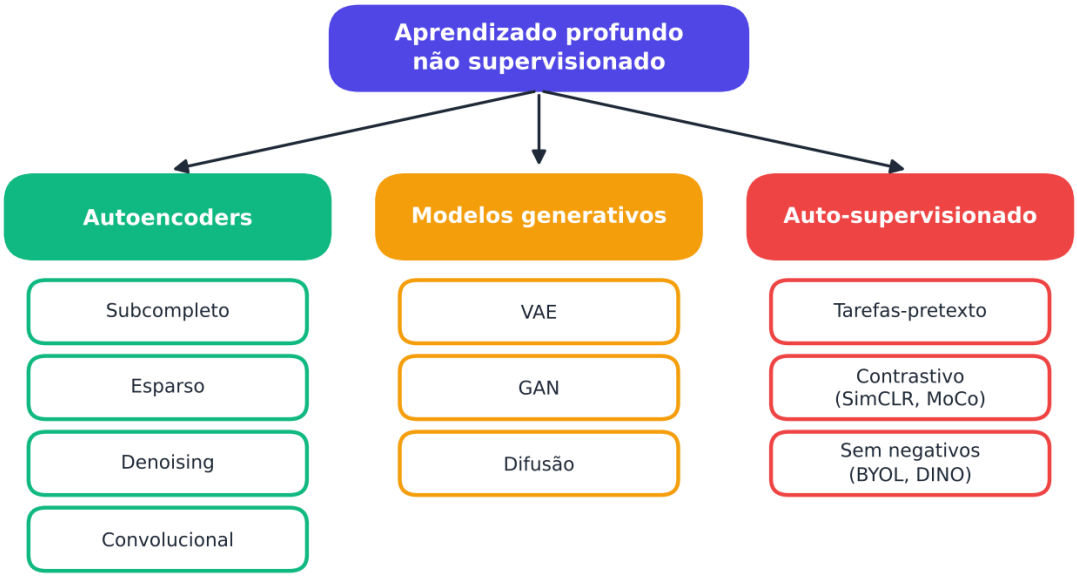


Figura 12.1 — Árvore de métodos profundos não supervisionados: autoencoders, modelos generativos e aprendizado auto-supervisionado.

12.2 Redes neurais em cinco minutos

12.2.1 O neurônio artificial

Um **neurônio artificial** recebe números, multiplica cada um por um **peso**, soma tudo e aplica uma função não linear (a **ativação**).

$$\text{saída} = \text{ativação}(\, w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + b \,)$$

- **Pesos (w):** a "importância" de cada entrada. São aprendidos.
- **Viés (b):** um deslocamento, também aprendido.
- **Ativação:** por exemplo, a **ReLU**, que devolve 0 para negativos e o próprio valor para positivos.

12.2.2 Camadas e profundidade

Neurônios são organizados em **camadas**. A saída de uma camada alimenta a seguinte. Uma rede com muitas camadas é **profunda**.

Analogia: uma linha de montagem. A primeira estação detecta bordas; a seguinte, formas; a seguinte, partes de objetos; a última, o objeto inteiro.

12.2.3 Como a rede aprende

1. A rede faz uma **previsão** com os pesos atuais.
2. Uma **função de perda** mede o quanto ela errou.
3. A **retropropagação** (*backpropagation*) calcula como cada peso contribuiu para o erro (Rumelhart; Hinton; Williams, 1986).
4. A **descida do gradiente** ajusta levemente cada peso para diminuir o erro.
5. Repete-se milhares de vezes.

> MEMORIZE **Em uma frase**
>
> Aprender é ajustar pesos, aos poucos, para errar cada vez menos.

12.3 Autoencoders

12.3.1 A analogia do telegrama

Você precisa mandar uma mensagem longa por telegrama, em que cada palavra custa dinheiro. Então escreve um **resumo curtíssimo**. Quem recebe tenta **reconstruir** a mensagem original a partir do resumo.

Se a reconstrução for boa, o resumo guardou o essencial. O **autoencoder** faz exatamente isso (Hinton; Salakhutdinov, 2006).

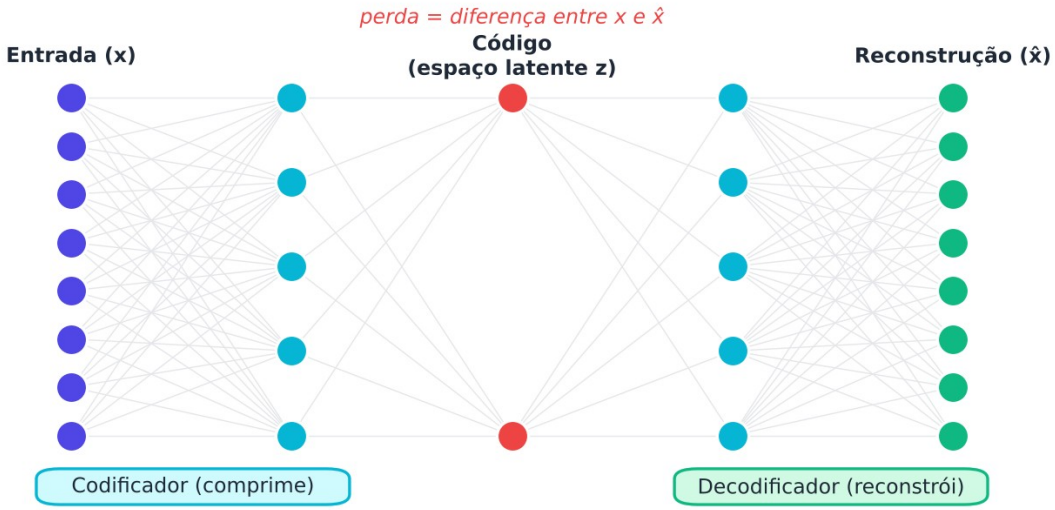


Figura 12.2 — Autoencoder: o codificador comprime a entrada num código curto (espaço latente); o decodificador tenta reconstruí-la.

12.3.2 As três partes

- **Codificador (*encoder*):** comprime a entrada **x** em um vetor curto **z**.
- **Espaço latente:** o conjunto dos códigos **z**. É um espaço de baixa dimensão com os "fatores essenciais".
- **Decodificador (*decoder*):** reconstrói **x̂** a partir de **z**.

A rede é treinada para minimizar o **erro de reconstrução**, normalmente o erro quadrático médio:

$$\text{perda} = (1/n) \cdot \Sigma \, \|x - \hat{x}\|^2$$

Note: **não há rótulos**. A entrada é, ela mesma, o alvo. É auto-supervisão no sentido mais simples.

12.3.3 Relação com o PCA

Um autoencoder **linear** (sem ativações não lineares), com perda quadrática, aprende o **mesmo subespaço do PCA** (Baldi; Hornik, 1989). Com ativações não lineares e várias camadas, ele consegue capturar **estruturas curvas**, algo além do PCA. Hinton e Salakhutdinov (2006) mostraram que autoencoders profundos reduzem dimensões muito melhor que o PCA em imagens e textos.

12.3.4 Implementação em PyTorch

> PYTHON

```python
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.preprocessing import MinMaxScaler

torch.manual_seed(0)

# 1) Dados: dígitos 8x8 (64 atributos) escalados em [0,1]
X, y = load_digits(return_X_y=True)
X = MinMaxScaler().fit_transform(X).astype("float32")
Xt = torch.from_numpy(X)

# 2) Modelo: 64 -> 32 -> 2 (código) -> 32 -> 64
class Autoencoder(nn.Module):
    def __init__(self, dim_latente=2):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(64, 32), nn.ReLU(),
                                 nn.Linear(32, dim_latente))
        self.dec = nn.Sequential(nn.Linear(dim_latente, 32), nn.ReLU(),
                                 nn.Linear(32, 64), nn.Sigmoid())
    def forward(self, x):
        z = self.enc(x)
        return self.dec(z), z

modelo = Autoencoder(dim_latente=2)
otimizador = torch.optim.Adam(modelo.parameters(), lr=1e-2)
perda_fn = nn.MSELoss()

# 3) Treino
for epoca in range(150):
    rec, z = modelo(Xt)
    perda = perda_fn(rec, Xt)
    otimizador.zero_grad()
    perda.backward()
    otimizador.step()
    if epoca % 50 == 0:
        print(f"época {epoca:3d}  perda = {perda.item():.4f}")

# 4) Usar o espaço latente (2D) para visualizar os dígitos
with torch.no_grad():
    Z = modelo.enc(Xt).numpy()
print("Formato do espaço latente:", Z.shape)
```

O que acontece em cada bloco?

1. Carregamos os dígitos e os escalamos para [0, 1].
2. Definimos o codificador (64 → 32 → 2) e o decodificador (2 → 32 → 64).
3. O laço de treino repete: reconstruir, medir o erro, calcular gradientes (`backward`) e atualizar os pesos (`step`).
4. O código **Z** de 2 dimensões pode ser desenhado, como fizemos com t-SNE.

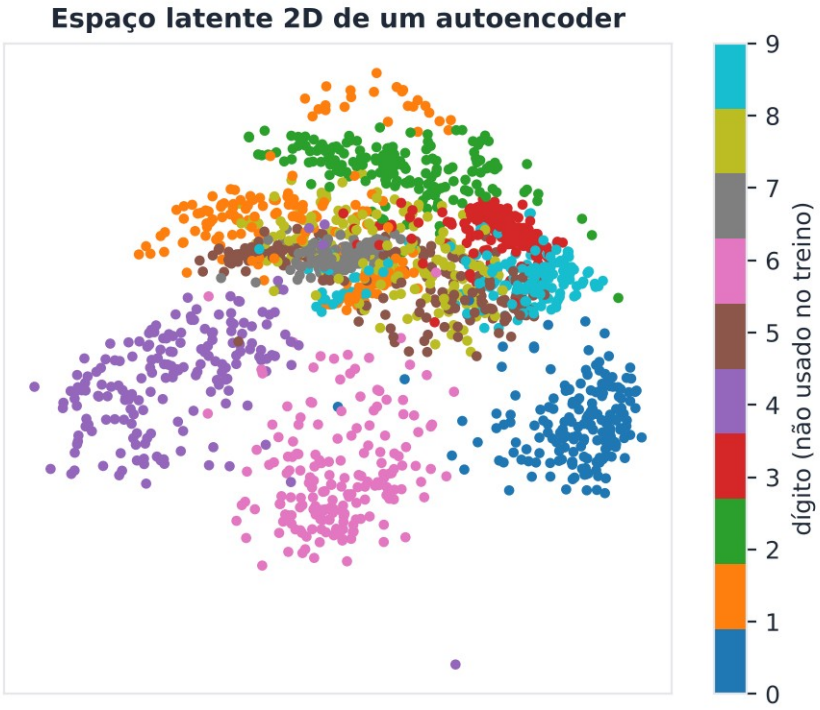


Figura 12.3 — Espaço latente de 2 dimensões de um autoencoder treinado nos dígitos: dígitos parecidos ficam próximos, sem que o modelo conheça os rótulos.

> **DICA** **Autoencoders só com o scikit-learn**
>
> Não é preciso instalar o PyTorch para experimentar. No scikit-learn, o `MLPRegressor` treinado com a **entrada como alvo** (`fit(X, X)`) funciona como um autoencoder simples, e o código pode ser extraído manualmente a partir de `coefs_` e `intercepts_`. O módulo `sklearn.neural_network` também traz a `BernoulliRBM`, uma rede que aprende atributos sem rótulos. Você usará as duas nos exercícios práticos deste capítulo. Para imagens maiores, redes convolucionais e treino em GPU, prefira o PyTorch ou o TensorFlow.

12.4 Variações do autoencoder

12.4.1 Subcompleto e com gargalo

O código tem dimensão **menor** que a entrada. O "gargalo" **força** a rede a aprender o essencial. É o caso que vimos.

12.4.2 Autoencoder esparso

Permite um código grande, mas **penaliza** o número de neurônios ativos, de modo que cada exemplo usa poucos. Gera "dicionários" de partes, como o NMF. A ideia remonta ao trabalho de Olshausen e Field (1996) sobre codificação esparsa.

12.4.3 Autoencoder de remoção de ruído (denoising)

Corrompe a entrada (adiciona ruído ou apaga pedaços) e pede que a rede reconstrua a **versão limpa** (Vincent et al., 2008; 2010). Assim, a rede não pode apenas copiar a entrada: precisa aprender a **estrutura** dos dados.

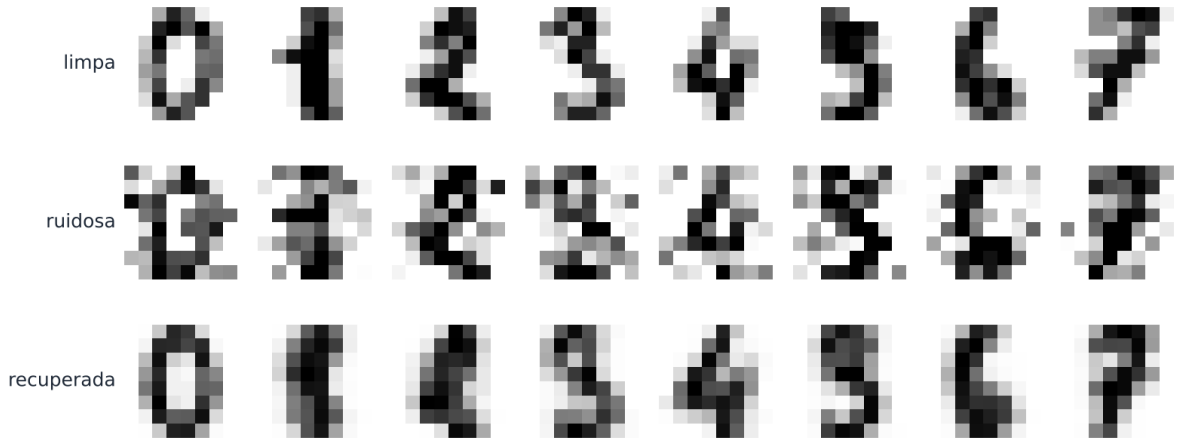


Figura 12.4 — Remoção de ruído: da esquerda para a direita, imagem limpa, imagem ruidosa e imagem recuperada pelo autoencoder.

```python
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.preprocessing import MinMaxScaler

torch.manual_seed(0)
X = MinMaxScaler().fit_transform(load_digits().data).astype("float32")
limpo = torch.from_numpy(X)

dae = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 16), nn.ReLU(),
                    nn.Linear(16, 32), nn.ReLU(), nn.Linear(32, 64), nn.Sigmoid())
opt = torch.optim.Adam(dae.parameters(), lr=5e-3)

for epoca in range(200):
    ruidoso = (limpo + 0.3 * torch.randn_like(limpo)).clamp(0, 1)   # corrompe
    perda = nn.functional.mse_loss(dae(ruidoso), limpo)             # alvo = versão limpa
    opt.zero_grad(); perda.backward(); opt.step()

with torch.no_grad():
    teste = (limpo + 0.3 * torch.randn_like(limpo)).clamp(0, 1)
    print("Erro da imagem ruidosa    :", nn.functional.mse_loss(teste, limpo).item())
    print("Erro da imagem recuperada :", nn.functional.mse_loss(dae(teste), limpo).item())
```

12.4.4 Autoencoder convolucional

Para imagens, usam-se **camadas convolucionais**, que aplicam pequenos filtros deslizantes e respeitam a estrutura espacial (LeCun et al., 1998).

12.4.5 Aplicações

- **Redução de dimensionalidade** não linear.
- **Detecção de anomalias:** treina-se só com exemplos normais; anomalias têm **erro de reconstrução alto**.
- **Remoção de ruído** em imagens e sinais.
- **Pré-treinamento** de representações.

```python
import torch
from torch import nn
```

```
from sklearn.datasets import load_digits
from sklearn.preprocessing import MinMaxScaler

torch.manual_seed(0)
X, y = load_digits(return_X_y=True)
X = MinMaxScaler().fit_transform(X).astype("float32")
normal = torch.from_numpy(X[y != 9])        # treina SEM o dígito 9
raro   = torch.from_numpy(X[y == 9])        # o "9" será a anomalia

ae = nn.Sequential(nn.Linear(64, 16), nn.ReLU(), nn.Linear(16, 8), nn.ReLU(),
                   nn.Linear(8, 16), nn.ReLU(), nn.Linear(16, 64), nn.Sigmoid())
opt = torch.optim.Adam(ae.parameters(), lr=5e-3)
for _ in range(300):
    perda = nn.functional.mse_loss(ae(normal), normal)
    opt.zero_grad(); perda.backward(); opt.step()

with torch.no_grad():
    e_normal = ((ae(normal) - normal) ** 2).mean(1)
    e_raro = ((ae(raro) - raro) ** 2).mean(1)
print("Erro médio nos dígitos normais:", e_normal.mean().item())
print("Erro médio nos dígitos '9'    :", e_raro.mean().item())
```

12.5 Autoencoders variacionais (VAE)

12.5.1 O limite do autoencoder comum

O espaço latente de um autoencoder comum pode ter "buracos". Se escolhermos um ponto qualquer e o decodificarmos, obteremos lixo. Ele **reconstrói**, mas não **gera** bem.

O **VAE** (Kingma; Welling, 2014) corrige isso com uma ideia probabilística.

12.5.2 A ideia

- O codificador não devolve um ponto **z**, e sim uma **distribuição** (uma média μ e um desvio σ).
- Sorteia-se **z** dessa distribuição.
- O decodificador reconstrói a partir de **z**.
- A perda tem **duas partes**:

1. **Reconstrução:** a saída deve se parecer com a entrada. 2. **Regularização (divergência KL):** a distribuição de cada exemplo deve ficar **perto de uma normal padrão**.

$$\text{perda} = \text{erro de reconstrução} + \mathrm{KL}(\ \mathrm{N}(\mu, \sigma^2) \parallel \mathrm{N}(0, 1)\ )$$

Analogia: o bibliotecário que não só guarda cada livro numa posição exata, mas organiza **regiões** de uma grande sala: ficção aqui, ciência ali, sem espaços vazios. Qualquer ponto da sala corresponde a algo plausível.

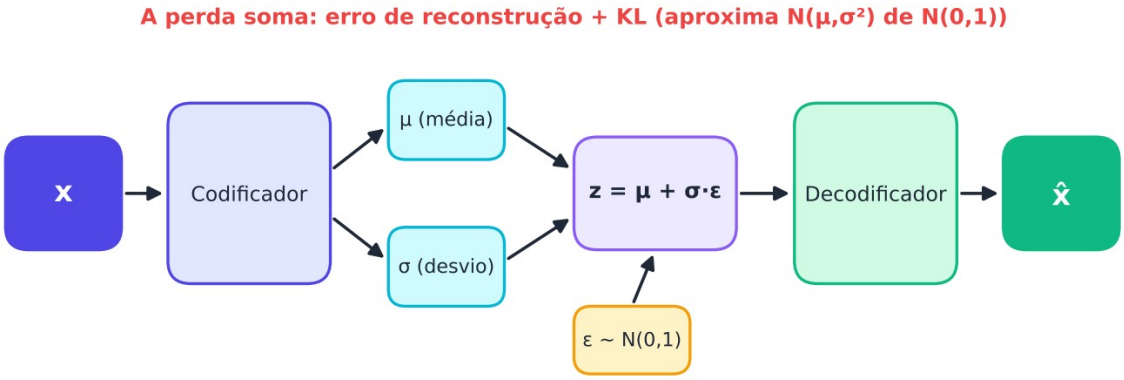


Figura 12.5 — VAE: o codificador gera média e desvio; sorteamos z; o decodificador gera a saída. O espaço latente fica contínuo e "preenchido".

12.5.3 O truque da reparametrização

Sortear **z** é um processo aleatório e não se pode retropropagar por um sorteio. O truque: escreva

$$z = \mu + \sigma \cdot \varepsilon, \text{ com } \varepsilon \sim N(0, 1)$$

O acaso fica todo em ε (que não precisa de gradiente). Agora μ e σ são contribuições determinísticas e a rede treina normalmente (Kingma; Welling, 2014).

12.5.4 VAE em PyTorch

```python
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.preprocessing import MinMaxScaler

torch.manual_seed(0)
X = MinMaxScaler().fit_transform(load_digits().data).astype("float32")
Xt = torch.from_numpy(X)

class VAE(nn.Module):
    def __init__(self, dim_z=2):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(64, 32), nn.ReLU())
        self.mu = nn.Linear(32, dim_z)
        self.logvar = nn.Linear(32, dim_z)          # log da variância (mais estável)
        self.dec = nn.Sequential(nn.Linear(dim_z, 32), nn.ReLU(),
                                 nn.Linear(32, 64), nn.Sigmoid())
    def forward(self, x):
        h = self.enc(x)
        mu, logvar = self.mu(h), self.logvar(h)
        eps = torch.randn_like(mu)
        z = mu + torch.exp(0.5 * logvar) * eps      # reparametrização
        return self.dec(z), mu, logvar

def perda_vae(rec, x, mu, logvar):
    r = nn.functional.binary_cross_entropy(rec, x, reduction="sum")
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return (r + kl) / x.shape[0]
```

```
vae = VAE()
opt = torch.optim.Adam(vae.parameters(), lr=5e-3)
for epoca in range(200):
    rec, mu, logvar = vae(Xt)
    perda = perda_vae(rec, Xt, mu, logvar)
    opt.zero_grad(); perda.backward(); opt.step()
print("Perda final por exemplo:", round(perda.item(), 2))

# GERAR dígitos novos: sorteie z ~ N(0,1) e decodifique
with torch.no_grad():
    novos = vae.dec(torch.randn(16, 2))
print("Imagens geradas:", novos.shape)
```

12.5.5 Por que o VAE é importante?

- Permite **gerar** dados novos e **interpolar** entre exemplos (caminhar no espaço latente: um "7" vira um "1" suavemente).
- Dá uma **representação probabilística**.
- Gerou extensões como o β-VAE, que busca fatores independentes e interpretáveis (Higgins et al., 2017).

12.6 Redes Adversariais Generativas (GANs)

12.6.1 A analogia do falsificador e da polícia

Goodfellow et al. (2014) propuseram um jogo entre duas redes.

- O **gerador** é um falsificador de quadros. Parte de ruído aleatório e tenta criar imagens que pareçam reais.
- O **discriminador** é um policial especializado. Recebe imagens (reais e falsas) e tenta dizer quais são falsas.

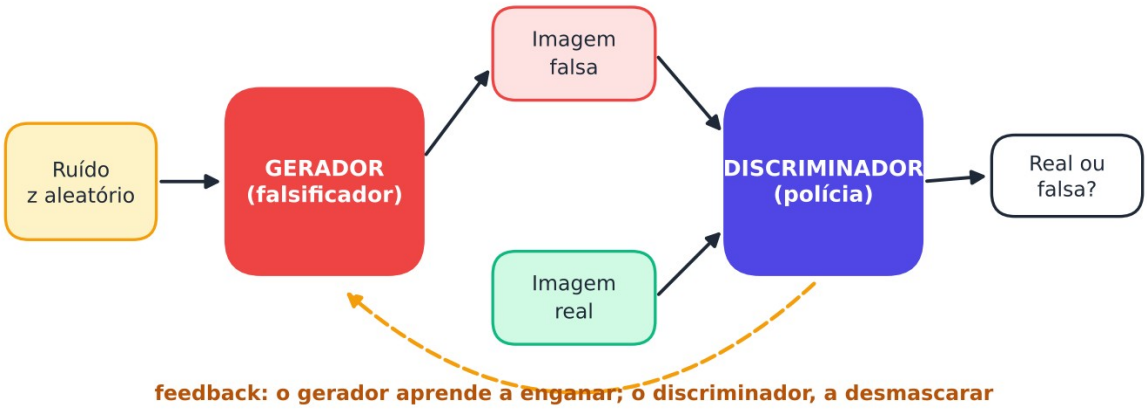


Figura 12.6 — GAN: o gerador tenta enganar o discriminador; o discriminador tenta desmascarar o gerador. Ambos melhoram.

Cada vez que o policial fica mais esperto, o falsificador precisa melhorar. Com o tempo, as falsificações ficam tão boas que o policial acerta apenas 50 % das vezes (o chute).

12.6.2 A formulação do jogo

$$\min_G \max_D \; E[\log D(x)] + E[\log(1 - D(G(z)))]$$

Em palavras: o discriminador **D** quer maximizar a chance de rotular corretamente reais e falsos; o gerador **G** quer minimizar essa chance.

12.6.3 Problemas conhecidos

- **Instabilidade do treino:** o equilíbrio entre as redes é delicado.
- **Colapso de modos:** o gerador produz poucas variedades de imagens.
- **Avaliação difícil:** como medir a qualidade de imagens geradas? Usam-se métricas como a FID (Heusel et al., 2017).

Melhorias: DCGAN (Radford; Metz; Chintala, 2016), Wasserstein GAN (Arjovsky; Chintala; Bottou, 2017) e StyleGAN (Karras; Laine; Aila, 2019).

12.6.4 Um esboço em PyTorch

```python
import torch
from torch import nn

torch.manual_seed(0)
# Dados "reais": uma distribuição 1D bimodal (duas corcovas)
def reais(n):
    metade = n // 2
    return torch.cat([torch.randn(metade, 1) * 0.5 - 2, torch.randn(n - metade, 1) * 0.5 + 2])

G = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 1))
D = nn.Sequential(nn.Linear(1, 16), nn.ReLU(), nn.Linear(16, 1), nn.Sigmoid())
optG = torch.optim.Adam(G.parameters(), lr=2e-3)
optD = torch.optim.Adam(D.parameters(), lr=2e-3)
bce = nn.BCELoss()

for passo in range(1500):
    # 1) treina o discriminador: reais -> 1, falsos -> 0
    real = reais(128)
    falso = G(torch.randn(128, 4)).detach()
    perdaD = bce(D(real), torch.ones(128, 1)) + bce(D(falso), torch.zeros(128, 1))
    optD.zero_grad(); perdaD.backward(); optD.step()
    # 2) treina o gerador: quer que o discriminador diga "1" para os falsos
    perdaG = bce(D(G(torch.randn(128, 4))), torch.ones(128, 1))
    optG.zero_grad(); perdaG.backward(); optG.step()

with torch.no_grad():
    amostra = G(torch.randn(2000, 4)).flatten()
print("Média do gerado:", amostra.mean().item(), "| desvio:", amostra.std().item())
frac = (amostra < 0).float().mean().item()
print("Fração negativa (próxima de 0,5 se houver duas corcovas):", frac)
```

12.7 Modelos de difusão (visão geral)

Hoje, muitos dos geradores de imagens mais impressionantes usam **modelos de difusão** (Ho; Jain; Abbeel, 2020). A ideia:

1. **Processo direto:** pegue uma imagem e vá **adicionando ruído** em muitos pequenos passos, até restar apenas ruído puro.
2. **Processo reverso:** treine uma rede para **remover um pouco de ruído** a cada passo.
3. **Gerar:** comece com ruído e aplique a rede repetidamente. Do caos, emerge uma imagem.

Analogia: uma gota de tinta se espalhando num copo d'água (direto). Imagine "desfazer" a difusão, passo a passo, para recuperar a gota.

Os modelos de difusão são treinados sem rótulos de classe (embora possam receber textos como orientação) e são parentes próximos dos autoencoders de remoção de ruído.

12.8 Aprendizado auto-supervisionado

12.8.1 A grande ideia

Em vez de depender de humanos para rotular, **criamos o rótulo a partir dos próprios dados**. Resolve-se uma tarefa artificial, chamada **tarefa-pretexto** (*pretext task*). Para resolvê-la bem, a rede é obrigada a aprender **representações úteis**.

Exemplos de tarefas-pretexto:

- **Prever palavras ocultas** (BERT; Devlin et al., 2019).
- **Prever a próxima palavra** (modelos de linguagem).
- **Prever a rotação** aplicada a uma imagem (Gidaris; Singh; Komodakis, 2018).
- **Reconstruir partes ocultas** de uma imagem (autoencoders mascarados; He et al., 2022).
- **Resolver quebra-cabeças** de pedaços de imagem.
- **Colorir** imagens em preto e branco.



Figura 12.7 — Tarefas-pretexto: esconder palavras, girar imagens, mascarar regiões. Os "rótulos" saem dos próprios dados.

12.8.2 Depois do pré-treino

Após treinar na tarefa-pretexto com milhões de exemplos sem rótulo, **reaproveitamos** a rede para tarefas reais (classificação, por exemplo) com **poucos rótulos**. Esse é o **ajuste fino** (*fine-tuning*) ou a **avaliação linear**.

Analogia: um estudante lê milhares de livros sem prova (pré-treino). Depois, com poucas aulas específicas, passa em concurso (ajuste fino).

12.9 Aprendizado contrastivo

12.9.1 A ideia

O aprendizado **contrastivo** ensina a rede o que é "parecido" e o que é "diferente".

1. Pegue uma imagem. Aplique **duas transformações aleatórias** (recortar, mudar cores, borrar). Essas duas versões formam um **par positivo**: devem ter representações **próximas**.
2. Imagens **diferentes** formam **pares negativos**: devem ter representações **distantes**.
3. Treine para **aproximar positivos e afastar negativos**.

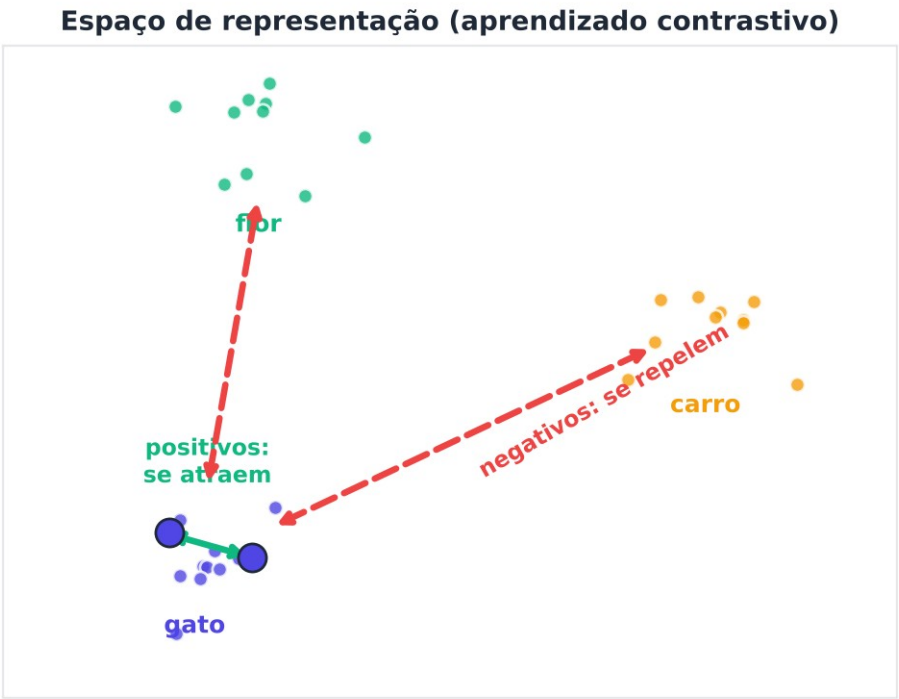


Figura 12.8 — Aprendizado contrastivo: visões diferentes da mesma imagem se atraem; imagens distintas se repelem no espaço de representação.

12.9.2 A função de perda InfoNCE (NT-Xent)

Para um par positivo (i, j) num lote com outros exemplos:

$$\text{perda} = -\log\left[\exp(\text{sim}(z_i, z_j)/\tau) / \Sigma_k \exp(\text{sim}(z_i, z_k)/\tau)\right]$$

- **sim** é a similaridade do cosseno (Capítulo 2).
- **τ** (temperatura) controla a "dureza" da comparação.
- O denominador soma sobre todos os outros exemplos do lote (os negativos).

Quanto mais parecidos os positivos e mais diferentes os negativos, menor a perda (Oord; Li; Vinyals, 2018).

12.9.3 Principais métodos

- **SimCLR** (Chen et al., 2020): usa lotes enormes; os negativos são os outros exemplos do lote.
- **MoCo** (He et al., 2020): mantém uma **fila** de representações antigas como negativos, evitando lotes gigantes.

- **CLIP** (Radford et al., 2021): aprende a alinhar **imagens e textos**; sustenta a busca de imagens por descrição.
- **BYOL** (Grill et al., 2020) e **DINO** (Caron et al., 2021): funcionam **sem negativos explícitos**, com uma rede "aluna" e outra "professora".

12.9.4 Um exemplo mínimo

```python
import torch
import torch.nn.functional as F

def nt_xent(z1, z2, tau=0.5):
    """Perda contrastiva para um lote: z1[i] e z2[i] são duas visões do mesmo exemplo."""
    z = F.normalize(torch.cat([z1, z2]), dim=1)          # normaliza (cosseno)
    sim = z @ z.T / tau                                   # similaridades
    n = z1.shape[0]
    sim.fill_diagonal_(float("-inf"))                     # ignora a similaridade consigo mesmo
    alvo = torch.cat([torch.arange(n, 2 * n), torch.arange(0, n)])   # o positivo de cada um
    return F.cross_entropy(sim, alvo)

torch.manual_seed(0)
base = torch.randn(8, 16)
visao1 = base + 0.05 * torch.randn(8, 16)                 # duas "visões" quase iguais
visao2 = base + 0.05 * torch.randn(8, 16)
aleatorio = torch.randn(8, 16)

print("Perda com positivos parecidos:", nt_xent(visao1, visao2).item())
print("Perda com pares aleatórios   :", nt_xent(visao1, aleatorio).item())
```

A perda é bem menor quando as duas visões vêm de fato do mesmo exemplo.

12.10 Usando representações aprendidas

Uma vez treinado, o codificador vira uma **ferramenta**:

- **Agrupamento** sobre os códigos (K-means em **z**): em imagens grandes e complexas, costuma superar o agrupamento sobre pixels. Xie, Girshick e Farhadi (2016) combinaram os dois passos no *Deep Embedded Clustering*.
- **Visualização:** t-SNE ou UMAP sobre os códigos.
- **Busca por similaridade:** vizinhos mais próximos no espaço de representação.
- **Anomalias:** distância ao centro dos códigos normais.
- **Transferência:** a base para classificadores com poucos rótulos.

```python
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

torch.manual_seed(0)
```

```
X, y = load_digits(return_X_y=True)
X = MinMaxScaler().fit_transform(X).astype("float32")
Xt = torch.from_numpy(X)

enc = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 10))
dec = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 64), nn.Sigmoid())
opt = torch.optim.Adam(list(enc.parameters()) + list(dec.parameters()), lr=5e-3)
for _ in range(300):
    perda = nn.functional.mse_loss(dec(enc(Xt)), Xt)
    opt.zero_grad(); perda.backward(); opt.step()

with torch.no_grad():
    Z = enc(Xt).numpy()

ari_pixels = adjusted_rand_score(y, KMeans(10, n_init=10, random_state=0).fit_predict(X))
ari_latente = adjusted_rand_score(y, KMeans(10, n_init=10, random_state=0).fit_predict(Z))
print("ARI K-means nos pixels  :", round(ari_pixels, 3))
print("ARI K-means no latente  :", round(ari_latente, 3))
```

ATENÇÃO **Resultado honesto**

Neste exemplo pequeno (imagens 8×8), o K-means sobre os **pixels** foi *melhor* que sobre o código aprendido. Isso é uma lição valiosa: em dados simples, o autoencoder pode perder informação útil. As vantagens do aprendizado profundo aparecem com dados maiores e mais complexos. **Sempre compare com uma linha de base simples.**

12.11 Boas práticas e limites

- **Comece simples.** Antes de uma rede profunda, teste PCA e UMAP. Muitas vezes bastam.
- **Dados:** redes profundas são "famintas". Precisam de muitos exemplos.
- **Reprodutibilidade:** fixe sementes e registre versões e hiperparâmetros.
- **Cuidado com o viés:** modelos generativos reproduzem preconceitos e estereótipos dos dados de treino.
- **Uso responsável:** conteúdo sintético (*deepfakes*) pode enganar e causar dano. Rotule e use com ética.
- **Interpretabilidade:** espaços latentes são difíceis de explicar.
- **Custo energético:** treinar grandes modelos consome muita energia (Strubell; Ganesh; McCallum, 2019).

12.12 Fechamento do livro: a grande figura

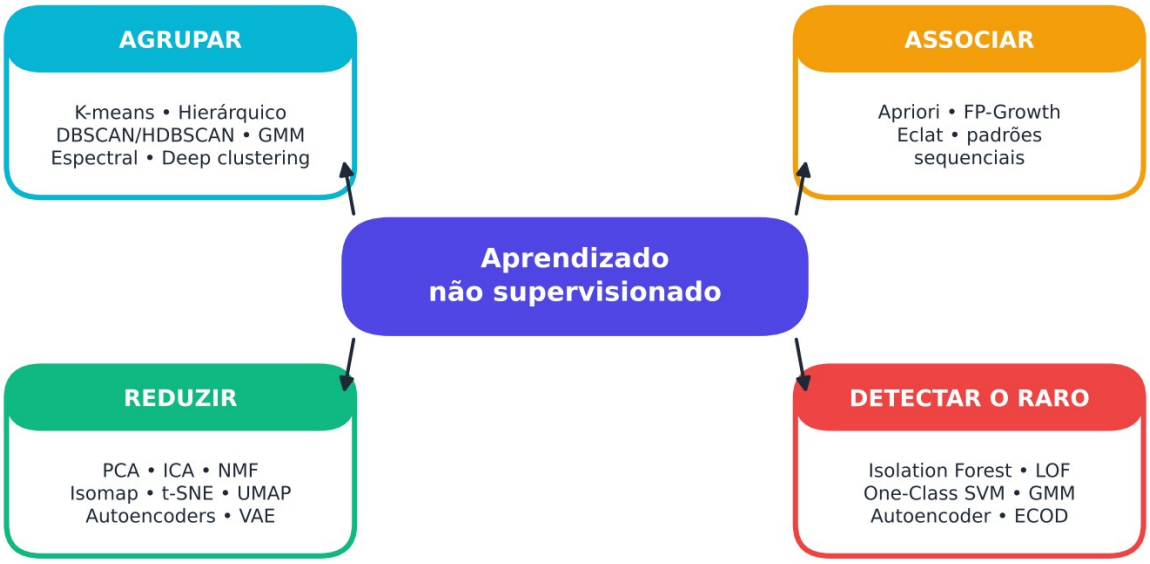


Figura 12.9 — Mapa final: das técnicas clássicas às profundas, os mesmos objetivos permanecem — agrupar, reduzir, associar e detectar o raro.

Você começou este livro sem saber o que era "aprendizado não supervisionado". Agora, você conhece:

- **Agrupar** (K-means, hierárquico, DBSCAN, GMM, espectral) e **avaliar** grupos.
- **Reduzir** dimensões (PCA, ICA, NMF) e **desenrolar** variedades (Isomap, t-SNE, UMAP).
- **Associar** (Apriori, FP-Growth).
- **Detectar anomalias** (Isolation Forest, LOF, One-Class SVM).
- **Aprender representações** (autoencoders, VAEs, GANs, contrastivo).

O próximo passo é **praticar** com dados reais. Escolha uma pergunta que lhe interesse, reúna dados, aplique, avalie, interprete e conte a história.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Autoencoder:** comprimir (codificador) e reconstruir (decodificador); o gargalo força o essencial.
- **Linear = PCA.** Não linear = mais poder.
- **Denoising:** aprende a limpar; **esparso:** poucas ativações.
- **VAE:** código probabilístico + KL; reparametrização; gera e interpola.
- **GAN:** falsificador contra polícia.
- **Difusão:** adicionar e remover ruído.
- **Auto-supervisionado:** o rótulo sai dos dados. **Contrastivo:** aproximar positivos, afastar negativos.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Em um neurônio artificial, a saída é calculada como:

a) o máximo das entradas.
b) uma função de ativação aplicada à soma ponderada das entradas mais um viés.
c) o produto das entradas.
d) a média das entradas.

Questão 2. A função de ativação ReLU devolve:

a) sempre 1.
b) 0 para valores negativos e o próprio valor para positivos.
c) o logaritmo do valor.
d) o quadrado do valor.

Questão 3. A retropropagação (backpropagation) é usada para:

a) gerar rótulos.
b) normalizar dados.
c) calcular como cada peso contribuiu para o erro.
d) escolher o número de neurônios.

Questão 4. A descida do gradiente:

a) calcula distâncias.
b) remove camadas.
c) aumenta a perda.
d) ajusta levemente os pesos para diminuir a perda.

Questão 5. Um autoencoder é treinado para:

a) calcular regras de associação.
b) maximizar recompensas.
c) reconstruir a própria entrada a partir de um código comprimido.
d) classificar imagens com rótulos.

Questão 6. O espaço latente é:

a) a imagem original.
b) o conjunto dos rótulos.
c) o conjunto dos códigos de baixa dimensão aprendidos pelo codificador.
d) o conjunto das distâncias.

Questão 7. Um autoencoder linear com perda quadrática aprende o mesmo subespaço do:

a) PCA.
b) DBSCAN.
c) Apriori.
d) K-means.

Questão 8. O gargalo (bottleneck) de um autoencoder subcompleto serve para:

a) rotular dados.
b) apagar o erro.
c) aumentar a dimensão.
d) forçar a rede a aprender o essencial.

Questão 9. Em um autoencoder esparso, o código:

a) tem sempre 2 dimensões.
b) é binário.
c) não existe.
d) pode ser grande, mas poucos neurônios ficam ativos por exemplo.

Questão 10. Em um autoencoder de remoção de ruído (denoising):

a) o alvo é a versão ruidosa.
b) o ruído é removido antes do treino.
c) não há alvo.
d) a entrada é corrompida e o alvo é a versão limpa.

Questão 11. Em detecção de anomalias com autoencoder, a anomalia é sinalizada por:

a) erro de reconstrução zero.
b) erro de reconstrução alto.
c) número de grupos.
d) alta densidade.

Questão 12. Para imagens, os autoencoders costumam usar camadas:

a) apenas lineares.
b) de regras.
c) convolucionais.
d) de poda.

Questão 13. O VAE (Kingma e Welling, 2014) difere do autoencoder comum porque o codificador devolve:

a) uma distribuição (média e desvio), da qual se sorteia o código.
b) um rótulo.
c) um único ponto fixo.
d) uma árvore.

Questão 14. A perda do VAE é composta por:

a) suporte e confiança.
b) precisão e revocação.
c) erro de reconstrução e divergência KL.
d) inércia e silhueta.

Questão 15. O truque da reparametrização escreve z como:

a) $\mu + \sigma \cdot \varepsilon$, com $\varepsilon \sim N(0, 1)$.
b) σ / μ.
c) $\mu \cdot \sigma$.
d) $\mu - \sigma$.

Questão 16. A principal vantagem do VAE sobre o autoencoder comum é:

a) não ter perda.
b) permitir gerar dados novos e interpolar no espaço latente.
c) ser mais simples.
d) não precisar de dados.

Questão 17. Na GAN, o gerador:

a) calcula a perda.
b) remove ruído.
c) cria dados falsos a partir de ruído, tentando enganar o discriminador.
d) classifica dados reais.

Questão 18. O discriminador de uma GAN:

a) tenta distinguir dados reais de falsos.
b) escolhe k.
c) calcula distâncias.
d) cria imagens.

Questão 19. Qual problema é característico do treino de GANs?

a) Uso de rótulos obrigatórios.
b) Instabilidade e colapso de modos.
c) Falta de dados em todos os casos.
d) Ausência de gradiente por definição.

Questão 20. Os modelos de difusão geram dados:

a) comparando perplexidades.
b) removendo ruído passo a passo, a partir de ruído puro.
c) pesquisando regras.
d) agrupando pixels.

Questão 21. Aprendizado auto-supervisionado cria o rótulo:

a) a partir dos próprios dados, por meio de uma tarefa-pretexto.
b) com especialistas humanos.
c) com recompensas.
d) a partir de regras.

Questão 22. O BERT (Devlin et al., 2019) usa como tarefa-pretexto principal:

a) reconstruir séries temporais.
b) comprimir áudio.
c) prever palavras ocultas (mascaradas).
d) classificar imagens.

Questão 23. Na aprendizagem contrastiva, um par positivo é formado por:

a) duas visões diferentes da mesma imagem.
b) duas imagens de classes diferentes.
c) duas distâncias.
d) uma imagem e seu rótulo.

Questão 24. O objetivo da perda contrastiva é:

a) afastar positivos e aproximar negativos.
b) aproximar positivos e afastar negativos no espaço de representação.
c) igualar todos os pontos.
d) maximizar o ruído.

Questão 25. O SimCLR (Chen et al., 2020) usa como negativos:

a) os outros exemplos do lote.
b) rótulos.
c) apenas imagens duplicadas.
d) nenhum exemplo.

Questão 26. O MoCo (He et al., 2020) mantém:

a) uma matriz de contingência.
b) uma fila de representações antigas como negativos.
c) um dendrograma.
d) uma árvore de prefixos.

Questão 27. O CLIP (Radford et al., 2021) aprende a:

a) calcular regras.
b) alinhar imagens e textos.
c) agrupar clientes.
d) detectar anomalias.

Questão 28. Os métodos BYOL e DINO caracterizam-se por funcionar:

a) sem imagens.
b) sem negativos explícitos.
c) sem redes neurais.
d) sem dados.

Questão 29. No exemplo do capítulo, o K-means no espaço latente do autoencoder:

a) deu ARI igual a 1.
b) não pôde ser executado.
c) superou sempre o K-means nos pixels.
d) não superou o K-means nos pixels, mostrando a importância de uma linha de base simples.

Questão 30. Qual das alternativas descreve corretamente a perda de um autoencoder simples?

a) O número de camadas.
b) A soma dos rótulos.
c) A inércia dos grupos.
d) O erro quadrático médio entre a entrada e a reconstrução.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique o que é um neurônio artificial, uma camada e como a rede aprende.
2. Descreva a arquitetura de um autoencoder e use a analogia do telegrama.
3. Explique a relação entre autoencoder linear e PCA, e a vantagem do autoencoder não linear.
4. Diferencie autoencoders subcompleto, esparso e de remoção de ruído.
5. Explique como usar um autoencoder para detectar anomalias.
6. Descreva o VAE e a função de cada termo da perda.
7. Explique o truque da reparametrização e por que ele é necessário.
8. Explique como funciona uma GAN e o jogo entre gerador e discriminador.
9. Cite três dificuldades de treinar GANs e soluções propostas na literatura.
10. Explique em linhas gerais como funcionam os modelos de difusão.
11. Defina aprendizado auto-supervisionado e dê três exemplos de tarefas-pretexto.
12. Descreva o aprendizado contrastivo e a perda NT-Xent.
13. Compare SimCLR, MoCo, CLIP e BYOL/DINO.
14. Explique como o aprendizado auto-supervisionado é aproveitado em tarefas com poucos rótulos.
15. Como as representações aprendidas podem ser usadas para agrupar, visualizar e buscar?
16. Interprete: no exemplo dos dígitos 8×8, o K-means nos pixels teve ARI de 0,67 e no espaço latente 0,58. O que isso ensina?
17. Cite riscos éticos e limites dos modelos generativos.
18. Explique a diferença entre autoencoder e VAE quanto à capacidade de gerar dados novos.
19. Faça um resumo da trajetória do livro, relacionando as quatro famílias de tarefas aos métodos clássicos e aos profundos.
20. Explique por que, em um autoencoder, a entrada é também o alvo e em que sentido isso é auto-supervisão.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Um autoencoder com MLPRegressor

NÍVEL: BÁSICO

Nos **dígitos** (UCI, via scikit-learn), escale os pixels para [0, 1] e divida em treino e teste (70/30). Treine um autoencoder com `MLPRegressor(hidden_layer_sizes=(32, 8, 32))`, usando a **própria entrada como alvo** (`fit(Xtr, Xtr)`). Compare o erro quadrático médio de reconstrução no teste com o de um PCA de 8 componentes.

Exercício 2 — Abrindo o autoencoder: extraindo o código

NÍVEL: INTERMEDIÁRIO

O `MLPRegressor` não tem um método "encoder", mas guarda os pesos em `coefs_` e `intercepts_`. Calcule manualmente as ativações das duas primeiras camadas (ReLU) para obter o **código de 8 dimensões** do teste. Depois passe esse código pela última parte da rede e verifique se o resultado é igual a `predict` (`np.allclose`).

Exercício 3 — Autoencoder de remoção de ruído

NÍVEL: INTERMEDIÁRIO

Adicione ruído gaussiano (desvio 0,3, com valores cortados em [0, 1]) às imagens. Treine `MLPRegressor(hidden_layer_sizes=(64, 16, 64))` com as imagens **ruidosas** de treino como entrada e as **limpas** como alvo. No teste, compare o MSE (contra a imagem limpa) da imagem ruidosa, da reconstrução do autoencoder e da reconstrução com PCA de 15 componentes.

Exercício 4 — Usando a RBM como extratora de atributos

NÍVEL: INTERMEDIÁRIO

A `BernoulliRBM` (máquina de Boltzmann restrita) aprende atributos sem rótulos. Num `Pipeline`, use `BernoulliRBM(n_components=100, learning_rate=0.05, n_iter=20, random_state=0)` seguida de `LogisticRegression` e compare, no teste, a acurácia com a de uma regressão logística diretamente nos pixels.

Exercício 5 — Anomalias pelo erro de reconstrução

NÍVEL: AVANÇADO

Treine o autoencoder `(32, 8, 32)` apenas com dígitos de 0 a 8 (treino). No teste, junte 30 imagens do dígito 9 aos normais. Use o erro de reconstrução por imagem como escore e calcule AUC-ROC e AUPRC. Compare com o erro de reconstrução de um PCA de 8 componentes.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Optical Recognition of Handwritten Digits (versão completa) e Pen-Based Recognition of Handwritten Digits.
> - **Kaggle:** Fashion-MNIST e MNIST (Digit Recognizer). Com imagens maiores, vale usar PyTorch ou TensorFlow.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Que representação ajuda mais a agrupar?

Contexto. Sua equipe quer saber se vale a pena trocar a representação dos dados (pixels) por uma representação aprendida antes de agrupar. Você comparará quatro representações dos dígitos manuscritos: **pixels**, **PCA**, **código de um autoencoder** e **atributos de uma RBM**.

Fonte dos dados. *Optical Recognition of Handwritten Digits* (**UCI Machine Learning**), via `load_digits`.

Tarefas.

1. Escale os pixels para [0, 1] e divida em treino e teste (70/30, estratificado). Ajuste **todas** as representações no treino.
2. Gere quatro representações: (a) pixels, (b) PCA com 16 componentes, (c) código de 16 dimensões de um autoencoder `(64, 16, 64)` (extraia o código com `coefs_` e `intercepts_`, como no Exercício 2) e (d) 64 atributos de uma `BernoulliRBM`.
3. Em cada representação, padronize e agrupe o **teste** com `KMeans(10)` (modelo ajustado no treino e aplicado ao teste). Calcule ARI, NMI e a homogeneidade.
4. Avalie também a **qualidade supervisionada** da representação: acurácia de uma regressão logística treinada no treino e avaliada no teste.
5. Monte uma tabela e escreva uma discussão (até 10 linhas): a representação aprendida compensa? Em que condições você esperaria um resultado diferente (mais dados, imagens maiores, outra arquitetura)?

Entregáveis. Script, tabela e discussão.

A solução de referência está no Apêndice E.

PARTE VI

Parte VI — Projeto Final

Um estudo completo que integra agrupamento, visualização, representações, anomalias, regras de associação e rotulagem com poucos exemplos, da pergunta de negócio ao relatório.

Nesta parte:

- Cap. 13 — Projeto Final: Integrando Tudo



Capítulo 13 — Projeto Final: Integrando Tudo

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Planejar um projeto completo de aprendizado não supervisionado, da pergunta de negócio ao relatório.
- Integrar agrupamento, redução de dimensionalidade, regras de associação, detecção de anomalias e aprendizado de representações em um único estudo.
- Comparar métodos de modo justo, sem vazamento de rótulos, e justificar escolhas com mais de um critério.
- Organizar o código para que outras pessoas consigam reproduzir os resultados.
- Comunicar resultados, limitações e riscos éticos em um relatório claro.

13.1 Chegou a hora de juntar as peças

Ao longo do livro, você aprendeu ferramentas isoladas. É como aprender a usar martelo, serra e furadeira. Agora você vai **construir a casa**.

Um projeto real raramente usa só uma técnica. Ele começa com uma pergunta, passa por vários métodos, exige comparações honestas e termina em uma **decisão**, que alguém precisa entender e aprovar.

O projeto final deste livro integra as quatro famílias de tarefas e o aprendizado de representações em um só estudo, sobre um mesmo conjunto de dados: as imagens de dígitos manuscritos do repositório UCI Machine Learning.

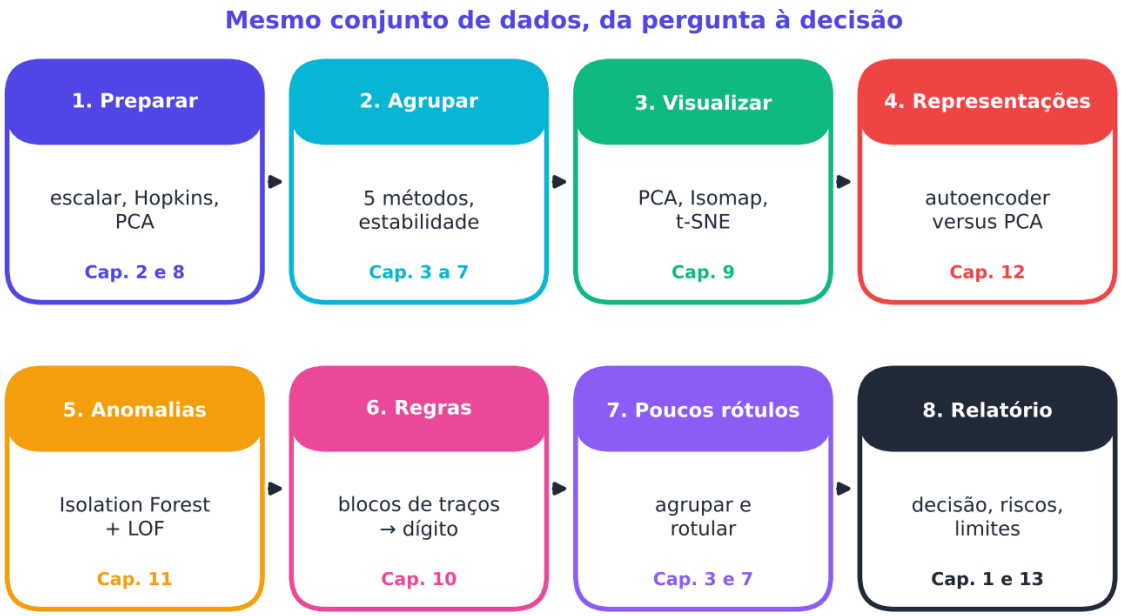


Figura 13.1 — As oito etapas do projeto final e os capítulos que sustentam cada uma.

13.2 A pergunta vem primeiro

Todo bom projeto começa com uma pergunta de negócio. No nosso caso:

> *"Podemos reduzir o custo de rotulagem e encontrar imagens problemáticas usando apenas as imagens?"*

Repare que a pergunta **não** menciona algoritmos. Ela fala de **custo** e de **qualidade**. Os métodos são meios, não fins.

> **Memorize**
>
> Primeiro defina a decisão que será apoiada. Depois escolha as ferramentas. Nunca o contrário.

Uma boa pergunta de projeto tem quatro ingredientes:

1. **Quem decide?** (a diretoria, a equipe de operações, um analista)
2. **O que será decidido?** (investir em rotulagem, revisar imagens, mudar um processo)
3. **Como saberemos que deu certo?** (um critério mensurável)
4. **Quais os riscos de errar?** (para pessoas e para a organização)

13.3 Organizando o trabalho

13.3.1 Estrutura sugerida

Mantenha o projeto em uma pasta com a estrutura abaixo.

```bash
projeto_digitos/
├── dados/            # dados brutos (nunca altere os originais)
├── notebooks/        # explorações e gráficos
├── src/              # funções reutilizáveis (hopkins, estabilidade...)
├── resultados/       # tabelas e figuras geradas
```

```
├── relatorio/        # texto final
└── requirements.txt  # versões das bibliotecas
```

13.3.2 Reprodutibilidade em cinco hábitos

1. **Fixe as sementes** (`random_state`) em tudo que for aleatório.
2. **Registre as versões** das bibliotecas (`pip freeze > requirements.txt`).
3. **Separe preparação, modelagem e avaliação** em funções pequenas.
4. **Ajuste as transformações só no treino** quando houver divisão treino/teste.
5. **Guarde as decisões**: por que escolheu k = 10? Por que descartou o método X?

13.3.3 Notebook ou script?

Notebooks são ótimos para explorar e contar a história. Scripts e funções são melhores para repetir e testar. Uma boa prática é **explorar no notebook** e **mover o código estável para funções** em arquivos `.py`.

13.4 Comparar com justiça

Comparar algoritmos é mais difícil do que parece. Cuidados essenciais:

- **Mesmos dados e mesmo pré-processamento** para todos os métodos.
- **Mesmos critérios de avaliação**, escolhidos *antes* de ver os resultados.
- **Rótulos reais só no final.** Se você escolher o método olhando o ARI, o rótulo "vazou" para a decisão.
- **Várias sementes ou reamostragens.** Um resultado que muda com a semente não é confiável.
- **Uma linha de base simples.** Se um método sofisticado não supera o PCA ou o K-means, você precisa saber disso.

> ATENÇÃO **Armadilha clássica**
>
> Escolher hiperparâmetros olhando o desempenho em rótulos e depois dizer que o método é "não supervisionado". Se usou rótulos para ajustar, declare isso no relatório. Na prática, use uma **pequena amostra rotulada** como validação, ou critérios internos e de estabilidade.

13.5 O relatório final

Um bom relatório tem uma estrutura previsível. Use este roteiro.

1. **Título e resumo executivo** (3 linhas): pergunta, resultado principal, recomendação.
2. **Dados:** origem, tamanho, preparação e limitações.
3. **Método:** o que foi feito, em ordem, e por quê.
4. **Resultados:** tabelas e gráficos legíveis, com interpretação.
5. **Limitações:** o que pode estar errado, o que não foi testado.
6. **Riscos éticos e operacionais.**
7. **Recomendação:** o que fazer, e o que monitorar depois.
8. **Apêndice técnico:** parâmetros, versões, sementes.

> **Dica de escrita**
>
> Escreva o resumo executivo **por último**, mas coloque-o **no início**. Se uma pessoa ocupada ler apenas essas três linhas, ela já deve saber o que fazer.

13.5.1 Gráficos que comunicam

- Cada gráfico responde a **uma** pergunta, que está no título.
- Rotule eixos e unidades.
- Evite excesso de cores; use cores consistentes para os mesmos grupos em todo o relatório.
- Mostre incerteza (variação entre sementes ou reamostragens) quando possível.

13.6 Ética e responsabilidade

Mesmo em um projeto "inocente" como o de dígitos, é preciso pensar nas consequências.

- **Quem pode ser prejudicado por um erro?** Um dígito errado em um cheque pode mover dinheiro para a conta errada.
- **Há viés?** Estilos de escrita variam por idade, país, escolaridade e deficiência. Um modelo treinado em poucos estilos pode errar mais com outros.
- **Privacidade.** Imagens de documentos podem conter dados pessoais. Trate-as segundo a LGPD e use o mínimo necessário.
- **Transparência.** As pessoas afetadas podem contestar uma decisão automatizada?
- **Humano no circuito.** Casos duvidosos devem ser revisados por pessoas. O detector de anomalias do projeto serve a isso.

> MEMORIZE **Lista de verificação ética**
>
> - Identifiquei quem é afetado pelas decisões?
> - Medi o desempenho em subgrupos relevantes?
> - Há revisão humana para os casos incertos?
> - Documentei limites e usos proibidos do modelo?
> - Os dados pessoais estão protegidos?

13.7 Como será avaliado o seu projeto

A rubrica abaixo é um modelo. Adapte-a ao seu curso ou equipe.

Critério	Peso	O que se espera
Pergunta e planejamento	10%	Pergunta clara, critérios definidos antes de rodar
Preparação dos dados	10%	Limpeza, escala, ausência de vazamento
Agrupamento e avaliação	20%	Vários métodos, critérios internos, estabilidade e conferência externa no fim
Visualização e representações	15%	Projeções comparadas por métricas; representação aprendida comparada com linha de base

Critério	Peso	O que se espera
Anomalias e regras	15%	Detecção, inspeção qualitativa e regras interpretadas com cuidado
Uso de poucos rótulos	10%	Comparação justa com repetições
Relatório e ética	20%	Clareza, limitações, riscos e recomendação

13.8 Extensões para quem quer ir além

- **Repita o projeto em outro conjunto**, como o *Pen-Based Recognition of Handwritten Digits* (UCI) ou o *Fashion-MNIST* (Kaggle), e compare as conclusões.
- **Troque o autoencoder** por um autoencoder convolucional em PyTorch.
- **Aplique um aprendizado contrastivo** simples (Capítulo 12) e compare com o PCA.
- **Crie um painel** (por exemplo, com Streamlit) para uma pessoa revisar as imagens mais estranhas.
- **Monitore a deriva**: simule uma mudança de estilo (por exemplo, imagens mais borradas) e veja quais métricas detectam o problema.

Resumo do capítulo

MEMORIZE **Cartões de memorização**

- **Pergunta de negócio primeiro**; algoritmos são meios.
- **Oito etapas:** preparar, agrupar, visualizar, aprender representações, achar anomalias, extrair regras, rotular com pouco orçamento, relatar.
- **Comparação justa:** mesmos dados, critérios definidos antes, rótulos só no fim, várias sementes, linha de base simples.
- **Reprodutibilidade:** sementes, versões, funções, decisões registradas.
- **Relatório:** resumo executivo, método, resultados, limitações, riscos, recomendação.
- **Ética:** quem é afetado, viés, privacidade, humano no circuito.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Em um projeto de dados bem conduzido, o primeiro passo é:

a) treinar um autoencoder.
b) escolher o algoritmo mais sofisticado.
c) calcular o ARI.
d) definir a pergunta de negócio e a decisão que será apoiada.

Questão 2. Qual destes NÃO é um ingrediente de uma boa pergunta de projeto?

a) O nome do algoritmo que será usado.
b) O que será decidido.
c) Quem decide.
d) Como saberemos que deu certo.

Questão 3. Por que os rótulos reais só devem ser usados no final de um estudo não supervisionado?

a) Para evitar que a escolha do método seja influenciada por informação que não estaria disponível na prática.
b) Porque o scikit-learn proíbe.
c) Porque tornam o algoritmo mais lento.
d) Porque rótulos sempre estão errados.

Questão 4. Fixar random_state em tudo que é aleatório serve principalmente para:

a) tornar os resultados reproduzíveis.
b) melhorar a acurácia.
c) evitar o sobreajuste.
d) acelerar o código.

Questão 5. Ajustar o StandardScaler com todos os dados antes de dividir em treino e teste causa:

a) vazamento de informação do teste para o treino.
b) redução do tempo de execução.
c) nenhum efeito possível.
d) melhora legítima do desempenho.

Questão 6. Entre as etapas do projeto, qual técnica serve para reduzir 64 pixels a poucas dimensões antes de agrupar?

a) Apriori.
b) One-Class SVM.
c) Silhueta.
d) PCA.

Questão 7. Na Etapa 2 do projeto, a escolha do melhor método de agrupamento foi feita com:

a) silhueta e estabilidade, sem usar rótulos.
b) sorteio.
c) o tempo de execução apenas.
d) o ARI contra os dígitos reais.

Questão 8. No estudo com os dígitos, o agrupamento espectral destacou-se por:

a) maior estabilidade entre reamostragens.
b) ser o mais rápido.
c) detectar ruído.
d) não precisar de parâmetros.

Questão 9. O HDBSCAN, nos dígitos, marcou muitas imagens como ruído e formou grupos muito puros. Isso ilustra:

a) que ele não funciona em imagens.
b) um erro do algoritmo.
c) que eixos devem ser trocados.
d) uma troca entre cobertura e pureza.

Questão 10. A estatística de Hopkins alta (por exemplo, 0,74) indica que os dados:

a) não têm estrutura.
b) são ruído.
c) são uniformes.
d) têm tendência de agrupamento.

Questão 11. Na comparação de projeções 2D dos dígitos, a técnica de maior confiança (trustworthiness) foi:

a) t-SNE.
b) Isomap.
c) MDS.
d) PCA.

Questão 12. Por que não se deve interpretar a distância entre grupos distantes em um mapa t-SNE?

a) Porque o t-SNE preserva vizinhanças locais, não distâncias globais.
b) Porque o t-SNE não usa distâncias.
c) Porque o t-SNE exige rótulos.
d) Porque o t-SNE é determinístico.

Questão 13. No projeto, o autoencoder com código de 16 dimensões, comparado ao PCA de 16 componentes, teve:

a) desempenho idêntico.
b) erro de execução.
c) desempenho de agrupamento melhor.
d) desempenho de agrupamento pior.

Questão 14. A lição principal do resultado do autoencoder versus PCA é:

a) nunca usar autoencoders.
b) autoencoders não funcionam em imagens.
c) PCA é sempre melhor.
d) comparar sempre com uma linha de base simples.

Questão 15. Para encontrar imagens "estranhas", o projeto combinou Isolation Forest e LOF por meio:

a) do produto dos escores brutos.
b) da média dos postos (ranks) dos escores.
c) da escolha do menor escore.
d) de uma votação de rótulos reais.

Questão 16. Por que combinar detectores por postos é preferível à média dos escores brutos?

a) Postos são sempre mais precisos.
b) Postos eliminam falsos positivos.
c) Os escores têm escalas diferentes.
d) Escores brutos não existem.

Questão 17. Em regras do tipo "blocos → dígito", a confiança de 0,84 e o lift de 8,4 indicam que:

a) o bloco causa o dígito.
b) a regra é inválida.
c) o dígito é raro.
d) o padrão de blocos é muito mais frequente no dígito do que o esperado por acaso.

Questão 18. Os blocos das colunas das bordas quase não aparecem nas regras porque:

a) têm lift negativo.
b) quase nunca estão ativos nas imagens.
c) são ruído.
d) não existem.

Questão 19. Na estratégia "agrupar e rotular", cada rótulo humano é:

a) propagado a todas as imagens do mesmo grupo.
b) descartado depois.
c) usado apenas para uma imagem.
d) usado para treinar o autoencoder.

Questão 20. Com 30 rótulos, "agrupar e rotular" teve acurácia de cerca de 0,93, contra cerca de 0,66 da amostra aleatória com regressão logística. Isso sugere que:

a) o sorteio foi errado.
b) não vale a pena rotular.
c) a regressão logística é um modelo ruim.
d) a estrutura dos grupos aproveita melhor poucos rótulos.

Questão 21. Uma limitação do "agrupar e rotular" é que:

a) só funciona com 2 grupos.
b) não permite rótulos.
c) exige redes neurais.
d) depende da qualidade dos grupos.

Questão 22. Um relatório final deve começar com:

a) um resumo executivo curto com pergunta, resultado e recomendação.
b) o código completo.
c) a lista de bibliotecas.
d) a tabela de hiperparâmetros.

Questão 23. Qual alternativa descreve um risco ético no projeto dos dígitos?

a) Algoritmos não erram.
b) Imagens nunca contêm dados pessoais.
c) Um dígito errado em um cheque pode mover dinheiro para a conta errada.
d) Dígitos não têm relação com pessoas.

Questão 24. Humano no circuito significa que:

a) casos duvidosos são revisados por pessoas.
b) pessoas treinam os algoritmos todos os dias.
c) o sistema funciona sem supervisão.
d) as pessoas escrevem o código.

Questão 25. Escolher hiperparâmetros olhando o desempenho em rótulos e depois chamar o método de "não supervisionado" é:

a) uma armadilha: se rótulos foram usados, isso deve ser declarado.
b) obrigatório.
c) irrelevante.
d) uma boa prática.

Questão 26. Em uma rubrica de projeto, o peso para "Relatório e ética" foi de:

a) 20%.
b) 50%.
c) 5%.
d) 0%.

Questão 27. O que é "deriva dos dados" (data drift)?

a) A mudança, ao longo do tempo, da distribuição dos dados em produção.
b) A perda de arquivos.
c) A redução do número de atributos.
d) O aumento da semente aleatória.

Questão 28. Qual hábito de reprodutibilidade registra as versões das bibliotecas?

a) Salvar um arquivo requirements.txt (por exemplo, com pip freeze).
b) Fixar random_state.
c) Usar notebooks.
d) Rodar o código duas vezes.

Questão 29. Quando um resultado muda bastante com a semente aleatória, o correto é:

a) escolher a melhor semente e esconder as demais.
b) ignorar o problema.
c) trocar de computador.
d) repetir com várias sementes e reportar a variação.

Questão 30. Uma extensão sugerida para o projeto final é:

a) repetir o estudo em outro conjunto, como Fashion-MNIST, e comparar as conclusões.
b) usar apenas um método.
c) apagar os resultados.
d) trocar o idioma do relatório.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique por que o projeto final começa pela pergunta de negócio e não pela escolha do algoritmo.
2. Liste os quatro ingredientes de uma boa pergunta de projeto e dê um exemplo para um caso de sua escolha.
3. Descreva cinco hábitos de reprodutibilidade e o problema que cada um evita.
4. O que significa "comparar com justiça" métodos de agrupamento? Cite quatro cuidados.
5. Explique por que usar o ARI para escolher o método de agrupamento contradiz a ideia de um estudo não supervisionado.
6. Interprete a tabela de resultados do projeto: K-means (ARI 0,67), Ward (0,72), espectral (0,76) e HDBSCAN (0,87 com 873 ruídos de 1.797). O HDBSCAN é o melhor?
7. Explique por que o autoencoder não superou o PCA no projeto e o que isso ensina.
8. Descreva como o escore combinado por postos foi construído na Etapa 5 e por que ele é útil.
9. Interprete uma regra "ativo_21, vazio_11, vazio_32 → dígito 7" com confiança 0,84 e lift 8,4.
10. Explique a estratégia "agrupar e rotular" e por que ela superou a amostra aleatória com 30 rótulos.
11. Quais são os riscos da estratégia "agrupar e rotular"?
12. Descreva a estrutura sugerida para um relatório final de projeto de dados.
13. Por que o resumo executivo deve ser escrito por último, mas colocado no início?
14. Cite quatro riscos éticos do projeto dos dígitos e uma ação de mitigação para cada.
15. O que é humano no circuito e como ele aparece neste projeto?
16. Como você detectaria a deriva dos dados em um sistema de leitura de dígitos em produção?
17. Proponha uma extensão do projeto usando Fashion-MNIST ou outro conjunto público e diga quais conclusões você espera que mudem.
18. Discuta qual etapa do projeto você considera mais crítica para a qualidade da decisão final e justifique.
19. Compare os custos e benefícios de rotular imagens manualmente versus agrupar e rotular.
20. Resuma em um parágrafo as principais lições do livro para um colega que vai começar um projeto de aprendizado não supervisionado.

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** Optical Recognition of Handwritten Digits (a versão completa tem 3.823 imagens de treino e 1.797 de teste), Pen-Based Recognition of Handwritten Digits, Dry Bean e Wine Quality.
> - **Kaggle:** Fashion-MNIST e MNIST (Digit Recognizer). Repita o projeto final em outro conjunto e compare as conclusões.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador final

Laboratório de Dígitos: do dado bruto à decisão

Contexto. Um banco (fictício) digitaliza cheques e formulários escritos à mão. Existem milhares de imagens de dígitos, mas **só alguém caro e lento pode rotulá-las**. A diretoria de dados pediu a você um **estudo completo** de aprendizado não supervisionado para responder, ao final, a pergunta: *"Podemos reduzir o custo de rotulagem e encontrar imagens problemáticas usando apenas as imagens?"*

Fonte dos dados. *Optical Recognition of Handwritten Digits*, do repositório **UCI Machine Learning**, incluído no scikit-learn (`load_digits`): 1.797 imagens de 8×8 pixels, com valores de 0 a 16. Os rótulos (dígitos de 0 a 9) **só podem ser usados para avaliar** os resultados, exceto na Etapa 7, em que você simulará um orçamento pequeno de rotulagem.

Etapa 1 — Entender e preparar. Escale os pixels para [0, 1]. Calcule a estatística de Hopkins e o número de componentes do PCA para reter 95% da variância. Reduza os dados a 30 componentes para as próximas etapas.

Etapa 2 — Agrupar. Agrupe com K-means, Ward, GMM (covariância `diag`), agrupamento espectral e HDBSCAN. Para os métodos com k fixo, use k = 10. Compare silhueta (interna), estabilidade (ARI entre reamostragens) e, **só no final**, ARI e NMI contra os dígitos reais.

Etapa 3 — Visualizar. Gere projeções 2D com PCA, Isomap e t-SNE e compare-as por confiança (`trustworthiness`). Desenhe o t-SNE colorido pelos grupos do melhor método.

Etapa 4 — Representações aprendidas. Treine um autoencoder (`MLPRegressor`) com código de 16 dimensões e verifique se agrupar o código supera agrupar o PCA de 16 componentes.

Etapa 5 — Anomalias. Use um escore combinado (Isolation Forest e LOF, média dos postos) para encontrar as 12 imagens "mais estranhas". Desenhe-as e descreva, em uma frase cada, o que parece estranho. Quais dígitos dominam os 30 principais alertas?

Etapa 6 — Regras de traços. Divida cada imagem em 16 blocos (grade 4×4 de blocos 2×2). Para cada bloco, crie dois itens: `ativo_rc` (média maior que 8, na escala de 0 a 16) e `vazio_rc` (caso contrário). Inclua também um item por dígito. Use `fpgrowth` (suporte mínimo de 0,03, no máximo 4 itens) para encontrar regras do tipo *blocos* → *dígito* com confiança de pelo menos 0,6 e lift de pelo menos 1,5. Liste a melhor regra de cada dígito para o qual existir alguma. Por que os blocos das bordas quase nunca aparecem nas regras?

Etapa 7 — Rotular com pouco orçamento. Suponha que você só possa rotular **30 imagens**. Estratégia A: agrupe em 30 grupos (K-means), rotule a imagem mais próxima do centro de cada grupo e propague o rótulo ao grupo ("agrupar e rotular"). Estratégia B: sorteie 30 imagens, rotule-as e treine uma regressão logística. Compare a acurácia (contra os rótulos reais, em todas as imagens) das duas estratégias, repetindo o sorteio da estratégia B 20 vezes.

Etapa 8 — Relatório final. Escreva um relatório de 1 a 2 páginas com: resumo executivo (3 linhas), método, principais resultados (tabela), limitações, riscos éticos (erros em cheques afetam pessoas; viés de escrita; privacidade) e recomendação ao banco.

Entregáveis. Um script (ou notebook) organizado por etapas, os gráficos e o relatório final.

A solução de referência está no Apêndice E.

Apêndice A — Gabarito dos exercícios

Esta seção traz a resposta de todas as questões. Nas questões de múltipla escolha, aparecem a alternativa correta e uma breve justificativa. Nas discursivas, aparece uma **resposta-modelo**: a sua resposta pode ser redigida de outro modo, desde que contemple as mesmas ideias.

Capítulo 1 — Bem-vindo ao Aprendizado Não Supervisionado

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	O aprendizado não supervisionado trabalha só com X e busca estrutura nos dados.
2	b)	Agrupar por semelhança, sem etiquetas prévias, é a essência do agrupamento.
3	c)	Linhas são exemplos; colunas são atributos.
4	d)	y é o rótulo (alvo), quando existe.
5	c)	O supervisionado compara a previsão com a resposta correta.
6	a)	Há um alvo numérico conhecido (preço), logo é regressão.
7	b)	O agente age no ambiente e recebe recompensas ou punições.
8	d)	Mistura uma pequena parte rotulada com uma grande parte sem rótulo.
9	c)	A máquina inventa a tarefa-pretexto e seus rótulos a partir dos dados.
10	b)	As quatro famílias são agrupamento, redução de dimensionalidade, regras de associação e detecção de anomalias.
11	a)	Agrupamento organiza exemplos em grupos de semelhantes.
12	d)	Reduzir o número de atributos preservando o essencial é redução de dimensionalidade.
13	b)	É a análise da cesta de compras (Agrawal et al., 1993).
14	b)	Anomalias são exemplos que destoam do padrão esperado.
15	a)	Rotular em grande escala custa tempo e dinheiro; às vezes nem existe rótulo.
16	a)	Sem gabarito, a avaliação depende de critérios internos, estabilidade e julgamento humano.
17	c)	Hotelling (1933) formalizou o método; Pearson (1901) é o precursor.
18	a)	Ester, Kriegel, Sander e Xu publicaram o DBSCAN em 1996.
19	b)	Antes de tudo, é preciso saber qual decisão os resultados vão apoiar.
20	b)	Verifica-se a qualidade dos dados antes de modelar.
21	d)	O scikit-learn (Pedregosa et al., 2011) oferece os algoritmos.
22	c)	O Iris, com 150 flores e 4 medidas, foi usado para o primeiro agrupamento.
23	b)	n_clusters define o número de grupos desejados.
24	c)	O PCA resume as medidas em 2 dimensões apenas para podermos desenhar.
25	a)	Viés, privacidade e rótulos que estigmatizam exigem cuidado.
26	b)	A responsabilidade pelas decisões permanece humana.
27	c)	A Lei Geral de Proteção de Dados (LGPD) exige proteção e base legal para dados pessoais.
28	b)	Representações úteis, muitas vezes aprendidas sem rótulos, são centrais.
29	d)	A Parte V (Capítulo 12) cobre autoencoders, VAEs, GANs e auto-supervisão.
30	a)	Não há rótulos: o objetivo é descobrir grupos (perfis) nos dados.

Discursivas (respostas-modelo)

1. É aprender estrutura a partir de dados que só têm atributos (X), sem rótulos. Analogia: organizar uma biblioteca sem etiquetas, agrupando livros parecidos; ninguém dá a resposta, a organização é descoberta.

2. No supervisionado há X e y e o feedback é o gabarito, usado para medir o erro. No não supervisionado há só X e não há gabarito; o objetivo é descobrir grupos, padrões ou exceções.
3. Supervisionado (classificar e-mails em spam), não supervisionado (segmentar clientes), semissupervisionado (poucos rótulos e muitos dados sem rótulo), auto-supervisionado (prever palavra oculta) e por reforço (agente que aprende por recompensas, como em jogos).
4. Rótulos são caros; rótulos podem não existir; exploração dos dados; pré-processamento e representações melhores; detecção de eventos raros e mutáveis, como fraudes e falhas.
5. Agrupamento (perfis de clientes); redução de dimensionalidade (mapa 2D de fotos); regras de associação (cesta de compras); detecção de anomalias (fraude em cartão).
6. Exemplo é uma linha (um cliente); atributo é uma coluna (idade, gasto, visitas). Resposta deve apresentar tabela com linhas de clientes e colunas de características.
7. Não há gabarito; o algoritmo sempre devolve algum agrupamento, mesmo em dados aleatórios. É preciso usar critérios internos, estabilidade e interpretação por especialistas.
8. Entender o problema, coletar dados, explorar e limpar, preparar atributos, escolher e rodar o algoritmo, avaliar, interpretar e comunicar, agir e repetir.
9. Porque, sem gabarito, o valor dos grupos depende de fazerem sentido para especialistas, de terem nomes e perfis claros e de apoiarem ações concretas.
10. Usa dados sem rótulos humanos (como o não supervisionado), mas treina com rótulos criados automaticamente a partir dos dados (como o supervisionado).
11. Para conferir o resultado. Na prática, não teríamos y; o algoritmo descobre os grupos sem ele. Comparar com y é apenas um exercício didático de verificação.
12. Com 2, duas espécies tendem a se fundir; com 5, grupos naturais são subdivididos. Para decidir, usam-se critérios como cotovelo, silhueta e gap (Capítulos 3 e 7), além de interpretação.
13. Viés nos dados (auditar e rebalancear); violação de privacidade (anonimizar, seguir a LGPD); rótulos que estigmatizam (nomear grupos de forma neutra e revisar com especialistas).
14. Regressão prevê um valor numérico (preço de imóvel); classificação prevê uma categoria (gato ou cachorro). Ambas são supervisionadas.
15. Pearson/Hotelling (PCA) com redução de dimensionalidade; Lloyd/MacQueen (K-means) e DBSCAN com agrupamento; Agrawal et al. (1993) com regras de associação.
16. Porque diferentes algoritmos, parâmetros e escolhas de atributos geram divisões distintas; os grupos dependem do objetivo e devem ser validados, não aceitos cegamente.
17. Fraude em cartão e falha de máquina. São raras, mudam de forma ao longo do tempo e são difíceis de rotular, então detectar o "diferente" sem rótulos é mais prático.
18. Resposta pessoal coerente com o roteiro: Parte I conceitos e dados; Parte II agrupamento; Parte III redução de dimensionalidade; Parte IV associação e anomalias; Parte V aprendizado profundo.
19. Resposta pessoal. Deve indicar a família de tarefa (por exemplo, agrupamento de clientes), descrever os atributos necessários e como os resultados apoiariam uma decisão.
20. Dados são o conjunto de fotos; atributos são as características que descrevem cada foto (pixels, cores, formas); rótulo é a resposta correta, como "gato" ou "cachorro", quando existe.

Capítulo 2 — Dados, Distâncias e Pré-processamento

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	Cada linha é um exemplo e cada coluna é um atributo.
2	d)	O CEP é um número "disfarçado": não representa uma quantidade, e sim uma categoria.
3	a)	São categorias com uma ordem natural.
4	b)	A distância pode assumir qualquer valor não negativo; não há limite superior obrigatório.
5	c)	raiz($3^2 + 4^2$) = raiz(25) = 5.
6	d)	$\|3 - 0\| + \|4 - 0\| = 7$.
7	c)	É a maior diferença entre atributos: máx(3, 4) = 4.
8	c)	Com p = 1, soma-se o valor absoluto das diferenças.
9	a)	Em L1, a bola unitária é um losango.
10	a)	Ângulo zero implica cosseno igual a 1, independentemente do tamanho dos vetores.
11	b)	Ela ignora o tamanho e compara a direção (o "perfil") dos vetores.
12	c)	As palavras diferem em apenas uma posição.
13	d)	Interseção = 2, união = 4, similaridade = 0,5; distância = 1 − 0,5 = 0,5.
14	c)	Ela calcula distâncias parciais por atributo e faz a média.
15	c)	Diferenças grandes em valor absoluto pesam mais na soma dos quadrados.
16	c)	z = (x − média) / desvio padrão.
17	c)	z = (70 − 50) / 10 = 2.
18	d)	(30 − 10) / (110 − 10) = 0,2.
19	a)	É menos sensível a valores extremos.
20	d)	O teste só deve receber a transformação já ajustada (transform).
21	b)	A mediana é robusta a extremos.
22	a)	Usa os vizinhos mais próximos para estimar o valor faltante.
23	d)	Evita criar uma ordem inexistente entre as cores.
24	c)	Faria o vermelho parecer "duas vezes mais longe" do azul.
25	a)	O logaritmo aproxima os extremos.
26	a)	Richard Bellman usou a expressão em 1957.
27	c)	As distâncias ficam parecidas, e "vizinho mais próximo" perde significado (Beyer et al., 1999).
28	a)	Normas com p menor sofrem menos com a concentração de distâncias.
29	a)	Garante consistência e evita esquecer passos.
30	a)	Podem ser removidos ou combinados.

Discursivas (respostas-modelo)

1. Cada exemplo é um ponto no espaço de atributos; exemplos parecidos ficam próximos. Agrupamento, redução de dimensionalidade e detecção de anomalias dependem de medir distância ou vizinhança.
2. Diferenças: 6, 25, 4. Euclidiana = raiz(36+625+16) = raiz(677) ≈ 26,0. Manhattan = 6+25+4 = 35. Chebyshev = 25.
3. Euclidiana: régua em linha reta. Manhattan: táxi em quarteirões, andando em "L". Chebyshev: rei do xadrez, que anda em qualquer direção por casa, contando só a maior diferença.

4. Quando a direção importa mais que o tamanho, como textos de comprimentos diferentes sobre o mesmo assunto: as proporções de palavras são parecidas, embora as contagens difiram.
5. Interseção {pão, café} = 2 itens; união = 4 itens. Similaridade = 2/4 = 0,5; distância = 0,5.
6. O gasto varia em centenas de reais e as visitas em unidades; na distância Euclidiana o gasto domina e as visitas quase não pesam. Padronizar iguala a contribuição dos atributos.
7. Padronização: média 0 e desvio 1; padrão para K-means, PCA e GMM. Min-max: intervalo [0,1], útil quando se precisam limites fixos. Robusto: mediana e IQR; indicado com muitos outliers.
8. É quando informação do teste influencia o treino, por exemplo ajustando o escalonador com todos os dados. Evita-se ajustando (fit) só no treino e aplicando (transform) no teste, de preferência com pipelines.
9. Remover (perde dados); imputar média ou mediana (rápido, mas reduz variância); imputar moda (para categorias); imputar por KNN (usa estrutura, mais caro); indicador de ausência (preserva a informação de que faltava).
10. Cria uma coluna binária por categoria. Assim nenhuma categoria parece maior ou mais distante que outra, evitando uma ordem artificial.
11. Atributos constantes não distinguem exemplos; redundantes repetem informação. Mantê-los adiciona ruído às distâncias e aumenta a dimensão sem ganho.
12. Com muitas dimensões, o espaço fica vazio e as distâncias se concentram. Consequências: vizinho mais próximo perde sentido e grupos ficam difíceis de separar; é preciso muito mais exemplos.
13. Selecionar atributos relevantes, reduzir dimensionalidade (PCA, UMAP), usar métricas mais robustas (Manhattan, cosseno) e coletar mais dados.
14. Nominal: categorias sem ordem (cidade). Ordinal: categorias com ordem (baixo, médio, alto). Binário: apenas dois valores (comprou ou não).
15. Não negatividade, identidade, simetria e desigualdade triangular. Exemplo: a distância ao quadrado viola a desigualdade triangular; medidas assimétricas violam a simetria.
16. Para numéricos, diferença dividida pela amplitude do atributo; para categóricos, 0 se iguais e 1 se diferentes. A distância final é a média das distâncias parciais.
17. A renda tem cauda longa e poucos valores muito altos dominam as distâncias. O logaritmo comprime a escala e aproxima os extremos do restante.
18. Gera pontos uniformes e mede (maior − menor)/menor distância a um ponto de referência. O contraste cai rapidamente com a dimensão, mostrando que as distâncias ficam parecidas.
19. Pipeline com SimpleImputer (mediana), StandardScaler e KMeans, na ordem; chamar fit_predict nos dados.
20. Eles puxam a média e inflam o desvio, distorcendo z-scores. Alternativas robustas: mediana, MAD e IQR.

Capítulo 3 — K-means

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	As pizzarias (centros) mudam de endereço para o centro dos moradores que atendem.
2	b)	Inicialização, atribuição, atualização e repetição até convergir.
3	c)	O centroide é a média (por isso "k-means").
4	b)	J = soma, em todos os grupos, de $\|x - \mu\|^2$.
5	d)	Os centroides convergem para 2 e 11, separando os dois blocos naturais.
6	a)	A inércia não aumenta a cada iteração, mas o resultado depende da inicialização.
7	d)	As fronteiras entre regiões formam retas (diagrama de Voronoi).
8	b)	inertia_ guarda o valor de J.
9	b)	cluster_centers_ contém os centros finais.
10	b)	Os centros são sorteados com probabilidade proporcional ao quadrado da distância ao centro mais próximo.
11	c)	Rodar várias vezes reduz o risco de mínimos locais ruins.
12	d)	No limite, com k = n, a inércia é zero.
13	c)	O cotovelo é onde acrescentar grupos traz ganhos cada vez menores.
14	c)	A estatística gap compara a inércia observada com a esperada em dados sem estrutura.
15	b)	Ele busca grupos esféricos; formas não convexas são mal separadas.
16	a)	Um único valor extremo pode deslocar muito o centroide.
17	c)	Atributos de grande escala dominariam o cálculo das distâncias.
18	c)	É muito mais rápido, com pequena perda de qualidade.
19	d)	O medoide é robusto a outliers e aceita qualquer distância.
20	c)	Usa a moda no lugar da média e uma medida de discordância no lugar da distância.
21	d)	A pertinência é parcial, como um cliente 70% "econômico" e 30% "premium".
22	b)	Recência (dias desde a última compra), Frequência e valor (Monetary).
23	d)	Cada pixel vira um ponto no espaço de cores; os centroides formam a paleta.
24	d)	O índice aponta para a cor correspondente na paleta.
25	a)	Escalar, rodar várias inicializações, testar k e inspecionar os grupos.
26	c)	MacQueen (1967) nomeou o método; Lloyd o propôs em 1957 (publicado em 1982).
27	c)	Não se conhece algoritmo rápido que garanta o ótimo; usam-se heurísticas.
28	a)	Indica quantas iterações foram necessárias.
29	b)	km.predict(novos_pontos) devolve o grupo mais próximo.
30	c)	Ele supõe grupos de tamanho e densidade semelhantes.

Discursivas (respostas-modelo)

1. Pizzarias são centroides; bairros atendidos são os grupos. Cada morador escolhe a pizzaria mais próxima e cada pizzaria se muda para o centro dos seus moradores. Quando ninguém muda de pizzaria, houve convergência.
2. Inicializar k centros; atribuir cada ponto ao centro mais próximo; recalcular cada centro como a média do seu grupo; repetir até as atribuições não mudarem ou atingir o limite de iterações.

3. Rodada 1: A={1}, B={2,3,10,11,12}; centros 1 e 7,6. Rodada 2: A={1,2,3}, B={10,11,12}; centros 2 e 11. Rodada 3: nenhuma mudança, convergiu.
4. Inércia é a soma dos quadrados das distâncias dos pontos ao centro do seu grupo. A atribuição ao centro mais próximo não aumenta J, e a média minimiza J dentro de cada grupo; logo, J não aumenta.
5. O resultado depende dos centros iniciais, e a otimização é local. Reduz-se usando k-means++, várias inicializações (n_init) e escolhendo a menor inércia.
6. O primeiro centro é aleatório; cada novo centro é sorteado com probabilidade proporcional ao quadrado da distância ao centro mais próximo. Isso espalha os centros e, em média, o resultado é O(log k) vezes pior que o ótimo, com garantia teórica.
7. Calcula-se a inércia para vários k e procura-se onde a queda desacelera. Cuidados: o cotovelo pode ser suave ou ambíguo; combinar com silhueta, gap e interpretação.
8. Grupos esféricos (GMM, espectral); tamanhos desiguais (GMM); densidades diferentes (DBSCAN); outliers (K-medoids); necessidade de k (cotovelo, silhueta); só numérico (K-modes ou Gower).
9. K-means usa a média e distância Euclidiana; K-medoids usa um ponto real central, robusto a outliers e flexível quanto à distância; K-modes usa a moda e é indicado para dados categóricos.
10. Cada ponto tem grau de pertinência a todos os grupos. É vantajoso quando as fronteiras são ambíguas, como clientes com perfil misto.
11. Usa pequenos lotes aleatórios a cada passo em vez de todos os dados. O custo é uma pequena perda de qualidade da solução.
12. Analisa-se o perfil médio de cada grupo: recência baixa com frequência e valor altos indica clientes fiéis; recência alta e baixo valor indica inativos. Cada grupo recebe ação própria: fidelidade, cupons, reativação.
13. Cada pixel é um ponto (R, G, B); K-means encontra k cores representativas; cada pixel é substituído pela cor do centroide. O arquivo passa a guardar índices e uma paleta, reduzindo o tamanho.
14. Distância Euclidiana é dominada por atributos de grande amplitude. Em dados de clientes, o gasto em reais domina o número de visitas; padronizando, ambos contribuem igualmente.
15. É a região dos pontos mais próximos de um dado centroide. As fronteiras entre grupos são retas equidistantes de dois centroides, o que explica grupos convexos.
16. Supõe grupos esféricos de tamanhos similares. Falha em luas, anéis concêntricos ou elipses alongadas, cortando o grupo real ao meio.
17. A média minimiza a soma dos quadrados das distâncias dentro do grupo. Com outras distâncias, o centro ótimo muda (por exemplo, mediana ou medoide).
18. Limpar e escalar os dados; rodar várias inicializações; testar k com cotovelo, silhueta e gap; verificar a estabilidade; perfilar e nomear os grupos; documentar as escolhas.
19. K-means escala bem para grandes bases, mas exige k de antemão; o hierárquico não exige k antes (corta-se a árvore), porém tem custo quadrático ou cúbico.
20. Resposta pessoal coerente, como agrupar estações meteorológicas por padrão de chuva; justificativa: atributos numéricos padronizados e grupos aproximadamente esféricos.

Capítulo 4 — Agrupamento Hierárquico

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Entrega uma árvore de fusões, e não apenas uma divisão.
2	c)	É um processo de baixo para cima (bottom-up): junta os mais próximos.
3	c)	Começa com tudo em um grupo e vai dividindo.
4	b)	O dendrograma mostra a sequência de fusões e as alturas.
5	b)	Fusões baixas indicam grupos muito parecidos; altas, grupos diferentes.
6	b)	A ordem horizontal das folhas não tem significado por si só.
7	d)	Cada ramo cortado forma um grupo.
8	d)	Um grande salto indica que a próxima fusão juntaria coisas bem diferentes.
9	c)	É a ligação do vizinho mais próximo (single linkage).
10	d)	Uma fileira de pontos intermediários pode unir grupos distintos.
11	a)	Tende a gerar grupos compactos, mas é sensível a outliers.
12	a)	É um meio-termo entre a simples e a completa.
13	a)	É a "irmã hierárquica" do K-means, pois minimiza somas de quadrados.
14	a)	Ward é definida em termos de variância e requer a distância Euclidiana.
15	b)	A menor distância é 1 (A–B e C–D).
16	d)	A menor distância entre os grupos é B a C = 4.
17	b)	A menor distância entre {A,B,C,D} e E é D a E = 8.
18	c)	Armazena a matriz de distâncias n × n.
19	d)	O custo quadrático ou cúbico limita sua escalabilidade.
20	d)	Expressa as várias ligações em uma única fórmula com parâmetros diferentes.
21	b)	Zhang, Ramakrishnan e Livny (1996) propuseram o resumo compacto dos dados.
22	d)	Z = linkage(X, method="ward") devolve a matriz de ligações.
23	d)	Com criterion="maxclust", obtém-se k grupos.
24	c)	Quanto mais próxima de 1, mais fiel é a árvore.
25	b)	distance_threshold define a altura do corte.
26	d)	Mostra grupos de linhas e colunas com comportamento parecido.
27	d)	Uma fusão errada nunca é desfeita.
28	c)	Os mesmos dados produzem sempre a mesma árvore.
29	b)	Sokal e Michener (1958) usaram a ligação média em taxonomia numérica.
30	b)	O trabalho é muito citado na biologia computacional.

Discursivas (respostas-modelo)

1. Como numa família, dois irmãos se juntam num ramo (pais), que se une ao dos primos (avós) e assim por diante. O método junta exemplos semelhantes em níveis sucessivos até formar um único grupo.
2. Aglomerativo junta de baixo para cima; divisivo separa de cima para baixo. O aglomerativo é mais usado por ser mais simples e menos custoso.

3. Folhas são os exemplos; cada junção é uma fusão; a altura é a distância entre os grupos fundidos; a ordem horizontal não tem significado por si só.
4. Traça-se um corte horizontal, de preferência onde há um grande salto vertical. Cuidado: depende da ligação; validar com critérios de avaliação e interpretação.
5. Simples: par mais próximo; capta formas alongadas, mas sofre de efeito corrente. Completa: par mais distante; grupos compactos, sensível a outliers. Média: média dos pares; equilíbrio. Ward: minimiza aumento da variância; grupos equilibrados, exige Euclidiana.
6. Na ligação simples, pontos intermediários formam uma ponte entre grupos distintos; como basta um par próximo, os grupos se unem em cadeia.
7. Fusões nas alturas 1 (A–B), 1 (C–D), 4 ({A,B}–{C,D}) e 8 (E). Corte entre 4 e 8 produz {A,B,C,D} e {E}.
8. Memória da ordem de n^2 e tempo entre $n^2 \log n$ e n^3. Pode-se amostrar, agrupar com K-means e aplicar o hierárquico aos centroides ou usar o BIRCH.
9. Mede a correlação entre as distâncias originais e as distâncias implícitas no dendrograma. Escolhe-se a ligação com maior correlação, indicando melhor representação das distâncias.
10. Reordena linhas e colunas segundo as árvores, formando blocos de padrão semelhante, como grupos de genes com expressão parecida em certos experimentos ou clientes que compram certos produtos.
11. Dados e ligação fixos geram sempre a mesma árvore. O K-means depende da inicialização aleatória e pode dar resultados diferentes.
12. Vantagens: sem k prévio, hierarquia completa, dendrograma comunicável, flexível em distâncias. Limitações: custo alto, fusões irreversíveis, sensível a ruído e à ligação escolhida.
13. Ambas minimizam somas de quadrados dentro dos grupos: Ward o faz de modo hierárquico e guloso; o K-means, por partição iterativa.
14. K-means exige k, dá uma partição, escala bem e é não determinístico; hierárquico não exige k prévio, dá árvore de partições, escala mal e é determinístico.
15. Em cada junção, os dois ramos podem trocar de lado; o que define o agrupamento é quem se junta com quem e a que altura.
16. Com SciPy: `Z = linkage(X, "ward"); fcluster(Z, t=3, criterion="maxclust")`. Com scikit-learn: `AgglomerativeClustering(n_clusters=3, linkage="ward").fit_predict(X)`.
17. Quando os grupos são alongados ou de forma irregular e bem separados por vazios, pois a ligação simples os acompanha; Ward favorece grupos esféricos.
18. Classificar espécies por características morfológicas: a árvore mostra relações em vários níveis, de espécies a gêneros e famílias, algo que uma partição única não mostra.
19. O algoritmo é guloso: uma vez unidos, dois grupos nunca se separam. Um erro de fusão no início se propaga, o que torna o método sensível a ruído e a outliers.
20. É uma fórmula única para atualizar as distâncias entre o novo grupo e os demais após uma fusão; variando seus parâmetros, obtêm-se as ligações simples, completa, média e de Ward (Lance e Williams, 1967).

Capítulo 5 — Agrupamento por Densidade

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	Grupos são regiões densas cercadas por vazio.
2	a)	Não precisa de k, aceita formas livres e rotula ruído.
3	b)	eps é o raio da vizinhança; min_samples é o mínimo de pontos para ser denso.
4	d)	Define o tamanho do círculo em volta de cada ponto.
5	a)	Está "no coração" de uma região densa.
6	b)	Fica na periferia de um grupo.
7	d)	Pontos que não pertencem a nenhum grupo recebem −1.
8	d)	A vizinhança é tão estreita que poucos pontos são centrais.
9	c)	A vizinhança abrange regiões que deveriam ficar separadas.
10	b)	O valor de y no joelho é um bom candidato a eps.
11	a)	Em 2D, valores entre 4 e 10 costumam funcionar.
12	c)	O raio eps vale para todos os atributos igualmente.
13	d)	Com um só eps, um grupo difuso pode virar ruído ou grupos densos se fundirem.
14	b)	Árvores espaciais aceleram a busca de vizinhos.
15	a)	O gráfico de alcançabilidade mostra a estrutura de densidade.
16	c)	Picos são fronteiras; vales são grupos.
17	b)	Constrói uma hierarquia de densidades e escolhe os grupos mais estáveis.
18	a)	Intuitivo: o menor grupo que interessa.
19	d)	Antes, era distribuído no pacote independente hdbscan.
20	c)	Dá uma medida de quão bem cada ponto pertence ao grupo.
21	c)	Os picos de densidade são os centros dos grupos.
22	a)	Coordenadas geográficas exigem a distância sobre a esfera.
23	a)	A distância haversine opera em radianos sobre a esfera.
24	c)	O DBSCAN segue a forma dos grupos; o K-means os corta ao meio.
25	b)	Ele escolhe grupos estáveis em várias escalas de densidade.
26	c)	A maldição da dimensionalidade afeta as distâncias.
27	b)	Ruído nunca é centro de grupo.
28	c)	O K-means exige k e supõe grupos esféricos.
29	d)	Pontos de ruído aparecem na subida acentuada da curva.
30	c)	Escolhe-se onde cortar para definir os grupos.

Discursivas (respostas-modelo)

1. Manchas brilhantes (cidades) separadas por áreas escuras correspondem a grupos (regiões densas) e vazio; luzes isoladas são ruído. Percebem-se as manchas sem fixar a quantidade.
2. Com min_samples = 4, é central o ponto com pelo menos 4 pontos (incluindo ele) no raio eps; borda é não central dentro do raio de um central; ruído é o que não é nenhum dos dois.

3. Escolher ponto não visitado; se é central, criar grupo e expandir pelos vizinhos centrais; se não, marcar como ruído provisório (pode virar borda); repetir até visitar todos.
4. Quem tem muitos amigos próximos (central) repassa o boato; quem está na periferia (borda) o recebe, mas não repassa. O grupo é a cadeia de quem recebeu.
5. min_samples ≥ d + 1 (4 a 10 em 2D). Para eps, ordenam-se as distâncias ao k-ésimo vizinho e procura-se o joelho da curva; o valor nesse ponto é candidato a eps.
6. Muito pequeno: quase tudo vira ruído e micro-grupos. Muito grande: grupos distintos se fundem em um só.
7. Um único eps não serve a densidades distintas: ou o grupo difuso vira ruído ou o denso se funde. Alternativas: OPTICS e HDBSCAN.
8. Mostra a distância de alcançabilidade dos pontos na ordem calculada; vales profundos são grupos densos, vales rasos são difusos, picos são fronteiras.
9. Distância de alcançabilidade mútua; árvore geradora mínima e hierarquia; condensação da árvore descartando divisões minúsculas; seleção dos grupos mais estáveis.
10. DBSCAN: eps e min_samples; fraco com densidades variáveis. OPTICS: min_samples e xi; lida bem, com gráfico. HDBSCAN: min_cluster_size; muito bom com densidades variáveis, dá probabilidades.
11. Usa um raio único; atributos de maior escala dominariam a vizinhança, distorcendo os grupos.
12. Embarques de táxi: pontos quentes têm formas irregulares, há ruído e k é desconhecido. DBSCAN com distância haversine encontra regiões densas e isola embarques isolados.
13. Em alta dimensão as distâncias se concentram e o espaço fica vazio; a noção de vizinhança perde sentido, tornando eps difícil de calibrar.
14. −1 marca ruído. Grupos = número de rótulos distintos menos 1 se −1 existir; ruído = soma de labels_ == −1.
15. Cada ponto sobe o gradiente da densidade até um pico; os picos são os centros. Não exige k, ao contrário do K-means.
16. Um ponto de borda pode estar no raio de centrais de dois grupos; será do primeiro que o alcançar na ordem de visita.
17. Limpar e padronizar; escolher min_samples; plotar k-distâncias para eps; rodar; verificar ruído e grupos; comparar com HDBSCAN; interpretar.
18. Densidades variáveis e dificuldade de escolher eps; HDBSCAN só pede min_cluster_size e dá probabilidades de pertinência.
19. Pontos de ruído podem indicar anomalias, erros ou eventos raros; identificá-los evita forçá-los a grupos e pode orientar investigações.
20. K-means: esféricos. Hierárquico simples: formas alongadas, mas sofre efeito corrente. DBSCAN: formas arbitrárias, resistente a ruído.

Capítulo 6 — Modelos de Mistura Gaussiana e Agrupamento Espectral

Múltipla escolha

Questão	Resposta	Justificativa
1	c)	A média dá o centro do sino e o desvio padrão a sua largura.
2	b)	Cerca de 68% dos valores ficam a até um desvio padrão da média.
3	d)	A matriz Σ descreve a variância de cada atributo e a covariância entre eles.
4	a)	A inclinação reflete a correlação entre os atributos.
5	b)	$p(x) = \Sigma\, \pi_k \cdot N(x \mid \mu_k, \Sigma_k)$.
6	c)	Indicam a fração dos dados vinda de cada componente.
7	d)	Passo E: responsabilidades; passo M: parâmetros.
8	a)	Dados os parâmetros atuais, estima-se a pertinência suave.
9	d)	Usam-se médias e covariâncias ponderadas pelas responsabilidades.
10	b)	O EM aumenta ou mantém a verossimilhança, convergindo a um máximo local.
11	d)	O GMM generaliza o K-means com covariâncias flexíveis e pertinências suaves.
12	a)	O GMM devolve probabilidades por componente.
13	d)	predict_proba dá o agrupamento suave.
14	d)	Cada componente pode ter elipse de forma e inclinação livres.
15	c)	É o tipo mais restrito, com poucos parâmetros.
16	c)	Pode memorizar o ruído dos dados.
17	c)	Penalizam a complexidade do modelo.
18	c)	O termo $p \cdot \ln(n)$ penaliza mais quando n é grande.
19	d)	Impede que uma gaussiana colapse sobre um único ponto.
20	d)	Devolve o log da densidade de cada ponto.
21	c)	Pontos são nós e semelhanças são arestas ponderadas.
22	c)	D é a matriz de graus e W a matriz de afinidade.
23	a)	A multiplicidade do autovalor zero é o número de componentes conexas.
24	d)	As linhas viram pontos compactos e o K-means os separa.
25	c)	Separa estruturas que o K-means não consegue.
26	a)	O número de autovalores antes do salto sugere k.
27	b)	Cortar só pelo menor número de arestas pode isolar um único ponto.
28	d)	Para bases grandes, usam-se aproximações como Nyström ou grafos esparsos.
29	a)	O peso é $\exp(-\gamma\|x - y\|^2)$.
30	d)	O GMM captura elipses e dá pertinência suave.

Discursivas (respostas-modelo)

1. O histograma de alturas tem duas corcovas: cada uma é uma normal. A mistura é a soma ponderada das duas; o GMM estima médias, desvios e pesos de cada componente.

2. Não se conhecem nem os grupos de cada ponto nem os parâmetros. O EM alterna: passo E calcula responsabilidades dados os parâmetros; passo M recalcula os parâmetros dadas as responsabilidades; repete até a verossimilhança estabilizar.
3. Rígido atribui um único grupo; suave dá probabilidades por grupo. É vantajoso com fronteiras ambíguas, como um cliente de perfil misto que pode receber duas campanhas.
4. spherical (círculos), diag (elipses alinhadas aos eixos), tied (mesma forma para todas) e full (livre). Mais flexibilidade ajusta melhor, mas aumenta parâmetros e o risco de sobreajuste.
5. Ajustam-se modelos com K variando e calcula-se o critério, que equilibra verossimilhança e número de parâmetros. Escolhe-se o K de menor valor; BIC penaliza mais a complexidade.
6. Inicialização (várias execuções), singularidades (reg_covar), alta dimensão (covariância full tem muitos parâmetros) e suposição gaussiana.
7. score_samples devolve a log-densidade; pontos de baixa densidade são suspeitos. O método sample gera novos dados a partir do modelo ajustado.
8. Construir o grafo de similaridade; calcular a Laplaciana; obter os k menores autovetores; aplicar K-means às linhas.
9. L = D − W codifica a conectividade do grafo; o número de autovalores zero igual ao de componentes conexas, e os menores autovetores revelam comunidades que se tornam nuvens compactas.
10. O grafo de vizinhos liga pontos de um mesmo anel, e a representação nos autovetores separa os anéis em nuvens compactas; o K-means, usando distância ao centro, corta os anéis.
11. Equilibra o custo do corte com o tamanho dos grupos, penalizando grupos minúsculos. O corte mínimo simples poderia isolar um único ponto.
12. Ordenam-se os autovalores da Laplaciana; o maior salto entre consecutivos indica o número de grupos (os autovalores antes do salto).
13. k vizinhos mais próximos; vizinhos dentro de um raio ε; kernel gaussiano (RBF).
14. Custo alto de autovetores, sensibilidade à similaridade e dificuldade com pontos novos. Contornos: aproximação de Nyström, grafos esparsos de vizinhos e extensões para novos pontos.
15. K-means: rígida, esféricos, exige k. GMM: suave, elípticos, exige K (BIC). Espectral: rígida, arbitrária, exige k (eigengap). DBSCAN: rígida com ruído, arbitrária, sem k.
16. Mais componentes ajustam melhor os dados, até decorar o ruído. Evita-se com BIC/AIC, validação e limitação do tipo de covariância.
17. É a probabilidade de um ponto vir de cada componente. Exemplo: 80% do grupo A, 15% do B e 5% do C; somam 100%.
18. Clientes com comportamento misto recebem probabilidades por segmento (por exemplo 60% econômico e 40% premium), permitindo campanhas combinadas ou análise de transição entre perfis.
19. Esféricos e base grande: K-means; elípticos e probabilidades: GMM; formas complicadas e base média: espectral; ruído e forma livre: DBSCAN ou HDBSCAN.
20. O GMM define uma distribuição de probabilidade para os dados e permite gerar novas amostras; o K-means só particiona os dados, sem modelo probabilístico.

Capítulo 7 — Como Saber se os Grupos São Bons? Avaliação e Validação

Múltipla escolha

Questão	Resposta	Justificativa
1	c)	Sem gabarito, é preciso usar critérios internos, relativos, externos e interpretação.
2	c)	Mede coesão e separação sem consultar rótulos.
3	d)	Índices como ARI e NMI exigem uma classificação de referência.
4	c)	Varia k ou parâmetros e compara índices.
5	a)	Verifica se os dados têm estrutura agrupável ou são uniformes.
6	b)	Perto de 1 indica forte agrupamento; perto de 0, dados muito regulares.
7	a)	Os pontos reais estão mais próximos entre si do que pontos aleatórios.
8	a)	b(i) é a menor distância média a outro grupo.
9	c)	O valor fica entre −1 e 1.
10	b)	Está mais próximo de outro grupo do que do seu.
11	c)	Abaixo de 0,25, a estrutura é considerada fraca (Kaufman e Rousseeuw, 1990).
12	b)	É a razão entre dispersão entre e dentro dos grupos.
13	d)	Mede a similaridade média entre cada grupo e seu mais parecido.
14	d)	Quanto maior, melhor; é muito sensível a outliers.
15	b)	Podem punir resultados corretos do DBSCAN em luas ou anéis.
16	b)	Moulavi et al. (2014) propuseram o DBCV para esse fim.
17	b)	Tibshirani, Walther e Hastie (2001).
18	c)	Se os grupos são reais, resistem a pequenas perturbações.
19	b)	O ARI é invariante à troca dos nomes dos grupos.
20	d)	Os números dos grupos são arbitrários.
21	a)	Mostra quantos pontos de cada classe caíram em cada grupo.
22	c)	Por isso é pouco informativa sem controle do número de grupos.
23	d)	O ARI desconta a concordância esperada ao acaso (Hubert e Arabie, 1985).
24	c)	Os agrupamentos coincidem, a menos dos nomes dos grupos.
25	b)	Mede quanto um agrupamento informa sobre o outro, entre 0 e 1.
26	c)	Rosenberg e Hirschberg (2007).
27	b)	A completude é o inverso: classes não divididas.
28	d)	Agrupamento não tem uma definição única.
29	d)	Ajuda a nomear e interpretar os grupos.
30	b)	Combinar critérios e interpretação.

Discursivas (respostas-modelo)

1. Os algoritmos particionam os dados por construção; isso não prova que haja grupos reais. Por isso é preciso avaliar a tendência de agrupamento e a qualidade dos grupos.

2. Interna: silhueta (só dados e grupos). Externa: ARI ou NMI (compara com rótulos). Relativa: gap ou estabilidade (compara soluções para escolher k).
3. Compara distâncias de pontos reais ao vizinho mais próximo com as de pontos aleatórios uniformes. H ≈ 0,5: sem grupos; H próximo de 1: forte tendência; H próximo de 0: dados regulares.
4. s = (6 − 2) / máx(2, 6) = 4/6 ≈ 0,67; o ponto está bem encaixado no seu grupo.
5. Quando a ≈ b, a distância média ao seu grupo é parecida com a do grupo vizinho, ficando no limite entre os dois.
6. Silhueta: encaixe individual (maior é melhor). CH: razão entre/dentro (maior). DB: similaridade entre grupos (menor). Todos tendem a favorecer grupos convexos e compactos.
7. Pressupõem compacidade e convexidade; grupos alongados ou curvos têm baixa coesão segundo esses índices, mesmo estando corretamente separados.
8. Gap(k) = E[log W_k de referência] − log W_k observado; escolhe-se o menor k tal que Gap(k) ≥ Gap(k+1) − desvio padrão de Gap(k+1).
9. Sortear subamostras, agrupar cada uma, comparar pares de soluções (no conjunto inteiro) com ARI e tomar a média; alta concordância indica agrupamento estável.
10. A diagonal domina, com poucos erros: o grupo 0 corresponde à classe A, o 1 à B e o 2 à C. Reproduz bem as classes, com pequenas confusões entre B e C.
11. Soma das maiores classes por grupo: 48 + 45 + 44 = 137; total = 150; pureza ≈ 0,913.
12. O Rand conta pares concordantes, mas inclui concordância ao acaso. O ARI desconta a concordância esperada ao acaso, indo de 1 (perfeita) a perto de 0 (acaso).
13. Homogeneidade: cada grupo tem uma só classe. Completude: cada classe está em um só grupo. V-measure: média harmônica das duas.
14. Os grupos fazem sentido para o especialista? São acionáveis? Têm tamanho útil? São estáveis no tempo? Diferem em variáveis de negócio?
15. Mostra, por atributo, se o grupo está acima (maior que 1) ou abaixo (menor que 1) da média geral, facilitando nomear e comparar os grupos.
16. Hopkins; variar k e calcular índices; testar estabilidade; comparar algoritmos; visualizar; perfilar e validar com especialistas; documentar escolhas.
17. Representam uma possível organização, vinculada a um objetivo; outros agrupamentos podem ser igualmente válidos para outros fins (von Luxburg et al., 2012).
18. k = 4 pelo maior valor de silhueta. Confirmar com CH, DB e gap, testar estabilidade, visualizar e verificar a interpretabilidade com especialistas.
19. O gap compara com dados de referência sem estrutura e fornece um critério quantitativo com desvio padrão, reduzindo a subjetividade de procurar o "cotovelo" a olho.
20. Coesão é a proximidade dos pontos dentro do mesmo grupo; separação é a distância entre grupos diferentes. Numa festa, cada rodinha de conversa deve ser unida (coesão) e as rodinhas devem ficar afastadas (separação).

Capítulo 8 — PCA e Métodos Lineares

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	O PCA é um método de extração.
2	a)	Hotelling (1933) formalizou e nomeou o método.
3	d)	O PCA busca direções de máxima variância.
4	a)	PC2 é perpendicular a PC1 e tem a maior variância restante.
5	d)	Cada componente é perpendicular às anteriores.
6	b)	Subtrai-se a média de cada coluna.
7	b)	Os autovalores são as variâncias ao longo das componentes.
8	c)	V contém as direções principais; U·S, os escores.
9	b)	Fundamenta a compressão por PCA/SVD.
10	b)	Somando-se, obtém-se a variância acumulada.
11	a)	Equilibra compressão e informação retida.
12	d)	Após o cotovelo, as componentes acrescentam pouco.
13	a)	Cada componente retida explica mais que um atributo original sozinho.
14	a)	Mantêm-se apenas as componentes que superam o acaso.
15	b)	Os escores são as novas coordenadas.
16	b)	Setas na mesma direção indicam atributos correlacionados.
17	d)	A direção é a mesma, só o sentido muda.
18	c)	Perde-se os detalhes das componentes descartadas.
19	d)	Para estruturas curvas, usam-se Kernel PCA ou aprendizado de variedades.
20	a)	Padronize os dados e considere versões robustas.
21	d)	Processa os dados em lotes.
22	a)	Zou, Hastie e Tibshirani (2006).
23	c)	Usa um kernel para fazer PCA implicitamente num espaço de maior dimensão.
24	d)	É um critério mais forte que a ausência de correlação do PCA.
25	b)	Separar vozes misturadas em microfones, sem conhecer a mistura.
26	a)	Pelo Teorema Central do Limite; o ICA maximiza a não gaussianidade.
27	d)	X ≈ W·H, com todos os elementos maiores ou iguais a zero.
28	d)	A natureza aditiva favorece partes interpretáveis.
29	a)	W indica o quanto cada documento fala de cada tópico.
30	d)	A direção comum concentra grande parte da variância.

Discursivas (respostas-modelo)

1. Projetar um objeto 3D em uma parede produz sombras diferentes conforme o ângulo; o PCA procura o ângulo cuja projeção preserva o máximo de variância, isto é, de informação.
2. Centralizar; calcular a covariância; obter autovalores e autovetores; ordenar do maior para o menor autovalor; escolher os k primeiros; projetar Z = X_centralizado × W.

3. Atributos com pouca variação distinguem pouco os exemplos. Falha quando uma direção de baixa variância separa classes importantes, pois o PCA não conhece rótulos.
4. As direções principais coincidem com os vetores singulares à direita; autovalor = $s^2/(n-1)$. A SVD é numericamente mais estável e rápida do que diagonalizar a covariância.
5. Variância acumulada: atingir limiar (90–95%). Scree: cotovelo. Kaiser: autovalor > 1 (simples, discutível). Análise paralela: superar o acaso (mais confiável).
6. Duas componentes somam 87%; três somam 95%. Para pelo menos 90%, mantêm-se três.
7. Escores são as coordenadas dos exemplos; cargas são os pesos dos atributos em cada componente. No biplot, pontos são exemplos e setas são atributos; setas próximas indicam correlação, opostas anticorrelação, perpendiculares independência.
8. Mantêm-se poucas componentes e reconstrói-se com inverse_transform; o ruído, em direções de baixa variância, é descartado, enquanto o essencial é preservado.
9. Linearidade; sensibilidade à escala; sensibilidade a outliers; variância não é importância; componentes de difícil interpretação; sinais arbitrários.
10. Incremental: lotes, para bases grandes. Aleatorizado: aproxima a SVD rapidamente. Esparso: cargas com zeros, interpretável. Kernel: captura relações não lineares.
11. Dois microfones gravam misturas de duas vozes. O PCA busca não correlação e não separa as fontes; o ICA maximiza a independência (não gaussianidade) e recupera as vozes.
12. H guarda peças básicas e W a quantidade de cada peça em cada exemplo; como só há adição (valores não negativos), as peças formam partes do todo.
13. Constrói-se matriz TF-IDF documento × palavra, aplica-se NMF com k tópicos; cada linha de H lista as palavras de maior peso de um tópico e W indica a presença de cada tópico por documento.
14. PCA: eixos ortogonais com valores positivos e negativos, componentes globais. NMF: valores não negativos, componentes aditivos como partes.
15. Atributos de maior escala teriam maior variância e dominariam as primeiras componentes, distorcendo a análise.
16. A seleção retira ingredientes originais; o PCA cria misturas prontas (combinações) que resumem vários ingredientes.
17. Padronizar, ajustar PCA com k componentes, medir variância acumulada e erro de reconstrução (inverse_transform), escolhendo k pelo equilíbrio entre compressão e erro.
18. Variância explicada é a fração da variância total capturada por cada componente. Com n_components=0.95, o PCA escolhe o menor número de componentes que somam pelo menos 95%.
19. Remoção de piscadas em eletroencefalograma: as fontes (cérebro e artefato) são independentes e não gaussianas; o ICA as separa, o PCA não.
20. Os escores das componentes têm covariância zero entre si, de modo que cada componente carrega informação não repetida; isso reduz redundância e facilita análises posteriores.

Capítulo 9 — Aprendizado de Variedades

Múltipla escolha

Questão	Resposta	Justificativa
1	b)	A estrutura real tem poucos graus de liberdade.
2	b)	A Terra é o exemplo clássico.
3	b)	A distância geodésica difere da euclidiana.
4	d)	É como a distância por rodovia, e não de voo.
5	c)	Constrói um mapa em baixa dimensão que respeite essas distâncias.
6	b)	Quanto menor, melhor o mapa.
7	a)	Usa autovetores e recupera o mesmo subespaço.
8	b)	Útil quando só há ranking de semelhança.
9	a)	Aproxima a distância geodésica pelo menor caminho no grafo.
10	a)	Também exige um grafo conectado.
11	b)	Preserva relações locais ao desenrolar a variedade.
12	b)	Quem era vizinho continua vizinho.
13	a)	As caudas pesadas evitam o amontoamento (crowding) no centro.
14	a)	Mede a diferença entre as probabilidades originais e as do mapa.
15	c)	Valores típicos: 5 a 50.
16	b)	Foca em estruturas muito locais.
17	d)	O t-SNE adapta a escala local; grupos densos e esparsos podem parecer iguais.
18	a)	Só a vizinhança local é confiável.
19	a)	Acelera e estabiliza o resultado.
20	d)	O método transform do UMAP projeta novos dados.
21	b)	Pequeno: detalhes locais; grande: estrutura global.
22	a)	Pequeno: blocos densos; grande: pontos mais espalhados.
23	d)	Com boa inicialização, o t-SNE se aproxima do UMAP na estrutura global.
24	b)	Relaciona-se ao agrupamento espectral (Capítulo 6).
25	b)	Venna e Kaski (2006).
26	c)	PCA, Kernel PCA e UMAP aprendem uma função de projeção.
27	c)	São mais escaláveis.
28	b)	Ambos buscam preservar distâncias globais (geodésicas no Isomap).
29	c)	O Isomap e o LLE inauguraram o aprendizado de variedades.
30	d)	Por isso se recomenda fixar a semente e testar mais de uma execução.

Discursivas (respostas-modelo)

1. Pontos em voltas vizinhas do tapete parecem próximos em linha reta, mas são distantes sobre a superfície. Fotos de rosto têm milhares de pixels, mas variam por um parâmetro (o ângulo): os dados vivem numa curva de baixa dimensão.
2. A euclidiana é a linha reta; a geodésica é ao longo da variedade. O Isomap liga cada ponto a k vizinhos e usa o menor caminho no grafo como aproximação da geodésica.

3. Busca posições 2D cujas distâncias reproduzam as da tabela, minimizando o stress; o mapa resultante pode sair girado ou espelhado, mas preserva as distâncias relativas.
4. Clássico: via autovetores (equivale ao PCA para distâncias euclidianas). Métrico por stress (SMACOF): otimiza iterativamente o stress. Não métrico: preserva apenas a ordem das distâncias.
5. Cada ponto é reconstruído como combinação de seus vizinhos; procura-se um mapa de baixa dimensão que mantenha esses mesmos pesos, preservando relações locais.
6. Calcular probabilidades de vizinhança no espaço original (gaussiana); inicializar o mapa e calcular probabilidades com t de Student; minimizar a divergência KL por gradiente; os pontos se atraem (vizinhos) e se repelem (não vizinhos).
7. Em baixa dimensão há menos espaço; as caudas pesadas permitem que pontos não vizinhos fiquem bem afastados, evitando o amontoamento no centro (crowding).
8. É o número aproximado de vizinhos efetivos de cada ponto. Valores de 5 a 50 são típicos; testam-se vários valores, pois baixos fragmentam grupos e altos podem fundi-los.
9. Tamanho dos grupos sem significado; distâncias entre grupos distantes pouco informativas; formas podem enganar; resultado varia com semente e perplexidade; versão original sem projeção de novos pontos.
10. Constrói um grafo difuso de vizinhos em alta dimensão e encontra um layout de baixa dimensão cujo grafo seja parecido. n_neighbors controla a escala local; min_dist, a compactação dos pontos no mapa.
11. UMAP é mais rápido, tem transform e costuma preservar mais estrutura global (muito devido à inicialização); t-SNE usa perplexidade e preserva sobretudo a estrutura local.
12. Pode criar estrutura aparente a partir de ruído e depende de parâmetros; deve-se confirmar com outros métodos e testes estatísticos.
13. Trustworthiness mede se vizinhos no mapa eram vizinhos no original; a correlação de distâncias mede a fidelidade da estrutura global. Combinam-se para comparar métodos.
14. Cada um otimiza um critério distinto: variância global (PCA), distâncias geodésicas (Isomap), vizinhança local probabilística (t-SNE) e grafo difuso (UMAP).
15. Visualizar rapidamente: PCA; transformar dados novos: PCA ou UMAP; base enorme: UMAP; geometria global: Isomap/MDS; componentes interpretáveis: PCA ou NMF.
16. Ambos usam autovetores do Laplaciano do grafo de vizinhos; Eigenmaps os usam como coordenadas de baixa dimensão e o espectral aplica K-means a essas coordenadas.
17. Remove ruído e reduz o custo; trabalhar com 30–50 componentes preserva o essencial e torna o t-SNE mais rápido e estável.
18. Visualização de células individuais em transcriptômica: cada ponto é uma célula e grupos são tipos celulares. Cuidados: não interpretar distâncias globais, testar parâmetros e validar com marcadores.
19. O PCA projeta em planos e sobrepõe voltas distintas; a estrutura é não linear e exige desenrolar a variedade (Isomap, LLE, UMAP).
20. A otimização é não convexa e depende da inicialização; estruturas que só aparecem em uma configuração podem ser artefatos. Estruturas que persistem em várias execuções são mais confiáveis.

Capítulo 10 — Regras de Associação

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	É a aplicação clássica das regras de associação.
2	c)	Cada cesta, sessão ou consulta é uma transação.
3	d)	B é o consequente.
4	b)	Sorvete e afogamentos coocorrem no verão, mas um não causa o outro.
5	d)	suporte(A) = transações com A / total de transações.
6	b)	O pão aparece em 8 das 10 compras.
7	d)	É P(B | A).
8	b)	Lift > 1 indica associação positiva; < 1, negativa.
9	b)	Lift = confiança / suporte(B).
10	a)	O lift compara com o que se esperaria por acaso.
11	a)	suporte({pão, manteiga}) = 0,5 e suporte(pão) = 0,8; 0,5/0,8 = 0,625.
12	c)	0,5 / 0,6 ≈ 0,83.
13	a)	Equivalentemente, se um itemset é infrequente, seus superconjuntos também são.
14	d)	Candidatos com subconjunto infrequente são descartados.
15	d)	Um subconjunto infrequente torna o conjunto maior infrequente.
16	b)	Por isso a explosão combinatória inviabiliza testar todos.
17	c)	Requer apenas duas leituras da base.
18	c)	A árvore comprime a base.
19	d)	Zaki (2000).
20	a)	A tabela one-hot é a entrada de apriori e fpgrowth.
21	b)	Permite filtrar por confiança ou lift.
22	c)	Diferença absoluta em relação à independência.
23	c)	Além de eliminar regras redundantes e triviais.
24	a)	Regras óbvias dão pouco valor de negócio.
25	a)	Por exemplo, ao separar por região ou tipo de cliente.
26	c)	Convém aplicar correções estatísticas (Webb, 2007).
27	a)	Agrawal e Srikant (1995).
28	d)	Srikant e Agrawal (1996).
29	d)	Valida o impacto real das regras.
30	a)	O FP-Growth foi proposto para contornar essa limitação (Han, Pei e Yin, 2000).

Discursivas (respostas-modelo)

1. Busca itens que ocorrem juntos nas transações. Aplicações: medicina (sintomas e diagnósticos), web (páginas visitadas na mesma sessão), bioinformática (genes expressos em conjunto), streaming (quem viu A também viu B).
2. Item: produto (pão). Transação: uma cesta ({pão, leite}). Itemset: qualquer conjunto de itens. Regra: {pão, manteiga} → {leite}, lida como "quem leva pão e manteiga costuma levar leite".

3. Indica coocorrência, não causalidade. Vendas de sorvete e afogamentos aumentam juntas no verão, mas o calor é a causa comum.
4. Suporte = 0,5; confiança = 0,5/0,8 = 0,625; lift = 0,625/0,7 ≈ 0,89, ou seja, associação ligeiramente negativa.
5. Fraldas e cerveja aparecem juntas 67% mais do que o esperado por acaso. O suporte de 20% indica que o padrão não é raro; é uma associação positiva relevante.
6. Se B é muito frequente, qualquer regra terminando em B terá confiança alta sem valor. O lift divide a confiança pelo suporte de B, comparando com a independência.
7. Se um itemset é infrequente, todos os seus superconjuntos também são. Exemplo: se {leite, manteiga} é infrequente, {pão, leite, manteiga} é descartado sem contagem.
8. Contar itens e manter os frequentes; gerar candidatos de tamanho k+1 e podar; contar e manter os frequentes; repetir até acabar; gerar regras dos itemsets frequentes e filtrar por confiança.
9. 1-itemsets: pão (0,8), leite (0,7), manteiga (0,6), café (0,5). 2-itemsets: {pão, leite} e {pão, manteiga} (0,5 cada). Não há 3-itemsets frequentes.
10. Duas leituras: conta itens e constrói a FP-tree, que comprime as transações; minera a árvore recursivamente com bases condicionais. Evita gerar e testar candidatos.
11. Apriori gera e testa candidatos, lê a base uma vez por nível, usa menos memória, mais lento. FP-Growth usa árvore, duas leituras, mais memória, em geral mais rápido.
12. Suporte mínimo realista; lift acima de um limiar; eliminar redundantes; descartar triviais e inexplicáveis; validar com especialistas e teste A/B.
13. Pedidos como transações; minerar regras com suporte baixo e lift alto; recomendar os consequentes das melhores regras; validar com teste A/B; reavaliar periodicamente.
14. Uma associação observada nos dados agregados pode desaparecer ou inverter-se ao segmentar por região, tipo de cliente etc.; por isso convém analisar por segmentos.
15. Aparecem em quase todas as transações, produzindo regras de alta confiança e lift próximo de 1, sem informação.
16. Com muitos testes, algumas regras aparecem por acaso; correções estatísticas (Webb, 2007) controlam falsos positivos.
17. Padrões sequenciais (a ordem importa, como celular → capinha → película) e regras com hierarquia de itens (refrigerante e refrigerante de cola), ou quantitativas.
18. Preferências, sazonalidade e produtos mudam com o tempo; regras antigas podem perder validade.
19. Lift é uma razão em relação à independência; alavancagem é a diferença absoluta entre o suporte observado e o esperado por independência.
20. Usa-se TransactionEncoder para criar colunas por item com valores verdadeiro/falso; é a entrada de apriori e fpgrowth.

Capítulo 11 — Detecção de Anomalias

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	A definição enfatiza a diferença de mecanismo gerador.
2	c)	Um único ponto muito diferente do resto.
3	d)	O valor só é estranho em certo contexto.
4	b)	O conjunto é anormal, mesmo que cada ponto não seja.
5	c)	Qualquer coisa muito diferente do normal é suspeita.
6	a)	O z-score mede o afastamento em desvios padrão.
7	b)	Outliers inflam o desvio e escondem a si mesmos.
8	b)	É robusto a valores extremos.
9	c)	É a regra do boxplot.
10	c)	Percebe que a combinação 1,90 m e 50 kg é incomum, mesmo que cada valor seja comum.
11	a)	Pontos anômalos são isolados com menos cortes.
12	a)	Caminho curto significa isolamento rápido.
13	a)	Escolhem-se um atributo e um valor de corte aleatórios.
14	c)	score_samples dá o escore contínuo (quanto menor, mais anômalo).
15	a)	Define o limiar de decisão com base na proporção esperada.
16	b)	Breunig et al. (2000).
17	a)	LOF muito maior que 1 sugere anomalia.
18	d)	A anomalia depende da densidade local.
19	c)	Schölkopf et al. (2001).
20	a)	Fica entre 0 e 1.
21	b)	O erro de reconstrução é alto para anomalias.
22	b)	Ele aprendeu a reconstruir apenas o padrão normal.
23	a)	Um modelo que sempre diz "normal" teria 99,9% de acurácia e seria inútil.
24	a)	Revocação é a fração das anomalias reais que foram pegas.
25	a)	Combina-se com a precisão no F1.
26	c)	Saito e Rehmsmeier (2015).
27	d)	No exemplo, 20 anomalias em 1020 dão cerca de 0,02.
28	b)	O limiar é uma decisão de negócio.
29	d)	Lundberg e Lee (2017).
30	d)	Pontos distantes de seus vizinhos recebem escores altos.

Discursivas (respostas-modelo)

1. Anomalia é um dado que destoa do padrão. Pontual: um único valor muito diferente (compra de R$ 50.000). Contextual: normal em um contexto e anormal em outro (30 °C em julho). Coletiva: conjunto anormal (batimentos que juntos revelam arritmia).
2. Supervisionado: exemplos rotulados de normal e anômalo (classificação desbalanceada). Semissupervisionado: só normais no treino. Não supervisionado: sem rótulos, supondo anomalias raras e diferentes.

3. z = (150 − 100)/20 = 2,5. Pela regra $|z| > 3$, não seria marcado como suspeito, embora mereça atenção.
4. O z-score usa média e desvio, afetados por outliers. O MAD usa a mediana e é robusto. O IQR usa quartis, também robusto e a base do boxplot.
5. Altura e peso são correlacionados; a Mahalanobis leva em conta a covariância e mede o afastamento na direção do ponto em unidades do desvio, revelando a combinação incomum.
6. Para achar uma pessoa comum numa multidão são necessárias muitas perguntas; para achar a única de smoking, basta uma. Pontos anômalos são isolados com poucos cortes.
7. Constroem-se muitas árvores de isolamento com cortes aleatórios em subamostras; mede-se a profundidade média de isolamento de cada ponto; profundidades pequenas geram escores maiores (mais anômalos).
8. Compara a densidade local de um ponto com a de seus vizinhos. Um ponto numa região esparsa cercada por vizinhos mais densos tem LOF alto; em região naturalmente esparsa, LOF ≈ 1.
9. Mapeia os dados via kernel para um espaço de maior dimensão e separa os dados da origem com margem máxima, criando uma fronteira flexível em volta da região normal.
10. IF: facilidade de isolar; escalável. LOF: densidade local; bom com densidades variáveis, depende de k. OCSVM: fronteira em espaço kernel; flexível, sensível a gamma e nu.
11. Treina-se com dados normais e usa-se o erro de reconstrução como escore; anomalias não seguem a estrutura aprendida e têm erro maior.
12. Fração esperada (contaminação); percentil dos escores; teoria de valores extremos; custo de falsos positivos e negativos; capacidade operacional da equipe.
13. Com 0,1% de fraude, dizer sempre "normal" dá 99,9% de acurácia. Usar precisão, revocação, F1, AUPRC e precisão@k.
14. O detector separa muito bem anomalias de normais; a AUPRC é muito superior ao acaso (0,02), confirmando alto desempenho mesmo com forte desbalanceamento.
15. Usar regras de negócio e verificação em segundo estágio, explicar os alertas, coletar feedback dos analistas e treinar depois um modelo supervisionado com os casos confirmados.
16. O que é normal muda com o tempo; um detector treinado no passado pode gerar alertas falsos ou perder anomalias novas.
17. Viés contra grupos minoritários, alertas injustos sobre pessoas, adversários que imitam o normal e uso indevido de dados pessoais.
18. GMM: baixa log-densidade (score_samples) indica anomalia. DBSCAN: pontos de ruído (rótulo −1) são candidatos a anomalias.
19. Ruído é variação aleatória sem interesse; anomalia pode ser um evento relevante (fraude, falha). Distingui-las evita descartar sinais importantes ou alertar sobre variações triviais.
20. Coletar e limpar séries; extrair atributos; padronizar; treinar Isolation Forest ou autoencoder com dados normais; definir limiar por capacidade; validar com especialistas; monitorar drift.

Capítulo 12 — Aprendizado Profundo Não Supervisionado

Múltipla escolha

Questão	Resposta	Justificativa
1	b)	saída = ativação($w_1x_1 + w_2x_2 + \ldots + b$).
2	b)	É simples e muito usada em redes profundas.
3	c)	Rumelhart, Hinton e Williams (1986).
4	d)	É repetida milhares de vezes.
5	c)	O alvo é a própria entrada; não há rótulos.
6	c)	Concentra os fatores essenciais dos dados.
7	a)	Baldi e Hornik (1989).
8	d)	O código tem dimensão menor que a entrada.
9	d)	Penaliza-se o número de neurônios ativos.
10	d)	Vincent et al. (2008; 2010).
11	b)	A rede só aprendeu a reconstruir o padrão normal.
12	c)	Respeitam a estrutura espacial (LeCun et al., 1998).
13	a)	O espaço latente fica contínuo e preenchido.
14	c)	A KL aproxima a distribuição de cada exemplo de uma normal padrão.
15	a)	Permite retropropagar o gradiente apesar do sorteio.
16	b)	Qualquer ponto do espaço latente corresponde a algo plausível.
17	c)	É o "falsificador".
18	a)	É o "policial".
19	b)	O gerador pode produzir pouca variedade de imagens.
20	b)	Ho, Jain e Abbeel (2020).
21	a)	Exemplo: prever palavras ocultas.
22	c)	Aprende representações de linguagem sem rótulos.
23	a)	Devem ter representações próximas.
24	b)	Ex.: NT-Xent ou InfoNCE.
25	a)	Usa lotes grandes para ter muitos negativos.
26	b)	Evita a necessidade de lotes gigantes.
27	b)	Possibilita busca de imagens por descrição.
28	b)	Usam uma rede aluna e uma professora.
29	d)	Em imagens 8×8, os pixels já eram informativos (ARI de 0,67 contra 0,58).
30	d)	perda = $(1/n) \cdot \Sigma \, \|x - \hat{x}\|^2$.

Discursivas (respostas-modelo)

1. Neurônio: soma ponderada das entradas mais viés e ativação. Camada: conjunto de neurônios; várias camadas formam uma rede profunda. Aprende por retropropagação e descida do gradiente, ajustando pesos para reduzir a perda.
2. Codificador comprime a entrada em um código; decodificador reconstrói. Como um resumo curto de telegrama que o receptor tenta expandir; se a reconstrução é boa, o resumo guardou o essencial.

3. O linear com perda quadrática aprende o mesmo subespaço do PCA; com ativações não lineares e várias camadas, captura estruturas curvas que o PCA não captura.
4. Subcompleto: código menor que a entrada. Esparso: código grande com poucas ativações. Denoising: entrada corrompida, alvo limpo, aprendendo estrutura robusta.
5. Treinar apenas com dados normais; calcular o erro de reconstrução; erros altos indicam anomalias; definir limiar adequado.
6. O codificador produz μ e σ; sorteia-se z; o decodificador reconstrói. O erro de reconstrução faz a saída parecer a entrada; a KL aproxima a distribuição latente de $N(0,1)$, tornando o espaço contínuo.
7. Escreve $z = \mu + \sigma\varepsilon$ com $\varepsilon \sim N(0,1)$, tirando a aleatoriedade do caminho dos parâmetros; permite retropropagar gradientes até μ e σ.
8. O gerador cria falsificações a partir de ruído; o discriminador diferencia real de falso; cada um melhora para vencer o outro, até o discriminador acertar cerca de 50%.
9. Instabilidade, colapso de modos e avaliação difícil; soluções: DCGAN, Wasserstein GAN, StyleGAN e métricas como FID.
10. Processo direto adiciona ruído em muitos passos até restar ruído; uma rede aprende a remover um pouco de ruído por passo; para gerar, parte-se de ruído e aplica-se a rede repetidamente.
11. O rótulo é criado a partir dos dados. Exemplos: prever palavras ocultas, prever a rotação de uma imagem, reconstruir regiões mascaradas, resolver quebra-cabeças, colorir imagens.
12. Duas visões da mesma imagem formam par positivo (aproximar) e imagens diferentes formam pares negativos (afastar). A perda usa similaridade do cosseno com temperatura e compara o positivo com todos os negativos do lote.
13. SimCLR: negativos do lote. MoCo: fila de negativos. CLIP: alinha imagens e textos. BYOL/DINO: sem negativos explícitos, com rede aluna e professora.
14. Pré-treina-se em muitos dados sem rótulo; depois reaproveita-se a rede com ajuste fino ou avaliação linear usando poucos rótulos.
15. Agrupar com K-means nos códigos; visualizar com t-SNE ou UMAP; buscar por vizinhos mais próximos no espaço; detectar anomalias por distância; transferir para classificadores.
16. Que o aprendizado profundo nem sempre supera uma linha de base simples, sobretudo em dados pequenos e simples; é preciso comparar com alternativas.
17. Reproduzem vieses dos dados; deepfakes podem enganar; interpretabilidade limitada do espaço latente; alto consumo de energia (Strubell et al., 2019).
18. O autoencoder comum pode ter buracos no espaço latente e decodificar lixo fora dos códigos vistos; o VAE regulariza o espaço, permitindo amostrar e interpolar.
19. Agrupar: K-means, hierárquico, DBSCAN, GMM e espectral; reduzir: PCA, ICA, NMF, Isomap, t-SNE, UMAP e autoencoders; associar: Apriori e FP-Growth; detectar anomalias: IF, LOF, OCSVM e autoencoders. Os métodos profundos aprendem as representações.
20. Não há rótulos humanos: a própria entrada serve de resposta que a rede tenta reproduzir. O "rótulo" é criado automaticamente, característica da auto-supervisão no sentido mais simples.

Capítulo 13 — Projeto Final

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	Primeiro a decisão; depois as ferramentas.
2	a)	O algoritmo é um meio, e não parte da pergunta.
3	a)	Usar rótulos para escolher o método é uma forma de vazamento.
4	a)	Outras pessoas conseguem repetir seus números.
5	a)	O escalonador deve ser ajustado só com o treino.
6	d)	O PCA resume os dados em poucas componentes.
7	a)	Assim a escolha não depende de rótulos.
8	a)	Teve a estabilidade mais alta entre os métodos testados.
9	d)	Ele deixa de agrupar pontos difíceis, mantendo grupos mais coesos.
10	d)	Valores próximos de 1 indicam estrutura agrupável.
11	a)	Preservou melhor as vizinhanças locais.
12	a)	Só a vizinhança local é confiável.
13	d)	O ARI do autoencoder foi menor que o do PCA nesta execução.
14	d)	Métodos sofisticados nem sempre superam os simples.
15	b)	Os postos tornam escores de escalas diferentes comparáveis.
16	c)	Postos colocam todos na mesma escala.
17	d)	Lift acima de 1 indica associação positiva; não implica causalidade.
18	b)	As bordas das imagens 8×8 são quase sempre vazias.
19	a)	Isso multiplica o efeito de cada rótulo.
20	d)	Cada rótulo informa dezenas de imagens parecidas.
21	d)	Se um grupo mistura dígitos, o rótulo propagado erra para parte deles.
22	a)	Quem lê só o início já deve saber o que fazer.
23	c)	Erros de leitura afetam pessoas.
24	a)	O detector de anomalias ajuda a encaminhar casos à revisão.
25	a)	A transparência sobre o uso de rótulos é essencial.
26	a)	É um dos critérios de maior peso.
27	a)	Com o tempo, o "normal" muda e os modelos podem perder validade.
28	a)	Versões diferentes podem gerar resultados diferentes.
29	d)	Reportar a variação é mais honesto.
30	a)	Comparar conjuntos ajuda a avaliar a generalidade das conclusões.

Discursivas (respostas-modelo)

1. A pergunta define a decisão a ser apoiada, os critérios de sucesso e os riscos. Algoritmos são meios; escolhê-los antes leva a respostas tecnicamente bonitas, mas inúteis para a decisão.
2. Quem decide, o que será decidido, como saber que deu certo e quais os riscos de errar. Exemplo: gerente de loja (quem), reorganizar prateleiras (o quê), aumento das vendas conjuntas em teste A/B (critério), perda de vendas em itens deslocados (risco).

3. Fixar sementes (resultados não repetíveis); registrar versões (diferenças entre ambientes); separar etapas em funções (código confuso); ajustar transformações só no treino (vazamento); registrar decisões (escolhas sem justificativa).
4. Mesmos dados e pré-processamento; critérios definidos antes; rótulos só no final; várias sementes ou reamostragens; linha de base simples.
5. Na prática, os rótulos não existem; escolher o método olhando o ARI usa informação indisponível e superestima o desempenho. Deve-se escolher por critérios internos, estabilidade e interpretação, e conferir com rótulos no final.
6. Não necessariamente: seu ARI foi calculado só nos pontos não ruidosos (cerca de metade). Ele troca cobertura por pureza. Para decidir, é preciso considerar o custo de deixar quase metade das imagens sem grupo.
7. Em imagens pequenas e simples, o PCA já captura bem a estrutura, e o autoencoder precisa de ajustes (arquitetura, treino). Ensina a comparar com linhas de base simples antes de adotar métodos complexos.
8. Calcula-se o escore de cada detector (Isolation Forest e LOF), converte-se cada um em postos e tira-se a média dos postos. Isso evita que escalas diferentes dominem e aproveita o acordo entre detectores.
9. Quando o bloco 21 está ativo e os blocos 11 e 32 estão vazios, em 84% dos casos o dígito é 7; isso é 8,4 vezes mais que o esperado ao acaso. É coocorrência de traços, não causa.
10. Agrupa-se em 30 grupos, rotula-se o representante de cada grupo e propaga-se o rótulo aos membros. Como cada rótulo cobre dezenas de imagens parecidas, a acurácia é maior do que com 30 imagens sorteadas isoladamente.
11. Grupos que misturam classes propagam rótulos errados; o representante pode ser atípico; erros se concentram em classes parecidas. É preciso revisar amostras e medir o erro por classe.
12. Título e resumo executivo, dados, método, resultados, limitações, riscos éticos e operacionais, recomendação e apêndice técnico.
13. Só no fim se sabe o resultado completo; mas leitores ocupados leem o início primeiro e precisam de pergunta, resultado e recomendação.
14. Erro que move dinheiro (revisão humana); viés contra estilos de escrita (avaliar por subgrupos); privacidade (minimizar e proteger dados); falta de transparência (explicar e permitir contestação).
15. Pessoas revisam casos duvidosos. No projeto, o detector de anomalias seleciona as imagens estranhas para revisão humana.
16. Monitorar a distribuição dos atributos e dos escores de anomalia ao longo do tempo, a taxa de alertas e o desempenho em uma pequena amostra rotulada periodicamente; disparar revisão quando houver mudança.
17. Resposta pessoal coerente: por exemplo, em imagens de roupas as classes são mais variadas e o autoencoder ou o aprendizado contrastivo podem superar o PCA com mais dados; o t-SNE ainda deve separar bem os grupos; o número de regras de traços deve diminuir.
18. Resposta pessoal fundamentada. Exemplos de respostas boas: a definição da pergunta e dos critérios (guia todo o resto), a avaliação sem vazamento de rótulos (evita conclusões otimistas) ou a revisão humana (reduz danos).
19. A rotulagem manual completa é cara, mas dá o melhor gabarito; agrupar e rotular reduz muito o custo, mas depende da qualidade dos grupos e propaga erros. Uma estratégia mista, com revisão por amostragem, equilibra custo e qualidade.
20. Resposta pessoal coerente com: definir a pergunta primeiro; preparar bem os dados e escalar; escolher a distância certa; testar vários métodos; avaliar com critérios internos, estabilidade e interpretação; usar rótulos só no final; comparar com linhas de base; visualizar com cuidado; pensar nos riscos éticos; documentar tudo.

Apêndice B — Glossário

Termos técnicos usados no livro, explicados de forma simples.

Agrupamento (clustering) — tarefa de dividir exemplos em grupos de modo que os de um mesmo grupo sejam parecidos entre si e diferentes dos demais.

Agrupamento rígido — cada exemplo pertence a um único grupo.

Agrupamento suave — cada exemplo tem uma probabilidade ou grau de pertinência a cada grupo.

AIC / BIC — critérios de informação que equilibram ajuste e complexidade de um modelo; menor é melhor.

Alavancagem (leverage) — diferença entre o suporte observado de A e B juntos e o esperado se fossem independentes.

Anomalia (outlier) — exemplo muito diferente do comportamento normal dos dados.

Antecedente / consequente — lado esquerdo (A) e lado direito (B) de uma regra A → B.

Apriori — algoritmo de regras de associação que gera candidatos e os poda com a propriedade de anti-monotonicidade do suporte.

ARI (índice de Rand ajustado) — medida de concordância entre dois agrupamentos corrigida pelo acaso; 1 indica concordância perfeita.

Atributo (feature) — característica que descreve um exemplo; uma coluna da tabela de dados.

Autoencoder — rede neural que comprime a entrada em um código e tenta reconstruí-la.

Autovalor e autovetor — um autovetor é uma direção que uma matriz apenas estica; o autovalor é o fator de esticamento.

Biplot — gráfico que mostra, juntos, exemplos (pontos) e atributos (setas) de uma análise de componentes principais.

Centroide — ponto que representa o centro de um grupo, em geral a média de seus pontos.

Chebyshev (distância de) — maior diferença absoluta entre os atributos de dois pontos.

Confiança — probabilidade de B aparecer dado que A apareceu; suporte(A ∪ B) / suporte(A).

Convicção (conviction) — medida que compara a frequência esperada de A sem B com a observada.

Correlação cofenética — correlação entre as distâncias originais e as distâncias implícitas em um dendrograma.

Cosseno (similaridade do) — medida do ângulo entre dois vetores; ignora seu tamanho.

Covariância — medida de quanto dois atributos variam juntos.

DBSCAN — algoritmo de agrupamento por densidade com os parâmetros eps e min_samples.

Dendrograma — gráfico em árvore que registra as fusões do agrupamento hierárquico.

Descida do gradiente — método que ajusta parâmetros aos poucos para reduzir uma função de perda.

Dimensionalidade — número de atributos dos dados.

Distância de Gower — distância para dados mistos, que combina diferenças parciais por atributo.

Distância de Mahalanobis — distância que leva em conta a covariância entre atributos.

Distância de Manhattan — soma das diferenças absolutas entre atributos (distância do quarteirão).

Distância geodésica — distância medida ao longo de uma superfície (variedade).

Eigengap — maior salto entre autovalores consecutivos da Laplaciana, usado para escolher k no agrupamento espectral.

EM (Esperança-Maximização) — algoritmo que alterna um passo de cálculo de responsabilidades (E) e de atualização de parâmetros (M).

Escore (score) — coordenada de um exemplo no espaço das componentes principais; também, valor numérico de anomalia.

Espaço latente — espaço de baixa dimensão formado pelos códigos aprendidos por um modelo.

Estatística gap — método para escolher k comparando a inércia observada com a de dados sem estrutura.

Euclidiana (distância) — distância em linha reta entre dois pontos.

Exemplo (instância, amostra) — uma linha da tabela de dados.

FP-Growth — algoritmo de itemsets frequentes que usa uma árvore de prefixos e não gera candidatos.

GAN — rede adversarial generativa, formada por um gerador e um discriminador que competem.

GMM (modelo de mistura gaussiana) — modelo que descreve os dados como soma ponderada de distribuições normais.

Inércia — soma dos quadrados das distâncias dos pontos ao centro de seus grupos; função objetivo do K-means.

Isolation Forest — detector de anomalias que mede a facilidade de isolar cada ponto com cortes aleatórios.

Itemset — conjunto de itens que ocorrem juntos.

K-means — algoritmo que divide os dados em k grupos representados por suas médias.

K-medoids — variante do K-means que usa um ponto real do conjunto como centro.

Kernel — função que mede similaridade e permite trabalhar implicitamente em um espaço de maior dimensão.

Laplaciana de um grafo — matriz $L = D - W$ cujos autovetores revelam a estrutura de comunidades do grafo.

Lift — razão entre a confiança de uma regra e o suporte do consequente; acima de 1 indica associação positiva.

LOF (Local Outlier Factor) — escore de anomalia que compara a densidade local de um ponto com a de seus vizinhos.

Maldição da dimensionalidade — conjunto de dificuldades que surgem quando há muitos atributos, como o espaço vazio e a concentração de distâncias.

MDS (escalonamento multidimensional) — técnica que reconstrói um mapa a partir de uma tabela de distâncias.

Métrica — distância que respeita não negatividade, identidade, simetria e desigualdade triangular.

NMF (fatoração de matrizes não negativas) — decomposição de uma matriz não negativa em duas matrizes não negativas.

NMI (informação mútua normalizada) — medida, entre 0 e 1, da informação compartilhada por dois agrupamentos.

One-Class SVM — detector de anomalias que aprende a fronteira em torno dos dados normais.

One-hot — codificação de categorias em colunas binárias, uma por categoria.

Padronização (z-score) — transformação que subtrai a média e divide pelo desvio padrão.

PCA (análise de componentes principais) — técnica linear que encontra as direções de máxima variância.

Perplexidade — parâmetro do t-SNE que equivale, aproximadamente, ao número de vizinhos considerados por ponto.

Pipeline — sequência encadeada de etapas de pré-processamento e modelo.

Rótulo (label) — resposta correta associada a um exemplo, usada no aprendizado supervisionado.

Silhueta — medida, entre −1 e 1, de quão bem cada ponto se encaixa em seu grupo.

Suporte — fração das transações que contêm um itemset.

SVD (decomposição em valores singulares) — fatoração $X = U \cdot S \cdot V^T$ usada para calcular o PCA.

t-SNE — técnica de visualização que preserva vizinhanças locais por meio de probabilidades.

Transação — conjunto de itens que ocorrem juntos, como uma compra.

UMAP — técnica de redução de dimensionalidade baseada em grafos difusos de vizinhos.

VAE (autoencoder variacional) — autoencoder probabilístico cujo espaço latente é regularizado para ser contínuo.

Variância explicada — fração da variância total dos dados capturada por uma componente principal.

Variedade (manifold) — superfície que, localmente, parece plana, mesmo sendo curva em escala global.

Apêndice C — Guia rápido de algoritmos e códigos

C.1 Mapa de decisão

Se você quer...	Considere	Capítulo
Agrupar grandes bases com grupos esféricos	K-means, Mini-Batch K-means	3
Obter uma hierarquia de grupos	Aglomerativo (Ward, média)	4
Achar grupos de forma livre e ruído	DBSCAN, HDBSCAN, OPTICS	5
Grupos elípticos e probabilidades	Mistura gaussiana (GMM)	6
Separar formas complexas em bases médias	Agrupamento espectral	6
Avaliar agrupamentos	Silhueta, CH, DB, gap, ARI, NMI	7
Resumir e visualizar rapidamente	PCA	8
Separar fontes misturadas	ICA	8
Partes aditivas e tópicos	NMF	8
Mapas 2D de estruturas curvas	t-SNE, UMAP, Isomap	9
Itens que ocorrem juntos	Apriori, FP-Growth	10
Encontrar casos raros	Isolation Forest, LOF, One-Class SVM	11
Aprender representações	Autoencoder, VAE, contrastivo	12

C.2 Classes do scikit-learn e parâmetros principais

Técnica	Classe ou função	Parâmetros-chave
K-means	sklearn.cluster.KMeans	n_clusters, n_init, init
Mini-Batch K-means	sklearn.cluster.MiniBatchKMeans	n_clusters, batch_size
Hierárquico	sklearn.cluster.AgglomerativeClustering	n_clusters, linkage, distance_threshold
DBSCAN	sklearn.cluster.DBSCAN	eps, min_samples, metric
OPTICS	sklearn.cluster.OPTICS	min_samples, xi, min_cluster_size
HDBSCAN	sklearn.cluster.HDBSCAN	min_cluster_size, min_samples
GMM	sklearn.mixture.GaussianMixture	n_components, covariance_type, n_init
Espectral	sklearn.cluster.SpectralClustering	n_clusters, affinity, n_neighbors
Silhueta	sklearn.metrics.silhouette_score	X, rótulos
ARI e NMI	sklearn.metrics.adjusted_rand_score, normalized_mutual_info_score	rótulos de referência e previstos
PCA	sklearn.decomposition.PCA	n_components, svd_solver
ICA	sklearn.decomposition.FastICA	n_components
NMF	sklearn.decomposition.NMF	n_components, init
Kernel PCA	sklearn.decomposition.KernelPCA	kernel, gamma
MDS	sklearn.manifold.MDS	n_components, dissimilarity

Técnica	Classe ou função	Parâmetros-chave
Isomap	sklearn.manifold.Isomap	n_neighbors, n_components
t-SNE	sklearn.manifold.TSNE	perplexity, init, learning_rate
UMAP	umap.UMAP	n_neighbors, min_dist, metric
Isolation Forest	sklearn.ensemble.IsolationForest	n_estimators, contamination
LOF	sklearn.neighbors.LocalOutlierFactor	n_neighbors, contamination, novelty
One-Class SVM	sklearn.svm.OneClassSVM	kernel, gamma, nu
Apriori e FP-Growth	mlxtend.frequent_patterns.apriori, fpgrowth	min_support
Regras	mlxtend.frequent_patterns.association_rules	metric, min_threshold

C.3 Modelo de código para um projeto completo

PYTHON

```python
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1) Dados (aqui, um exemplo embutido)
from sklearn.datasets import load_wine
X = load_wine(as_frame=True).data

# 2) Pré-processar
pre = Pipeline([("imputar", SimpleImputer(strategy="median")),
                ("escalar", StandardScaler())])
Xp = pre.fit_transform(X)

# 3) Testar vários k e avaliar
for k in range(2, 8):
    r = KMeans(n_clusters=k, n_init=10, random_state=0).fit_predict(Xp)
    print(k, round(silhouette_score(Xp, r), 3))

# 4) Escolher k, treinar e perfilar os grupos
rotulos = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(Xp)
print(X.assign(grupo=rotulos).groupby("grupo").mean().round(1).T)
```

C.4 Lista de verificação final

MEMORIZE **Antes de apresentar seus resultados**

- Os dados foram limpos, e os atributos padronizados quando necessário?
- Existe tendência de agrupamento (Hopkins)?
- Testei mais de um valor de k ou de parâmetros, e mais de um algoritmo?
- Os grupos são estáveis em reamostragens?
- Os grupos têm perfil claro, nome e utilidade para decisão?
- Considerei riscos éticos, privacidade e viés?
- Documentei sementes, versões e escolhas?

Apêndice D — Soluções dos exercícios práticos

Cada solução traz o código, a **saída obtida** na elaboração do livro e um comentário sobre o que observar. Há várias maneiras corretas de resolver um exercício: o importante é que o seu resultado seja coerente e que você consiga explicá-lo. Pequenas diferenças numéricas podem ocorrer com outras versões das bibliotecas. Os gráficos aparecem quando você executa o código.

Capítulo 1 — Bem-vindo ao Aprendizado Não Supervisionado

Exercício 1 — Primeiro contato com o conjunto Wine

PYTHON

```python
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.cluster import KMeans

dados = load_wine()
X, y = dados.data, dados.target
print("Formato de X:", X.shape)
print("Atributos:", list(dados.feature_names)[:5], "...")
print("Exemplos por classe:", np.bincount(y))

grupos = KMeans(n_clusters=3, n_init=10, random_state=0).fit_predict(X)
print(pd.crosstab(grupos, y, rownames=["grupo"], colnames=["classe"]))
```

SAÍDA

```
Formato de X: (178, 13)
Atributos: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium'] ...
Exemplos por classe: [59 71 48]
classe   0   1   2
grupo
0        0  50  19
1       46   1   0
2       13  20  29
```

EXEMPLO **O que observar**

Sem padronizar, o atributo "proline" (valores de centenas a mil) domina a distância. Por isso os grupos se misturam com as classes. Você corrigirá isso no próximo exercício.

Exercício 2 — O efeito da escala

PYTHON

```python
from sklearn.datasets import load_wine
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score

X, y = load_wine(return_X_y=True)
bruto = KMeans(3, n_init=10, random_state=0).fit_predict(X)
Xp = StandardScaler().fit_transform(X)
padr = KMeans(3, n_init=10, random_state=0).fit_predict(Xp)

print("ARI sem padronizar:", round(adjusted_rand_score(y, bruto), 3))
print("ARI padronizado   :", round(adjusted_rand_score(y, padr), 3))
print("Desvio padrão de alcohol e proline:", X.std(axis=0).round(1)[[0, 12]])
```

SAÍDA

```
ARI sem padronizar: 0.371
ARI padronizado   : 0.897
Desvio padrão de alcohol e proline: [  0.8 314. ]
```

EXEMPLO **O que observar**

O ARI sobe de cerca de 0,37 para cerca de 0,90 depois da padronização. O atributo "proline" (desvio padrão acima de 300, contra menos de 1 do álcool) dominava as distâncias.

Exercício 3 — Enxergando os grupos com PCA

PYTHON

```python
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X, y = load_wine(return_X_y=True)
Xp = StandardScaler().fit_transform(X)
grupos = KMeans(3, n_init=10, random_state=0).fit_predict(Xp)

pca = PCA(n_components=2).fit(Xp)
Z = pca.transform(Xp)
print(
    "Variância explicada:",
    pca.explained_variance_ratio_.round(3),
    "| soma:",
    pca.explained_variance_ratio_.sum().round(3),
)

fig, ax = plt.subplots(1, 2, figsize=(9, 4))
ax[0].scatter(Z[:, 0], Z[:, 1], c=grupos, cmap="viridis", s=25)
ax[0].set_title("Grupos do K-means")
ax[1].scatter(Z[:, 0], Z[:, 1], c=y, cmap="viridis", s=25)
ax[1].set_title("Classes reais")
plt.show()
```

SAÍDA

```
Variância explicada: [0.362 0.192] | soma: 0.554
```

EXEMPLO **O que observar**

As duas componentes retêm cerca de 55% da variância. Mesmo assim, os grupos são bem visíveis e muito parecidos com as classes.

Exercício 4 — A sorte da inicialização

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
```

```python
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(load_wine().data)
ines = [KMeans(3, n_init=1, random_state=s).fit(X).inertia_ for s in range(30)]
print("Inércia mínima:", round(min(ines), 1), "| máxima:", round(max(ines), 1))
print("Soluções distintas:", len(set(np.round(ines, 1))))
print("Com n_init=10   :", round(KMeans(3, n_init=10, random_state=0).fit(X).inertia_, 1))
```

SAÍDA

```
Inércia mínima: 1277.9 | máxima: 1282.5
Soluções distintas: 5
Com n_init=10   : 1277.9
```

EXEMPLO O que observar

Aparecem várias soluções distintas (mínimos locais). Neste conjunto a diferença é pequena, mas em outros dados pode ser grande. Com `n_init=10`, escolhe-se a melhor entre dez tentativas, o que reduz esse risco.

Exercício 5 — O algoritmo sempre responde

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

rng = np.random.default_rng(0)
aleatorio = rng.random((178, 13))
Xw = StandardScaler().fit_transform(load_wine().data)

for nome, X in [("Aleatório", aleatorio), ("Wine", Xw)]:
    g = KMeans(3, n_init=10, random_state=0).fit_predict(X)
    print(
        f"{nome:10s} tamanhos dos grupos: {np.bincount(g)}  silhueta: {silhouette_score(X, g):.3f}"
    )
```

SAÍDA

```
Aleatório  tamanhos dos grupos: [55 69 54]  silhueta: 0.078
Wine       tamanhos dos grupos: [65 51 62]  silhueta: 0.285
```

EXEMPLO O que observar

O K-means sempre devolve k grupos. A silhueta dos dados aleatórios é bem menor que a do Wine, o que ajuda a perceber a ausência de estrutura. Esse tema volta no Capítulo 7.

Capítulo 2 — Dados, Distâncias e Pré-processamento

Exercício 1 — Matriz de distâncias entre clientes

PYTHON

```python
import pandas as pd
import numpy as np
from sklearn.metrics import pairwise_distances

df = pd.DataFrame(
    {"idade": [25, 62, 31, 58], "gasto": [120, 340, 95, 310], "visitas": [8, 12, 4, 10]},
    index=["Ana", "Bruno", "Carla", "Diego"],
)

for metrica in ["euclidean", "manhattan", "cosine"]:
    D = pd.DataFrame(pairwise_distances(df, metric=metrica), index=df.index, columns=df.index)
    M = D.where(~np.eye(len(D), dtype=bool))  # ignora a diagonal (zeros)
    i, j = np.unravel_index(np.nanargmin(M.values), M.shape)
    print(
        f"{metrica:10s} par mais próximo: {df.index[i]} e {df.index[j]}  (distância = {M.values[i,
j]:.3f})"
    )
```

SAÍDA

```
euclidean  par mais próximo: Ana e Carla  (distância = 26.019)
manhattan  par mais próximo: Ana e Carla  (distância = 35.000)
cosine     par mais próximo: Bruno e Diego  (distância = 0.000)
```

EXEMPLO **O que observar**

Nas distâncias Euclidiana e de Manhattan, o par mais próximo é Ana e Carla (poucas diferenças absolutas). No cosseno, é Bruno e Diego, cujos vetores têm quase a mesma direção. A escolha da métrica muda quem é "parecido" com quem. (Os dados aqui não foram padronizados; a escala também pesa.)

Exercício 2 — Três escalonadores diante de um outlier

PYTHON

```python
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

x = np.array([10, 12, 11, 13, 12, 11, 10, 500], dtype=float).reshape(-1, 1)
for nome, esc in [
    ("Padronização", StandardScaler()),
    ("Min-Max", MinMaxScaler()),
    ("Robusto", RobustScaler()),
]:
    z = esc.fit_transform(x).ravel()
    print(f"{nome:13s} 7 primeiros: {np.round(z[:7], 2)}  | outlier: {z[7]:.2f}")
```

SAÍDA

```
Padronização  7 primeiros: [-0.39 -0.37 -0.38 -0.37 -0.37 -0.38 -0.39]  | outlier: 2.65
```

```
Min-Max       7 primeiros: [0.   0.   0.   0.01 0.   0.   0.  ]  | outlier: 1.00
Robusto       7 primeiros: [-1.    0.33 -0.33  1.    0.33 -0.33 -1.  ]  | outlier: 325.67
```

EXEMPLO O que observar

Padronização e Min-Max colocam os sete pontos normais quase no mesmo lugar (diferenças mínimas), pois o outlier domina média, desvio, mínimo e máximo. O escalonador robusto mantém os pontos normais bem espalhados.

Exercício 3 — Imputar: mediana ou vizinhos?

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer, KNNImputer

X = StandardScaler().fit_transform(load_wine().data)
rng = np.random.default_rng(0)
mascara = rng.random(X.shape) < 0.10
Xn = X.copy()
Xn[mascara] = np.nan
print("Valores apagados:", int(mascara.sum()))

for nome, imp in [
    ("Mediana", SimpleImputer(strategy="median")),
    ("KNN (k=5)", KNNImputer(n_neighbors=5)),
]:
    Xi = imp.fit_transform(Xn)
    rmse = np.sqrt(np.mean((Xi[mascara] - X[mascara]) ** 2))
    print(f"{nome:10s} RMSE = {rmse:.3f}")
```

SAÍDA

```
Valores apagados: 249
Mediana    RMSE = 1.082
KNN (k=5)  RMSE = 0.757
```

EXEMPLO O que observar

O KNN costuma errar menos, porque usa os exemplos mais parecidos, enquanto a mediana ignora as relações entre atributos.

Exercício 4 — Codificando variáveis categóricas

PYTHON

```python
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler

df = pd.DataFrame(
    {
```

```
        "cidade": ["SP", "RJ", "BH", "SP", "RJ"],
        "plano": ["básico", "premium", "médio", "médio", "básico"],
        "gasto": [120.0, 340.0, 210.0, 190.0, 95.0],
    }
)

ct = ColumnTransformer(
    [
        ("cidade", OneHotEncoder(), ["cidade"]),
        ("plano", OrdinalEncoder(categories=[["básico", "médio", "premium"]]), ["plano"]),
        ("gasto", StandardScaler(), ["gasto"]),
    ]
)
M = ct.fit_transform(df)
print(ct.get_feature_names_out())
print(M.round(2))
```

SAÍDA

```
['cidade__cidade_BH' 'cidade__cidade_RJ' 'cidade__cidade_SP'
 'plano__plano' 'gasto__gasto']
[[ 0.    0.    1.    0.   -0.83]
 [ 0.    1.    0.    2.    1.74]
 [ 1.    0.    0.    1.    0.22]
 [ 0.    0.    1.    1.   -0.01]
 [ 0.    1.    0.    0.   -1.12]]
```

EXEMPLO **O que observar**

A cidade vira três colunas binárias (sem ordem). O plano vira 0, 1 ou 2 (com ordem). O gasto fica com média 0 e desvio padrão 1.

Exercício 5 — A maldição da dimensionalidade em números

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.metrics import pairwise_distances

rng = np.random.default_rng(0)
linhas = []
for d in [2, 10, 100, 1000]:
    X = rng.random((500, d))
    linha = {"d": d}
    for metrica in ["euclidean", "manhattan"]:
        dist = pairwise_distances(X[:1], X[1:], metric=metrica).ravel()
        linha[metrica] = (dist.max() - dist.min()) / dist.min()
    linhas.append(linha)
print(pd.DataFrame(linhas).round(3).to_string(index=False))
```

SAÍDA

```
   d  euclidean  manhattan
   2     66.130     73.751
  10      2.316      3.158
 100      0.363      0.445
```

```
1000      0.108      0.142
```

EXEMPLO O que observar

O contraste cai rapidamente nas duas métricas. A de Manhattan mantém, em geral, um contraste um pouco maior que a Euclidiana, o que concorda com Aggarwal, Hinneburg e Keim (2001).

Capítulo 3 — K-means

Exercício 1 — K-means "do zero" com NumPy

PYTHON

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(300, centers=3, cluster_std=1.0, random_state=7)
rng = np.random.default_rng(0)
C = X[rng.choice(len(X), 3, replace=False)]  # centros iniciais

for _ in range(20):
    D = ((X[:, None, :] - C[None, :, :]) ** 2).sum(axis=2)  # distâncias ao quadrado
    rot = D.argmin(axis=1)  # atribuição
    C = np.array(
        [X[rot == k].mean(axis=0) if np.any(rot == k) else C[k] for k in range(3)]
    )  # atualização

inercia = ((X - C[rot]) ** 2).sum()
sk = KMeans(3, init="random", n_init=1, random_state=0).fit(X)
print("Inércia (NumPy)       :", round(inercia, 1))
print("Inércia (scikit-learn):", round(sk.inertia_, 1))
```

SAÍDA

```
Inércia (NumPy)       : 573.4
Inércia (scikit-learn): 573.4
```

EXEMPLO **O que observar**

Os valores ficam iguais ou muito próximos. Se forem diferentes, a causa é a inicialização (mínimo local). Rode com outra semente e veja.

Exercício 2 — Escolhendo k com inércia e silhueta

PYTHON

```python
import pandas as pd
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X, _ = make_blobs(500, centers=5, cluster_std=0.9, random_state=3)
linhas = []
for k in range(2, 10):
    km = KMeans(k, n_init=10, random_state=0).fit(X)
    linhas.append({"k": k, "inercia": km.inertia_, "silhueta": silhouette_score(X, km.labels_)})
tab = pd.DataFrame(linhas)
print(tab.round(3).to_string(index=False))
print("k de maior silhueta:", int(tab.loc[tab.silhueta.idxmax(), "k"]))
```

SAÍDA

```
 k  inercia  silhueta
 2 7244.159     0.653
 3 3873.697     0.588
 4 2017.481     0.619
 5  815.308     0.703
 6  739.237     0.635
 7  672.670     0.571
 8  612.019     0.481
 9  558.299     0.403
k de maior silhueta: 5
```

EXEMPLO O que observar

A inércia cai sempre; a silhueta tem um máximo claro em k = 5 se os grupos estão bem separados. Se houver grupos sobrepostos, os dois critérios podem discordar.

Exercício 3 — Aleatória ou k-means++?

PYTHON

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(1000, centers=8, random_state=1)
res = {}
for init in ["random", "k-means++"]:
    res[init] = np.array(
        [KMeans(8, init=init, n_init=1, random_state=s).fit(X).inertia_ for s in range(100)]
    )
melhor = min(r.min() for r in res.values())
for init, r in res.items():
    print(
        f"{init:10s} média = {r.mean():8.1f}  desvio = {r.std():7.1f}  acertou o melhor = {(r <= melhor
* 1.01).mean():.0%}"
    )
```

SAÍDA

```
random     média =   2123.3  desvio =   632.3  acertou o melhor = 23%
k-means++  média =   1758.2  desvio =    38.4  acertou o melhor = 40%
```

EXEMPLO O que observar

O k-means++ costuma ter média menor, menos variação e mais execuções boas, como previsto por Arthur e Vassilvitskii (2007).

Exercício 4 — Mini-Batch K-means em dados grandes

PYTHON

```python
import time
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans, MiniBatchKMeans
```

```
X, _ = make_blobs(200_000, centers=6, n_features=10, random_state=0)
for nome, modelo in [
    ("KMeans", KMeans(6, n_init=3, random_state=0)),
    ("MiniBatchKMeans", MiniBatchKMeans(6, batch_size=2048, n_init=3, random_state=0)),
]:
    t0 = time.perf_counter()
    modelo.fit(X)
    dt = time.perf_counter() - t0
    print(f"{nome:16s} tempo = {dt:5.2f} s   inércia = {modelo.inertia_:,.0f}")
```

SAÍDA

```
KMeans           tempo =  0.39 s   inércia = 1,997,537
MiniBatchKMeans  tempo =  0.26 s   inércia = 1,997,641
```

EXEMPLO O que observar

O Mini-Batch costuma ser bem mais rápido e ter inércia apenas ligeiramente maior. Os tempos exatos dependem do seu computador.

Exercício 5 — Quantização de cores de uma foto

PYTHON

```python
import numpy as np
from sklearn.datasets import load_sample_images
from sklearn.cluster import KMeans

img = load_sample_images().images[0] / 255.0  # (altura, largura, 3)
h, w, _ = img.shape
px = img.reshape(-1, 3)
rng = np.random.default_rng(0)
amostra = px[rng.choice(len(px), 10_000, replace=False)]

for k in [8, 16, 32]:
    km = KMeans(k, n_init=3, random_state=0).fit(amostra)
    q = km.cluster_centers_[km.predict(px)].reshape(h, w, 3)
    mse = np.mean((img - q) ** 2)
    print(f"k = {k:2d}  MSE = {mse:.5f}  compressão ≈ {24 / np.log2(k):.1f}x")
```

SAÍDA

```
k =  8  MSE = 0.00326  compressão ≈ 8.0x
k = 16  MSE = 0.00177  compressão ≈ 6.0x
k = 32  MSE = 0.00100  compressão ≈ 4.8x
```

EXEMPLO O que observar

Mais cores significam menor erro e menos compressão. A amostragem de pixels torna o ajuste rápido sem perda visível.

Capítulo 4 — Agrupamento Hierárquico

Exercício 1 — O dendrograma à mão, conferido pelo computador

PYTHON

```python
import numpy as np
from scipy.cluster.hierarchy import linkage, fcluster

pts = np.array([1, 2, 6, 7, 15], dtype=float).reshape(-1, 1)
Z = linkage(pts, method="single")
print("Matriz de ligações (grupo A, grupo B, altura, tamanho):")
print(Z)
print("Alturas das fusões:", Z[:, 2])
print("Rótulos com 2 grupos:", fcluster(Z, t=2, criterion="maxclust"))
```

SAÍDA

```
Matriz de ligações (grupo A, grupo B, altura, tamanho):
[[0. 1. 1. 2.]
 [2. 3. 1. 2.]
 [5. 6. 4. 4.]
 [4. 7. 8. 5.]]
Alturas das fusões: [1. 1. 4. 8.]
Rótulos com 2 grupos: [1 1 1 1 2]
```

EXEMPLO **O que observar**

As alturas são 1, 1, 4 e 8, como no cálculo manual do Capítulo 4. Com 2 grupos, o ponto 15 fica sozinho.

Exercício 2 — Qual ligação reproduz melhor as classes?

PYTHON

```python
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score
from scipy.cluster.hierarchy import linkage, cophenet
from scipy.spatial.distance import pdist

X, y = load_wine(return_X_y=True)
X = StandardScaler().fit_transform(X)
d = pdist(X)
for lig in ["single", "complete", "average", "ward"]:
    rot = AgglomerativeClustering(n_clusters=3, linkage=lig).fit_predict(X)
    coph, _ = cophenet(linkage(X, method=lig), d)
    print(f"{lig:9s} ARI = {adjusted_rand_score(y, rot):.3f}   cofenética = {coph:.3f}")
```

SAÍDA

```
single    ARI = -0.007   cofenética = 0.544
complete  ARI = 0.577   cofenética = 0.592
average   ARI = -0.005   cofenética = 0.759
ward      ARI = 0.790   cofenética = 0.662
```

EXEMPLO **O que observar**

Ward obtém o melhor ARI (cerca de 0,79) e a completa vem em seguida (cerca de 0,58). As ligações simples e média têm ARI próximo de zero: produzem grupos muito desequilibrados. A correlação cofenética é maior para a ligação média (cerca de 0,76), mas isso não garante grupos melhores.

Exercício 3 — Cortando a árvore por altura

PYTHON

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import linkage

X = StandardScaler().fit_transform(load_iris().data)
for h in [5, 8, 12, 20]:
    n = (
        AgglomerativeClustering(n_clusters=None, distance_threshold=h, linkage="ward")
        .fit(X)
        .n_clusters_
    )
    print(f"altura = {h:2d}  ->  {n} grupos")

alturas = linkage(X, method="ward")[:, 2]
n = len(X)
saltos = np.diff(alturas)
j = int(np.argmax(saltos))  # maior salto entre a fusão j e a fusão j+1
print("Últimos saltos entre fusões:", saltos[-5:].round(2))
print("O maior salto sugere cortar para obter", n - (j + 1), "grupos")
```

SAÍDA

```
altura =  5  ->  5 grupos
altura =  8  ->  4 grupos
altura = 12  ->  3 grupos
altura = 20  ->  2 grupos
Últimos saltos entre fusões: [ 0.02  2.36  1.4   4.63 14.61]
O maior salto sugere cortar para obter 2 grupos
```

EXEMPLO **O que observar**

Alturas maiores produzem menos grupos. O maior salto está entre as duas últimas fusões e sugere 2 grupos; o salto seguinte sugere 3. No Iris, é comum haver essa ambiguidade, pois duas espécies se sobrepõem.

Exercício 4 — O efeito corrente

PYTHON

```python
import numpy as np
from sklearn.cluster import AgglomerativeClustering

rng = np.random.default_rng(0)
A = rng.normal([0, 0], 0.4, (100, 2))
```

```
B = rng.normal([6, 0], 0.4, (100, 2))
ponte = np.c_[np.linspace(0.8, 5.2, 12), rng.normal(0, 0.05, 12)]
X = np.vstack([A, B, ponte])
for lig in ["single", "ward"]:
    rot = AgglomerativeClustering(2, linkage=lig).fit_predict(X)
    print(f"{lig:7s} tamanhos dos grupos: {np.bincount(rot)}")
```

SAÍDA

```
single  tamanhos dos grupos: [211   1]
ward    tamanhos dos grupos: [108 104]
```

EXEMPLO O que observar

Com a ligação simples, os pontos da ponte unem os dois blocos e a solução costuma isolar poucos pontos (tamanhos desequilibrados). Ward divide em dois blocos equilibrados (108 e 104 pontos). Atenção: com a ligação simples, um único ponto ficou isolado.

Exercício 5 — Agrupando exemplos e atributos ao mesmo tempo

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import linkage, dendrogram
from scipy.spatial.distance import squareform

d = load_wine()
X = StandardScaler().fit_transform(d.data)

Zl = linkage(X, method="ward")
ordem_l = dendrogram(Zl, no_plot=True)["leaves"]

corr = np.corrcoef(X.T)
dist = 1 - np.abs(corr)
np.fill_diagonal(dist, 0)
Zc = linkage(squareform(dist, checks=False), method="average")
ordem_c = dendrogram(Zc, no_plot=True)["leaves"]
print("Ordem dos atributos:", [d.feature_names[i] for i in ordem_c])

plt.imshow(X[ordem_l][:, ordem_c], aspect="auto", cmap="viridis")
plt.xticks(range(13), [d.feature_names[i][:8] for i in ordem_c], rotation=90)
plt.title("Wine: linhas e colunas reordenadas")
plt.show()
```

SAÍDA

```
Ordem dos atributos: ['malic_acid', 'hue', 'nonflavanoid_phenols', 'proanthocyanins',
'od280/od315_of_diluted_wines',…
```

EXEMPLO O que observar

Atributos fortemente correlacionados (como flavanoids, total_phenols e od280/od315) ficam adjacentes. O mapa mostra blocos de vinhos com perfis químicos parecidos.

Capítulo 5 — Agrupamento por Densidade

Exercício 1 — Varrendo o parâmetro eps

PYTHON

```python
import numpy as np
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN

X = StandardScaler().fit_transform(make_moons(500, noise=0.07, random_state=0)[0])
for eps in [0.05, 0.1, 0.2, 0.3, 0.6]:
    rot = DBSCAN(eps=eps, min_samples=5).fit_predict(X)
    print(
        f"eps = {eps:4.2f}  grupos = {len(set(rot)) - (1 if -1 in rot else 0):2d}  ruído = {(rot == -1).sum():3d}"
    )
```

SAÍDA

```
eps = 0.05  grupos =  8  ruído = 454
eps = 0.10  grupos = 25  ruído = 123
eps = 0.20  grupos =  2  ruído =   3
eps = 0.30  grupos =  2  ruído =   0
eps = 0.60  grupos =  1  ruído =   0
```

EXEMPLO **O que observar**

Com eps muito pequeno, quase tudo vira ruído e surgem muitos grupos pequenos. Com valores intermediários, aparecem os 2 grupos corretos. Com eps grande, os dois grupos se fundem em um só.

Exercício 2 — Achando o joelho do gráfico de k-distâncias

PYTHON

```python
import numpy as np
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN

X = StandardScaler().fit_transform(make_moons(500, noise=0.07, random_state=0)[0])
k = 5
dist, _ = NearestNeighbors(n_neighbors=k).fit(X).kneighbors(X)
dk = np.sort(dist[:, -1])

x = np.arange(len(dk))
p1, p2 = np.array([0, dk[0]]), np.array([len(dk) - 1, dk[-1]])
v = (p2 - p1) / np.linalg.norm(p2 - p1)
pts = np.c_[x, dk] - p1
distancia = np.abs(pts[:, 0] * v[1] - pts[:, 1] * v[0])  # distância à reta
eps = dk[np.argmax(distancia)]
print("eps sugerido pelo joelho:", round(eps, 3))
```

```
rot = DBSCAN(eps=eps, min_samples=k).fit_predict(X)
print("grupos:", len(set(rot)) - (1 if -1 in rot else 0), "| ruído:", (rot == -1).sum())
```

SAÍDA

```
eps sugerido pelo joelho: 0.147
grupos: 4 | ruído: 24
```

EXEMPLO O que observar

Neste caso, o joelho automático (eps ≈ 0,15) resulta em 4 grupos e 24 pontos de ruído: um pouco pequeno demais. Valores próximos de 0,2 (veja o Exercício 1) dão os 2 grupos corretos. Moral da história: o joelho é um ponto de partida; confira o gráfico e varie o eps.

Exercício 3 — DBSCAN, K-means e espectral em anéis

PYTHON

```python
from sklearn.datasets import make_circles
from sklearn.cluster import KMeans, SpectralClustering, DBSCAN
from sklearn.metrics import adjusted_rand_score

X, y = make_circles(600, factor=0.4, noise=0.05, random_state=0)
modelos = {
    "K-means": KMeans(2, n_init=10, random_state=0),
    "Espectral": SpectralClustering(
        2, affinity="nearest_neighbors", n_neighbors=10, random_state=0
    ),
    "DBSCAN": DBSCAN(eps=0.15, min_samples=5),
}
for nome, m in modelos.items():
    rot = m.fit_predict(X)
    print(f"{nome:10s} ARI = {adjusted_rand_score(y, rot):.3f}")
```

SAÍDA

```
K-means    ARI = -0.001
Espectral  ARI = 1.000
DBSCAN     ARI = 1.000
```

EXEMPLO O que observar

O K-means falha (ARI perto de zero). O espectral e o DBSCAN reconhecem os anéis (ARI perto de 1).

Exercício 4 — Densidades variáveis: OPTICS e HDBSCAN

PYTHON

```python
import numpy as np
from sklearn.cluster import DBSCAN, OPTICS, HDBSCAN
from sklearn.metrics import adjusted_rand_score

rng = np.random.default_rng(0)
G = [
    rng.normal([0, 0], 0.3, (250, 2)),
```

```
    rng.normal([5, 5], 0.6, (250, 2)),
    rng.normal([-5, 5], 1.2, (250, 2)),
]
y = np.repeat([0, 1, 2], 250)
ruido = rng.uniform(-9, 9, (40, 2))
X = np.vstack(G + [ruido])
y = np.r_[y, np.full(40, -1)]
real = y >= 0

for nome, m in [
    ("DBSCAN", DBSCAN(eps=0.5, min_samples=10)),
    ("OPTICS", OPTICS(min_samples=10, xi=0.05, min_cluster_size=0.05)),
    ("HDBSCAN", HDBSCAN(min_cluster_size=30)),
]:
    rot = m.fit_predict(X)
    print(
        f"{nome:8s} grupos = {len(set(rot)) - (1 if -1 in rot else 0)}  ruído = {(rot == -1).sum():3d}
ARI = {adjusted_rand_score(y[real], rot[real]):.3f}"
    )
```

SAÍDA

```
DBSCAN   grupos = 3  ruído =  95  ARI = 0.900
OPTICS   grupos = 3  ruído = 398  ARI = 0.546
HDBSCAN  grupos = 3  ruído =  30  ARI = 0.992
```

EXEMPLO O que observar

O HDBSCAN obtém o melhor resultado (ARI ≈ 0,99, com 30 pontos de ruído). Com um único eps, o DBSCAN tem ARI ≈ 0,90 e marca 95 pontos como ruído, pois o grupo mais difuso é parcialmente descartado. O OPTICS, com esses parâmetros, extrai grupos pelo método xi e marca muito ruído (ARI ≈ 0,55): ele exige ajuste fino de xi e min_cluster_size. Densidades variáveis são um ponto forte do HDBSCAN.

Exercício 5 — Pontos quentes com a distância haversine

PYTHON

```
import numpy as np
from sklearn.cluster import DBSCAN

rng = np.random.default_rng(0)
centros = np.array([[-23.55, -46.63], [-23.50, -46.70], [-23.60, -46.55]])
quentes = np.vstack([c + rng.normal(0, 0.004, (120, 2)) for c in centros])
disp = np.c_[rng.uniform(-23.70, -23.40, 80), rng.uniform(-46.80, -46.45, 80)]
coords = np.vstack([quentes, disp])

R = 6371.0  # raio da Terra em km
db = DBSCAN(eps=0.4 / R, min_samples=15, metric="haversine", algorithm="ball_tree").fit(
    np.radians(coords)
)
for g in sorted(set(db.labels_) - {-1}):
    pts = coords[db.labels_ == g]
    print(
        f"ponto quente {g}: {len(pts):3d} ocorrências, centro = ({pts[:, 0].mean():.3f}, {pts[:,
1].mean():.3f})"
    )
```

```
print("ruído:", int((db.labels_ == -1).sum()))
```

SAÍDA

```
ponto quente 0: 114 ocorrências, centro = (-23.550, -46.630)
ponto quente 1: 110 ocorrências, centro = (-23.500, -46.700)
ponto quente 2: 116 ocorrências, centro = (-23.600, -46.550)
ruído: 100
```

EXEMPLO **O que observar**

Os três centros simulados são recuperados com boa precisão (as coordenadas médias ficam muito próximas das originais). O eps precisa estar em radianos: distância em km dividida pelo raio da Terra.

Capítulo 6 — Modelos de Mistura Gaussiana e Agrupamento Espectral

Exercício 1 — Tipos de covariância no Iris

PYTHON

```python
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score

X, y = load_iris(return_X_y=True)
X = StandardScaler().fit_transform(X)
for tipo in ["spherical", "diag", "tied", "full"]:
    g = GaussianMixture(3, covariance_type=tipo, n_init=5, random_state=0).fit(X)
    print(
        f"{tipo:10s} BIC = {g.bic(X):7.1f}   ARI = {adjusted_rand_score(y, g.predict(X)):.3f}"
    )
```

SAÍDA

```
spherical  BIC =  1226.9   ARI = 0.620
diag       BIC =   964.7   ARI = 0.818
tied       BIC =   854.4   ARI = 0.904
full       BIC =   801.5   ARI = 0.904
```

EXEMPLO **O que observar**

Neste conjunto, o tipo `full` tem o menor BIC (cerca de 800), e `tied` obtém o mesmo ARI (cerca de 0,90), com menos parâmetros. O tipo `spherical` é o pior, pois obriga os grupos a serem círculos. Em outros dados, tipos mais simples podem vencer pelo BIC; vale comparar sempre.

Exercício 2 — Quantas gaussianas?

PYTHON

```python
import pandas as pd
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture

X, _ = make_blobs(600, centers=4, cluster_std=1.1, random_state=11)
linhas = []
for K in range(1, 9):
    g = GaussianMixture(K, n_init=3, random_state=0).fit(X)
    linhas.append({"K": K, "BIC": g.bic(X), "AIC": g.aic(X)})
t = pd.DataFrame(linhas)
print(t.round(1).to_string(index=False))
print(
    "K pelo BIC:",
    int(t.loc[t.BIC.idxmin(), "K"]),
    "| K pelo AIC:",
    int(t.loc[t.AIC.idxmin(), "K"]),
)
```

SAÍDA

```
 K    BIC    AIC
 1 6900.0 6878.0
 2 5994.4 5946.1
 3 5508.9 5434.2
 4 5399.8 5298.6
 5 5434.8 5307.3
 6 5460.7 5306.9
 7 5502.0 5321.7
 8 5533.7 5327.1
K pelo BIC: 4 | K pelo AIC: 4
```

EXEMPLO **O que observar**

Nos dois critérios, o mínimo ocorre em K = 4, o número verdadeiro de grupos. Com grupos sobrepostos, o AIC tende a escolher mais componentes que o BIC.

Exercício 3 — Pontos ambíguos e pertinência suave

PYTHON

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture

X, _ = make_blobs(400, centers=[[0, 0], [2.5, 0.5], [1.2, 3]], cluster_std=1.0, random_state=2)
g = GaussianMixture(3, n_init=5, random_state=0).fit(X)
P = g.predict_proba(X)
pmax = P.max(axis=1)
idx = np.argsort(pmax)[:5]
for i in idx:
    print(f"ponto {i:3d}  probabilidades = {P[i].round(2)}")
print("Pontos com probabilidade máxima < 0,8:", int((pmax < 0.8).sum()), "de", len(X))
```

SAÍDA

```
ponto 107  probabilidades = [0.26 0.34 0.4 ]
ponto 354  probabilidades = [0.28 0.4  0.32]
ponto 286  probabilidades = [0.22 0.41 0.36]
ponto 160  probabilidades = [0.42 0.39 0.19]
ponto  19  probabilidades = [0.17 0.44 0.39]
Pontos com probabilidade máxima < 0,8: 93 de 400
```

EXEMPLO **O que observar**

Os pontos mais ambíguos têm probabilidades quase iguais entre dois ou três grupos (cerca de 0,3 a 0,4 cada) e ficam nas regiões de sobreposição. Cerca de um quarto dos pontos (93 de 400) tem probabilidade máxima abaixo de 0,8. Em um negócio, esses clientes podem receber ações combinadas.

Exercício 4 — GMM como detector de anomalias

PYTHON

```python
import numpy as np
from sklearn.mixture import GaussianMixture
```

```
rng = np.random.default_rng(0)
normais = np.vstack([rng.normal([0, 0], 0.8, (400, 2)), rng.normal([4, 3], 0.6, (400, 2))])
anom = rng.uniform(-6, 10, (30, 2))
X = np.vstack([normais, anom])
y = np.r_[np.zeros(800), np.ones(30)]

g = GaussianMixture(2, n_init=3, random_state=0).fit(normais)  # treina só com normais
escore = g.score_samples(X)
limiar = np.percentile(escore, 3)
marcado = escore < limiar
print("Anomalias detectadas:", int(marcado[y == 1].sum()), "de 30")
print("Normais marcados por engano:", int(marcado[y == 0].sum()), "de 800")
```

SAÍDA

```
Anomalias detectadas: 24 de 30
Normais marcados por engano: 1 de 800
```

EXEMPLO **O que observar**

O GMM detecta 24 das 30 anomalias, com apenas 1 falso alarme entre os 800 pontos normais. As 6 anomalias perdidas caíram em regiões de densidade alta, perto dos grupos normais. O limiar de 3% é uma escolha: mudar o percentil troca falsos alarmes por anomalias perdidas.

Exercício 5 — Espectral: o papel da afinidade

PYTHON

```
from sklearn.datasets import make_moons
from sklearn.cluster import SpectralClustering, KMeans
from sklearn.metrics import adjusted_rand_score

X, y = make_moons(500, noise=0.07, random_state=0)
print(
    f"K-means                  ARI = {adjusted_rand_score(y, KMeans(2, n_init=10,
random_state=0).fit_predict(X)):.3f}"
)
for gamma in [0.1, 1, 10, 100]:
    r = SpectralClustering(2, affinity="rbf", gamma=gamma, random_state=0).fit_predict(X)
    print(f"Espectral rbf gamma={gamma:<5} ARI = {adjusted_rand_score(y, r):.3f}")
r = SpectralClustering(
    2, affinity="nearest_neighbors", n_neighbors=10, random_state=0
).fit_predict(X)
print(f"Espectral 10-vizinhos     ARI = {adjusted_rand_score(y, r):.3f}")
```

SAÍDA

```
K-means                  ARI = 0.245
Espectral rbf gamma=0.1   ARI = 0.229
Espectral rbf gamma=1     ARI = 0.290
Espectral rbf gamma=10    ARI = 0.795
Espectral rbf gamma=100   ARI = 1.000
Espectral 10-vizinhos     ARI = 1.000
```

EXEMPLO **O que observar**

O resultado depende muito da afinidade. Com gamma pequeno (0,1 ou 1), tudo parece similar e o ARI fica baixo (cerca de 0,23 a 0,29, como o K-means); com gamma = 100, o ARI chega a 1,0. O grafo de 10 vizinhos também atinge 1,0 e é uma escolha robusta para formas curvas.

Capítulo 7 — Como Saber se os Grupos São Bons? Avaliação e Validação

Exercício 1 — Há grupos para encontrar? A estatística de Hopkins

PYTHON

```python
import numpy as np
from sklearn.datasets import make_blobs, load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors


def hopkins(X, m=50, seed=42):
    r = np.random.default_rng(seed)
    n, d = X.shape
    nn = NearestNeighbors(n_neighbors=2).fit(X)
    idx = r.choice(n, m, replace=False)
    w = nn.kneighbors(X[idx], 2)[0][:, 1]  # vizinho mais próximo (reais)
    U = r.uniform(X.min(0), X.max(0), (m, d))
    u = nn.kneighbors(U, 1)[0][:, 0]  # vizinho mais próximo (artificiais)
    return (u**d).sum() / ((u**d).sum() + (w**d).sum())


rng = np.random.default_rng(0)
print("Uniforme:", round(hopkins(rng.random((500, 2))), 2))
print("4 blobs :", round(hopkins(make_blobs(500, centers=4, random_state=2)[0]), 2))
print("Wine    :", round(hopkins(StandardScaler().fit_transform(load_wine().data)), 2))
```

SAÍDA

```
Uniforme: 0.48
4 blobs : 0.96
Wine    : 1.0
```

EXEMPLO **O que observar**

Perto de 0,5 indica dados uniformes; valores perto de 1 indicam tendência de agrupamento. O Wine fica claramente acima de 0,5. Nesta versão, usamos a distância elevada à dimensão d, como na definição original.

Exercício 2 — Silhueta por grupo e por ponto

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_samples

X = StandardScaler().fit_transform(load_wine().data)
rot = KMeans(3, n_init=10, random_state=0).fit_predict(X)
s = silhouette_samples(X, rot)
for g in range(3):
```

```
    print(f"grupo {g}: n = {(rot == g).sum():3d}  silhueta média = {s[rot == g].mean():.3f}")
print(
    "Silhueta média geral:",
    round(s.mean(), 3),
    "| pontos com silhueta negativa:",
    int((s < 0).sum()),
)
```

SAÍDA

```
grupo 0: n =  65  silhueta média = 0.177
grupo 1: n =  51  silhueta média = 0.351
grupo 2: n =  62  silhueta média = 0.343
Silhueta média geral: 0.285 | pontos com silhueta negativa: 7
```

EXEMPLO **O que observar**

Grupos com silhueta média baixa e muitos pontos negativos são sinais de fronteira confusa. Observe também o equilíbrio entre os tamanhos dos grupos.

Exercício 3 — Um painel de índices internos

PYTHON

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score

X = StandardScaler().fit_transform(load_wine().data)
linhas = []
for k in range(2, 9):
    r = KMeans(k, n_init=10, random_state=0).fit_predict(X)
    linhas.append(
        {
            "k": k,
            "silhueta": silhouette_score(X, r),
            "CH": calinski_harabasz_score(X, r),
            "DB": davies_bouldin_score(X, r),
        }
    )
t = pd.DataFrame(linhas)
print(t.round(3).to_string(index=False))
print(
    "k preferido -> silhueta:",
    int(t.loc[t.silhueta.idxmax(), "k"]),
    "| CH:",
    int(t.loc[t.CH.idxmax(), "k"]),
    "| DB:",
    int(t.loc[t.DB.idxmin(), "k"]),
)
```

SAÍDA

```
 k  silhueta     CH    DB
 2     0.268 69.486 1.448
```

```
 3     0.285 70.940 1.389
 4     0.246 55.668 1.765
 5     0.203 46.883 1.820
 6     0.196 41.812 1.812
 7     0.139 37.714 1.922
 8     0.158 35.210 1.769
k preferido -> silhueta: 3 | CH: 3 | DB: 3
```

EXEMPLO O que observar

Em dados reais, os índices podem discordar. Neste conjunto, os três apontam para k = 3. Quando discordarem, combine com estabilidade e interpretação.

Exercício 4 — Avaliação externa de três algoritmos

PYTHON

```python
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (
    adjusted_rand_score,
    normalized_mutual_info_score,
    homogeneity_score,
    completeness_score,
    v_measure_score,
)

X, y = load_digits(return_X_y=True)
Z = PCA(20, random_state=0).fit_transform(X)
modelos = {
    "K-means": KMeans(10, n_init=10, random_state=0),
    "Ward": AgglomerativeClustering(10, linkage="ward"),
    "GMM": GaussianMixture(10, covariance_type="diag", n_init=3, random_state=0),
}
linhas = []
for nome, m in modelos.items():
    r = m.fit_predict(Z)
    linhas.append(
        {
            "modelo": nome,
            "ARI": adjusted_rand_score(y, r),
            "NMI": normalized_mutual_info_score(y, r),
            "homog.": homogeneity_score(y, r),
            "compl.": completeness_score(y, r),
            "V": v_measure_score(y, r),
        }
    )
print(pd.DataFrame(linhas).round(3).to_string(index=False))
```

SAÍDA

```
 modelo   ARI   NMI  homog.  compl.     V
K-means 0.668 0.743   0.739   0.747 0.743
   Ward 0.783 0.860   0.850   0.870 0.860
```

```
    GMM 0.581 0.747   0.732   0.762 0.747
```

EXEMPLO O que observar

Os três algoritmos têm desempenho parecido (ARI na faixa de 0,6 a 0,8). Nenhum é perfeito: o agrupamento nos dígitos mistura classes parecidas, como 3, 5, 8 e 9.

Exercício 5 — Estabilidade para escolher k

PYTHON

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

X, _ = make_blobs(600, centers=4, random_state=8)
rng = np.random.default_rng(0)
for k in range(2, 8):
    sc = []
    for _ in range(15):
        i1 = rng.choice(len(X), 300, replace=False)
        i2 = rng.choice(len(X), 300, replace=False)
        m1 = KMeans(k, n_init=3, random_state=0).fit(X[i1])
        m2 = KMeans(k, n_init=3, random_state=1).fit(X[i2])
        sc.append(adjusted_rand_score(m1.predict(X), m2.predict(X)))
    print(f"k = {k}  ARI médio = {np.mean(sc):.3f}  desvio = {np.std(sc):.3f}")
```

SAÍDA

```
k = 2  ARI médio = 1.000  desvio = 0.000
k = 3  ARI médio = 0.782  desvio = 0.262
k = 4  ARI médio = 1.000  desvio = 0.000
k = 5  ARI médio = 0.856  desvio = 0.051
k = 6  ARI médio = 0.803  desvio = 0.056
k = 7  ARI médio = 0.686  desvio = 0.073
```

EXEMPLO O que observar

Valores de k que cortam um grupo natural em pedaços arbitrários tendem a ser instáveis (ARI médio menor e desvio maior). Para k = 2 e k = 4 a estabilidade costuma ser alta, o que combina com a estrutura (grupos que se juntam em pares).

Capítulo 8 — PCA e Métodos Lineares

Exercício 1 — Quantas componentes para 90% da variância?

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

d = load_wine()
X = StandardScaler().fit_transform(d.data)
pca = PCA().fit(X)
acum = pca.explained_variance_ratio_.cumsum()
print("Variância acumulada:", acum.round(3))
print("Componentes para 90%:", int(np.argmax(acum >= 0.90)) + 1)

pesos = pca.components_[0]
top = np.argsort(-np.abs(pesos))[:3]
print("Maiores pesos na PC1:", [(d.feature_names[i], round(float(pesos[i]), 2)) for i in top])
```

SAÍDA

```
Variância acumulada: [0.362 0.554 0.665 0.736 0.802 0.851 0.893 0.92  0.942 0.962 0.979 0.992
 1.   ]
Componentes para 90%: 8
Maiores pesos na PC1: [('flavanoids', 0.42), ('total_phenols', 0.39), ('od280/od315_of_diluted_wines',
0.38)]
```

EXEMPLO **O que observar**

Em geral, 8 componentes bastam para 90% da variância dos 13 atributos. A PC1 combina, sobretudo, atributos ligados a fenóis (flavanoids, total_phenols) e à diluição do vinho (od280/od315).

Exercício 2 — Reconstrução e compressão de dígitos

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

X, _ = load_digits(return_X_y=True)
for k in [2, 5, 10, 20, 40]:
    p = PCA(k, random_state=0).fit(X)
    R = p.inverse_transform(p.transform(X))
    print(
        f"k = {k:2d}  variância = {p.explained_variance_ratio_.sum():.1%}  MSE = {np.mean((X - R) **
2):6.3f}  compressão = {64 / k:5.1f}x"
    )
```

SAÍDA

```
k =  2  variância = 28.5%  MSE = 13.421  compressão =  32.0x
```

```
k =  5  variância = 54.5%  MSE =  8.542  compressão =  12.8x
k = 10  variância = 73.8%  MSE =  4.914  compressão =   6.4x
k = 20  variância = 89.4%  MSE =  1.984  compressão =   3.2x
k = 40  variância = 98.8%  MSE =  0.221  compressão =   1.6x
```

EXEMPLO O que observar

O erro cai rápido com as primeiras componentes e depois diminui devagar. Com 20 componentes (compressão de 3,2 vezes) já se retém cerca de 90% da variância.

Exercício 3 — PCA "na unha" com autovetores

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X = StandardScaler().fit_transform(load_wine().data)
C = np.cov(X.T)
autoval, autovet = np.linalg.eigh(C)
ordem = np.argsort(autoval)[::-1]
autoval, autovet = autoval[ordem], autovet[:, ordem]
Z = X @ autovet[:, :2]

print("Variância explicada (manual)     :", (autoval / autoval.sum())[:3].round(4))
print(
    "Variância explicada (scikit-learn):", PCA().fit(X).explained_variance_ratio_[:3].round(4)
)
print("Componentes ortogonais? (V'V = I):", np.allclose(autovet.T @ autovet, np.eye(13)))
pc = PCA(2).fit_transform(X)
print("Projeção igual, a menos do sinal?", np.allclose(np.abs(Z), np.abs(pc), atol=1e-6))
```

SAÍDA

```
Variância explicada (manual)     : [0.362  0.1921 0.1112]
Variância explicada (scikit-learn): [0.362  0.1921 0.1112]
Componentes ortogonais? (V'V = I): True
Projeção igual, a menos do sinal? True
```

EXEMPLO O que observar

As variâncias coincidem. As projeções são iguais a menos do sinal de cada componente, pois o sinal é arbitrário.

Exercício 4 — Kernel PCA em anéis

PYTHON

```python
from sklearn.datasets import make_circles
from sklearn.decomposition import PCA, KernelPCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

X, y = make_circles(500, factor=0.3, noise=0.05, random_state=0)
```

```
for nome, red in [
    ("PCA linear", PCA(2)),
    ("Kernel PCA rbf", KernelPCA(2, kernel="rbf", gamma=10)),
]:
    z1 = red.fit_transform(X)[:, [0]]
    acc = cross_val_score(LogisticRegression(), z1, y, cv=5).mean()
    print(f"{nome:15s} acurácia usando só a 1ª componente = {acc:.3f}")
```

SAÍDA

```
PCA linear      acurácia usando só a 1ª componente = 0.474
Kernel PCA rbf  acurácia usando só a 1ª componente = 0.766
```

EXEMPLO O que observar

No PCA linear, a primeira componente não separa os anéis (acurácia em torno de 0,47, pior que o acaso por pouco). No Kernel PCA, a primeira componente já ajuda bastante (cerca de 0,77); com as duas componentes a separação seria ainda melhor.

Exercício 5 — Separando fontes com ICA

PYTHON

```
import numpy as np
from scipy import signal
from sklearn.decomposition import FastICA, PCA

t = np.linspace(0, 8, 3000)
S = np.c_[np.sin(2 * t), np.sign(np.sin(3 * t)), signal.sawtooth(2 * np.pi * 0.7 * t)]
S += 0.03 * np.random.default_rng(0).normal(size=S.shape)
A = np.random.default_rng(1).uniform(0.5, 1.5, (3, 3))
X = S @ A.T

ica = FastICA(3, random_state=0, max_iter=1000).fit_transform(X)
pca = PCA(3).fit_transform(X)


def melhor(R):
    c = np.abs(np.corrcoef(S.T, R.T)[:3, 3:])
    return c.max(axis=1).round(2)


print("ICA:", melhor(ica))
print("PCA:", melhor(pca))
```

SAÍDA

```
ICA: [0.99 1.   0.99]
PCA: [0.63 0.75 0.84]
```

EXEMPLO O que observar

O ICA recupera as três fontes (correlações próximas de 1), enquanto o PCA, que apenas decorrela e ordena por variância, mistura as fontes.

Capítulo 9 — Aprendizado de Variedades

Exercício 1 — O mapa do Brasil a partir de distâncias

PYTHON

```python
import numpy as np
from sklearn.manifold import MDS
from sklearn.metrics import pairwise_distances
from scipy.spatial.distance import pdist

cidades = {
    "São Paulo": (-23.55, -46.63),
    "Rio de Janeiro": (-22.91, -43.17),
    "Brasília": (-15.78, -47.93),
    "Salvador": (-12.97, -38.51),
    "Fortaleza": (-3.73, -38.53),
    "Manaus": (-3.12, -60.02),
    "Porto Alegre": (-30.03, -51.23),
    "Belo Horizonte": (-19.92, -43.94),
    "Recife": (-8.05, -34.88),
    "Curitiba": (-25.43, -49.27),
}
ll = np.radians(np.array(list(cidades.values())))
D = pairwise_distances(ll, metric="haversine") * 6371.0  # km

mds = MDS(2, dissimilarity="precomputed", random_state=0, n_init=8, normalized_stress="auto")
Y = mds.fit_transform(D)
d_orig = D[np.triu_indices(10, 1)]
d_mapa = pdist(Y)
stress = np.sqrt(np.sum((d_orig - d_mapa) ** 2) / np.sum(d_orig**2))
print("Stress normalizado:", round(float(stress), 4))
print(
    "Correlação entre distâncias originais e do mapa:",
    round(float(np.corrcoef(d_mapa, d_orig)[0, 1]), 4),
)
print(
    "Distância São Paulo - Rio: original",
    round(D[0, 1]),
    "km | mapa",
    round(float(np.linalg.norm(Y[0] - Y[1]))),
    "km",
)
```

SAÍDA

```
Stress normalizado: 0.0019
Correlação entre distâncias originais e do mapa: 1.0
Distância São Paulo - Rio: original 361 km | mapa 362 km
```

EXEMPLO **O que observar**

Como o mapa 2D de uma superfície esférica é quase plano para distâncias dessa escala, a reconstrução é muito boa (correlação acima de 0,99). O mapa pode sair girado ou espelhado: só as distâncias importam.

Exercício 2 — Desenrolando o rocambole

PYTHON

```python
import numpy as np
from scipy.stats import spearmanr
from sklearn.datasets import make_swiss_roll
from sklearn.decomposition import PCA
from sklearn.manifold import Isomap, LocallyLinearEmbedding

X, t = make_swiss_roll(1500, noise=0.05, random_state=0)
metodos = {
    "PCA": PCA(2),
    "Isomap": Isomap(n_neighbors=12, n_components=2),
    "LLE": LocallyLinearEmbedding(n_neighbors=12, n_components=2, random_state=0),
}
for nome, m in metodos.items():
    Z = m.fit_transform(X)
    r = max(abs(spearmanr(Z[:, j], t)[0]) for j in range(2))
    print(f"{nome:7s} |correlação de Spearman| com t = {r:.3f}")
```

SAÍDA

```
PCA     |correlação de Spearman| com t = 0.213
Isomap  |correlação de Spearman| com t = 1.000
LLE     |correlação de Spearman| com t = 1.000
```

EXEMPLO **O que observar**

O Isomap e o LLE recuperam quase perfeitamente a posição ao longo do rocambole (correlação de Spearman próxima de 1). O PCA, linear, sobrepõe as voltas e fica longe disso (cerca de 0,2).

Exercício 3 — Perplexidade e qualidade do t-SNE

PYTHON

```python
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE, trustworthiness

X, _ = load_digits(return_X_y=True)
for p in [5, 30, 100]:
    t = TSNE(2, perplexity=p, init="pca", learning_rate="auto", random_state=0)
    Z = t.fit_transform(X)
    print(
        f"perplexidade = {p:3d}  trustworthiness = {trustworthiness(X, Z, n_neighbors=10):.3f}  KL
final = {t.kl_divergence_:.3f}"
    )
```

SAÍDA

```
perplexidade =   5  trustworthiness = 0.991  KL final = 0.944
perplexidade =  30  trustworthiness = 0.993  KL final = 0.754
perplexidade = 100  trustworthiness = 0.991  KL final = 0.594
```

EXEMPLO **O que observar**

A confiança é alta nos três casos (acima de 0,98), mas a KL não é comparável entre perplexidades diferentes. Escolha a perplexidade pela qualidade visual e por métricas externas ao t-SNE, e não pela KL.

Exercício 4 — PCA antes do t-SNE: tempo e qualidade

PYTHON

```python
import time
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE, trustworthiness

X, _ = load_digits(return_X_y=True)
for nome, ent in [("64 pixels", X), ("PCA 20", PCA(20, random_state=0).fit_transform(X))]:
    t0 = time.perf_counter()
    Z = TSNE(2, perplexity=30, init="pca", random_state=0).fit_transform(ent)
    dt = time.perf_counter() - t0
    print(
        f"{nome:10s} tempo = {dt:5.1f} s   trustworthiness = {trustworthiness(X, Z,
n_neighbors=10):.3f}"
    )
```

SAÍDA

```
64 pixels  tempo =   3.9 s   trustworthiness = 0.993
PCA 20     tempo =   3.7 s   trustworthiness = 0.993
```

EXEMPLO **O que observar**

Nos dígitos, que têm só 64 atributos, o tempo e a qualidade ficam praticamente iguais. Com centenas ou milhares de atributos, reduzir antes com PCA acelera muito o t-SNE e remove ruído, por isso é uma etapa comum.

Exercício 5 — Quem preserva melhor os vizinhos?

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.manifold import Isomap, TSNE
from sklearn.neighbors import NearestNeighbors, KNeighborsClassifier
from sklearn.model_selection import cross_val_score

X, y = load_digits(return_X_y=True)


def vizinhos(A, k=10):
    return (
        NearestNeighbors(n_neighbors=k + 1).fit(A).kneighbors(A, return_distance=False)[:, 1:]
    )


orig = vizinhos(X)
proj = {
```

```
    "PCA": PCA(2).fit_transform(X),
    "Isomap": Isomap(n_neighbors=10).fit_transform(X),
    "t-SNE": TSNE(2, init="pca", random_state=0).fit_transform(X),
}
for nome, Z in proj.items():
    v = vizinhos(Z)
    pres = np.mean([len(set(orig[i]) & set(v[i])) / 10 for i in range(len(X))])
    acc = cross_val_score(KNeighborsClassifier(5), Z, y, cv=5).mean()
    print(f"{nome:7s} vizinhos preservados = {pres:.2f}   acurácia KNN em 2D = {acc:.3f}")
```

SAÍDA

```
PCA     vizinhos preservados = 0.12   acurácia KNN em 2D = 0.603
Isomap  vizinhos preservados = 0.18   acurácia KNN em 2D = 0.705
t-SNE   vizinhos preservados = 0.58   acurácia KNN em 2D = 0.976
```

EXEMPLO O que observar

O t-SNE preserva muito mais vizinhos que o PCA e o Isomap e produz grupos mais separados (acurácia KNN em 2D bem maior). Isso não significa que o t-SNE "seja melhor" em tudo: ele não preserva distâncias globais.

Capítulo 10 — Regras de Associação

Exercício 1 — Suporte, confiança e lift "na unha"

PYTHON

```python
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

compras = [
    ["pão", "leite"],
    ["pão", "manteiga", "leite"],
    ["pão", "manteiga"],
    ["leite", "café"],
    ["pão", "leite", "café"],
    ["pão", "manteiga", "café"],
    ["leite", "manteiga"],
    ["pão", "manteiga", "leite", "café"],
    ["pão", "café"],
    ["pão", "leite", "manteiga"],
]
te = TransactionEncoder()
df = pd.DataFrame(te.fit(compras).transform(compras), columns=te.columns_)

sup_a = df["pão"].mean()
sup_b = df["manteiga"].mean()
sup_ab = (df["pão"] & df["manteiga"]).mean()
print(
    f"suporte(A→B) = {sup_ab:.2f} | confiança = {sup_ab / sup_a:.3f} | lift = {sup_ab / (sup_a *
sup_b):.3f}"
)

regras = association_rules(
    apriori(df, min_support=0.3, use_colnames=True), metric="confidence", min_threshold=0.0
)
r = regras[
    (regras.antecedents == frozenset({"pão"})) & (regras.consequents == frozenset({"manteiga"}))
]
print(r[["support", "confidence", "lift"]].round(3).to_string(index=False))
```

SAÍDA

```
suporte(A→B) = 0.50 | confiança = 0.625 | lift = 1.042
 support  confidence  lift
     0.5       0.625 1.042
```

EXEMPLO **O que observar**

Os dois cálculos coincidem: suporte 0,50; confiança 0,625; lift de cerca de 1,04. O lift próximo de 1 indica associação fraca: pão e manteiga aparecem juntos quase tanto quanto se esperaria por acaso.

Exercício 2 — Apriori ou FP-Growth?

PYTHON

```python
import time
import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import apriori, fpgrowth


def gerar_cestas(n=20000, seed=0):
    rng = np.random.default_rng(seed)
    p = np.array(
        [0.5, 0.4, 0.35, 0.3, 0.25, 0.2, 0.15, 0.12, 0.1, 0.08, 0.06, 0.05, 0.04, 0.03, 0.02]
    )
    M = rng.random((n, len(p))) < p
    M[M[:, 0], 1] |= rng.random(M[:, 0].sum()) < 0.5
    return pd.DataFrame(M, columns=[f"item{i}" for i in range(len(p))])


df = gerar_cestas()
res = {}
for nome, f in [("apriori", apriori), ("fpgrowth", fpgrowth)]:
    t0 = time.perf_counter()
    r = f(df, min_support=0.02, use_colnames=True)
    dt = time.perf_counter() - t0
    res[nome] = set(map(frozenset, r["itemsets"]))
    print(f"{nome:9s} tempo = {dt:5.2f} s   itemsets = {len(r)}")
print("Mesmos itemsets?", res["apriori"] == res["fpgrowth"])
```

SAÍDA

```
apriori   tempo =  0.03 s   itemsets = 92
fpgrowth  tempo =  0.05 s   itemsets = 92
Mesmos itemsets? True
```

EXEMPLO **O que observar**

Os dois algoritmos devolvem exatamente os mesmos itemsets (92). Neste teste, com apenas 15 itens, o Apriori foi mais rápido que o FP-Growth. A vantagem do FP-Growth aparece com muitos itens e suporte mínimo baixo, quando o número de candidatos explode. Tempos dependem do computador e da implementação.

Exercício 3 — Como o suporte mínimo controla a explosão de regras

PYTHON

```python
import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import fpgrowth, association_rules


def gerar_cestas(n=20000, seed=0):
    rng = np.random.default_rng(seed)
    p = np.array(
        [0.5, 0.4, 0.35, 0.3, 0.25, 0.2, 0.15, 0.12, 0.1, 0.08, 0.06, 0.05, 0.04, 0.03, 0.02]
    )
    M = rng.random((n, len(p))) < p
    M[M[:, 0], 1] |= rng.random(M[:, 0].sum()) < 0.5
    return pd.DataFrame(M, columns=[f"item{i}" for i in range(len(p))])
```

```
df = gerar_cestas()
for s in [0.10, 0.05, 0.03, 0.02]:
    f = fpgrowth(df, min_support=s, use_colnames=True)
    n_regras = (
        len(association_rules(f, metric="confidence", min_threshold=0.3)) if len(f) > 1 else 0
    )
    print(f"min_support = {s:.2f}  itemsets = {len(f):3d}  regras = {n_regras:4d}")
```

SAÍDA

```
min_support = 0.10  itemsets =  20  regras =   21
min_support = 0.05  itemsets =  39  regras =   42
min_support = 0.03  itemsets =  63  regras =   73
min_support = 0.02  itemsets =  92  regras =  118
```

EXEMPLO O que observar

Quanto menor o suporte mínimo, mais itemsets e regras aparecem, em crescimento rápido. Um suporte mínimo muito baixo gera ruído; muito alto esconde padrões raros porém valiosos.

Exercício 4 — Regras em dados reais: o conjunto Wine

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.preprocessing import KBinsDiscretizer
from mlxtend.frequent_patterns import fpgrowth, association_rules

d = load_wine()
X = (
    KBinsDiscretizer(n_bins=3, encode="ordinal", strategy="quantile")
    .fit_transform(d.data)
    .astype(int)
)
nomes = ["baixo", "médio", "alto"]
itens = {}
for j, f in enumerate(d.feature_names):
    for b in range(3):
        itens[f"{f[:12]}={nomes[b]}"] = X[:, j] == b
for c in range(3):
    itens[f"classe_{c}"] = d.target == c
df = pd.DataFrame(itens)

freq = fpgrowth(df, min_support=0.1, use_colnames=True, max_len=3)
regras = association_rules(freq, metric="confidence", min_threshold=0.9)
alvo = regras[
    regras.consequents.apply(lambda s: len(s) == 1 and next(iter(s)).startswith("classe_"))
]
alvo = (
    alvo[alvo.antecedents.apply(len) <= 2]
    .sort_values(["lift", "support"], ascending=False)
    .head(5)
)
```

```
for _, r in alvo.iterrows():
    print(
        f"{sorted(r.antecedents)} -> {sorted(r.consequents)}  sup = {r.support:.2f}  conf =
{r.confidence:.2f}  lift = {r.lift:.2f}"
    )
```

SAÍDA

```
['color_intens=alto', 'total_phenol=baixo'] -> ['classe_2']  sup = 0.15  conf = 1.00  lift = 3.71
['color_intens=alto', 'malic_acid=alto'] -> ['classe_2']  sup = 0.13  conf = 1.00  lift = 3.71
['flavanoids=baixo', 'hue=baixo'] -> ['classe_2']  sup = 0.24  conf = 0.98  lift = 3.62
['color_intens=alto', 'od280/od315_=baixo'] -> ['classe_2']  sup = 0.18  conf = 0.97  lift = 3.60
['color_intens=alto', 'hue=baixo'] -> ['classe_2']  sup = 0.17  conf = 0.97  lift = 3.59
```

EXEMPLO O que observar

Aparecem regras como "flavanoids = baixo e hue = baixo → classe_2" com confiança alta e lift acima de 3. É uma forma interpretável de descrever o que distingue as classes, mas lembre-se: regras de associação descrevem coocorrência, não causalidade, e variam com a discretização.

Exercício 5 — Filtrando regras e recomendando produtos

PYTHON

```
import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import fpgrowth, association_rules


def gerar_cestas(n=20000, seed=0):
    rng = np.random.default_rng(seed)
    p = np.array(
        [0.5, 0.4, 0.35, 0.3, 0.25, 0.2, 0.15, 0.12, 0.1, 0.08, 0.06, 0.05, 0.04, 0.03, 0.02]
    )
    M = rng.random((n, len(p))) < p
    M[M[:, 0], 1] |= rng.random(M[:, 0].sum()) < 0.5
    return pd.DataFrame(M, columns=[f"item{i}" for i in range(len(p))])


df = gerar_cestas()
f = fpgrowth(df, min_support=0.02, use_colnames=True)
regras = association_rules(f, metric="lift", min_threshold=1.2)
regras = regras[regras.confidence >= 0.3].reset_index(drop=True)
print("Regras antes de remover redundâncias:", len(regras))


def redundante(r, todas):
    for _, o in todas.iterrows():
        if (
            o.consequents == r.consequents
            and o.antecedents < r.antecedents
            and o.confidence >= r.confidence
        ):
            return True
    return False
```

```
manter = [not redundante(r, regras) for _, r in regras.iterrows()]
limpas = regras[manter].sort_values(["lift", "confidence"], ascending=False)
print("Regras depois:", len(limpas))
print(
    limpas[["antecedents", "consequents", "support", "confidence", "lift"]]
    .head(5)
    .round(2)
    .to_string(index=False)
)


def recomendar(cesta, regras, n=3):
    cesta = frozenset(cesta)
    ok = regras[
        regras.antecedents.apply(lambda a: a <= cesta)
        & ~regras.consequents.apply(lambda c: c <= cesta)
    ]
    ok = ok.sort_values(["lift", "confidence"], ascending=False)
    saida = []
    for c in ok.consequents:
        for item in c:
            if item not in saida:
                saida.append(item)
    return saida[:n]


print("Recomendações para {item0}:", recomendar({"item0"}, limpas))
```

SAÍDA

```
Regras antes de remover redundâncias: 30
Regras depois: 16
                    antecedents        consequents  support  confidence  lift
frozenset({item3, item1, item5}) frozenset({item0})     0.02        0.66  1.32
       frozenset({item8, item0}) frozenset({item1})     0.03        0.72  1.31
      frozenset({item10, item0}) frozenset({item1})     0.02        0.72  1.30
       frozenset({item6, item1}) frozenset({item0})     0.05        0.65  1.30
frozenset({item2, item3, item1}) frozenset({item0})     0.04        0.65  1.30
Recomendações para {item0}: ['item1']
```

EXEMPLO **O que observar**

A remoção de redundâncias reduz as regras de 30 para 16. A recomendação para quem tem o item0 na cesta é o item1, o par plantado. Repare que várias regras de maior lift envolvem item0 e item1 com antecedentes maiores: a dependência plantada "contamina" combinações maiores. Por isso se preferem regras curtas e se removem as redundantes.

Capítulo 11 — Detecção de Anomalias

Exercício 1 — Regras estatísticas simples em uma medida real

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer

d = load_breast_cancer()
rng = np.random.default_rng(0)
benigno = np.where(d.target == 1)[0]
maligno = rng.choice(np.where(d.target == 0)[0], 18, replace=False)
idx = np.r_[benigno, maligno]
x = d.data[idx, list(d.feature_names).index("mean area")]
anom = np.r_[np.zeros(len(benigno)), np.ones(len(maligno))]

z = (x - x.mean()) / x.std()
mad = np.median(np.abs(x - np.median(x)))
zmod = 0.6745 * (x - np.median(x)) / mad
q1, q3 = np.percentile(x, [25, 75])
iqr = q3 - q1
regras = {
    "|z| > 3": np.abs(z) > 3,
    "MAD > 3,5": np.abs(zmod) > 3.5,
    "IQR 1,5": (x < q1 - 1.5 * iqr) | (x > q3 + 1.5 * iqr),
}
for nome, m in regras.items():
    print(
        f"{nome:10s} marcados = {m.sum():3d}   malignos entre os marcados = {int(anom[m].sum()):2d} de
{len(maligno)}"
    )
```

SAÍDA

```
|z| > 3    marcados =   5   malignos entre os marcados =  5 de 18
MAD > 3,5  marcados =   8   malignos entre os marcados =  7 de 18
IQR 1,5    marcados =  13   malignos entre os marcados = 10 de 18
```

EXEMPLO **O que observar**

As regras marcam poucos pontos e quase todos são malignos (alta precisão), mas deixam passar a maioria dos malignos (baixa revocação), pois uma única medida não basta. Por isso se usam métodos multivariados.

Exercício 2 — Isolation Forest

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.metrics import roc_auc_score, average_precision_score

d = load_breast_cancer()
```

```
rng = np.random.default_rng(0)
idx = np.r_[
    np.where(d.target == 1)[0], rng.choice(np.where(d.target == 0)[0], 18, replace=False)
]
X = StandardScaler().fit_transform(d.data[idx])
y = (d.target[idx] == 0).astype(int)

escore = -IsolationForest(n_estimators=300, random_state=0).fit(X).score_samples(X)
print("AUC-ROC:", round(roc_auc_score(y, escore), 3))
print(
    "AUPRC  :",
    round(average_precision_score(y, escore), 3),
    "| linha de base:",
    round(y.mean(), 3),
)
```

SAÍDA

```
AUC-ROC: 0.957
AUPRC  : 0.573 | linha de base: 0.048
```

EXEMPLO **O que observar**

O AUC-ROC é alto (acima de 0,9). O AUPRC é bem menor, mas muito acima da linha de base (cerca de 0,05), o que mostra que o detector concentra anomalias no topo do ranking. Em dados muito desbalanceados, o AUPRC é a medida mais informativa.

Exercício 3 — LOF e a escolha de n_neighbors

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import average_precision_score

d = load_breast_cancer()
rng = np.random.default_rng(0)
idx = np.r_[
    np.where(d.target == 1)[0], rng.choice(np.where(d.target == 0)[0], 18, replace=False)
]
X = StandardScaler().fit_transform(d.data[idx])
y = (d.target[idx] == 0).astype(int)

for k in [5, 10, 20, 50]:
    escore = -LocalOutlierFactor(n_neighbors=k).fit(X).negative_outlier_factor_
    print(f"n_neighbors = {k:2d}  AUPRC = {average_precision_score(y, escore):.3f}")
```

SAÍDA

```
n_neighbors =  5  AUPRC = 0.236
n_neighbors = 10  AUPRC = 0.378
n_neighbors = 20  AUPRC = 0.443
n_neighbors = 50  AUPRC = 0.528
```

EXEMPLO **O que observar**

O LOF é sensível a n_neighbors. Valores muito baixos tornam o escore ruidoso; o melhor valor depende dos dados e deve ser escolhido com cuidado e, se possível, com validação.

Exercício 4 — One-Class SVM treinado só com o normal

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM
from sklearn.metrics import precision_score, recall_score

d = load_breast_cancer()
rng = np.random.default_rng(0)
ben = d.data[d.target == 1]
mal = d.data[d.target == 0][rng.choice(212, 18, replace=False)]
ben_tr, ben_te = train_test_split(ben, test_size=0.3, random_state=0)
esc = StandardScaler().fit(ben_tr)
Xte = esc.transform(np.vstack([ben_te, mal]))
yte = np.r_[np.zeros(len(ben_te)), np.ones(len(mal))]

for gamma in [0.002, "scale"]:
    for nu in [0.01, 0.05, 0.1, 0.3]:
        m = OneClassSVM(gamma=gamma, nu=nu).fit(esc.transform(ben_tr))
        pred = (m.predict(Xte) == -1).astype(int)
        print(
            f"gamma = {str(gamma):6s} nu = {nu:4.2f}  precisão = {precision_score(yte, pred):.2f}
revocação = {recall_score(yte, pred):.2f}  alertas = {pred.sum()}"
        )
```

SAÍDA

```
gamma = 0.002  nu = 0.01  precisão = 0.94  revocação = 0.83  alertas = 16
gamma = 0.002  nu = 0.05  precisão = 0.94  revocação = 0.83  alertas = 16
gamma = 0.002  nu = 0.10  precisão = 0.64  revocação = 0.89  alertas = 25
gamma = 0.002  nu = 0.30  precisão = 0.38  revocação = 1.00  alertas = 47
gamma = scale  nu = 0.01  precisão = 0.47  revocação = 1.00  alertas = 38
gamma = scale  nu = 0.05  precisão = 0.47  revocação = 1.00  alertas = 38
gamma = scale  nu = 0.10  precisão = 0.47  revocação = 1.00  alertas = 38
gamma = scale  nu = 0.30  precisão = 0.35  revocação = 1.00  alertas = 52
```

EXEMPLO **O que observar**

Com gamma = 0,002, aumentar nu eleva a revocação (de 0,83 para 1,0), mas a precisão cai de 0,94 para 0,38: é o compromisso do limiar. Com `gamma="scale"` (cerca de 0,033 para 30 atributos), o kernel é estreito demais: o modelo marca cerca de 10% dos benignos como anômalos qualquer que seja nu (precisão de 0,47). Os hiperparâmetros do One-Class SVM, sobretudo gamma, importam muito.

Exercício 5 — Cinco detectores nos dígitos

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from sklearn.covariance import EllipticEnvelope
from sklearn.metrics import roc_auc_score, average_precision_score

X, y = load_digits(return_X_y=True)
rng = np.random.default_rng(0)
normais = X[y != 9]
nove = X[y == 9][rng.choice((y == 9).sum(), 30, replace=False)]
tr, te = train_test_split(normais, test_size=0.3, random_state=0)
Xte = np.vstack([te, nove])
yte = np.r_[np.zeros(len(te)), np.ones(len(nove))]

pca = PCA(15, random_state=0).fit(tr)
Ztr, Zte = pca.transform(tr), pca.transform(Xte)

escores = {}
escores["Isolation Forest"] = (
    -IsolationForest(n_estimators=300, random_state=0).fit(Ztr).score_samples(Zte)
)
escores["LOF"] = -LocalOutlierFactor(n_neighbors=20, novelty=True).fit(Ztr).score_samples(Zte)
escores["One-Class SVM (gamma=scale)"] = (
    -OneClassSVM(gamma="scale", nu=0.05).fit(Ztr).score_samples(Zte)
)
escores["One-Class SVM (gamma=0,005)"] = (
    -OneClassSVM(gamma=0.005, nu=0.05).fit(Ztr).score_samples(Zte)
)
escores["Elíptico"] = -EllipticEnvelope(random_state=0).fit(Ztr).score_samples(Zte)
rec = pca.inverse_transform(Zte)
escores["Erro de reconstrução"] = ((Xte - rec) ** 2).sum(axis=1)

linhas = [
    {"detector": n, "AUC-ROC": roc_auc_score(yte, e), "AUPRC": average_precision_score(yte, e)}
    for n, e in escores.items()
]
print(pd.DataFrame(linhas).round(3).to_string(index=False))
print("Linha de base AUPRC:", round(yte.mean(), 3))
```

SAÍDA

```
                  detector  AUC-ROC  AUPRC
          Isolation Forest    0.702  0.123
                       LOF    0.884  0.331
One-Class SVM (gamma=scale)    0.440  0.058
One-Class SVM (gamma=0,005)    0.954  0.563
                  Elíptico    0.616  0.070
       Erro de reconstrução    0.777  0.187
Linha de base AUPRC: 0.058
```

EXEMPLO **O que observar**

Os desempenhos variam muito (todos os valores abaixo são aproximados). O One-Class SVM com gamma = 0,005 e o LOF vão muito bem (AUC-ROC de 0,95 e 0,88). O mesmo One-Class SVM com `gamma="scale"` fica abaixo do acaso (AUC-ROC em torno de 0,44), e o Isolation Forest e o detector elíptico têm resultados fracos (0,70 e 0,62). Moral: sem ajustar hiperparâmetros e comparar vários detectores com AUPRC, é fácil tirar conclusões erradas. O dígito 9 se parece com 4, 7 e 8, o que o torna uma anomalia "difícil".

Capítulo 12 — Aprendizado Profundo Não Supervisionado

Exercício 1 — Um autoencoder com MLPRegressor

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.decomposition import PCA

X = load_digits().data / 16.0
Xtr, Xte = train_test_split(X, test_size=0.3, random_state=0)

ae = MLPRegressor(
    hidden_layer_sizes=(32, 8, 32), activation="relu", max_iter=1000, random_state=0
).fit(Xtr, Xtr)
mse_ae = np.mean((ae.predict(Xte) - Xte) ** 2)

pca = PCA(8, random_state=0).fit(Xtr)
mse_pca = np.mean((pca.inverse_transform(pca.transform(Xte)) - Xte) ** 2)
print(f"MSE do autoencoder (código de 8 dimensões): {mse_ae:.4f}")
print(f"MSE do PCA com 8 componentes              : {mse_pca:.4f}")
```

SAÍDA

```
MSE do autoencoder (código de 8 dimensões): 0.0361
MSE do PCA com 8 componentes              : 0.0249
```

EXEMPLO **O que observar**

Aqui o PCA com 8 componentes teve erro menor (cerca de 0,025) que o autoencoder (cerca de 0,036). O autoencoder não linear pode ser melhor ou pior que o PCA, dependendo da arquitetura, do tempo de treino e da semente. Em dados pequenos como esse, o PCA é uma linha de base muito forte.

Exercício 2 — Abrindo o autoencoder: extraindo o código

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor

X = load_digits().data / 16.0
Xtr, Xte = train_test_split(X, test_size=0.3, random_state=0)
ae = MLPRegressor(hidden_layer_sizes=(32, 8, 32), max_iter=1000, random_state=0).fit(Xtr, Xtr)

relu = lambda a: np.maximum(a, 0)
W, b = ae.coefs_, ae.intercepts_
h1 = relu(Xte @ W[0] + b[0])  # 64 -> 32
codigo = relu(h1 @ W[1] + b[1])  # 32 -> 8  (o "código")
h3 = relu(codigo @ W[2] + b[2])  # 8 -> 32
saida = h3 @ W[3] + b[3]  # 32 -> 64 (saída linear no MLPRegressor)
```

```
print("Formato do código:", codigo.shape)
print("Reconstrução manual igual ao predict?", np.allclose(saida, ae.predict(Xte)))
print("Fração de neurônios do código inativos (zero):", round(float((codigo == 0).mean()), 2))
```

SAÍDA

```
Formato do código: (540, 8)
Reconstrução manual igual ao predict? True
Fração de neurônios do código inativos (zero): 0.38
```

EXEMPLO O que observar

Os resultados coincidem, o que confirma que a camada de 8 neurônios é o "gargalo" (código). A ReLU deixa vários neurônios do código em zero, o que se comporta como uma representação esparsa.

Exercício 3 — Autoencoder de remoção de ruído

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.decomposition import PCA

X = load_digits().data / 16.0
Xtr, Xte = train_test_split(X, test_size=0.3, random_state=0)
rng = np.random.default_rng(0)
ruido = lambda A: np.clip(A + rng.normal(0, 0.3, A.shape), 0, 1)
Xtr_r, Xte_r = ruido(Xtr), ruido(Xte)

dae = MLPRegressor(hidden_layer_sizes=(64, 16, 64), max_iter=1500, random_state=0).fit(
    Xtr_r, Xtr
)
pca = PCA(15, random_state=0).fit(Xtr)
mse = lambda A: np.mean((A - Xte) ** 2)
print("MSE da imagem ruidosa      :", round(mse(Xte_r), 4))
print("MSE após autoencoder       :", round(mse(dae.predict(Xte_r)), 4))
print(
    "MSE após PCA (15 comp.)    :", round(mse(pca.inverse_transform(pca.transform(Xte_r))), 4)
)
```

SAÍDA

```
MSE da imagem ruidosa      : 0.0522
MSE após autoencoder       : 0.0321
MSE após PCA (15 comp.)    : 0.028
```

EXEMPLO O que observar

Os dois reduzem o erro da imagem ruidosa (de cerca de 0,052 para cerca de 0,032 com o autoencoder e cerca de 0,028 com o PCA). Em dados simples e pequenos, o PCA fica ligeiramente à frente. A vantagem dos autoencoders cresce com dados maiores e estruturas não lineares.

Exercício 4 — Usando a RBM como extratora de atributos

PYTHON

```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import BernoulliRBM
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

X = load_digits().data / 16.0
y = load_digits().target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

base = LogisticRegression(max_iter=3000).fit(Xtr, ytr)
rbm = Pipeline(
    [
        ("rbm", BernoulliRBM(n_components=100, learning_rate=0.05, n_iter=20, random_state=0)),
        ("log", LogisticRegression(max_iter=3000)),
    ]
).fit(Xtr, ytr)
print("Acurácia (pixels)      :", round(base.score(Xte, yte), 3))
print("Acurácia (RBM + logist.):", round(rbm.score(Xte, yte), 3))
```

SAÍDA

```
Acurácia (pixels)      : 0.97
Acurácia (RBM + logist.): 0.883
```

EXEMPLO **O que observar**

A RBM aprende os atributos **sem usar os rótulos**; os rótulos entram só no classificador final. Aqui, a regressão logística nos pixels (cerca de 0,97) foi melhor que a combinação com a RBM (cerca de 0,88). Com mais ajuste (componentes, taxa de aprendizado, iterações) a RBM pode melhorar, mas é uma boa lição: atributos aprendidos sem rótulos não garantem vantagem sobre a linha de base.

Exercício 5 — Anomalias pelo erro de reconstrução

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.decomposition import PCA
from sklearn.metrics import roc_auc_score, average_precision_score

X, y = load_digits(return_X_y=True)
X = X / 16.0
rng = np.random.default_rng(0)
normais = X[y != 9]
nove = X[y == 9][rng.choice((y == 9).sum(), 30, replace=False)]
tr, te = train_test_split(normais, test_size=0.3, random_state=0)
Xte = np.vstack([te, nove])
yte = np.r_[np.zeros(len(te)), np.ones(len(nove))]

ae = MLPRegressor(hidden_layer_sizes=(32, 8, 32), max_iter=1500, random_state=0).fit(tr, tr)
```

```
pca = PCA(8, random_state=0).fit(tr)
esc = {
    "Autoencoder": ((ae.predict(Xte) - Xte) ** 2).sum(axis=1),
    "PCA (8 comp.)": ((pca.inverse_transform(pca.transform(Xte)) - Xte) ** 2).sum(axis=1),
}
for nome, e in esc.items():
    print(
        f"{nome:14s} AUC-ROC = {roc_auc_score(yte, e):.3f}   AUPRC = {average_precision_score(yte,
e):.3f}"
    )
print("Linha de base AUPRC:", round(yte.mean(), 3))
```

SAÍDA

```
Autoencoder    AUC-ROC = 0.792   AUPRC = 0.177
PCA (8 comp.)  AUC-ROC = 0.849   AUPRC = 0.265
Linha de base AUPRC: 0.058
```

EXEMPLO O que observar

O 9 é uma anomalia difícil: se parece com 4, 7 e 8, que o autoencoder aprendeu a reconstruir. O desempenho é modesto para os dois métodos: o PCA (AUC-ROC de cerca de 0,85) ficou à frente do autoencoder (cerca de 0,79). Quanto mais "diferente" é a anomalia, mais fácil fica detectá-la pelo erro de reconstrução.

Apêndice E — Soluções de referência dos projetos integradores

Estas são **soluções de referência**: caminhos possíveis, e não os únicos. Compare com o seu trabalho, observe as decisões e discuta as diferenças. A avaliação de um projeto deve considerar também a qualidade da argumentação, e não apenas o código. Os critérios de avaliação de cada projeto aparecem antes do código.

Capítulo 1 — Bem-vindo ao Aprendizado Não Supervisionado

Primeiro diagnóstico de um conjunto de dados

Critérios de avaliação

- Os dados foram inspecionados (formato, ausentes) antes da modelagem.
- Os atributos foram padronizados e a escolha foi justificada.
- Os grupos foram avaliados por silhueta e visualizados em 2D.
- O rótulo real só foi usado no final, para conferência.
- O texto discute limites da análise e cuidados éticos (saúde, privacidade, viés, não substituir o diagnóstico).

PYTHON

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, adjusted_rand_score

dados = load_breast_cancer(as_frame=True)
X, y = dados.data, dados.target
print("Formato:", X.shape, "| ausentes:", int(X.isna().sum().sum()))

Xp = StandardScaler().fit_transform(X)
km = KMeans(n_clusters=2, n_init=10, random_state=0).fit(Xp)
print("Silhueta média:", round(silhouette_score(Xp, km.labels_), 3))

Z = PCA(n_components=2).fit_transform(Xp)
plt.scatter(Z[:, 0], Z[:, 1], c=km.labels_, cmap="coolwarm", s=18)
plt.title("Câncer de mama: grupos do K-means em 2 componentes")
plt.show()

print(pd.crosstab(km.labels_, y, rownames=["grupo"], colnames=["rótulo real"]))
print("ARI:", round(adjusted_rand_score(y, km.labels_), 3))
```

SAÍDA

```
Formato: (569, 30) | ausentes: 0
Silhueta média: 0.343
rótulo real    0    1
grupo
0             36  339
1            176   18
ARI: 0.654
```

EXEMPLO Discussão dos resultados

Com a padronização, os dois grupos reproduzem bem o rótulo real (ARI em torno de 0,65). Mesmo assim, é uma análise exploratória: não serve para decidir sobre pacientes.

Capítulo 2 — Dados, Distâncias e Pré-processamento

Pipeline de preparação para uma carteira de clientes

Critérios de avaliação

- Todas as linhas foram aproveitadas (imputação), e não descartadas.
- A renda recebeu transformação logarítmica e escalonamento robusto, por causa dos outliers.
- A cidade foi tratada com one-hot e o plano com codificação ordinal.
- Foram comparadas silhueta e ARI nas duas abordagens.
- O texto explica o papel de cada etapa.

```python
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    FunctionTransformer,
    RobustScaler,
    OneHotEncoder,
    OrdinalEncoder,
)
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score


def gerar_clientes(n=600, seed=0):
    rng = np.random.default_rng(seed)
    g = rng.integers(0, 3, n)
    idade = np.where(
        g == 0,
        rng.normal(28, 4, n),
        np.where(g == 1, rng.normal(45, 6, n), rng.normal(63, 5, n)),
    )
    renda = np.exp(rng.normal(np.where(g == 0, 8.0, np.where(g == 1, 8.8, 8.4)), 0.35, n))
    sinistros = rng.poisson(np.where(g == 0, 2.0, np.where(g == 1, 0.6, 1.2)))
    cidade = rng.choice(["SP", "RJ", "BH", "POA"], n, p=[0.4, 0.3, 0.2, 0.1])
    plano = np.where(g == 0, "básico", np.where(g == 1, "premium", "médio"))
    df = pd.DataFrame(
        {
            "idade": idade,
            "renda": renda,
            "sinistros": sinistros,
            "cidade": cidade,
            "plano": plano,
        }
    )
    df.loc[rng.random(n) < 0.08, "idade"] = np.nan
    df.loc[rng.random(n) < 0.05, "cidade"] = np.nan
    df.loc[rng.choice(n, 5, replace=False), "renda"] *= 40
    return df, g
```

```
df, g = gerar_clientes()
print("Ausentes por coluna:\n", df.isna().sum().to_string())

# 1) Abordagem ingênua
ok = df.notna().all(axis=1).values
d1 = df[ok].copy()
for c in ["cidade", "plano"]:
    d1[c] = d1[c].astype("category").cat.codes
X1 = d1.values.astype(float)
r1 = KMeans(3, n_init=10, random_state=0).fit_predict(X1)
print(
    f"Ingênuo : linhas usadas = {ok.sum()}, silhueta = {silhouette_score(X1, r1):.3f}, ARI =
{adjusted_rand_score(g[ok], r1):.3f}, tamanhos = {np.bincount(r1)}"
)

# 2) Pipeline
num = Pipeline([("imp", SimpleImputer(strategy="median")), ("esc", RobustScaler())])
renda = Pipeline(
    [
        ("imp", SimpleImputer(strategy="median")),
        ("log", FunctionTransformer(np.log1p)),
        ("esc", RobustScaler()),
    ]
)
cat = Pipeline([("imp", SimpleImputer(strategy="most_frequent")), ("oh", OneHotEncoder())])
pre = ColumnTransformer(
    [
        ("num", num, ["idade", "sinistros"]),
        ("renda", renda, ["renda"]),
        ("cidade", cat, ["cidade"]),
        ("plano", OrdinalEncoder(categories=[["básico", "médio", "premium"]]), ["plano"]),
    ]
)
X2 = pre.fit_transform(df)
r2 = KMeans(3, n_init=10, random_state=0).fit_predict(X2)
print(
    f"Pipeline: linhas usadas = {len(df)}, silhueta = {silhouette_score(X2, r2):.3f}, ARI =
{adjusted_rand_score(g, r2):.3f}, tamanhos = {np.bincount(r2)}"
)
```

SAÍDA

```
Ausentes por coluna:
 idade        52
renda         0
sinistros     0
cidade       33
plano         0
Ingênuo : linhas usadas = 516, silhueta = 0.983, ARI = 0.000, tamanhos = [512   1   3]
Pipeline: linhas usadas = 600, silhueta = 0.257, ARI = 0.940, tamanhos = [218 187 195]
```

EXEMPLO **Discussão dos resultados**

Repare no contraste: a abordagem ingênua tem silhueta altíssima (cerca de 0,98), mas ARI zero. Sem escalar, os outliers de renda dominam: o K-means separa só os pontos extremos e coloca 512 dos 516 clientes em um único grupo. O pipeline tem silhueta menor (cerca de 0,26) e ARI alto (cerca de 0,94), recuperando os perfis ocultos. Lição: uma silhueta alta não garante grupos úteis; confira tamanhos dos grupos, perfis e, quando possível, estabilidade.

Capítulo 3 — K-means

Segmentação RFM de uma loja

Critérios de avaliação

- RFM calculado corretamente (recência = hoje − último dia).
- Transformação logarítmica e padronização justificadas.
- k escolhido com mais de um critério e discutido.
- Segmentos nomeados a partir do perfil médio em unidades originais.
- Estabilidade avaliada com reamostragens.
- Cada segmento recebe uma ação coerente (por exemplo: fidelidade para VIPs, cupom de reativação para inativos).

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score


def gerar_compras(seed=0):
    rng = np.random.default_rng(seed)
    perfis = [
        ("fiel", 120, 22, 90.0),
        ("ocasional", 260, 5, 70.0),
        ("inativo", 150, 1.5, 55.0),
        ("vip", 60, 40, 220.0),
    ]
    linhas, cid = [], 0
    for nome, n, freq, ticket in perfis:
        for _ in range(n):
            cid += 1
            k = max(1, rng.poisson(freq))
            dias = np.sort(rng.integers(0, 365 if nome != "inativo" else 150, k))
            for d in dias:
                linhas.append((cid, int(d), float(rng.gamma(4, ticket / 4)), nome))
    return pd.DataFrame(linhas, columns=["cliente", "dia", "valor", "perfil_oculto"])


compras = gerar_compras()
rfm = compras.groupby("cliente").agg(
    recencia=("dia", lambda d: 365 - d.max()),
    frequencia=("dia", "size"),
    valor=("valor", "sum"),
    perfil=("perfil_oculto", "first"),
)
print("Clientes:", len(rfm))

Xr = rfm[["recencia", "frequencia", "valor"]].copy()
Xr[["frequencia", "valor"]] = np.log1p(Xr[["frequencia", "valor"]])
Xp = StandardScaler().fit_transform(Xr)
```

```
for k in range(2, 9):
    km = KMeans(k, n_init=10, random_state=0).fit(Xp)
    print(f"k={k}  inércia={km.inertia_:7.1f}  silhueta={silhouette_score(Xp, km.labels_):.3f}")

K = 4
km = KMeans(K, n_init=10, random_state=0).fit(Xp)
rfm["segmento"] = km.labels_
print(
    rfm.groupby("segmento")[["recencia", "frequencia", "valor"]]
    .mean()
    .round(1)
    .assign(n=rfm.segmento.value_counts().sort_index())
)

rng = np.random.default_rng(1)
aris = []
for _ in range(20):
    i1 = rng.choice(len(Xp), int(0.8 * len(Xp)), replace=False)
    i2 = rng.choice(len(Xp), int(0.8 * len(Xp)), replace=False)
    m1 = KMeans(K, n_init=5, random_state=0).fit(Xp[i1])
    m2 = KMeans(K, n_init=5, random_state=1).fit(Xp[i2])
    aris.append(adjusted_rand_score(m1.predict(Xp), m2.predict(Xp)))
print("Estabilidade (ARI médio):", round(float(np.mean(aris)), 3))
print("ARI contra o perfil oculto:", round(adjusted_rand_score(rfm.perfil, rfm.segmento), 3))
print(pd.crosstab(rfm.segmento, rfm.perfil))
```

SAÍDA

```
Clientes: 590
k=2  inércia=  671.6  silhueta=0.523
k=3  inércia=  253.9  silhueta=0.597
k=4  inércia=  197.4  silhueta=0.588
k=5  inércia=  150.8  silhueta=0.515
k=6  inércia=  117.8  silhueta=0.472
k=7  inércia=   99.5  silhueta=0.470
k=8  inércia=   86.9  silhueta=0.465
          recencia  frequencia   valor    n
segmento
0             10.9        41.6  9270.6   60
1            259.7         2.1   117.3  183
2             14.8        21.9  1969.4  122
3             53.7         5.5   390.0  225
Estabilidade (ARI médio): 0.955
ARI contra o perfil oculto: 0.821
perfil    fiel  inativo  ocasional  vip
segmento
0            0        0          0   60
1            0      150         33    0
2          120        0          2    0
3            0        0        225    0
```

EXEMPLO Discussão dos resultados

Repare que a silhueta é ligeiramente maior para k = 3 (cerca de 0,60) do que para k = 4 (cerca de 0,59). Com k = 4, porém, o grupo dos clientes VIP se separa dos fiéis, o que tem grande valor de negócio. A escolha de k combina critérios numéricos e utilidade. Com k = 4, a estabilidade é alta (ARI médio de cerca de 0,95) e o ARI contra os perfis ocultos fica em torno de 0,82: os "ocasionais" e "inativos" se misturam em parte (33 ocasionais caem no grupo dos inativos). Discuta isso no relatório.

Capítulo 4 — Agrupamento Hierárquico

A árvore dos dígitos

Critérios de avaliação

- Imagens médias calculadas corretamente (uma por dígito).
- Dendrograma legível, com rótulos.
- Corte da árvore descrito e interpretado (dígitos "primos").
- ARI e NMI calculados para as três ligações e comparados.
- Matriz de contingência analisada, com os dígitos mais confundidos apontados.
- Parágrafo final relaciona a árvore das médias às confusões do agrupamento.

PYTHON

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

X, y = load_digits(return_X_y=True)

medias = np.array([X[y == d].mean(axis=0) for d in range(10)])
Z = linkage(medias, method="average")
dendrogram(Z, labels=[str(d) for d in range(10)])
plt.title("Árvore dos dígitos (imagens médias)")
plt.show()
cortes = fcluster(Z, t=3, criterion="maxclust")
print("Grupos dos dígitos 0..9 (corte em 3):", cortes)

for lig in ["ward", "average", "single"]:
    rot = AgglomerativeClustering(10, linkage=lig).fit_predict(X)
    print(
        f"{lig:8s} ARI = {adjusted_rand_score(y, rot):.3f}   NMI = {normalized_mutual_info_score(y,
rot):.3f}"
    )

rot = AgglomerativeClustering(10, linkage="ward").fit_predict(X)
tab = pd.crosstab(rot, y, rownames=["grupo"], colnames=["dígito"])
print(tab)
conf = {}
for g in range(10):
    ordem = np.argsort(-tab.values[g])
    conf[g] = (int(ordem[0]), int(ordem[1]), int(tab.values[g, ordem[1]]))
print("Maior e segunda maior classe em cada grupo (classe1, classe2, nº da 2ª):", conf)
```

SAÍDA

```
Grupos dos dígitos 0..9 (corte em 3): [3 1 1 1 2 1 2 1 1 1]
ward     ARI = 0.794   NMI = 0.868
average  ARI = 0.514   NMI = 0.713
single   ARI = 0.000   NMI = 0.010
```

```
dígito    0   1    2    3    4    5    6    7    8    9
grupo
0         0   0    0    0    0  179    0    0    0    2
1         0   0    0  169    0    2    0    0    1  145
2         0  27  166    0    0    0    0    0    4    0
3         0   0    1    1    3    0    0  179    1   11
4         0   0   10   13    0    0    1    0  165    2
5         0   0    0    0  178    0    0    0    0    0
6         0   0    0    0    0    1  180    0    0    0
7       178   0    0    0    0    0    0    0    0    0
8         0  59    0    0    0    0    0    0    1   20
9         0  96    0    0    0    0    0    0    2    0
Maior e segunda maior classe em cada grupo (classe1, classe2, nº da 2ª): {0: (5, 9, 2), 1: (3, 9, 145),
2: (2, 1, 27)…
```

EXEMPLO Discussão dos resultados

Ward tem o melhor ARI (cerca de 0,79); a ligação média, cerca de 0,51; e a simples, zero (efeito corrente). Na matriz de contingência, o grupo 1 mistura 169 dígitos "3" com 145 dígitos "9", e o dígito "1" se divide entre vários grupos (inclusive com o "8" e o "9"). Na árvore das imagens médias com corte em 3, o "0" fica isolado, e "4" e "6" formam outro grupo. Discuta por que imagens médias e imagens individuais contam histórias diferentes.

Capítulo 5 — Agrupamento por Densidade

Pontos quentes de ocorrências urbanas

Critérios de avaliação

- Gráfico exploratório inicial e leitura qualitativa.
- Escolha de eps e min_samples justificada com k-distâncias.
- DBSCAN e HDBSCAN comparados.
- Tabela dos pontos quentes com tamanho, centro e densidade.
- Análise por turno.
- Proposta de patrulhas proporcional e reflexão ética (viés de policiamento, privacidade, uso de dados por território).

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN, HDBSCAN
from sklearn.neighbors import NearestNeighbors


def gerar_ocorrencias(seed=0):
    rng = np.random.default_rng(seed)
    quentes = [
        (-23.550, -46.633, 0.003, 220, "dia"),
        (-23.520, -46.690, 0.006, 260, "noite"),
        (-23.610, -46.560, 0.004, 150, "dia"),
        (-23.480, -46.620, 0.010, 200, "noite"),
    ]
    linhas = []
    for lat, lon, s, n, turno in quentes:
        P = np.c_[rng.normal(lat, s, n), rng.normal(lon, s, n)]
        linhas += [(a, b, turno) for a, b in P]
    for _ in range(120):
        linhas.append(
            (
                rng.uniform(-23.70, -23.40),
                rng.uniform(-46.80, -46.45),
                rng.choice(["dia", "noite"]),
            )
        )
    return pd.DataFrame(linhas, columns=["lat", "lon", "turno"])


df = gerar_ocorrencias()
R = 6371.0
rad = np.radians(df[["lat", "lon"]].values)

k = 15
dist, _ = NearestNeighbors(n_neighbors=k, metric="haversine").fit(rad).kneighbors(rad)
dk_km = np.sort(dist[:, -1]) * R
print(
    "Distância ao 15º vizinho (km) - percentis 50/80/90/95:",
```

```
    np.percentile(dk_km, [50, 80, 90, 95]).round(2),
)

eps_km = 0.8
db = DBSCAN(eps=eps_km / R, min_samples=k, metric="haversine", algorithm="ball_tree").fit(rad)
df["dbscan"] = db.labels_
hd = HDBSCAN(min_cluster_size=40, metric="haversine").fit(rad)
df["hdbscan"] = hd.labels_
for nome in ["dbscan", "hdbscan"]:
    r = df[nome]
    print(
        f"{nome:8s} pontos quentes = {len(set(r)) - (1 if -1 in set(r) else 0)}  ruído = {(r == -
1).sum()}"
    )

tabela = []
for g in sorted(set(db.labels_) - {-1}):
    P = df[df.dbscan == g]
    c = P[["lat", "lon"]].mean().values
    # raio (km) que contém 90% dos pontos, em aproximação plana
    dx = (P.lat - c[0]) * 111.0
    dy = (P.lon - c[1]) * 111.0 * np.cos(np.radians(c[0]))
    r90 = np.percentile(np.hypot(dx, dy), 90)
    area = np.pi * r90**2
    tabela.append(
        {
            "ponto": g,
            "n": len(P),
            "lat": round(c[0], 3),
            "lon": round(c[1], 3),
            "raio90_km": round(r90, 2),
            "dens_por_km2": round(0.9 * len(P) / area),
            "dia": int((P.turno == "dia").sum()),
            "noite": int((P.turno == "noite").sum()),
        }
    )
tab = pd.DataFrame(tabela)
print(tab.to_string(index=False))

quentes = tab.n.sum()
tab["patrulhas"] = (20 * tab.n / quentes).round().astype(int)
print(
    "Patrulhas por ponto quente:",
    dict(zip(tab.ponto, tab.patrulhas)),
    "| total:",
    tab.patrulhas.sum(),
)

for turno in ["dia", "noite"]:
    sub = df[df.turno == turno]
    r = (
        DBSCAN(eps=eps_km / R, min_samples=8, metric="haversine", algorithm="ball_tree")
        .fit(np.radians(sub[["lat", "lon"]].values))
        .labels_
    )
    print(f"turno {turno}: pontos quentes = {len(set(r)) - (1 if -1 in set(r) else 0)}")

plt.scatter(df.lon, df.lat, c=np.where(df.dbscan >= 0, df.dbscan, -1), cmap="tab10", s=8)
```

```
plt.title("Pontos quentes (DBSCAN haversine); cinza = ruído")
plt.show()
```

SAÍDA

```
Distância ao 15º vizinho (km) - percentis 50/80/90/95: [0.33 0.77 3.54 6.35]
dbscan   pontos quentes = 4  ruído = 127
hdbscan  pontos quentes = 4  ruído = 105
 ponto   n     lat     lon  raio90_km  dens_por_km2  dia  noite
     0 220 -23.550 -46.633       0.65           150  220      0
     1 264 -23.520 -46.690       1.32            44    4    260
     2 150 -23.610 -46.560       0.88            55  150      0
     3 189 -23.478 -46.620       2.13            12    1    188
Patrulhas por ponto quente: {0: 5, 1: 6, 2: 4, 3: 5} | total: 20
turno dia: pontos quentes = 2
turno noite: pontos quentes = 2
```

EXEMPLO Discussão dos resultados

O DBSCAN com eps de 0,8 km encontra os quatro pontos quentes, e o HDBSCAN também. As densidades diferem muito: o ponto 0 (cerca de 150 ocorrências por km²) é bem mais concentrado que o ponto 3 (cerca de 12 por km²), que é difuso. O ponto 1 só ocorre à noite e o ponto 0, só de dia. Por isso, a distribuição de patrulhas deve considerar também o turno, e não apenas o total. Sempre discuta o viés: ocorrências registradas refletem onde há mais policiamento e mais registro, não necessariamente onde há mais crime.

Capítulo 6 — Modelos de Mistura Gaussiana e Agrupamento Espectral

Perfis com probabilidades: uma operadora de telecomunicações

Critérios de avaliação

- Comparação justificada entre K-means e GMM, com ARI.
- Escolha de K por BIC discutida.
- Clientes de fronteira identificados e ação de marketing coerente.
- Dados sintéticos gerados e comparados com os reais.
- Atípicos descritos em unidades originais.
- Texto reconhece as limitações (perfis sobrepostos, suposição gaussiana).

```python
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score


def gerar_telecom(seed=0):
    rng = np.random.default_rng(seed)
    a = rng.multivariate_normal([4, 300], [[3, 40], [40, 4000]], 350)
    b = rng.multivariate_normal([25, 120], [[30, -30], [-30, 1500]], 450)
    c = rng.multivariate_normal([14, 210], [[20, 10], [10, 2500]], 300)
    X = np.vstack([a, b, c])
    y = np.repeat([0, 1, 2], [350, 450, 300])
    return np.clip(X, 0, None), y


X, y = gerar_telecom()
esc = StandardScaler().fit(X)
Xp = esc.transform(X)

km = KMeans(3, n_init=10, random_state=0).fit(Xp)
gm = GaussianMixture(3, covariance_type="full", n_init=5, random_state=0).fit(Xp)
print(
    "ARI K-means:",
    round(adjusted_rand_score(y, km.labels_), 3),
    "| ARI GMM:",
    round(adjusted_rand_score(y, gm.predict(Xp)), 3),
)

bics = {K: GaussianMixture(K, n_init=5, random_state=0).fit(Xp).bic(Xp) for K in range(1, 8)}
print(
    "BIC por K:",
    {k: round(v, 1) for k, v in bics.items()},
    "-> melhor K =",
    min(bics, key=bics.get),
)
```

```
P = gm.predict_proba(Xp)
fronteira = P.max(axis=1) < 0.7
print("Clientes de fronteira (prob. máx. < 0,7):", int(fronteira.sum()), "de", len(X))

sint = esc.inverse_transform(gm.sample(300)[0])
print("Médias reais     :", X.mean(axis=0).round(1))
print("Médias sintéticas:", sint.mean(axis=0).round(1))

esc_dens = gm.score_samples(Xp)
atip = esc_dens <= np.percentile(esc_dens, 1)
print(
    "Atípicos (1%): n =", int(atip.sum()), "| média [GB, min] =", X[atip].mean(axis=0).round(1)
)
```

SAÍDA

```
ARI K-means: 0.676 | ARI GMM: 0.795
BIC por K: {1: 5396.4, 2: 5055.7, 3: 4695.1, 4: 4738.0, 5: 4773.4, 6: 4811.2, 7: 4843.0} -> melhor K =
3
Clientes de fronteira (prob. máx. < 0,7): 83 de 1100
Médias reais     : [ 15.2 199.9]
Médias sintéticas: [ 15.1 194.1]
Atípicos (1%): n = 11 | média [GB, min] = [ 19.  199.7]
```

EXEMPLO Discussão dos resultados

Em perfis elípticos e sobrepostos, o GMM tende a superar o K-means, pois modela covariâncias diferentes. Os clientes de fronteira são ótimos candidatos a ofertas combinadas (voz + dados). Os dados sintéticos preservam as médias, mas não substituem dados reais em decisões.

Capítulo 7 — Como Saber se os Grupos São Bons? Avaliação e Validação

Painel de avaliação de agrupamentos

Critérios de avaliação

- Função de avaliação reutilizável, com os quatro critérios.
- Hopkins calculado antes da modelagem.
- Ranking combinado construído de modo explícito (postos somados).
- Resultados comparados com os rótulos reais apenas no final.
- O parágrafo defende o uso de vários critérios, estabilidade e interpretação, e não de um único número.

```python
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine, load_digits, load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import (
    silhouette_score,
    calinski_harabasz_score,
    davies_bouldin_score,
    adjusted_rand_score,
)


def hopkins(X, m=50, seed=42):
    r = np.random.default_rng(seed)
    n, d = X.shape
    nn = NearestNeighbors(n_neighbors=2).fit(X)
    idx = r.choice(n, m, replace=False)
    w = nn.kneighbors(X[idx], 2)[0][:, 1]
    U = r.uniform(X.min(0), X.max(0), (m, d))
    u = nn.kneighbors(U, 1)[0][:, 0]
    return (u**d).sum() / ((u**d).sum() + (w**d).sum())


FABRICAS = {
    "K-means": lambda k: KMeans(k, n_init=5, random_state=0),
    "Ward": lambda k: AgglomerativeClustering(k, linkage="ward"),
    "GMM": lambda k: GaussianMixture(k, covariance_type="diag", n_init=2, random_state=0),
}


def estabilidade(X, fab, k, pares=10, seed=0):
    rng = np.random.default_rng(seed)
    n = len(X)
    sc = []
    for _ in range(pares):
```

```
        i1 = rng.choice(n, int(0.7 * n), replace=False)
        i2 = rng.choice(n, int(0.7 * n), replace=False)

        # Ward não tem predict: usa-se 1-NN para estender a solução ao conjunto todo
        def ajustar(idx):
            r = fab(k).fit_predict(X[idx])
            return (
                NearestNeighbors(n_neighbors=1)
                .fit(X[idx])
                .kneighbors(X, return_distance=False)
                .ravel(),
                r,
            )

        a1, r1 = ajustar(i1)
        a2, r2 = ajustar(i2)
        sc.append(adjusted_rand_score(r1[a1], r2[a2]))
    return float(np.mean(sc))


def avaliar(X, ks=range(2, 7)):
    linhas = []
    for nome, fab in FABRICAS.items():
        for k in ks:
            rot = fab(k).fit_predict(X)
            linhas.append(
                {
                    "algoritmo": nome,
                    "k": k,
                    "sil": silhouette_score(X, rot),
                    "CH": calinski_harabasz_score(X, rot),
                    "DB": davies_bouldin_score(X, rot),
                    "estab": estabilidade(X, fab, k),
                    "rotulos": rot,
                }
            )
    t = pd.DataFrame(linhas)
    t["rank"] = (
        t.sil.rank(ascending=False)
        + t.CH.rank(ascending=False)
        + t.DB.rank(ascending=True)
        + t.estab.rank(ascending=False)
    )
    return t


conjuntos = {}
Xw, yw = load_wine(return_X_y=True)
conjuntos["Wine"] = (StandardScaler().fit_transform(Xw), yw)
Xd, yd = load_digits(return_X_y=True)
conjuntos["Dígitos"] = (PCA(20, random_state=0).fit_transform(Xd), yd)
Xi, yi = load_iris(return_X_y=True)
conjuntos["Iris"] = (StandardScaler().fit_transform(Xi), yi)

for nome, (X, y) in conjuntos.items():
    ks = range(2, 7) if nome != "Dígitos" else range(6, 13)
    t = avaliar(X, ks)
    melhor = t.loc[t["rank"].idxmin()]
```

```
    ari = adjusted_rand_score(y, melhor.rotulos)
    print(
        f"{nome:8s} Hopkins = {hopkins(X):.2f} | melhor pelo ranking: {melhor.algoritmo}, k =
{int(melhor.k)} "
        f"(sil {melhor.sil:.2f}, estab {melhor.estab:.2f}) | ARI com os rótulos reais = {ari:.2f}"
    )
    print(t.drop(columns="rotulos").sort_values("rank").head(3).round(2).to_string(index=False))
```

SAÍDA

```
Wine     Hopkins = 1.00 | melhor pelo ranking: K-means, k = 3 (sil 0.28, estab 0.88) | ARI com os
rótulos reais = 0.90
algoritmo  k  sil    CH   DB  estab  rank
  K-means  3 0.28 70.94 1.39   0.88   4.0
      GMM  3 0.28 69.22 1.42   0.86  10.0
     Ward  3 0.28 67.65 1.42   0.82  15.0
Dígitos  Hopkins = 1.00 | melhor pelo ranking: K-means, k = 9 (sil 0.22, estab 0.87) | ARI com os
rótulos reais = 0.59
algoritmo  k  sil     CH   DB  estab  rank
  K-means  9 0.22 217.58 1.59   0.87   6.0
  K-means  8 0.21 218.67 1.68   0.84  13.0
  K-means 10 0.22 207.20 1.64   0.80  17.0
Iris     Hopkins = 1.00 | melhor pelo ranking: K-means, k = 2 (sil 0.58, estab 0.99) | ARI com os
rótulos reais = 0.57
algoritmo  k  sil     CH   DB  estab  rank
  K-means  2 0.58 251.35 0.59   0.99   7.0
      GMM  2 0.58 251.35 0.59   0.99   7.0
     Ward  2 0.58 240.25 0.59   0.98  11.0
```

EXEMPLO **Discussão dos resultados**

Para o Iris, é comum o ranking preferir k = 2, pois duas espécies se sobrepõem, embora existam 3 classes reais. Para os dígitos, o intervalo de k é de 6 a 12 para ficar perto do número real (10). Observe que o k "melhor" pelos índices internos nem sempre coincide com o número de classes: isso é esperado e deve ser discutido, e não escondido.

Capítulo 8 — PCA e Métodos Lineares

Compressão, limpeza e partes de dígitos

Critérios de avaliação

- Nenhuma transformação ajustada com dados de teste (sem vazamento).
- Linha de base e acurácia por k apresentadas com a razão de compressão.
- Limpeza de ruído avaliada com MSE antes e depois.
- Peças do NMF mostradas e interpretadas.
- A recomendação equilibra compressão e acurácia.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA, NMF
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)


def acuracia(Ztr, Zte):
    return LogisticRegression(max_iter=2000).fit(Ztr, ytr).score(Zte, yte)


print("Linha de base (64 pixels): acurácia =", round(acuracia(Xtr, Xte), 3))
ks = [5, 10, 20, 30]
accs = []
for k in ks:
    p = PCA(k, random_state=0).fit(Xtr)
    accs.append(acuracia(p.transform(Xtr), p.transform(Xte)))
    print(f"PCA k = {k:2d}  acurácia = {accs[-1]:.3f}  compressão = {64 / k:.1f}x")
plt.plot(ks, accs, "o-")
plt.xlabel("k")
plt.ylabel("Acurácia no teste")
plt.title("Acurácia × componentes")
plt.show()

rng = np.random.default_rng(0)
ruidoso = Xte + rng.normal(0, 4, Xte.shape)
p15 = PCA(15, random_state=0).fit(Xtr)
rec = p15.inverse_transform(p15.transform(ruidoso))
print("MSE da imagem ruidosa    :", round(np.mean((ruidoso - Xte) ** 2), 2))
print("MSE após PCA (15 comp.)  :", round(np.mean((rec - Xte) ** 2), 2))

nmf = NMF(12, init="nndsvda", random_state=0, max_iter=800).fit(Xtr)
fig, ax = plt.subplots(2, 6, figsize=(9, 3.2))
for a, comp in zip(ax.ravel(), nmf.components_):
    a.imshow(comp.reshape(8, 8), cmap="gray_r")
    a.axis("off")
plt.suptitle("As 12 peças do NMF")
```

```
plt.show()
print("NMF (12): acurácia =", round(acuracia(nmf.transform(Xtr), nmf.transform(Xte)), 3))
p12 = PCA(12, random_state=0).fit(Xtr)
print("PCA (12): acurácia =", round(acuracia(p12.transform(Xtr), p12.transform(Xte)), 3))
```

SAÍDA

```
Linha de base (64 pixels): acurácia = 0.961
PCA k =  5  acurácia = 0.852  compressão = 12.8x
PCA k = 10  acurácia = 0.941  compressão = 6.4x
PCA k = 20  acurácia = 0.948  compressão = 3.2x
PCA k = 30  acurácia = 0.959  compressão = 2.1x
MSE da imagem ruidosa    : 15.97
MSE após PCA (15 comp.)  : 6.97
NMF (12): acurácia = 0.748
PCA (12): acurácia = 0.944
```

EXEMPLO Discussão dos resultados

A linha de base com 64 pixels tem acurácia de cerca de 0,96. Com apenas 10 componentes (compressão de 6,4 vezes) a acurácia é cerca de 0,94, e com 30 componentes chega a cerca de 0,96. A reconstrução com 15 componentes reduz o MSE do ruído de cerca de 16 para cerca de 7. As peças do NMF lembram traços de dígitos, mas as ativações do NMF com 12 peças deram acurácia bem menor (cerca de 0,75) que o PCA com 12 componentes (cerca de 0,94) quando usadas com um classificador linear: interpretabilidade e desempenho nem sempre andam juntos.

Capítulo 9 — Aprendizado de Variedades

Comparando mapas dos dígitos e dos vinhos

Critérios de avaliação

- Quatro ou mais técnicas comparadas nos dois conjuntos.
- Confiança, continuidade e acurácia KNN calculadas e comparadas.
- Estabilidade do t-SNE avaliada com várias sementes.
- A recomendação diferencia usos (exploração, comunicação, pré-processamento) e destaca armadilhas (tamanho e distância entre grupos no t-SNE, ausência de transform, parâmetros).
- O código funciona sem o UMAP.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits, load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import Isomap, LocallyLinearEmbedding, TSNE, trustworthiness
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

try:
    import umap
except ImportError:
    umap = None


def projecoes(X):
    d = {
        "PCA": PCA(2).fit_transform(X),
        "Isomap": Isomap(n_neighbors=10).fit_transform(X),
        "LLE": LocallyLinearEmbedding(n_neighbors=10, random_state=0).fit_transform(X),
        "t-SNE": TSNE(2, init="pca", random_state=0).fit_transform(X),
    }
    if umap is not None:
        d["UMAP"] = umap.UMAP(random_state=0).fit_transform(X)
    return d


conj = {
    "Dígitos": load_digits(return_X_y=True),
    "Wine": (StandardScaler().fit_transform(load_wine().data), load_wine().target),
}
fig, axes = plt.subplots(2, 5, figsize=(14, 6))
for r, (nome, (X, y)) in enumerate(conj.items()):
    linhas = []
    for c, (m, Z) in enumerate(projecoes(X).items()):
        linhas.append(
            {
                "método": m,
                "confiança": trustworthiness(X, Z, n_neighbors=10),
```

```
                "continuidade": trustworthiness(Z, X, n_neighbors=10),
                "KNN 2D": cross_val_score(KNeighborsClassifier(5), Z, y, cv=5).mean(),
            }
        )
        axes[r, c].scatter(Z[:, 0], Z[:, 1], c=y, cmap="tab10", s=4)
        axes[r, c].set_title(f"{nome}: {m}", fontsize=8)
        axes[r, c].axis("off")
    print(nome)
    print(pd.DataFrame(linhas).round(3).to_string(index=False))
    conf = [
        trustworthiness(
            X, TSNE(2, init="random", random_state=s).fit_transform(X), n_neighbors=10
        )
        for s in range(5)
    ]
    print(
        f"Estabilidade do t-SNE (5 sementes): confiança média = {np.mean(conf):.3f}, desvio =
{np.std(conf):.4f}\n"
    )
plt.tight_layout()
plt.show()
```

SAÍDA

```
Dígitos
método  confiança  continuidade  KNN 2D
   PCA      0.830         0.951   0.603
Isomap      0.837         0.969   0.705
   LLE      0.925         0.970   0.911
 t-SNE      0.993         0.988   0.976
  UMAP      0.989         0.986   0.976
Estabilidade do t-SNE (5 sementes): confiança média = 0.993, desvio = 0.0005

Wine
método  confiança  continuidade  KNN 2D
   PCA      0.888         0.941   0.966
Isomap      0.880         0.942   0.955
   LLE      0.821         0.863   0.815
 t-SNE      0.956         0.953   0.961
  UMAP      0.951         0.949   0.933
Estabilidade do t-SNE (5 sementes): confiança média = 0.958, desvio = 0.0042
```

EXEMPLO Discussão dos resultados

No Wine, que é mais simples, o PCA já separa bem as classes e preserva bem a estrutura global. Nos dígitos, as técnicas não lineares (t-SNE, Isomap) revelam melhor os grupos. A confiança do t-SNE varia muito pouco entre sementes, mas a *posição* dos grupos no mapa muda, por isso as distâncias globais não devem ser interpretadas.

Capítulo 10 — Regras de Associação

Cesta de supermercado: encontrando regras de verdade

Critérios de avaliação

- Produtos mais frequentes e suporte calculados.
- Regras filtradas por lift, mantendo apenas regras simples.
- Precisão e revocação calculadas para vários limiares e interpretadas.
- Três combos justificados com suporte, confiança e lift.
- O texto discute causalidade, regras espúrias, itens muito frequentes e a necessidade de teste A/B.

```python
import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import fpgrowth, association_rules

PRODUTOS = [
    "pão",
    "manteiga",
    "leite",
    "café",
    "açúcar",
    "arroz",
    "feijão",
    "macarrão",
    "molho",
    "queijo",
    "presunto",
    "cerveja",
    "salgadinho",
    "fralda",
    "refrigerante",
    "sabão",
    "detergente",
    "esponja",
]
PLANTADAS = [
    ("pão", "manteiga"),
    ("café", "açúcar"),
    ("macarrão", "molho"),
    ("cerveja", "salgadinho"),
    ("sabão", "detergente"),
]


def gerar_mercado(n=12000, seed=0):
    rng = np.random.default_rng(seed)
    p = rng.uniform(0.04, 0.35, len(PRODUTOS))
    M = rng.random((n, len(PRODUTOS))) < p
    ix = {nome: i for i, nome in enumerate(PRODUTOS)}
    for a, b in PLANTADAS:
        tem = M[:, ix[a]]
        M[tem, ix[b]] |= rng.random(tem.sum()) < 0.55
```

```
    return pd.DataFrame(M, columns=PRODUTOS)


df = gerar_mercado()
print("Mais frequentes:", df.mean().sort_values(ascending=False).head(5).round(3).to_dict())

freq = fpgrowth(df, min_support=0.02, use_colnames=True)
todas = association_rules(freq, metric="confidence", min_threshold=0.3)
print("Regras (confiança ≥ 0,3):", len(todas))

simples = todas[(todas.antecedents.apply(len) == 1) & (todas.consequents.apply(len) == 1)]
planta = {frozenset(p) for p in PLANTADAS}
for lift_min in [1.2, 1.5, 2.0]:
    r = simples[simples.lift >= lift_min]
    pares = {frozenset(list(a) + list(c)) for a, c in zip(r.antecedents, r.consequents)}
    acertos = pares & planta
    prec = len(acertos) / len(pares) if pares else 0
    rev = len(acertos) / len(planta)
    print(
        f"lift ≥ {lift_min}: pares distintos = {len(pares):2d}  precisão = {prec:.2f}  revocação =
{rev:.2f}"
    )

melhores = simples[simples.lift >= 1.5].sort_values("lift", ascending=False)
print(
    melhores[["antecedents", "consequents", "support", "confidence", "lift"]]
    .head(6)
    .round(2)
    .to_string(index=False)
)
```

SAÍDA

```
Mais frequentes: {'detergente': 0.344, 'queijo': 0.333, 'molho': 0.327, 'arroz': 0.323, 'açúcar':
0.316}
Regras (confiança ≥ 0,3): 499
lift ≥ 1.2: pares distintos =  5  precisão = 1.00  revocação = 1.00
lift ≥ 1.5: pares distintos =  5  precisão = 1.00  revocação = 1.00
lift ≥ 2.0: pares distintos =  4  precisão = 1.00  revocação = 0.80
          antecedents             consequents  support  confidence  lift
     frozenset({pão})   frozenset({manteiga})     0.14        0.60  2.52
frozenset({manteiga})        frozenset({pão})     0.14        0.59  2.52
 frozenset({cerveja}) frozenset({salgadinho})     0.03        0.69  2.19
    frozenset({café})     frozenset({açúcar})     0.03        0.68  2.16
   frozenset({sabão}) frozenset({detergente})     0.07        0.70  2.04
frozenset({macarrão})      frozenset({molho})     0.17        0.64  1.94
```

EXEMPLO **Discussão dos resultados**

Neste conjunto simulado, com lift ≥ 1,2 e com lift ≥ 1,5, o método recupera as cinco regras plantadas sem nenhum par espúrio (precisão e revocação iguais a 1,0). Com lift ≥ 2,0, perde-se uma regra plantada (revocação de 0,8). Em dados reais não há gabarito, e a precisão tende a ser bem menor: o limiar de lift deve ser escolhido pelo custo de agir sobre uma regra falsa e validado com teste A/B.

Capítulo 11 — Detecção de Anomalias

Detecção de fraude em transações de cartão

Critérios de avaliação

- Transformações adequadas (logaritmo, hora circular, padronização) e treino sem rótulos.
- Quatro detectores avaliados com AUC-ROC e AUPRC, comparados com a linha de base.
- Escore combinado construído e avaliado.
- Limiar por capacidade e por custo, com precisão e revocação interpretadas.
- Explicação dos alertas por atributos de maior desvio robusto.
- Parágrafo sobre riscos: viés, deriva, adversários, impacto sobre clientes.

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.mixture import GaussianMixture
from sklearn.svm import OneClassSVM
from sklearn.metrics import roc_auc_score, average_precision_score


def gerar_transacoes(n=40000, taxa_fraude=0.01, seed=0):
    rng = np.random.default_rng(seed)
    fraude = rng.random(n) < taxa_fraude
    valor = np.where(fraude, np.exp(rng.normal(5.8, 0.9, n)), np.exp(rng.normal(3.9, 0.8, n)))
    hora = np.where(
        fraude,
        rng.choice([0, 1, 2, 3, 4, 23], n),
        rng.choice(24, n, p=np.r_[np.full(6, 0.01), np.full(18, 0.94 / 18)]),
    )
    distancia = np.where(fraude, rng.gamma(3, 120, n), rng.gamma(2, 6, n))
    n24h = np.where(fraude, rng.poisson(6, n), rng.poisson(1.5, n))
    minutos = np.where(fraude, rng.exponential(15, n), rng.exponential(400, n))
    exterior = np.where(fraude, rng.random(n) < 0.35, rng.random(n) < 0.03).astype(int)
    df = pd.DataFrame(
        {
            "valor": valor,
            "hora": hora,
            "distancia_km": distancia,
            "n_24h": n24h,
            "min_desde_ultima": minutos,
            "exterior": exterior,
        }
    )
    return df, fraude.astype(int)


df, y = gerar_transacoes()
F = pd.DataFrame(
    {
```

```
        "log_valor": np.log1p(df.valor),
        "log_dist": np.log1p(df.distancia_km),
        "log_min": np.log1p(df.min_desde_ultima),
        "n_24h": df.n_24h,
        "exterior": df.exterior,
        "hora_sen": np.sin(2 * np.pi * df.hora / 24),
        "hora_cos": np.cos(2 * np.pi * df.hora / 24),
    }
)
Ftr, Fte, ytr, yte = train_test_split(F, y, test_size=0.5, random_state=0, stratify=y)
esc = StandardScaler().fit(Ftr)
Xtr, Xte = esc.transform(Ftr), esc.transform(Fte)
print("Fraudes no teste:", int(yte.sum()), "de", len(yte))

rng = np.random.default_rng(0)
sub = Xtr[rng.choice(len(Xtr), 5000, replace=False)]
for k in [30, 100, 300]:
    e_k = -LocalOutlierFactor(n_neighbors=k, novelty=True).fit(Xtr).score_samples(Xte)
    print(f"LOF n_neighbors = {k:3d}  AUPRC = {average_precision_score(yte, e_k):.3f}")
escores = {
    "Isolation Forest": -IsolationForest(n_estimators=200, random_state=0)
    .fit(Xtr)
    .score_samples(Xte),
    "LOF (k=300)": -LocalOutlierFactor(n_neighbors=300, novelty=True)
    .fit(Xtr)
    .score_samples(Xte),
    "GMM": -GaussianMixture(4, random_state=0).fit(Xtr).score_samples(Xte),
    "One-Class SVM": -OneClassSVM(gamma=0.05, nu=0.05).fit(sub).score_samples(Xte),
}
escores["Combinado (ranks)"] = np.mean(
    [pd.Series(e).rank().values for e in escores.values()], axis=0
)
linhas = [
    {"detector": n, "AUC-ROC": roc_auc_score(yte, e), "AUPRC": average_precision_score(yte, e)}
    for n, e in escores.items()
]
print(
    pd.DataFrame(linhas).round(3).to_string(index=False),
    "\nLinha de base AUPRC:",
    round(yte.mean(), 4),
)

e = escores["Combinado (ranks)"]
n_alertas = int(round(50 * len(yte) / 10000))
corte = np.sort(e)[::-1][n_alertas - 1]
alerta = e >= corte
print(
    f"\nCapacidade: {n_alertas} alertas | fraudes capturadas = {int(yte[alerta].sum())} | precisão =
{yte[alerta].mean():.2f} | revocação = {yte[alerta].sum() / yte.sum():.2f}"
)

med = np.median(Xte, axis=0)
mad = np.median(np.abs(Xte - med), axis=0)
escala = np.where(
    mad > 0, mad / 0.6745, Xte.std(axis=0)
)  # variáveis binárias têm MAD = 0: usa o desvio padrão
top5 = np.argsort(-e)[:5]
for i in top5:
```

```
    z = (Xte[i] - med) / escala
    cols = np.argsort(-np.abs(z))[:3]
    print(
        "alerta",
        i,
        "-> motivos:",
        [(F.columns[c], round(float(z[c]), 1)) for c in cols],
        "| fraude real?",
        bool(yte[i]),
    )

ordem = np.argsort(-e)
fn_custo, fp_custo = 100, 1
custos = [
    fn_custo * (yte.sum() - yte[ordem[:k]].sum()) + fp_custo * (k - yte[ordem[:k]].sum())
    for k in range(0, 3001)
]
k_otimo = int(np.argmin(custos))
print(
    f"Limiar por custo (FN = 100 × FP): {k_otimo} alertas | fraudes capturadas =
{int(yte[ordem[:k_otimo]].sum())}"
)
```

SAÍDA

```
Fraudes no teste: 198 de 20000
LOF n_neighbors =  30  AUPRC = 0.015
LOF n_neighbors = 100  AUPRC = 0.051
LOF n_neighbors = 300  AUPRC = 0.913
         detector  AUC-ROC  AUPRC
 Isolation Forest    0.998  0.894
      LOF (k=300)    0.999  0.913
              GMM    0.995  0.774
    One-Class SVM    0.999  0.943
Combinado (ranks)    1.000  0.980
Linha de base AUPRC: 0.0099

Capacidade: 100 alertas | fraudes capturadas = 99 | precisão = 0.99 | revocação = 0.50
alerta 13898 -> motivos: [('n_24h', 8.1), ('log_dist', 5.4), ('log_min', -3.7)] | fraude real? True
alerta 1350 -> motivos: [('n_24h', 5.4), ('log_valor', 5.2), ('log_dist', 3.8)] | fraude real? True
alerta 13476 -> motivos: [('n_24h', 8.1), ('log_dist', 4.4), ('log_min', -2.6)] | fraude real? True
alerta 8378 -> motivos: [('n_24h', 7.4), ('log_dist', 5.7), ('log_valor', 2.7)] | fraude real? True
alerta 10599 -> motivos: [('n_24h', 8.1), ('exterior', 5.7), ('log_dist', 3.9)] | fraude real? True
Limiar por custo (FN = 100 × FP): 290 alertas | fraudes capturadas = 197
```

EXEMPLO Discussão dos resultados

Um achado importante: o LOF com n_neighbors = 30 falha (AUPRC de cerca de 0,015, pior que a linha de base de 1%), pois o treino contém cerca de 200 fraudes parecidas entre si, que formam um grupo denso: com 30 vizinhos, cada fraude só enxerga outras fraudes e parece normal. Com n_neighbors = 300 (maior que o grupo de fraudes), o LOF vai muito bem (AUPRC de cerca de 0,91). O escore combinado por postos atinge AUPRC de cerca de 0,98. Com a capacidade de 100 alertas no teste (50 por 10.000), são capturadas 99 das 198 fraudes (precisão de 0,99 e revocação de 0,50): a revocação fica limitada pela capacidade da equipe. O limiar por custo (FN = 100 × FP) pede cerca de 290 alertas e captura 197 das 198 fraudes. Atenção: nesta simulação, escolhi hiperparâmetros (k do LOF, gamma e nu do One-Class SVM) usando os rótulos como validação. Em produção, use uma pequena amostra rotulada ou critérios de estabilidade, e monitore a deriva.

Capítulo 12 — Aprendizado Profundo Não Supervisionado

Que representação ajuda mais a agrupar?

Critérios de avaliação

- Todas as representações foram ajustadas no treino e aplicadas ao teste (sem vazamento).
- O código do autoencoder foi extraído corretamente da rede.
- Agrupamento e avaliação supervisionada comparados em uma tabela.
- A discussão relaciona o resultado ao tamanho do problema e a alternativas (outras arquiteturas, mais dados, PyTorch).
- A discussão evita concluir que "profundo é sempre melhor".

```python
import numpy as np
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neural_network import MLPRegressor, BernoulliRBM
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score, homogeneity_score

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

relu = lambda a: np.maximum(a, 0)
ae = MLPRegressor(hidden_layer_sizes=(64, 16, 64), max_iter=1500, random_state=0).fit(Xtr, Xtr)


def codigo(A):
    W, b = ae.coefs_, ae.intercepts_
    return relu(relu(A @ W[0] + b[0]) @ W[1] + b[1])


pca = PCA(16, random_state=0).fit(Xtr)
rbm = BernoulliRBM(n_components=64, learning_rate=0.05, n_iter=30, random_state=0).fit(Xtr)

reps = {
    "Pixels": (lambda A: A),
    "PCA (16)": pca.transform,
    "Autoencoder (16)": codigo,
    "RBM (64)": rbm.transform,
}
linhas = []
for nome, f in reps.items():
    Ztr, Zte = f(Xtr), f(Xte)
    esc = StandardScaler().fit(Ztr)
    Ztr, Zte = esc.transform(Ztr), esc.transform(Zte)
    km = KMeans(10, n_init=10, random_state=0).fit(Ztr)
    r = km.predict(Zte)
```

```
    acc = LogisticRegression(max_iter=3000).fit(Ztr, ytr).score(Zte, yte)
    linhas.append(
        {
            "representação": nome,
            "dim": Ztr.shape[1],
            "ARI": adjusted_rand_score(yte, r),
            "NMI": normalized_mutual_info_score(yte, r),
            "homog.": homogeneity_score(yte, r),
            "acurácia (logist.)": acc,
        }
    )
print(pd.DataFrame(linhas).round(3).to_string(index=False))
```

SAÍDA

```
   representação  dim   ARI   NMI  homog.  acurácia (logist.)
          Pixels   64 0.487 0.665   0.627               0.972
       PCA (16)    16 0.636 0.737   0.733               0.959
Autoencoder (16)   16 0.529 0.657   0.654               0.933
       RBM (64)    64 0.573 0.692   0.682               0.937
```

EXEMPLO **Discussão dos resultados**

Neste teste, o PCA com 16 componentes foi a melhor representação para agrupar (ARI de cerca de 0,64, contra 0,49 dos pixels, 0,53 do autoencoder e 0,57 da RBM). Para classificar com regressão logística, os próprios pixels foram os melhores (acurácia de cerca de 0,97). Ou seja: a representação aprendida não é automaticamente melhor. Cada uma tem custo de treino, hiperparâmetros e variabilidade entre sementes. A lição do capítulo se confirma: **compare sempre com uma linha de base simples**. Para imagens maiores e coloridas, redes convolucionais e o aprendizado auto-supervisionado costumam fazer grande diferença.

Capítulo 13 — Projeto Final

Laboratório de Dígitos: do dado bruto à decisão

Critérios de avaliação

- Preparação adequada e justificativa do número de componentes.
- Cinco métodos de agrupamento comparados por critérios internos, estabilidade e (apenas no final) externos.
- Projeções comparadas por confiança, com gráfico interpretado com cuidado.
- Autoencoder comparado a uma linha de base (PCA) sem concluir que "profundo é melhor" sem evidência.
- Imagens estranhas descritas e dígitos predominantes nos alertas identificados.
- Regras de traços interpretadas como coocorrência, não causalidade.
- Comparação justa entre rotular por grupos e rotular ao acaso, com repetições.
- Relatório com resumo executivo, limitações, riscos éticos e recomendação clara.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import KMeans, AgglomerativeClustering, SpectralClustering, HDBSCAN
from sklearn.mixture import GaussianMixture
from sklearn.manifold import Isomap, TSNE, trustworthiness
from sklearn.neural_network import MLPRegressor
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import silhouette_score, adjusted_rand_score, normalized_mutual_info_score
from mlxtend.frequent_patterns import fpgrowth, association_rules

# ------------------------------------------------------------------ Etapa 1
dig = load_digits()
X = dig.data / 16.0
y = dig.target  # só para avaliar (e na Etapa 7, como "oráculo" de rotulagem)


def hopkins(A, m=100, seed=42):
    r = np.random.default_rng(seed)
    n, d = A.shape
    nn = NearestNeighbors(n_neighbors=2).fit(A)
    idx = r.choice(n, m, replace=False)
    w = nn.kneighbors(A[idx], 2)[0][:, 1]
    U = r.uniform(A.min(0), A.max(0), (m, d))
    u = nn.kneighbors(U, 1)[0][:, 0]
    return u.sum() / (u.sum() + w.sum())


pca_full = PCA().fit(X)
n95 = int(np.argmax(pca_full.explained_variance_ratio_.cumsum() >= 0.95)) + 1
```

```
Z = PCA(30, random_state=0).fit_transform(X)
print(f"[1] Hopkins = {hopkins(X):.2f} | componentes para 95% da variância = {n95}")


# ---------------------------------------------------------------- Etapa 2
def estabilidade(fab, A, pares=8, seed=0):
    rng = np.random.default_rng(seed)
    n = len(A)
    sc = []
    for _ in range(pares):
        i1 = rng.choice(n, int(0.7 * n), replace=False)
        i2 = rng.choice(n, int(0.7 * n), replace=False)
        v1 = (
            NearestNeighbors(n_neighbors=1)
            .fit(A[i1])
            .kneighbors(A, return_distance=False)
            .ravel()
        )
        v2 = (
            NearestNeighbors(n_neighbors=1)
            .fit(A[i2])
            .kneighbors(A, return_distance=False)
            .ravel()
        )
        r1 = fab().fit_predict(A[i1])
        r2 = fab().fit_predict(A[i2])
        sc.append(adjusted_rand_score(r1[v1], r2[v2]))
    return float(np.mean(sc))


metodos = {
    "K-means": lambda: KMeans(10, n_init=10, random_state=0),
    "Ward": lambda: AgglomerativeClustering(10, linkage="ward"),
    "GMM (diag)": lambda: GaussianMixture(10, covariance_type="diag", n_init=3, random_state=0),
    "Espectral": lambda: SpectralClustering(
        10, affinity="nearest_neighbors", n_neighbors=15, random_state=0
    ),
    "HDBSCAN": lambda: HDBSCAN(min_cluster_size=20),
}
linhas = []
rotulos = {}
for nome, fab in metodos.items():
    r = fab().fit_predict(Z)
    rotulos[nome] = r
    ok = r >= 0
    sil = silhouette_score(Z[ok], r[ok]) if len(set(r[ok])) > 1 else np.nan
    linhas.append(
        {
            "método": nome,
            "grupos": len(set(r[ok])),
            "ruído": int((~ok).sum()),
            "silhueta": sil,
            "estabilidade": estabilidade(fab, Z),
            "ARI": adjusted_rand_score(y[ok], r[ok]),
            "NMI": normalized_mutual_info_score(y[ok], r[ok]),
        }
    )
tab = pd.DataFrame(linhas)
```

```
print("[2] Comparação de agrupamentos (ARI e NMI só para conferência):")
print(tab.round(3).to_string(index=False))
dez = tab[tab.grupos == 10].copy()  # escolha SEM usar rótulos: silhueta + estabilidade
dez["postos"] = dez.silhueta.rank(ascending=False) + dez.estabilidade.rank(ascending=False)
melhor = dez.loc[dez.postos.idxmin(), "método"]
print("    Escolhido por silhueta e estabilidade (sem rótulos):", melhor)

# ---------------------------------------------------------------- Etapa 3
proj = {
    "PCA": PCA(2).fit_transform(X),
    "Isomap": Isomap(n_neighbors=10).fit_transform(X),
    "t-SNE": TSNE(2, init="pca", random_state=0).fit_transform(X),
}
print(
    "[3] Confiança (k=10):",
    {k: round(float(trustworthiness(X, v, n_neighbors=10)), 3) for k, v in proj.items()},
)
plt.scatter(proj["t-SNE"][:, 0], proj["t-SNE"][:, 1], c=rotulos[melhor], cmap="tab10", s=6)
plt.title(f"t-SNE colorido por {melhor}")
plt.show()

# ---------------------------------------------------------------- Etapa 4
Xtr_ae = X
ae = MLPRegressor(hidden_layer_sizes=(64, 16, 64), max_iter=1500, random_state=0).fit(X, X)
relu = lambda a: np.maximum(a, 0)
W, b = ae.coefs_, ae.intercepts_
codigo = relu(relu(X @ W[0] + b[0]) @ W[1] + b[1])
pca16 = PCA(16, random_state=0).fit_transform(X)
for nome, R in [("PCA (16)", pca16), ("Autoencoder (16)", codigo)]:
    r = KMeans(10, n_init=10, random_state=0).fit_predict(StandardScaler().fit_transform(R))
    print(
        f"[4] {nome:17s} K-means: ARI = {adjusted_rand_score(y, r):.3f}  NMI =
{normalized_mutual_info_score(y, r):.3f}"
    )

# ---------------------------------------------------------------- Etapa 5
Z20 = PCA(20, random_state=0).fit_transform(X)
s_if = -IsolationForest(n_estimators=300, random_state=0).fit(Z20).score_samples(Z20)
s_lof = -LocalOutlierFactor(n_neighbors=20).fit(Z20).negative_outlier_factor_
comb = (pd.Series(s_if).rank() + pd.Series(s_lof).rank()).values / 2
topo = np.argsort(-comb)
print(
    "[5] Dígitos entre os 30 maiores alertas:",
    np.bincount(y[topo[:30]], minlength=10),
    "| índices dos 12 primeiros:",
    topo[:12].tolist(),
)
fig, ax = plt.subplots(2, 6, figsize=(9, 3.4))
for a, i in zip(ax.ravel(), topo[:12]):
    a.imshow(dig.images[i], cmap="gray_r")
    a.set_title(f"#{i} (dígito {y[i]})", fontsize=8)
    a.axis("off")
plt.suptitle("As 12 imagens mais estranhas")
plt.show()

# ---------------------------------------------------------------- Etapa 6
blocos = X.reshape(-1, 4, 2, 4, 2).mean(axis=(2, 4)).reshape(len(X), 16) > 0.5
itens = {}
```

```
for r in range(4):
    for c in range(4):
        itens[f"ativo_{r}{c}"] = blocos[:, 4 * r + c]
        itens[f"vazio_{r}{c}"] = ~blocos[:, 4 * r + c]
for d in range(10):
    itens[f"digito_{d}"] = y == d
df_i = pd.DataFrame(itens)
freq = fpgrowth(df_i, min_support=0.03, use_colnames=True, max_len=4)
regras = association_rules(freq, metric="lift", min_threshold=1.5)
so_digito = regras.consequents.apply(
    lambda s: len(s) == 1 and next(iter(s)).startswith("digito_")
)
so_blocos = regras.antecedents.apply(lambda s: all(not i.startswith("digito") for i in s))
r2 = regras[so_digito & so_blocos & (regras.confidence >= 0.6)].sort_values(
    ["lift", "confidence"], ascending=False
)
print(
    f"[6] {len(r2)} regras blocos → dígito (confiança ≥ 0,6, lift ≥ 1,5). Melhor regra por dígito:"
)
for d in range(10):
    sub = r2[r2.consequents.apply(lambda s: f"digito_{d}" in s)].head(1)
    for _, r in sub.iterrows():
        print(
            f"    dígito {d}: {sorted(r.antecedents)}  conf = {r.confidence:.2f}  lift = {r.lift:.1f}"
        )
print(
    "    Fração de imagens com bloco ativo nas colunas das bordas (0 e 3):",
    round(float(blocos.reshape(-1, 4, 4)[:, :, [0, 3]].mean()), 3),
)

# ------------------------------------------------------------------ Etapa 7
km30 = KMeans(30, n_init=10, random_state=0).fit(Z)
d2c = np.linalg.norm(Z[:, None, :] - km30.cluster_centers_[None], axis=2)
representantes = d2c.argmin(axis=0)  # imagem mais próxima de cada centro
rot_grupo = {g: y[representantes[g]] for g in range(30)}  # "oráculo" rotula só 30 imagens
pred_A = np.array([rot_grupo[g] for g in km30.labels_])
acc_A = (pred_A == y).mean()
accs_B = []
for s in range(20):
    rng = np.random.default_rng(s)
    idx = rng.choice(len(X), 30, replace=False)
    if len(set(y[idx])) < 2:
        continue
    accs_B.append(LogisticRegression(max_iter=2000).fit(X[idx], y[idx]).score(X, y))
print(
    f"[7] Acurácia com 30 rótulos: agrupar e rotular = {acc_A:.3f} | amostra aleatória + regressão
logística = {np.mean(accs_B):.3f} ± {np.std(accs_B):.3f}"
)
```

SAÍDA

```
[1] Hopkins = 0.74 | componentes para 95% da variância = 29
[2] Comparação de agrupamentos (ARI e NMI só para conferência):
    método  grupos  ruído  silhueta  estabilidade   ARI   NMI
   K-means      10      0     0.194         0.792 0.666 0.740
      Ward      10      0     0.188         0.807 0.721 0.810
GMM (diag)      10      0     0.159         0.614 0.585 0.751
 Espectral      10      0     0.193         0.885 0.758 0.857
```

```
   HDBSCAN       7    873     0.300         0.657 0.870 0.935
    Escolhido por silhueta e estabilidade (sem rótulos): Espectral
[3] Confiança (k=10): {'PCA': 0.83, 'Isomap': 0.837, 't-SNE': 0.993}
[4] PCA (16)          K-means: ARI = 0.691  NMI = 0.758
[4] Autoencoder (16)  K-means: ARI = 0.438  NMI = 0.574
[5] Dígitos entre os 30 maiores alertas: [1 3 4 1 3 5 1 7 2 3] | índices dos 12 primeiros: [1113, 689,
1595, 1274, 12…
[6] 25 regras blocos → dígito (confiança ≥ 0,6, lift ≥ 1,5). Melhor regra por dígito:
    dígito 4: ['ativo_22', 'vazio_02', 'vazio_32']  conf = 0.77  lift = 7.6
    dígito 5: ['ativo_31', 'vazio_12', 'vazio_32']  conf = 0.61  lift = 6.0
    dígito 7: ['ativo_21', 'vazio_11', 'vazio_32']  conf = 0.84  lift = 8.4
    Fração de imagens com bloco ativo nas colunas das bordas (0 e 3): 0.006
[7] Acurácia com 30 rótulos: agrupar e rotular = 0.929 | amostra aleatória + regressão logística =
0.663 ± 0.073
```

EXEMPLO Discussão dos resultados

O resultado exato depende do computador e das versões das bibliotecas. Nesta execução: (i) a estatística de Hopkins é alta (cerca de 0,74) e são necessárias 29 componentes para reter 95% da variância; (ii) o agrupamento espectral é o mais estável (cerca de 0,89) e tem ARI de cerca de 0,76, seguido por Ward (ARI de cerca de 0,72); o GMM é o menos estável; o HDBSCAN encontra 7 grupos e marca 873 imagens como ruído, mas os grupos que forma são muito puros (ARI de cerca de 0,87 só nos pontos não ruidosos), o que mostra que ele troca cobertura por pureza; (iii) o t-SNE tem a maior confiança (cerca de 0,99, contra cerca de 0,83 do PCA e do Isomap); (iv) o autoencoder (ARI de cerca de 0,44) é pior que o PCA de 16 componentes (cerca de 0,69); (v) com apenas 30 rótulos, "agrupar e rotular" alcança acurácia de cerca de 0,93, contra cerca de 0,66 da amostra aleatória com regressão logística, porque cada rótulo é propagado a dezenas de imagens parecidas. (vi) com blocos de 2×2 pixels, só se encontram regras confiáveis para os dígitos 4, 5 e 7, e os blocos das colunas das bordas quase nunca ficam ativos (cerca de 0,6% das vezes), por isso quase não aparecem nas regras. Atenção: no passo (v), escolher a imagem mais próxima do centro para rotular depende de um "oráculo" humano; o resultado depende da qualidade dos grupos.

Referências

Lista de artigos de periódicos e conferências de referência, além de livros reconhecidos, citados ao longo do texto.

AGGARWAL, C. C. Data Mining: The Textbook. Cham: Springer, 2015.

AGGARWAL, C. C.; HINNEBURG, A.; KEIM, D. A. On the surprising behavior of distance metrics in high dimensional space. In: International Conference on Database Theory (ICDT), 2001. Lecture Notes in Computer Science, v. 1973, p. 420-434.

AGRAWAL, R.; IMIELINSKI, T.; SWAMI, A. Mining association rules between sets of items in large databases. In: ACM SIGMOD International Conference on Management of Data, 1993. p. 207-216.

AGRAWAL, R.; SRIKANT, R. Fast algorithms for mining association rules. In: International Conference on Very Large Data Bases (VLDB), 1994. p. 487-499.

AGRAWAL, R.; SRIKANT, R. Mining sequential patterns. In: IEEE International Conference on Data Engineering (ICDE), 1995. p. 3-14.

AKAIKE, H. A new look at the statistical model identification. IEEE Transactions on Automatic Control, v. 19, n. 6, p. 716-723, 1974.

ANKERST, M.; BREUNIG, M. M.; KRIEGEL, H.-P.; SANDER, J. OPTICS: ordering points to identify the clustering structure. In: ACM SIGMOD International Conference on Management of Data, 1999. p. 49-60.

ARJOVSKY, M.; CHINTALA, S.; BOTTOU, L. Wasserstein generative adversarial networks. In: International Conference on Machine Learning (ICML), 2017. PMLR, v. 70, p. 214-223.

ARTHUR, D.; VASSILVITSKII, S. k-means++: the advantages of careful seeding. In: ACM-SIAM Symposium on Discrete Algorithms (SODA), 2007. p. 1027-1035.

BALDI, P.; HORNIK, K. Neural networks and principal component analysis: learning from examples without local minima. Neural Networks, v. 2, n. 1, p. 53-58, 1989.

BANERJEE, A.; DAVE, R. N. Validating clusters using the Hopkins statistic. In: IEEE International Conference on Fuzzy Systems, 2004. p. 149-153.

BECHT, E. et al. Dimensionality reduction for visualizing single-cell data using UMAP. Nature Biotechnology, v. 37, p. 38-44, 2019.

BELKIN, M.; NIYOGI, P. Laplacian eigenmaps for dimensionality reduction and data representation. Neural Computation, v. 15, n. 6, p. 1373-1396, 2003.

BELLMAN, R. Dynamic Programming. Princeton: Princeton University Press, 1957.

BEN-HUR, A.; ELISSEEFF, A.; GUYON, I. A stability based method for discovering structure in clustered data. In: Pacific Symposium on Biocomputing, v. 7, p. 6-17, 2002.

BENGIO, Y.; COURVILLE, A.; VINCENT, P. Representation learning: a review and new perspectives. IEEE Transactions on Pattern Analysis and Machine Intelligence, v. 35, n. 8, p. 1798-1828, 2013.

BEYER, K.; GOLDSTEIN, J.; RAMAKRISHNAN, R.; SHAFT, U. When is "nearest neighbor" meaningful? In: International Conference on Database Theory (ICDT), 1999. Lecture Notes in Computer Science, v. 1540, p. 217-235.

BEZDEK, J. C. Pattern Recognition with Fuzzy Objective Function Algorithms. New York: Plenum Press, 1981.

BISHOP, C. M. Pattern Recognition and Machine Learning. New York: Springer, 2006.

BREUNIG, M. M.; KRIEGEL, H.-P.; NG, R. T.; SANDER, J. LOF: identifying density-based local outliers. In: ACM SIGMOD International Conference on Management of Data, 2000. p. 93-104.

BRIN, S.; MOTWANI, R.; ULLMAN, J. D.; TSUR, S. Dynamic itemset counting and implication rules for market basket data. ACM SIGMOD Record, v. 26, n. 2, p. 255-264, 1997.

CALINSKI, T.; HARABASZ, J. A dendrite method for cluster analysis. Communications in Statistics, v. 3, n. 1, p. 1-27, 1974.

CAMPELLO, R. J. G. B.; MOULAVI, D.; SANDER, J. Density-based clustering based on hierarchical density estimates. In: Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD), 2013. p. 160-172.

CARON, M. et al. Emerging properties in self-supervised vision transformers. In: IEEE/CVF International Conference on Computer Vision (ICCV), 2021. p. 9650-9660.

CATTELL, R. B. The scree test for the number of factors. Multivariate Behavioral Research, v. 1, n. 2, p. 245-276, 1966.

CHANDOLA, V.; BANERJEE, A.; KUMAR, V. Anomaly detection: a survey. ACM Computing Surveys, v. 41, n. 3, art. 15, 2009.

CHEN, T.; KORNBLITH, S.; NOROUZI, M.; HINTON, G. A simple framework for contrastive learning of visual representations. In: International Conference on Machine Learning (ICML), 2020. PMLR, v. 119, p. 1597-1607.

COIFMAN, R. R.; LAFON, S. Diffusion maps. Applied and Computational Harmonic Analysis, v. 21, n. 1, p. 5-30, 2006.

COMANICIU, D.; MEER, P. Mean shift: a robust approach toward feature space analysis. IEEE Transactions on Pattern Analysis and Machine Intelligence, v. 24, n. 5, p. 603-619, 2002.

COX, T. F.; COX, M. A. A. Multidimensional Scaling. 2. ed. Boca Raton: Chapman & Hall/CRC, 2001.

DAVIES, D. L.; BOULDIN, D. W. A cluster separation measure. IEEE Transactions on Pattern Analysis and Machine Intelligence, v. PAMI-1, n. 2, p. 224-227, 1979.

DEMPSTER, A. P.; LAIRD, N. M.; RUBIN, D. B. Maximum likelihood from incomplete data via the EM algorithm. Journal of the Royal Statistical Society: Series B, v. 39, n. 1, p. 1-38, 1977.

DEVLIN, J.; CHANG, M.-W.; LEE, K.; TOUTANOVA, K. BERT: pre-training of deep bidirectional transformers for language understanding. In: Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2019. p. 4171-4186.

DUNN, J. C. Well-separated clusters and optimal fuzzy partitions. Journal of Cybernetics, v. 4, n. 1, p. 95-104, 1974.

ECKART, C.; YOUNG, G. The approximation of one matrix by another of lower rank. Psychometrika, v. 1, n. 3, p. 211-218, 1936.

EISEN, M. B.; SPELLMAN, P. T.; BROWN, P. O.; BOTSTEIN, D. Cluster analysis and display of genome-wide expression patterns. Proceedings of the National Academy of Sciences, v. 95, n. 25, p. 14863-14868, 1998.

ESTER, M.; KRIEGEL, H.-P.; SANDER, J.; XU, X. A density-based algorithm for discovering clusters in large spatial databases with noise. In: International Conference on Knowledge Discovery and Data Mining (KDD), 1996. p. 226-231.

GABRIEL, K. R. The biplot graphic display of matrices with application to principal component analysis. Biometrika, v. 58, n. 3, p. 453-467, 1971.

GENG, L.; HAMILTON, H. J. Interestingness measures for data mining: a survey. ACM Computing Surveys, v. 38, n. 3, art. 9, 2006.

GIDARIS, S.; SINGH, P.; KOMODAKIS, N. Unsupervised representation learning by predicting image rotations. In: International Conference on Learning Representations (ICLR), 2018.

GOODFELLOW, I. et al. Generative adversarial nets. In: Advances in Neural Information Processing Systems (NeurIPS), 2014. v. 27.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. Deep Learning. Cambridge: MIT Press, 2016.

GOWER, J. C. A general coefficient of similarity and some of its properties. Biometrics, v. 27, n. 4, p. 857-871, 1971.

GRILL, J.-B. et al. Bootstrap your own latent: a new approach to self-supervised learning. In: Advances in Neural Information Processing Systems (NeurIPS), 2020. v. 33, p. 21271-21284.

HALKIDI, M.; BATISTAKIS, Y.; VAZIRGIANNIS, M. On clustering validation techniques. Journal of Intelligent Information Systems, v. 17, n. 2-3, p. 107-145, 2001.

HALKO, N.; MARTINSSON, P. G.; TROPP, J. A. Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. SIAM Review, v. 53, n. 2, p. 217-288, 2011.

HAN, J.; PEI, J.; YIN, Y. Mining frequent patterns without candidate generation. In: ACM SIGMOD International Conference on Management of Data, 2000. p. 1-12.

HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. The Elements of Statistical Learning. 2. ed. New York: Springer, 2009.

HAWKINS, D. M. Identification of Outliers. London: Chapman and Hall, 1980.

HE, K. et al. Masked autoencoders are scalable vision learners. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022. p. 16000-16009.

HE, K. et al. Momentum contrast for unsupervised visual representation learning. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020. p. 9729-9738.

HEUSEL, M. et al. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In: Advances in Neural Information Processing Systems (NeurIPS), 2017. v. 30, p. 6626-6637.

HIGGINS, I. et al. beta-VAE: learning basic visual concepts with a constrained variational framework. In: International Conference on Learning Representations (ICLR), 2017.

HINTON, G. E.; SALAKHUTDINOV, R. R. Reducing the dimensionality of data with neural networks. Science, v. 313, n. 5786, p. 504-507, 2006.

HO, J.; JAIN, A.; ABBEEL, P. Denoising diffusion probabilistic models. In: Advances in Neural Information Processing Systems (NeurIPS), 2020. v. 33, p. 6840-6851.

HOPKINS, B.; SKELLAM, J. G. A new method for determining the type of distribution of plant individuals. Annals of Botany, v. 18, n. 70, p. 213-227, 1954.

HORN, J. L. A rationale and test for the number of factors in factor analysis. Psychometrika, v. 30, n. 2, p. 179-185, 1965.

HOTELLING, H. Analysis of a complex of statistical variables into principal components. Journal of Educational Psychology, v. 24, n. 6, p. 417-441, 1933.

HUANG, Z. Extensions to the k-means algorithm for clustering large data sets with categorical values. Data Mining and Knowledge Discovery, v. 2, n. 3, p. 283-304, 1998.

HUBERT, L.; ARABIE, P. Comparing partitions. Journal of Classification, v. 2, n. 1, p. 193-218, 1985.

HYVÄRINEN, A.; OJA, E. Independent component analysis: algorithms and applications. Neural Networks, v. 13, n. 4-5, p. 411-430, 2000.

IGLEWICZ, B.; HOAGLIN, D. C. How to Detect and Handle Outliers. Milwaukee: ASQC Quality Press, 1993.

JAIN, A. K. Data clustering: 50 years beyond K-means. Pattern Recognition Letters, v. 31, n. 8, p. 651-666, 2010.

JAIN, A. K.; DUBES, R. C. Algorithms for Clustering Data. Englewood Cliffs: Prentice Hall, 1988.

JAIN, A. K.; MURTY, M. N.; FLYNN, P. J. Data clustering: a review. ACM Computing Surveys, v. 31, n. 3, p. 264-323, 1999.

JOLLIFFE, I. T.; CADIMA, J. Principal component analysis: a review and recent developments. Philosophical Transactions of the Royal Society A, v. 374, n. 2065, art. 20150202, 2016.

KAISER, H. F. The application of electronic computers to factor analysis. Educational and Psychological Measurement, v. 20, n. 1, p. 141-151, 1960.

KARRAS, T.; LAINE, S.; AILA, T. A style-based generator architecture for generative adversarial networks. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019. p. 4401-4410.

KAUFMAN, L.; ROUSSEEUW, P. J. Finding Groups in Data: An Introduction to Cluster Analysis. New York: Wiley, 1990.

KINGMA, D. P.; WELLING, M. Auto-encoding variational Bayes. In: International Conference on Learning Representations (ICLR), 2014.

KOBAK, D.; BERENS, P. The art of using t-SNE for single-cell transcriptomics. Nature Communications, v. 10, art. 5416, 2019.

KOBAK, D.; LINDERMAN, G. C. Initialization is critical for preserving global data structure in both t-SNE and UMAP. Nature Biotechnology, v. 39, p. 156-157, 2021.

KRUSKAL, J. B. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. Psychometrika, v. 29, n. 1, p. 1-27, 1964.

LANCE, G. N.; WILLIAMS, W. T. A general theory of classificatory sorting strategies: 1. Hierarchical systems. The Computer Journal, v. 9, n. 4, p. 373-380, 1967.

LANGE, T.; ROTH, V.; BRAUN, M. L.; BUHMANN, J. M. Stability-based validation of clustering solutions. Neural Computation, v. 16, n. 6, p. 1299-1323, 2004.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. Nature, v. 521, p. 436-444, 2015.

LECUN, Y.; BOTTOU, L.; BENGIO, Y.; HAFFNER, P. Gradient-based learning applied to document recognition. Proceedings of the IEEE, v. 86, n. 11, p. 2278-2324, 1998.

LEE, D. D.; SEUNG, H. S. Learning the parts of objects by non-negative matrix factorization. Nature, v. 401, p. 788-791, 1999.

LI, Z.; ZHAO, Y.; BOTTA, N.; IONESCU, C.; HU, X. COPOD: copula-based outlier detection. In: IEEE International Conference on Data Mining (ICDM), 2020. p. 1118-1123.

LI, Z.; ZHAO, Y.; HU, X.; BOTTA, N.; IONESCU, C.; CHEN, G. H. ECOD: unsupervised outlier detection using empirical cumulative distribution functions. IEEE Transactions on Knowledge and Data Engineering, v. 35, n. 12, p. 12181-12193, 2023.

LINDERMAN, G. C. et al. Fast interpolation-based t-SNE for improved visualization of single-cell RNA-seq data. Nature Methods, v. 16, p. 243-245, 2019.

LIU, F. T.; TING, K. M.; ZHOU, Z.-H. Isolation forest. In: IEEE International Conference on Data Mining (ICDM), 2008. p. 413-422.

LLOYD, S. P. Least squares quantization in PCM. IEEE Transactions on Information Theory, v. 28, n. 2, p. 129-137, 1982.

LUNDBERG, S. M.; LEE, S.-I. A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems (NeurIPS), 2017. v. 30, p. 4765-4774.

MACQUEEN, J. Some methods for classification and analysis of multivariate observations. In: Berkeley Symposium on Mathematical Statistics and Probability, 5., 1967. v. 1, p. 281-297.

MAHALANOBIS, P. C. On the generalised distance in statistics. Proceedings of the National Institute of Sciences of India, v. 2, n. 1, p. 49-55, 1936.

MCINNES, L.; HEALY, J.; ASTELS, S. hdbscan: hierarchical density based clustering. Journal of Open Source Software, v. 2, n. 11, art. 205, 2017.

MCINNES, L.; HEALY, J.; MELVILLE, J. UMAP: uniform manifold approximation and projection for dimension reduction. arXiv:1802.03426, 2018.

MCLACHLAN, G.; PEEL, D. Finite Mixture Models. New York: Wiley, 2000.

MOON, K. R. et al. Visualizing structure and transitions in high-dimensional biological data. Nature Biotechnology, v. 37, p. 1482-1492, 2019.

MOULAVI, D.; JASKOWIAK, P. A.; CAMPELLO, R. J. G. B.; ZIMEK, A.; SANDER, J. Density-based clustering validation. In: SIAM International Conference on Data Mining (SDM), 2014. p. 839-847.

MURPHY, K. P. Machine Learning: A Probabilistic Perspective. Cambridge: MIT Press, 2012.

NG, A. Y.; JORDAN, M. I.; WEISS, Y. On spectral clustering: analysis and an algorithm. In: Advances in Neural Information Processing Systems (NIPS), 2001. v. 14, p. 849-856.

OLSHAUSEN, B. A.; FIELD, D. J. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. Nature, v. 381, p. 607-609, 1996.

OORD, A. van den; LI, Y.; VINYALS, O. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018.

PEARSON, K. On lines and planes of closest fit to systems of points in space. Philosophical Magazine, v. 2, n. 11, p. 559-572, 1901.

PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. Journal of Machine Learning Research, v. 12, p. 2825-2830, 2011.

RADFORD, A. et al. Learning transferable visual models from natural language supervision. In: International Conference on Machine Learning (ICML), 2021. PMLR, v. 139, p. 8748-8763.

RADFORD, A.; METZ, L.; CHINTALA, S. Unsupervised representation learning with deep convolutional generative adversarial networks. In: International Conference on Learning Representations (ICLR), 2016.

RAMASWAMY, S.; RASTOGI, R.; SHIM, K. Efficient algorithms for mining outliers from large data sets. In: ACM SIGMOD International Conference on Management of Data, 2000. p. 427-438.

ROSENBERG, A.; HIRSCHBERG, J. V-measure: a conditional entropy-based external cluster evaluation measure. In: Conference on Empirical Methods in Natural Language Processing (EMNLP-CoNLL), 2007. p. 410-420.

ROUSSEEUW, P. J. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, v. 20, p. 53-65, 1987.

ROWEIS, S. T.; SAUL, L. K. Nonlinear dimensionality reduction by locally linear embedding. Science, v. 290, n. 5500, p. 2323-2326, 2000.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. Nature, v. 323, p. 533-536, 1986.

SAITO, T.; REHMSMEIER, M. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE, v. 10, n. 3, art. e0118432, 2015.

SCHÖLKOPF, B.; PLATT, J. C.; SHAWE-TAYLOR, J.; SMOLA, A. J.; WILLIAMSON, R. C. Estimating the support of a high-dimensional distribution. Neural Computation, v. 13, n. 7, p. 1443-1471, 2001.

SCHÖLKOPF, B.; SMOLA, A.; MÜLLER, K.-R. Nonlinear component analysis as a kernel eigenvalue problem. Neural Computation, v. 10, n. 5, p. 1299-1319, 1998.

SCHUBERT, E.; SANDER, J.; ESTER, M.; KRIEGEL, H.-P.; XU, X. DBSCAN revisited, revisited: why and how you should (still) use DBSCAN. ACM Transactions on Database Systems, v. 42, n. 3, art. 19, 2017.

SCHWARZ, G. Estimating the dimension of a model. The Annals of Statistics, v. 6, n. 2, p. 461-464, 1978.

SCULLEY, D. Web-scale k-means clustering. In: International World Wide Web Conference (WWW), 2010. p. 1177-1178.

SHI, J.; MALIK, J. Normalized cuts and image segmentation. IEEE Transactions on Pattern Analysis and Machine Intelligence, v. 22, n. 8, p. 888-905, 2000.

SIFFER, A.; FOUQUE, P.-A.; TERMIER, A.; LARGOUËT, C. Anomaly detection in streams with extreme value theory. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2017. p. 1067-1075.

SOKAL, R. R.; MICHENER, C. D. A statistical method for evaluating systematic relationships. University of Kansas Science Bulletin, v. 38, p. 1409-1438, 1958.

SOKAL, R. R.; ROHLF, F. J. The comparison of dendrograms by objective methods. Taxon, v. 11, n. 2, p. 33-40, 1962.

SRIKANT, R.; AGRAWAL, R. Mining generalized association rules. In: International Conference on Very Large Data Bases (VLDB), 1995. p. 407-419.

SRIKANT, R.; AGRAWAL, R. Mining quantitative association rules in large relational tables. In: ACM SIGMOD International Conference on Management of Data, 1996. p. 1-12.

STREHL, A.; GHOSH, J. Cluster ensembles: a knowledge reuse framework for combining multiple partitions. Journal of Machine Learning Research, v. 3, p. 583-617, 2002.

STRUBELL, E.; GANESH, A.; MCCALLUM, A. Energy and policy considerations for deep learning in NLP. In: Annual Meeting of the Association for Computational Linguistics (ACL), 2019. p. 3645-3650.

TAN, P.-N.; KUMAR, V.; SRIVASTAVA, J. Selecting the right objective measure for association analysis. Information Systems, v. 29, n. 4, p. 293-313, 2004.

TENENBAUM, J. B.; DE SILVA, V.; LANGFORD, J. C. A global geometric framework for nonlinear dimensionality reduction. Science, v. 290, n. 5500, p. 2319-2323, 2000.

TIBSHIRANI, R.; WALTHER, G.; HASTIE, T. Estimating the number of clusters in a data set via the gap statistic. Journal of the Royal Statistical Society: Series B, v. 63, n. 2, p. 411-423, 2001.

TUKEY, J. W. Exploratory Data Analysis. Reading: Addison-Wesley, 1977.

VAN DER MAATEN, L.; HINTON, G. Visualizing data using t-SNE. Journal of Machine Learning Research, v. 9, p. 2579-2605, 2008.

VENNA, J.; KASKI, S. Local multidimensional scaling. Neural Networks, v. 19, n. 6-7, p. 889-899, 2006.

VINCENT, P. et al. Stacked denoising autoencoders: learning useful representations in a deep network with a local denoising criterion. Journal of Machine Learning Research, v. 11, p. 3371-3408, 2010.

VINCENT, P.; LAROCHELLE, H.; BENGIO, Y.; MANZAGOL, P.-A. Extracting and composing robust features with denoising autoencoders. In: International Conference on Machine Learning (ICML), 2008. p. 1096-1103.

VINH, N. X.; EPPS, J.; BAILEY, J. Information theoretic measures for clusterings comparison: variants, properties, normalization and correction for chance. Journal of Machine Learning Research, v. 11, p. 2837-2854, 2010.

VON LUXBURG, U. A tutorial on spectral clustering. Statistics and Computing, v. 17, n. 4, p. 395-416, 2007.

VON LUXBURG, U.; WILLIAMSON, R. C.; GUYON, I. Clustering: science or art? In: ICML Workshop on Unsupervised and Transfer Learning. JMLR Workshop and Conference Proceedings, v. 27, p. 65-80, 2012.

WARD, J. H. Hierarchical grouping to optimize an objective function. Journal of the American Statistical Association, v. 58, n. 301, p. 236-244, 1963.

WEBB, G. I. Discovering significant patterns. Machine Learning, v. 68, n. 1, p. 1-33, 2007.

XIE, J.; GIRSHICK, R.; FARHADI, A. Unsupervised deep embedding for clustering analysis. In: International Conference on Machine Learning (ICML), 2016. PMLR, v. 48, p. 478-487.

ZAKI, M. J. Scalable algorithms for association mining. IEEE Transactions on Knowledge and Data Engineering, v. 12, n. 3, p. 372-390, 2000.

ZHANG, T.; RAMAKRISHNAN, R.; LIVNY, M. BIRCH: an efficient data clustering method for very large databases. In: ACM SIGMOD International Conference on Management of Data, 1996. p. 103-114.

ZOU, H.; HASTIE, T.; TIBSHIRANI, R. Sparse principal component analysis. Journal of Computational and Graphical Statistics, v. 15, n. 2, p. 265-286, 2006.